Virtual University	CS201-Introduction to Programming (Solved Macq's) LECTURE FROM (23 to 45)	
Junaidfazal08@gmail.com Bc190202640@vu.edu.pk	FOR MORE VISIT VULMSHELP.COME	**JUNAID MALIK** **0304-1659294**



AL-JUNAID INSTITUTE GROUP

Question no.1

Which of the following function call is “call by Reference” for the following function prototype?

a. Func(&num)

b. Func(name)

c. A&b

d. None of the given option

Question no.2

When break statement is encountered in switch statement,it_____

a. Exits from switch statement

b. enter from switches statement

c. both a&b

d. none of the given option

Question no.3

The operator ++ and – used to increment or decrement the value of a variable by_____

a.3

b.2

c. 1

d.4

Question no.4

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Which of the following operator is used to access the value of variable pointed by a pointer?

a. *operator
b.->operator
c. &&operator
d. & operator

Question no.5

In case of single dereferencing the value of the_____is the address of the _____

a. Pointer ,variable
b. pointer, constant
c. variable, pointer
d. constant, pointer

Question no.5

The remainder(%) operator is a _____operator.

a. logical
b. Arithmetic
c. Relational
d. Conditional

Question no.6

What will be the output of the following code?

Intx=10;
Cout<<"x="<<x;

a. x=10
b. 10
c. 10=x

Question no.7

The perpose of using cout<<is to_____

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Read the data from keyboard
b. Read the data from file
c. Write into a file
d. Display information on the screen

Question no.8

When an array element is passed to function, it is passed by _____

a.reference
b. data type
c. value
d. data

Question no.9

.C is a/an____language.

Ans.function oriented

Question no.10

_____of a function is also known as signature of a function.

a.
b. Declaration
c.
d.

Question no.11

While programming,it is good to provide an easy to understand and easy to use interface; this programming skill is called_____

a. Scalability
b. Reliability
c. sustainability
d. usability

Question no.12

How many dimensions does n-dimensional array has?

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. 2n dimension
b. (n+1) dimension
c. (n-1) dimension
d. n dimensions

Question no.13

Which of the following function call is"call by reference" for the following function protocol?

Void func(int*)

a. func(int&num)
b. func(&num)
c. func(*num)
d. func(num)

Question no.14

The loop which is most suitable to be used when the number of iteration is known is called_____

a. for
b. while
c. do-while
d. all looping processes require that the iterations be known

Question no.15

In C/C++, the string constant is enclosed in_____

a. curly braces()
b. parentheses()
c. single quotes' '
d. double quotes" "

Question no.16

In order to get the right most digit of a number, we divide this number by 10 and take_____

a. Its remainder

0304-1659294 www.vulmshelp.com

b. Its quotient
c. Its divisor
d. The number

Question no.17

What is the correct syntax to declare an array of size 10 of int data type?

a. int[10] name;
b. name[10] int;
c. int name[10];
d. int name[];

Question no.18

How many bytes of memory are occupied by array'star'?
Char str[]="programming";

a. 10
b. 11
c. 12
d. 13

Question no.19

When the if statement consist more than one statement then enclosing these statement in curly braces is,

a. Not required
b. Good programming
c. Relevant
d. Must

Question no.20

Suppose that an integer type pointer contains a memory address 0x22f230. What will be the new memory address if we increment this pointer by one?

a. 0x22f231

AL-JUNAID TECH INSTITUTE 0304-1639294 www.vulmshelp.com

b. 0x22f234
c. 0x22f226
d. 0x22f238

Question no.21

Which of the following if missing would result in infinite recursion in case of recursive function?

a. Recursive call
b. Base case
c. Function parameters
d. Local variables

Question no.22

Whenever we use a library function or a predefined object or macro, we need to use a_____

a. Source file
b. Object file
c. header file
d. exe file

Question no.23

Switch statement deals with _____type of data

a. Integer
b. Float
c. Character
d. Both Integer and Character

Question no.24

All a template function must have at least_____ generic dats tyoe?

a. Zero
b. One
c. Two
d. Three

0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.25

Which one of the following is the declaration of overloaded pre-increment operator implemented as number of function?

a. Class-name operator+()
b. Class-name operator+(int)
c. Class-name operator++()
d. Class-name++operator++(int)

Question no.26

Class is a user defined_____.

a. Data type
b. Memory reference
c. Value
d. None of the given option

Question no.27

How many bytes will the pointer intPtr of type int move in the following statement? intPtr+=3;

a. 3 bytes
b. 6 bytes
c. 12 bytes
d. 24 bytes

Question no.28

Which of the following is the correct C++ syntax to allocate space dynamically for an array 10 int?

a. new int(10);
b. new int[10];
c. int new(10);
d. int new[10];

Question no.29

AL-JUNAID TECH INSTITUTE
0304-1659294
www.ajmshelp.com

A pointer is a special variable that contain

a. Data values
b. Memory address
c. Both data and value
d. None of the given option

Question no.30

Reference Value Type Data the code is written to____

a. Implement
b. Design
c. Analysis
d. None of the given option

Question no.31

Operator overloading can be performed through_____

a. Classes
b. Operator
c. Function
d. Reference

Question no.32

When a value is referred by a normal variable then it is known as,

a. Direct reference
b. Indirect reference
c. Partial reference
d. Proper reference

Question no.33

Which of the following function is used to increase the size of already allocated memory chunk?

a. malloc
b. calloc
c. realloc

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

d. free

Question no.34

Which of the following is NOT a preprocessor directive?

a. #error
b. #define
c. #line
d. #ndefine

Question no.35

The Stream object **cin** and **cout** are included in which header file?

a. iostream.h
b. fstream.h
c. istream.h
d. ostream.h

Question no.36

Overloaded delete operator function takes the same parameter as an arguments returned by a new operator function.

a. True
b. False

Question no.37

When an array of object is created by dynamically then there is no way to provide parameterized constructors for array of object.

a. True
b. False

Question no,38

C is widely known as development language of___ operating system.

a. Linux
b. Unix

AL-JUNAID TECH INSTITUTE
www.vulmshelp.com

c. Windows
d. Mac os

Question no.39

Computer can understand only machine language code.

a. True
b. False

Question no.40

We cannot define a function as a friend of a Template class.

a. True
b. False

Question no.41

What will be the value of 'a' and 'b' after executing the following statement?

A=3;
b= a++;

a. 3,4
b. 4,4
c. 3,3
d. 4,3

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Question no.42

_____is used to trace the logic of the program and correct the logical errors.

a. Compiler
b. Editor
c. Linker
d. Debugger

Question no.43

New and delete are_____ whereas malloc and free are____.

a. Function, operator
b. Classes , operator
c. Operator , function
d. Operator, classes

Question no.45

Like member function,_____can also access the private data members of a class.

a. Non-member function
b. Friend function
c. Any function outside class
d. None of the given options

Question no.46

Which situation would require the use of non-member overloaded operator?

a. The overloaded operator is an Assignment operator
b. The left most operand is an object of a class.
c. The left operand is built-in data type.
d. The operator returns a reference.

Questions no.47

The stream insertion and stream extraction operator are already overloaded for_____.

a. User-defined data type
b. Built in data type
c. Both “a” and “b”
d. None of the given option

Question no.48

If we define an identifier with the statement **#define PI 3.1415926** then the execution of the program the value of PI_____.

AL-JUNAID TECH INSTITUTE
0304-1659291
www.vulmshelp.com

a. Can not be replaced
b. None of the given option
c. Remain constant
d. Can be changed by some option

Question no.49

Assignment operator is_____associative.

a. Right
b. Left
c. Binary
d. Unary

Question no.50

Whenever dynamic memory allocation is made in C/C==, it is freed___.

a. Explicitly
b. Implicitly
c. Both explicitly and implicitly
d. None of the given option

Question no.51

The appropriate data type to store the number of rows and columns of the matrix is _____.

a. Float
b. Int
c. Char
d. None of the given option

Question no.52

The function free() return back the allocated memory got through calloc and malloc to_____.

a. Stack
b. Heap

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

c. Stack and heap
d. None of the given option

Question no.53

Width() is member function of _____.

a. Cin object
b. Cout object
c. Both cin and cout object
d. None of the given object

Question no.54

Templates are not type safe.

a. True
b. false

Question no.57

A Matrix can be composed of int, float or doubles as their elements. Best way is to handle this,

a. write a separate class to handle each
b. use string to store all types
c. use templates
d. none of the given option

Question no.58

In if structure the block of statement is executed only,

a. when the condition is false
b. when it contain arithmetic operator
c. when it contain logical operator
d. when the condition is true

Question no.59

Header file **fstream.h** include the definition of the stream classes_____.

AL-JUNAID TECH INSTITUTE
0304-1659294
www.pucitshelp.com

a. Ifsream, fstream, ofstream
b. Ifstram, ofstream, cout
c. Fstream, cin, cout
d. None of the given option

Question no.60

To access the data members of structure_____is used.

a. Dot operator(.)
b. * operator
c. &operator
d. None of the given

Question no.61

Eof(), bad(), good(), clear() all are manipulators.

a. True
b. False

Question no.62

Which kind of function can access private member variables of a class?

a. Friend function of the class
b. Private member function of the class
c. Public member function of the class
d. Friend, private and public function

Question no.63

The return type of operator function must always be void.

a. True
b. False

Question no.64

Friend function of a class is____.

a. Member function

AL-JUNAID TECH INSTITUTE
www.aljunaidhelp.com

b. Non-member function
c. Private function
d. Public function

Question no.65

Function implementation of friend function must be defined outside the class.

a. True
b. False

Question no.66

The normal source of **cin** object is,

a. File
b. Disk
c. Keyboard
d. Ram

Question no.67

Which of the following is correct way to initialize a variable x of int type with value 10?

a. Int x; x = 10
b. Int x=10;
c. Int x, x = 10
d. X =10

Question no.68

With the function, the compiler automatically detects the passed data and generate a new copy of function using passed data.

a. True
b. False

Question no.69

What will be the correct syntax to declear two-dimensional array of float data type?

AL-JUNAID TECH INSTITUTE 0304-1659294 www.aljunaidtech.com

a. Float{2}{2};
b. Float arr[2][2];
c. Float arr[2,2]
d. Float[2][2]

Question no.70

Heap is a constantly changing in size.

a. True
b. False

Question no.71

While calling function, the arguments are assigned to the parameters from_____.

a. Left to right
b. Right to left
c. No space order is followed
d. None of the given option

Question no.72

Classes defined inside other classes are called_____.

a. Looped
b. Nested
c. Overloaded
d. None of the given option

Question no.73

If we define an identifier with the statement #**defined PI 3.1415926** then during the execution of the program the value of PI____.

a. Cannot be replaced
b. None of the given option
c. Remain constant
d. Can be changed by some operation

Question no.74

AL-JUNAID TECH INSTITUTE
0304-1659294
www.aljunaidtechinstitute.com

Which value is returned by the destructor of a class?

a. A pointer to the class.
b. An object of the class
c. A status code determining whether the class was destructed
d. Destructor do not return a value.

Question no.75

Every class contains_____.

a. Constructor
b. Destructor
c. Both a and b
d. None

Question no.76

A template function must have

a. One or more than one arguments
b. Only one arguments
c. Zero arguments
d. None

Question no.77

Structured query language is used for_____.

a. Database management
b. Network
c. Writing operating system
d. None

Question no.78

When a call to user defined function finishes the variable defined inside the function still in existence.

a. True
b. False

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.79

The precedence of an operator can be changed through operator overloading.

a. True

b. False

Question no.80

"delete" operator is used to return memory to free store, which is allocate by the "**new"** operator.

a. True

b. False

Question no.81

Pointer is a variable which store.

a. Data

b. Memory address

c. Data type

d. values

Question no.82

All preprocessor directives are started with the symbol____.

a. *

b. +

c. @

d. #

Question no. 83

The number 544.53 must be stored in_____data type.

a. Int

b. Short

c. Float

d. Char

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.84

A template function can have different types of arguments.

a. True
b. False

Question no.85

Templates class cannot have static variable.

a. True
b. False

Question no.86

Which of the following is used with bit manipulation?

a. Signed integer
b. Un signed integer
c. Signed double
d. Un-signed double

Question no.87

Structure is a collection of____ under a single name.

a. Only function
b. Only variable
c. Both function and variable
d. None

Question no,88

We can also do conditional compilation with preprocessor directives.

a. True
b. False

Question no.89

If a symbolic constant has been defined, it will be an error to define it again.

a. True

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. False

Question no.90

The code is written is to____the program.

a. Implement
b. Design
c. Analysis
d. None

Question no.91

Template are not type safe.

a. True
b. false

Question no.92

*doc is_____by type.

a. sequential file
b. Random Access File
c. Data File
d. Record File

Question no.93

Which of the following is NOT a preprocessor directive?

a. #error
b. #define
c. #line
d. #ndefine

Question no.94

The return type of operator function must always be void.

a. True
b. false

Question no.95

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

What does (*this) represent?

a. The current function of the class.
b. The current pointer of the class.
c. The current object of the class.
d. A value of a data number.

Question no.96

The statement **cin.get()** is used to

a. Read a string from keyboard
b. Read a char from keyboard
c. Read a string from file
d. Read a char from file

Question no.97

C++ is a sensitive language

a. True
b. False

Question no.98

Which one of the statement given below is equivalent to **sum+=j*j**?

a. sum=sum+j*j
b. sum= sum+j
c. sum= sum*j
d. sum*j=j

Question no.99

.All element of an array must be of_____

a.different data type
b.:same datatype
c.float data type
d. char data type

Question no.100

AL-JUNAID TECH INSTITUTE
0304-1659294
www.aihelp.com

AL-JUNAID INSTITUTE GROUP

In C/C++, null character is represented as_____

a. /n
b. %c
c. #d
d. *c

Question no.101

The dynamic memory allocation uses memory from the____

a. Heap
b. Stack
c. Values
d. none

Question no.102

When a value is reffered by a normal variable then it is known as___

a. Direct Referance
b. Indirect reference
c. Both a and b
d. none

Question no.103

Assignment operator is used for_____

a. assigning values to variables
b. assigning value to a constant
c. assigning constant and variables
d. none of the given option

Question no.104

Which of the following is not an example of int datatype?

a. -4.0
b. +4.2
c. -3.0
d. -2.0

AL JUNAID TECH INSTITUTE
AJ
Skype: live:.cid.1659294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.105

For which array,the size of the array should be one more than the number of elements in an array?

a. Char
b. Variable
c. Float
d. Intiger

Question no.106

Which header file should be included to use functions like malloc() and calloc()?

Ans:stdio.h

Question no.107

Using a segment of source code again by adding new functionalities with slight or no modification is termed as_____

Ans:code reusability

Question no.108

The program in which we allocate static memory run essentially on_____

Ans:Stack

Question no,109

TWAIN stand for___

Ans:Technology Without An Interesting Name

Question no.110

Which of the following will be the correct function call for function prototype given below?

Ans:func(&num)

AL-JUNAID TECH INSTITUTE
0304-1659294
www.aljunaidtechinstitute.com

AL-JUNAID INSTITUTE GROUP

Question no.111

The operator used for casting in C is standard_____

Ans:cast

Question no.112

The stream insertion and extraction operators____

Ans:must be overloaded as non-member function

Question no.113

The operator function for stream insertion(>>)and stream extraction(<<) must be_____

Ans:Non-Member function of Class

Question no.114

In statement Matrix m2=m1;

Ans:Assignment operator is being used

Question no.115

A template function must have at least_____generic datatype

Ans:One

Question no.116

Templates provide way of abstracting_____information.

Ans:type

Question no.117

ANSI stand for_____

Ans:American National Standards Institute

Question no.118

By using object as class members,_____is achieved.

Ans:Accessibility

0304-1639294
www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.119

In operator overloading,a non-member unary operator function takes_____arguments.

Ans:1

Question no.120

A class whose object is contained as const object, must have_____

Ans:Parameterized Constructor

Question no.121

It is possible to define a class within other class.

a. True
b. false

Question no.122

Where we can include a header file in the program?

a. Any where
b. In start
c. At the end
d. None of the given option

Question no.123

The return type of the operator function >> operator.

a. Class for which we overloaded this operator
b. Reference of ostream class
c. Reference of istream class
d. Void

Question no.124

With user defined data type variables(object) self assignment can produce_____.

a. Syntax error
b. Logical error

AL JUNAID TECH INSTITUTE
AJ
www.vulmshelp.com

c. Link error
d. None of the given option

Question no.125

We can also do conditional compilation with preprocessor directives.

a. True
b. False

Question no.126

What will be the output of the following statement?

Cout<<setbase(16)<<52;

a. 74
b. 52
c. 34
d. 64

Question no.127

Dec, her, oct are all_____.

a. Member function
b. Object of input/output stream
c. Parameterized manipulator
d. Non- parametrized manipulator

Question no.127

The input output stream; cin cout are___.

a. Operator
b. Function
c. Object
d. Structures

Question no.128

Friend function are_____of a class

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Member functions
b. Public member function
c. Private member function
d. Non-member function

Question no.129

Template class can not have static variables.

a. True
b. False

Question no.130

In a group of a nested loops, which loop is executed the most member of times?

a. The outermost loop
b. The innermost loop
c. All loops are executed
d. Cannot be determined

Question no.131

Which looping process checks the test condition at the end of the loop?

a. For
b. While
c. Do while
d. No looping process checks the test condition at the end.

Question no.131

We can delete an array of object without specifying[] bracket if a class is not dynamic memory allocation internally.

a. True
b. False

Question no.132

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

A stream is an ordered sequence of bytes.

a. True
b. False

Question no.133

The normal source of **cin** object is,

a. File
b. Disk
c. Keyboard
d. Ram

Question no.134

We can also create an array of user define data type.

a. True
b. False

Question no.135

Which of the following is valid class declaration?

a. Class A{int x;};
b. Class B{]
c. Public class A{ }
d. Object A{ int x;};

Question no.136

If Num is an integer variable then Num++ means,

a. Add 1 two times with Num
b. Add 1 with Num
c. Add 2 with Num
d. Subtract 2 from Num

Question no.137

Automatic variable are created on-_____.

a. Heap

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. Free storage
c. Static storage
d. Stack

Question no.138

We can also define a user-defines manipulator.

a. True
b. False

Question no.139

Class is a user defined_____.

a. Data type
b. Memory reference
c. Value
d. None of the given option

Question no.140

The static members of a class are initialized_____.

a. At file scope
b. Within class definition
c. Within member function
d. Within main function

Question no.141

The data members of the class are initialized_____

a. At runtime
b. Within main group
c. Outside the function
d. At compile time

Question no.142

In flow chart, flow of control is represented by____

a. Rectangle
b. Circle

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

c. Diamond
d. Arrow

Question no.143

A constructor has the same name as that of_____

a. Variable
b. Class
c. Structure
d. Function

Question no.144

Whenever **New** operator is called to create an object, the_____

a. Constructor
b. Destructor
c. Pointer
d. Array

Question no.145

The destructor for all the objects in the program in automatically called when_____

a. Memory is dynamically allocated to object
b. Object are used inside function
c. Array of object is decleared
d. Program is terminated

Question no.146

What will be the correct syntax of declearation of the following statement?

Ptr is a pointer to cons int

a. Const int*ptr
b. Int* const ptr,
c. Int const* ptr

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

d. Const* int ptr

Question no.148

Sequence of event(s) when deallocating memory using delete operator_____

a. Only block of memory is deallocated for object
b. Only destructor is called for object
c. Memory is deallocated first before calling destructor
d. Destructor is called first before deallocating memory

Question no.149

Consider the following segment'
Class M{
Public;
M&operator+ (constM&);

a. Member function
b. Non- member function
c. Friend function
d. Virtual function

Question no.150

if there is a symbol(& sign) used with the variable name following by data type then it refers to _____

a. Logic expression
b. Value of variable
c. Reference variable
d. Address of variable

Question no.151

New and delete are also used with _____and primitive data typed as well.

a. Loops
b. Sructures , pointer

0304-1659294 www.ulmshelp.com

c. Classes , structures
d. Constant

Question no.152

Users must not know about_____

a. Methods functionality with in class
b. Object of class
c. Functions name within class
d. Class implementation of functionality and interfaces

Question no.153

What should be the return type of the constructed?

a. Void
b. Int
c. Same as object type
d. Construction do not return any thing

Question no.154

Which is NOT a protection level provided by classes in C++?

a. Protected
b. Hidden
c. Private
d. Public

Question no.155

A macro when takes arguments is called_____

a. Function
b. Procedure
c. Parameterized macro
d. Simple macro

Question no.156

The order of destruction of an object is_____

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

a. Same as the construction order
b. Reverse as the construction order
c. Independent of the construction order
d. Same as the order of object decleared

Question no.157

We can use cin.peak() function to__-

a. See the next character that we are going to get
b. Get to the end of the object buffer
c. Get to the end of input buffer
d. See the first character of input buffer

Question no.158

The normal source of cin object is____

a. File
b. Disk
c. Keyboard
d. Ram

Question no.159

The initializer list is used to initialize the contained objects of a class at_____

a. Run time
b. Debugging time
c. Construction time
d. Compile time

Question no.160

_____data members of a contained object can be accessed from inside of the containing class.

a. Private
b. Public
c. Protected

0304-1659294
www.vulmshelp.com

d. All

Question no.161

A class whose object is contained as const object, must have_____

a. Getter/setter function
b. Default constructor
c. Parameterized constructor
d. Operator overloading function

Question no.162

The appropriate data type to store the number of rows and coloums of the matrix_____

a. Float
b. Strings
c. Int
d. Char

Question no.163

The code is written to_____ the program

a. Implement
b. Design
c. Analysis
d. Test

Question no.164

When a call to user-defined function finishes, the variable defined inside the function will_____

a. Still in existence
b. Not exist further
c. Finish but a copy will remain in memory
d. None of the given

Question no.165

The static data member of a class can be accessed by_____

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Only class
b. Only object
c. Both class and object
d. Function out side the class

Question no.166

The static data members of a class are initialized_____

a. At fill scope
b. Within class definition
c. Within member function
d. Within main function

Question no.167

A structure brings together a group of_____

a. Same data type
b. Different data type
c. Constant
d. Both a,b

Question no.168

The C language was developed in late 60's and 70's in_____

a. Richards laboratories
b. Microsoft laboratories
c. Bell laboratories
d. IBM laboratries

Question no.169

To assigned a value to a character type variable_____are used around the value.

a. Parenthesis()
b. Single quotes ' '
c. Double quotes " "
d. Curly braces{ }

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

AL-JUNAID INSTITUTE GROUP

Question no.170

The order of destruction of an object is _____

a. Same as the construction order
b. Reverse as the construction order
c. Independent of the construction order
d. Same as the order of object declare

Question no.171

In flow chart, flow of control is represented by_____.
(Please select correct option)

a. Circle
b. Diamond
c. Arrow
d. All

Question no.172

The operator ++ and – are used to increment or decrement the value of a variable by _____.

a. 3
b. 2
c. 1
d. 4

Question no.173

To assigned a value to a character type variable,_____are used around the value.

a. Parenthesis ( )
b. Single quotes ‘ ’
c. Double quotes “ ”
d. Curly braces { }

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vuhelp.com

AL-JUNAID INSTITUTE GROUP

Question no.174

Which of the following syntax will be used to initialize an array of size 5 of int data type to value 0?

a. Arr[5] = {0} ;
b. int arr[5] = 0 ;
c. int arr[5] = (0) ;
d. int arr[] = 0 ;

Question no.175

Let ptr 1 and ptr 2 are pointer variables that points to integer data types then which one of the following arithmetic is allowed?

a. Ptr1 + ptr2
b. Ptr1 – ptr2
c. Ptr1 * ptr2
d. Ptr1 / ptr2

Question no.176

When an argument is passed by reference,_____

a. The function cannot access the argument's value
b. A variable is created in the function to hold the argument's value
c. A temporary variable is created in the calling program to hold the argument's value
d. The function accesses the argument's original value in the calling program

Question no.177

In year_____C was approved as a standard language worldwide.

a. 1988
b. 1967

AL-JUNAID TECH INSTITUTE
©0304-1639294
www.vulmshelp.com

c. 1989

d. 1986

Question no.178

The C language was developed in late 60's and early 70's, in _____

a. Richards Laboratories
b. Microsoft Laboratory
c. Bell Laboratory
d. IBM Laboratory

Question no.179

If a friend function outside the class declares itself friend of the class, this may violate the concept of _____

a. Encapsulation and data accessing
b. Encapsulation and data hiding
c. Friend member functions of the class
d. Interface of the class

Question no.180

A structure brings together a group of _____

a. Same data type
b. Different data type
c. Constants
d. Both a, b

Question no.181

You can use a _____statement to transfer control elsewhere out of the nested structure.

a. Continue

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b. Goto
c. Pause
d. Break

Question no.182

Which of the given operator is not a valid compound assignment operator?

a. >>=
b. =>>
c. *=
d. &=

Question no.183

The purpose of using cout<< is to _____

a. Display information on the screen
b. Read the data from keyboard
c. Read the data from file
d. Write into a file

0304-1659294
www.Vulmshelp.com

Question no.184

Assignment operator is used for_____

a. Calculation
b. Reading
c. Assigning values to variables
d. Equality checking

Question no.185

Missing semicolon ‘ ;’ at the end of C++ statement causes a _____ error.

a. Logical
b. Syntax
c. Runtime
d. Compile time

Question no.186

Which of the following is not an example of int data type?

a. 0
b. -32
c. 65531
d. -4.0

Question no.189

For which array , the size of the array should be one more than the number of elements in array?

a. Int
b. Double
c. Float
d. Char

Question no.190

If an integer takes 4 bytes in the memory then how many bytes will be in the memory for following code?

#include<iostream.h>

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

```
#include< conio.h>

Int main()
```

a. 12 bytes
b. 24 bytes
c. 48 bytes
d. 32 bytes

Question no.191

Using a segment of source code again by adding new functionalities with slight or no modification is termed as _____

a. Code efficiency
b. Code reusability
c. Code notification
d. Code compatibility

Question no.192

A matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this_____

a. Wrire a separate class to handle each
b. Use templates
c. Use strings to store all types
d. None of the given option

Question no.193

When a call user defined function finishes, the variable defined inside the function is still in existence.

AL-JUNAID TECH INSTITUTE 0304 1659294 www.ajmshelp.com

a. True
b. false

Question no. 194

Within the statement **obj1 = obj2 ; *obj1*** will call the assignment operator function and ***obj2*** will be passed as an argument to function.

a. True
b. False

Question no.195

What is the sequence of events(s) when deallocating memory using delete operator?

a. Only block of memory is deallocated for object
b. Only destructor is called for objects
c. Memory is deallocated first before calling destructor
d. Destructor is called first before deallocating memory

Question no.196

The second parameter of operator function for << and >> are objects of the class for which we are overloading these operator.

a. True
b. False

Question no.197

0304-1659294
www.vulmshelp.com

For which values of the integer_ value will be the following code becomes an infinite loop?

```
Int number=1;

While(true) {

Cout<< number;

If(number==3) break;

Number+= integer_ value: }
```

a. Any number other than 1 or 2
b. Only 0
c. Only 2
d. Only 1

Question no.198

Unary operator implemented as member function takes___ arguments whereas non-member function takes_____arguments.

a. New int (10)
b. New int [10]
c. Int new (10)
d. Int new [10]

Question no.199

What will be the correct syntax to assign an array named ***arr*** of 5 elements to a pointer ***ptr***?

a. *ptr = arr;

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b. Ptr = arr;
c. *ptr = arr[5];
d. Ptr = arr[5];

Question no.200

We can also do conditional compilation with preprocessor directive?

a. True
b. false

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Virtual University	CS201-Intorduction to Programming (PRACTCAL) (Solved Macq's) LECTURE FROM (1 to 45)	VU LMS HELP vulmshelp.com
Junaidfazal08@gmail.com Bc190202640@vu.edu.pk	FOR MORE VISIT VULMSHELP.COME	JUNAID MALIK 0304-1659294

AL-JUNAID TECH INSTITUTE



For CS619 Project Feel Free To Contact With Me

Ph# 0304-1659294
Email: junaidfazal08@gmail.com

AL-JUNAID TECH INSTITUTE

**Q.1 We cannot declare a function as a a friend of ----------
classes**

Ans: istream or ostream

Q.2 Automatic variable are created constantly and destroyed

Ans: All the time

**Q.3 are a conditional complication process that is used to
replace a piece of code by its definition in a programe**

Ans: function

**Q.4: calloc () functions allocates multiple blocks of memory
and each block have---------- ?**

Ans: same size

Q.5: in C++ sizeof operator is used to determine----------

Ans: not confirm

Q.6:which of the following header file deals with streamextraction?

Ans: iostream.h

**Q.7: Type to use getch() function without including
#include<conio.h> header file in the program is a ---------- error**

Ans: compilation

Q.8: in C++ the default constructor takes----------

0304-1639994
www.vulmshelp.com

Ans: Only one argument

Q.9: in C++ the name of the ----------is the same as that of a class with a preceding tild (~) sign.

Ans: Destructor

Q10: which of the following option returns the state of end file? Ans: cin.eof()

Q.11: If the memory in the free store is not sufficient enough tofulfill the request , malloc () function return----------

Ans; Null

Q.12: in C/C++ language . memory at runtime is allocated from ----------

Ans: Stack

Q.13: When overloading assignment (=) operator, which of the following syntax is correct?

Ans: void operator(=) const string &other:

Q.14: which of the following code:

Ans :*7**

Q.15: For overloading minus (-) operator, which memberoperator function will be correct?

Ans: complex operator-(complex c)

Q:16: In C++ which operator is used far memory allocated ?

Ans: New

0304-1639294

AL-JUNAID TECH INSTITUTE

Q.17:The Statement char array [10]; is an example of a type of -- ---------- memory allocation

Ans: staic

Q.18:which of the following option not a source of the outputsteam?

Ans: file

Q.19: A sequare matrix has----------

Ans: Equal number of rows and columns

Q.20:The statement *double elements; * says that *elements*is a / an ---------- to double.**

Ans: Array of pointers

Q.21: A matrix is nothing but a---------- of numbers.

Ans: Two- dimensional array

Q.22: A programmer can ---------- the simple matrix class to handle the integer, float, or double data type as elements of the givenmatrix.

Ans: Hide

Q.23: in C and C++ language, every expression has a---------- of its own?

Ans: Value

Q.24: The cycle of testing and---------- continues until the program is working perfectly without any problem.

Ans: Bug fixing

0304-1659294

AL-JUNAID TECH INSTITUTE

Q.25: Default scope for members of structures is ----------and default visibility for class members is ----

Ans: Public, Private

Q.26: If A and B are two matrices of order M*N to be added thentheir resultant matrix will have an order of

Ans: M*N

Q.27: if *m* is a matrix object, then self-assignment can beavoided by using following code statement

Ans: If (&m!=this);

Q.28: Which of the following is correct formate of templatefunction?

Ans: Template <class T>

Q.29: In template class, compiler makes a copy of ---------- and convert in to -----

Ans: Source code , Object code

Q.30: Far a matrix, the variables of Rows and Columns shouldalways be a -------

Ans: Whole Number

Q:31: After implementation of code is completed, we do its ----- to verify that it is behaving properly in all scenarios.

Ans: Testing

Q.32: which three constructs do we need to solve any problem?

Ans: Classes, structures, functions

Q.33: Declaration and definition of a template function shouldbe in the ---------- file

Ans: same

Q.34: For template function, we must have at least --------- generic argument.

Ans: One

Q.35: In C++ , which of the following option has the correct meaning of the expression"a<=b"?

Ans: a is less than or equal to b

Q.36: which of the following code?

Ans: 2 4 6 8

Q.37: What will be the output of the following code?Int

x=15;

Iny y=10;

Int z= 5;

Cout<< x*x-4*y*z;

Ans: 25

Q.38:Which of the following expression is correct in C/C ++language?

Ans: X=X+3

0304-1659294

AL-JUNAID TECH INSTITUTE

Q.39: Which of the following C++ statements I used take inputfrom the user?

Ans: cin

Q.40: Which of the following is an example of logical operator?

Ans: &&,II

Q.41: in C/C ++ , which of the following data type is used tostore real numbers?

Ans: Float

Q.42: C/C++ has many libraries which contain variables andfunction names normly starting with ------

Ans: Underscore

Q.42: The coding of a program is translated into machinelanguage by ------

Ans: Compiler

Q.43:Learning "How to program" is important, because it develops --------- and problem solving abilities.

Ans: Analytical

Q.44: in C/C++ , the algebraic expression (b2-4ac)/2a cab bewritten as:

Ans: (b*b-4*a*c)(2*a)

Q.45: In C/C++, which of the following data type is used to storereal member?

Ans: Float

AL-JUNAID TECH INSTITUTE

Q.46: To understand programming concepts,----------
programming language being used as a vehicle in cs201"Introduction to programming" course.

Ans: C/C++

Q.47: The bit manipulators operator---------- is used to check weather a specific bit is set or not.

Ans: &

Q48.: in C++, which operator is used while accessing throughthe pointer to structure?

Ans: ->

Q.49: Function seekg() and seekp() require an argument of type to let them know how many bytes to move forward backword.

Ans: Long

Q:50: Macro is defined by ----------preprocessor directive.

Ans: #define

Q.51: In C++ , when accessing files randomly, which function isused at get current pointer position inside a file?

Ans: tellg()

Q.52:Which is not a preprocessor directive of C-Language?

Ans: # Startif

Q.53:The statement char arr [10]; is an example of a type of -----
-- memory allocation

Ans: Built-in

Q.54: What would be the output of the following statement?

C=13%2;

Ans= 1

Q.55: Which of the following two array are copy-able?

Ans: int a[5],int b[5]

Q.56: A variable declared inside a code block becomes the ------- variable for that block.

Ans: Local

Q.57: Which type of arrays are used while dealing withsentences?

Ans: Character arrays

Q.58: In the call by reference method, which operator is used toget the address of a variable?

Ans: &

Q.59: Which of the following is the correct method to declare anarray?

Ans: data-type array-name[size];

Q.60: char name []= "Hello World";

In the above statement, a memory of ----------characters will be allocated to the array name.

Ans: 12

0304-1659294 www.vulmshelp.com

AL-JUNAID TECH INSTITUTE

Q.61 In C++ , what is the correct syntax for accessing the fifthelement of an array?

Ans: a(4)

Q.62: Which of the following shows the memory address of thefirst element in an array?

Ans: array[0]

Q.63 In C++ , the precise way to declare a pointer is ,

Ans: int*ptr;

Q.64: Identify the invalid option.

Ans: int *pi=0;

Q.65 ----------character is used to terminate a string.

Ans: Null

Q.66: which of the following is an example of repetition structure?

Ans: while

Q.67: Function declaration is a one-line statement in which wewrite the return type,---------- of the function and the---------- of arguments

Ans: Name, Data type

Q.68: If the condition is not made false in while loop, what willhappen?

Ans: Program will terminate only

0304-1659294

AL-JUNAID TECH INSTITUTE

Q.69: Which of the following optio gives the [value] stored atthe address pointed to by the pointer "ptr"?

Ans: *ptr

Q.70: in C++ , cout is known as ----------

Ans: output stream

Q.71: Which of the following is not a file opening mode?

Ans: ios:: trunk

Q.72: In C++ , the precise way to declared a pointer is.

Ans: int*ptr;

Q.73 ---------- statement is necessary in switch structure.

Ans: Break

Q.74: Normally, the float data type is half of the size of ----------

Ans: Double

Q.75: which of the following option is used when working withcompleted expression in c++?

Ans: ()

Q.76: A binary operator has ---------- operands.

Ans: Two

Q.77:which of the following is the default function calling mechanism of C/C++?

Ans: Call by value

© 0304-1659294 www.vulmshelp.com

AL-JUNAID TECH INSTITUTE

Q.78: which function is automatically called when an object of aclass is initialed?

Ans: constructor

Q.79: in C++, the default constructor takes----------

Ans: No arguments

Q.80: In C++, which of the following option is used to declare aclass?

Ans: class

Q.81:in C++ which of the following function causes the full definition of the function to be inserted in each place when itgots called?

Ans: Inline function

Q.82:Bool data type takes---------- values.

Ans: 2

Q.83: In the syntax of the overloaded operator function givenbelow

Complex operator + (Complex &);

What is operator?

Ans: keyword

Q.84:which operator can be overloaded?

Ans: not confirm

Q.85:Complex codd (complex c1, complex c2);

03041659294
www.vulmshelp.com

Which statement is true far above code?

Ans: Creating object of class complex with name codd with parameters.

Q.86: Which of the following operator can be overloaded?

Ans: ?

Q.87:Which of the following is assignment operator?

Ans: =

Q.88: which option will be correct in order to free the space ofmemory allocated to a buffer (but)?

Ans: Delete but

Q.89: When overloading assignment (=) operator, which of thefollowing syntax is correct?

Ans: void operator=(const String &s);

Q.90:Complex codd (Complex c1, Complex c2);

Which of statement is true for above code?

Ans: It accepts two complex numbers as parameters and returns back the resultant complex number.

Q.91: What is correct example of binary operator?

Ans: -

Q.92: Which of the following options not a source of the outputstream?

Ans: region in the memory

Q.93: How many option are initialized for the class?Date

my Dates[10];

Ans: 10

Q.94:which statement about pointer is not true?

Ans: pointer can be void

Q.95:Representing logical depiction of the solution to theproblem is known as a ----------

Ans: Structure

Q.96: Which statement about manipulators is not true?

Ans: it automatically gets that object as parameter passed in to the function

Q.97:which of the following structure is correct for WHILEloop?

Ans: While (condition)

Q.98:which of the following is true about the switch statement?

www.vulmshelp.com

Ans: It cannot handle the compound conditions which use logical operators

Q.99: If the condition is not made false in while loop, what will happen?

Ans: Loop will become infinite

Q.100: If a function does not return anything, its return typewill be ----------

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Ans: void

Q.101:which of the following is the first step in FOR loop?

Ans: Initialization condition

Q.102: When we call a function and pass an argument, an objector variable to the function, it is called ?

Ans: call by value

Q.103: Stream insertion(<<) and extraction operators (>>) arealways implemented as ---------- function.

Ans: Non-member

Q104: which of the following statement is not true about overloading stream extraction operator(>>)?

Ans: it can be a member operator.

Q.105: endl is a type of ---------- manipulator.

Ans: Parameterized less

0304-1659294
www.vulmshelp.com

FINALTERM EXAMINATION 2011
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
All A template function must have at least ---------- generic data type

► Zero
► One (Page 499)
► Two
► Three

Question No: 2 (Marks: 1) – Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword
. ► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)

Question No: 3 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

► Class-name operator +() ;
► Class-name operator +(int) ;
► **Class-name operator ++() ; (Page 389)**
► Class-name operator ++(int) ;

Question No: 4 (Marks: 1) - Please choose one
Class is a user defined_____.

► **data type (Page 317)**
► memory referee
► value
► none of the given options.

Question No: 5 (Marks: 1) - Please choose one
How many bytes will the pointer intPtr of type int move in the following statement? intPtr += 3 ;

► 3 bytes
► 6 bytes
► **12 bytes**
► 24 bytes

Question No: 6 (Marks: 1) - Please choose one
Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

► new int(10) ;
► **new int[10] ;**
► int new(10) ;
► int new[10];

Question No: 7 (Marks: 1) - Please choose one
A Pointer is a special variable that contain

► Data values
► **Memory Address (Page 267)**
► Both data and values
► None of the given option

Question No: 8 (Marks: 1) - Please choose one
Reference Value Type Data The code is written to _____ the program.

► **implement**
► design
► analysis
► none of the given options

FINALTERM EXAMINATION
CS201 - Introduction to programming
Final Term Spring 2010

Operator overloading can be performed through_____.
► Classes
► **Function (Page 371)**
► Operators
► Reference

Question No: 2 (Marks: 1) - Please choose one
When a value is referred by a normal variable then it is known as,

► **Direct Reference (page 143)**
►Indirect Reference.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► Partial Reference
► Proper Reference

When a value is referred by a normal variable is known as direct reference

Question No: 3 (Marks: 1) - Please choose one

Which of the following function is used to increase the size of already allocated memory chunk?

► malloc
► calloc
► **realloc (Page 285)**
► free

Question No: 4 (Marks: 1) - Please choose one

Which of the following is NOT a preprocessor directive?

► #error
► #define
► #line
► **#ndefine (Page 272)**

Question No: 5 (Marks: 1) – Please choose one

The stream objects **cin** and **cout** are included in which header file?

► **iostream.h (Page 15)**
► fstream.h
► istream.h
► ostream.h

Question No: 6 (Marks: 1) – Please choose one

Overloaded delete operator function takes the same parameter as an argument returned by new operator function.

► **True (Page 414)**
► False

The same pointer that is returned by the new operator, is passed as an argument to the delete operator. These rules apply to both, if operators (new and delete) are overloaded as member or non-member operators (as global operators).

Question No: 7 (Marks: 1) – Please choose one

When an array of object is created dynamically then there is no way to provide parameterized constructors for array of objects.

► **True (Page 414)**
► False

Question No: 8 (Marks: 1) – Please choose one

C is widely known as development language of _____ operating system.

- ► Linux
- ► Windows
- ► **Unix (Page 12)**
- ► Mac OS

In the start C became widely known as the development language of the UNIX operating system, and the UNIX operating system was written by using this C language. The C language is so powerful that the compiler of C and other various operating systems are written in C.

Question No: 9 (Marks: 1) - Please choose one

Computer can understand only machine language code.

- ► **True**
- ► False

Question No: 10 (Marks: 1) - Please choose one

We can not define a function as a friend of a Template class.

- ► True
- ► **False (Page 151)**

Class templates can have friends. A class or class template, function, or function template can be a friend to a template class. Friends can also be specializations of a class template or function template, but not partial specializations.

Question No: 11 (Marks: 1) – Please choose one

What will be the value of 'a' and 'b' after executing the following statements?

```
A = 3;
b = a++;
```

- ► 3, 4
- ► 4, 4
- ► 3, 3
- ► **4, 3**

Question No: 12 (Marks: 1) – Please choose one

Consider the following code segment. What will be the output of following code?

```
Int addValue (int *a){
int b = (*a) + 2;
return b ;
}
main () {
int x =6 ;
cout <<  x << "," ;
cout << addValue(&x) << "," ;
cout <<  x ;
}
```

► **6,8,6**
► 6,6,8
► 6,8,8
► 6,6,6

Question No: 13 (Marks: 1) – Please choose one
_____ is used to trace the logic of the program and correct the logical errors.
► Compiler
► Editor
► Linker
► **Debugger (Page 13)**

Question No: 14 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Hence, we can call new and delete operators, P# 342
we have allocated a memory space for our use by malloc function. P# 285

Question No: 15 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 16 (Marks: 1) – Please choose one
Which situation would require the use of a non-member overloaded operator?

► The overloaded operator is an Assignment operator.
► **The left most operand is an object of a class.(Page 375)**
► The left operand is built-in data type.
► The operator returns a reference.

When an operator function is implemented as a non-member function, the left-most operand may be an object of the operator's class, an object of a different class, or a built-in type

Question No: 17 (Marks: 1) - Please choose one
The stream insertion and stream extraction operators are already overloaded for _____.
► User-defined data types
► **Built-in data types http://www.sstutor.com/cpp/stream1.htm**
► User-defined and built-in data types
► None of the given options

Question No: 18 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

- ► can not be replaced
- ► None of the given options
- ► **Remain constant. (Page 271)**
- ► can be changed by some operation

Question No: 19 (Marks: 1) - Please choose one
Assignment operator is ----------associative.

- ► **right**
- ► left
- ► binary
- ► unary

Question No: 20 (Marks: 1) - Please choose one
When ever dynamic memory allocation is made in C/C++, it is freed_____.

- ► **Explicitly (Page 527)**
- ► Implicitly
- ► Both explicitly and implicitly
- ► None of the given options

Question No: 21 (Marks: 1) – Please choose one
The appropriate data type to store the number of rows and colums of the matrix is_____.

- ► float
- ► **int (Not sure)**
- ► char
- ► none of the given options.

Question No: 22 (Marks: 1) – Please choose one
Which of the following function do NOT initialize the chunk of memory to all zero?

- ► calloc() function
- ► Both malloc() and calloc()
- ► None of the above
- ► **malloc() function** (Page 283)

The **malloc** function differs from **calloc** in the way that the space allocated by **malloc** is not initialized and contains any values initially.

Question No: 23 (Marks: 1) – Please choose one
The function free() returns back the allocated memory got thorough calloc and

malloc to _____ .

- ► stack
- ► **heap**
- ► stack and heap
- ► None of the given options

Question No: 24 (Marks: 1) – Please choose one
width() is member function of _____

- ► cin object
- ► cout object
- ► **Both cin and cout object (Page 440)**
- ► None of the given option

Question No: 25 (Marks: 1) - Please choose one
Templates are not type safe.

- ► true
- ► **false (Page 518)**

Templates are type-safe. This is because the types that templates act upon are known at compile time, so the compiler can perform type checking before errors occur.

Question No: 26 (Marks: 1) - Please choose one
A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this ,
_____

- ► Write a separate class to handle each
- ► **Use templates (Page 527)**
- ► Use strings to store all types
- ► None of the given options

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
In if structure the block of statements is executed only,

- ► When the condition is false
- ► When it contain arithmetic operators
- ► When it contain logical operators
- ► **When the condition is true (Page 38)**

Question No: 2 (Marks: 1) - Please choose one
Header file: **fstream.h** includes the definition of the stream classes _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► ifstream, fstream, cout
► **ifstream, fstream, ofstream (Page 199)**
► fstream, cin, cout
► None of the above

Question No: 3 (Marks: 1) - Please choose one
To access the data members of structure _____ is used.

► **dot operator (.) (Page 231)**
► * operator
► □ operator
► None of given.

Question No: 4 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 5 (Marks: 1) - Please choose one
Which kind of functions can access private member variables of a class?

► **Friend functions of the class (Page 374)**
► Private member functions of the class
► Public member functions of the class
► Friend, private and public functions

Question No: 6 (Marks: 1) - Please choose one
The return type of operator function must always be void.

► True
► **False (Page 79)**

Question No: 7 (Marks: 1) - Please choose one
Friend function of a class is _____ .

► Member function
► **Non-member function (Page 348)**
► Private function
► Public function

Question No: 8 (Marks: 1) - Please choose one
Function implementation of friend function must be defined outside the class.

► True
► **False (Page 346)**

Question No: 9 (Marks: 1) - Please choose one

The normal source of **cin** object is,

- ► File
- ► Disk
- ► **Keyboard (Page 424)**
- ► RAM

Question No: 10 (Marks: 1) - Please choose one
Which of the following is correct way to initialize a variable x of int type with value 10?

- ► int x ; x = 10;
- ► **int x = 10;**
- ► int x, x = 10;
- ► x = 10;

Question No: 11 (Marks: 1) - Please choose one
Consider the following code segment. What will be the output of the following program?

```
int func(int) ;
int num = 10 ;

int main(){
int num ;
num = 5 ;
cout << num ;
cout << func(num) ;
}
int func(int x){
return num ;
}
```

- ► 5, 5
- ► 10, 5
- ► **5, 10**
- ► 10, 10

Question No: 12 (Marks: 1) - Please choose one
With template function, the compiler automatically detects the passed data and generates a new copy of function using passed data.

- ► **True Page 510**
- ► False

Question No: 13 (Marks: 1) - Please choose one
What will be the correct syntax to declare two-dimensional array of float data type?

- ► float arr{2}{2} ;
- ► **float arr[2][2] ;**

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► float arr[2,2] ;
► float[2][2] arr ;

Question No: 14 (Marks: 1) - Please choose one

The first parameter of operator function for << operator,

► Must be passed by value
► **Must be passed by reference (Page 446)**
► Can be passed by value or reference
► Must be object of class

Question No: 15 (Marks: 1) - Please choose one
Heap is constantly changing in size.

► **True (Page 280)**
► False

Question No: 16 (Marks: 1) - Please choose one
While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (Page 295)**
► right to left
► no specific order is followed
► none of the given options.

Question No: 17 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

► looped
► **nested (Page 492)**
► overloaded
► none of the given options.

Question No: 18 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

► can not be replaced
► None of the given options
► **Remain constant. (Page 271)**
► can be changed by some operation

Question No: 19 (Marks: 1) - Please choose one
Which value is returned by the destructor of a class?

► A pointer to the class.

► An object of the class.
► A status code determining whether the class was destructed correctly
► **Destructors do not return a value. (page 323)**

Question No: 20 (Marks: 1) - Please choose one
Every class contains _____.

► Constructor
► Destructor
► **Both a constructor and a destructor (Lecture 27)**
► None of the given options

Question No: 21 (Marks: 1) - Please choose one
A template function must have

► **One or more than one arguments**
► Only one argument
► Zero arguments
► None of the given options
http://en.wikipedia.org/wiki/Template_%28programming%29

Question No: 22 (Marks: 1) - Please choose one
Structured Query Language is used for _____

► **Databases Management (page 563)**
► Networks
► Writing Operating System
► none of the given options

Question No: 23 (Marks: 1) - Please choose one
When a call to a user-defined function finishes, the variable defined inside the function is still in existence.

► True
► **False (Page 460)**

Question No: 24 (Marks: 1) - Please choose one
The precedence of an operator can be changed through operator overloading.

► True
► **False (Page 372)**

Question No: 25 (Marks: 1) - Please choose one
A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this ,
_____

► Write a separate class to handle each
► **Use templates (Page 527)**
► Use strings to store all types
► None of the given options

Question No: 26 (Marks: 1) - Please choose one
"delete" operator is used to return memory to free store, which is allocated by the **"new"** operator.

► **True (page 332)**
► False

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
Pointer is a variable which store,

► Data
► **Memory Address (Page 267)**
► Data Type
► Values

Question No: 2 (Marks: 1) - Please choose one
All preprocessor directives are started with the symbol_____.

► *
► +
► @
► **# (Page 269)**

Question No: 3 Marks: 1) - Please choose one

Within the statement **obj1=obj2;** ***obj1*** will call the assignment operator function and ***obj2*** will be passed as an argument to function.

► True
► False

Question No: 4 (Marks: 1) - Please choose one
What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects
► Only destructor is called for objects
► Memory is deallocated first before calling destructor
► **Destructor is called first before deallocating memory (Page 414)**

Question No: 5 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

overloading these operators.

► **True (page 453)**
► False

Question No: 6 (Marks: 1) - Please choose one
To include code from the library in the program, such as iostream, a directive would be called up using this command.

► #include “iostream.h”
► include <iostream.h>
► include <iostream.h>
► **#include <iostream.h> (sure)**

Question No: 7 (Marks: 1) - Please choose one
The number 544.53 must be stored in _____ data type.

► int
► short
► **float (Sure)**
► char

Question No: 8 (Marks: 1) - Please choose one
A template function can have different type of arguments.

► **True (Page 503)**
► False

Question No: 9 (Marks: 1) - Please choose one
For which values of the integer _value will the following code becomes an infinite loop?

```
int number=1;
while (true) {
        cout << number;
        if (number == 3) break;
        number +=  integer_value; }
```

► any number other than 1 or 2
► **only 0**
► only 1
► only 2

Question No: 10 (Marks: 1) - Please choose one
Template class cannot have static variables.

► True
► False

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 11 (Marks: 1) - Please choose one
Which of the following is used with bit manipulation?

- ► Signed integer
- ► **Un-signed integer (Page 245)**
- ► Signed double
- ► Un-signed double

Question No: 12 (Marks: 1) - Please choose one
Structure is a collection of _____ under a single name.

- ► Only Functions
- ► **Only Variables (Page 229)**
- ► Both Functions and Variables
- ► None of the given options

Question No: 13 (Marks: 1) - Please choose one
Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

- ► new int(10) ;
- ► **new int[10] ;**
- ► int new(10) ;
- ► int new[10];

Question No: 14 (Marks: 1) - Please choose one
Unary operator implemented as member function takes ____ arguments whereas non-member function takes _____ arguments.

- ► One, zero
- ► **Zero, one**
- ► One, two
- ► Two, one

Question No: 15 (Marks: 1) - Please choose one
The first parameter of overloaded stream insertion operator is _____ where second parameter is _____

- ► input stream, object of class
- ► object of class, output stream
- ► output stream, object of class
- ► object of class, input stream

Question No: 16 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- ► **True (Page 272)**
- ► False

Question No: 17 (Marks: 1) - Please choose one
If a symbolic constant has been defined, it will be an error to define it again.

► **True (Page 273)**
► False

Question No: 18 (Marks: 1) - Please choose one
While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (Page 295)**
► right to left
► no specific order is followed
► none of the given options.

Question No: 19 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

► looped
► **nested (Page 492)**
► overloaded
► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI _____.

► cannot be replaced
► None of the given options
► **Remain constant. (Page 271)**
► can be changed by some operation

Question No: 21 (Marks: 1) - Please choose one
Assignment operator is ----------associative.

► **right**
► left
► binary
► unary
http://msdn.microsoft.com/en-us/library/ms173145.aspx

Question No: 22 (Marks: 1) - Please choose one
If **text** is a pointer of class **String** then what is meant by the following statement?
text = new String [5];

► Creates an array of 5 string objects statically
► **creates an array of 5 string objects dynamically**

Mc100401285 moaaz.pk@gmail.com
Bc100400662 bc100400662asad@gmail.com

► Creates an array of pointers to string
► Creates a string Object

Question No: 23 (Marks: 1) - Please choose one
The return type of the operator function for << operator is _____.

► **class for which we overload operator**
► reference of ostream class (ostream&)
► reference of istream class (istream&)
► void

Question No: 24 (Marks: 1) - Please choose one
The code is written to _____ the program.

► **implement (Page 520)**
► design
► analysis
► none of the given options.

Question No: 25 (Marks: 1) - Please choose one
Memory allocated at run time is a system resource and it is the responsibility of _____ to de-allocate the memory.
► System
► **Programmer (Page 331)**
► User of program
► None of given options

Question No: 26 (Marks: 1) - Please choose one
Templates are not type safe.

► True
► **False (Page 518)**

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
*.doc is _____ by type.
.
► Sequential File
► Random Access File
► Data File
► Record File

Question No: 2 (Marks: 1) - Please choose one
Which of the following is NOT a preprocessor directive?

► #error

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► #define
► #line
► **#ndefine (Page 2727)**

Question No: 3 (Marks: 1) - Please choose one
The return type of operator function must always be void.

► True
► **False (Page 79)**

Question No: 4 (Marks: 1) - Please choose one
What does **(*this)** represents?

► The current function of the class
► The current pointer of the class
► **The current object of the class (Page 390)**
► A value of the data member

Question No: 5 (Marks: 1) - Please choose one
The statement **cin.get ();** is used to,

► Read a string from keyboard
► **Read a character from keyboard (Page 426)**
► Read a string from file
► Read a character from file

Question No: 6 (Marks: 1) - Please choose one
When we do dynamic memory allocation in the constructor of a class, then it is necessary to provide a destructor.

► **True (Page 479)**
► False

Question No: 7 (Marks: 1) - Please choose one
Overloaded new operator function takes parameter of type *size_t* and returns

► void (nothing)
► **void pointer (Page 415)**
► object pointer
► int pointer

Question No: 8 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

► **True (Page 453)**
► False

Question No: 9 (Marks: 1) - Please choose one

C++ is a case-sensitive language

► **True (Page 68)**
► False

Question No: 10 (Marks: 1) - Please choose one
To include code from the library in the program, such as iostream, a directive would be called up using this command.

► #include “iostream.h”
► include <iostream.h>
► include <iostream.h>
► **#include <iostream.h> (Sure)**

Question No: 11 (Marks: 1) - Please choose one
A template function must have only generic data types.

► True
► **False (Page 501)**

Question No: 12 (Marks: 1) - Please choose one
Template class can not have static variables.

► True
► **False**

Question No: 13 (Marks: 1) - Please choose one
What will be the correct syntax to assign an array named ***arr*** of 5 elements to a pointer ***ptr***?

► *ptr = arr ;
► ptr = arr ;
► *ptr = arr[5] ;
► ptr = arr[5] ;

Question No: 14 (Marks: 1) - Please choose one
What will be the correct syntax to access the value of fourth element of an array using pointer ptr?

► ptr[3]
► (ptr+3)
► *(ptr+3)
► **Both 1and 3**

Question No: 15 (Marks: 1) - Please choose one
If most significant bit of un-signed number is 1 then it represents a positive number.

► True
► **False (Page 251)**

Question No: 16 (Marks: 1) - Please choose one

If there is a symbol (& sign) used with the variable name followed by data type then it refers to _____ and if & is being used with variable name then it refers to _____.

- ► Address of variable, reference variable
- ► Reference variable, value of variable
- ► **Reference variable, address of variable (Page 367)**
- ► Address of variable, value of variable

Question No: 17 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- ► **True (Page 272)**
- ► False

Question No: 18 (Marks: 1) - Please choose one
The default value of a parameter can be provided inside the _____

- ► function prototype
- ► function definition
- ► **both function prototype or function definition (Page 294)**
- ► none of the given options.

Question No: 19 (Marks: 1) - Please choose one
Classes defined inside other classes are called _____ classes

- ► looped
- ► **nested (Page 492)**
- ► overloaded
- ► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
What purpose do classes serve?

- ► Data encapsulation
- ► Providing a convenient way of modeling real-world objects
- ► Simplifying code reuse
- ► **All of the given options**

Question No: 21 (Marks: 1) - Please choose one
Every class contains _____.

- ► Constructor
- ► Destructor
- ► **Both a constructor and a destructor (Lecture 27)**
- ► None of the given options

Question No: 22 (Marks: 1) - Please choose one
new operator is used to allocate memory from the free store during

► Compile Time
► **Run Time**
► Link Time
► None of the given options

Question No: 23 (Marks: 1) - Please choose one
When an object of a class is defined inside another class then,

► Destructor of enclosing class will be called first
► Destructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Question No: 24 (Marks: 1) - Please choose one
It is possible to define a class within another class.

► **True (Page 492)**
► False

Question No: 25 (Marks: 1) - Please choose one
New and Delete are also used with _____ and data types as well.

► **Class, Objects (Page 342)**
► Structures, Pointers
► Both Class and structures
► None of above

Question No: 26 (Marks: 1) - Please choose one
With New keyword, data types and class members are initialized with meaningful values instead of garbage.

► **True (Page 334)**
► False

FINALTERM EXAMINATION
Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
If it is required to copy an array to another array then,

► **Both arrays must be of the same size and data type (Page 105)**
► Both arrays may be of different size
► Both arrays may be of different data type
► Both arrays may be of different size and type

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 2 (Marks: 1) - Please choose one
Dealing with structures and functions passing by reference is the most economical method

► **True**
► False
http://mathbits.com/MathBits/CompSci/Structures/Functions.htm

Question No: 3 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 4 (Marks: 1) - Please choose one
Overloaded new operator function takes parameter of type *size_t* and returns

► void (nothing)
► **void pointer (Page 415)**
► object pointer
► int pointer

Question No: 5 (Marks: 1) - Please choose one
When new operator is overloaded at global level then corresponding built-in new operator will not be visible to whole of the program.

► **True (Page 413)**
► False

Question No: 6 (Marks: 1) - Please choose one
If there is more than one statement in the block of a for loop, which of the following must be placed at the beginning and the ending of the loop block?

► parentheses ( )
► **braces { } (Page 37)**
► brackets [ ]
► arrows < >

Question No: 7 (Marks: 1) - Please choose one
The return type of a function that do not return any value must be _____

► float
► int
► **void (Page 79)**
► double

Question No: 8 (Marks: 1) - Please choose one
UNIX has been developed in _____ language.

► JAVA
► B
► **C (Page 12)**
► FORTRAN

Question No: 9 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 10 (Marks: 1) - Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword.
► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► **It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)**

Question No: 11 (Marks: 1) - Please choose one
The operator function overloaded for an Assignment operator (=) must be

► Non-member function of class
► **Member function of class (Page 372)**
► Friend function of class
► None of the given options

Question No: 12 (Marks: 1) - Please choose one
The **endl** and **flush** are _____

► Functions
► Operators
► **Manipulators (Page 434)**
► Objects

Question No: 13 (Marks: 1) - Please choose one
If a symbolic constant has been defined, it will be an error to define it again.

► **True (Page 273) 15**
► False

Question No: 14 (Marks: 1) - Please choose one
The operator used for casting, in C, is standard _____ operator.

► none of the given options.
► **cast (Page 281)**
► cost
► const

Question No: 15 (Marks: 1) - Please choose one
Constructors can not be overloaded like ordinary functions.

► True
► **False (Page 323)**

Question No: 16 (Marks: 1) - Please choose one

Which of the following function call is correct for the function prototype?
defaultParameters (int a, int b = 7, char z = '*');

► **defaultParameters (5);**
► defaultParameters (5, '8');
► defaultParameters (6, '#');
► defaultParameters (0, 0, '*', 0);

Question No: 17 (Marks: 1) - Please choose one
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

► Zero
► One
► **Two (Page 371)**
► N arguments

Question No: 18 (Marks: 1) - Please choose one
We can not define a variable of user-defined data type in the class.

► True
► **False (Page 320)**

Question No: 19 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

► **Constructor of enclosing class will be called first (Page 464)**
► Constructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 20 (Marks: 1) - Please choose one
The appropriate data type to store the number of rows and colums of the matrix is_____.

- ► float
- ► **int (Not sure)**
- ► char
- ► none of the given options.

Question No: 21 (Marks: 1) - Please choose one
Class is a user defined_____.

- ► **data type (page 317)**
- ► memory referee
- ► value
- ► none of the given options.

Question No: 22 (Marks: 1) - Please choose one
A pointer variable can be,

- ► Decremented
- ► Incremented
- ► Multiplied
- ► **Both Iecremented and Decremented (Page 366)**

Question No: 23 (Marks: 1) - Please choose one

NULL value has been defined in _____ and _____ header files.

- ► strings.h and iostream.h
- ► ctype.h and conio.c
- ► conio.c and conio.h
- ► **stdlib.h and stddef.h (Page 281)**

Question No: 24 (Marks: 1) - Please choose one

A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this , _____

- ► Write a separate class to handle each
- ► **Use templates (Page 527)**
- ► Use strings to store all types
- ► None of the given options

Question No: 25 (Marks: 1) - Please choose one
"setprecision" manipulator will set

► **The number of digits after the decimal point (Page 440)**
► The number of digits before the decimal point
► The number of digits in a number
► None of the given options

Question No: 26 (Marks: 1) - Please choose one
Which of the following option will be true, if we overload "-=" operator?

► only - operator needs to be overloaded
► Minus (-) and = operators need to be overloaded
► **the -= operator need to be overloaded explicitly**
► the - and = operators need to be overloaded implicitly

Fall 2009
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
In C/C++ if we define an array of size eight (8) i.e. int Arr [8]; then the last element of this array will be stored at,

► Arr[0]
► Arr[8]
► **Arr[7]**
► Arr[-1]

Question No: 2 (Marks: 1) - Please choose one
When an array is passed to a function then default way of passing this array is,

► By data
► **By reference (Page 154)**
► By value
► By data type

Question No: 3 (Marks: 1) - Please choose one
Array is a data structure which store

► Memory addresses
► Variables
► **Data Type (Page 100)**
► Data

Question No: 4 (Marks: 1) - Please choose one
We can also create an array of user define data type.

► **True (Page 408)**
► False

Question No: 5 (Marks: 1) - Please choose one
When we define an array of objects then,

► Destructor will call once for whole array
► **Destructor will call for each object of the array**
► Destructor will never call
► Depends on the size of array

Question No: 6 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

► Only block of memory is allocated for objects
► Only constructor is called for objects
► **Memory is allocated first before calling constructor (Page 414)**
► Constructor is called first before allocating memory

Question No: 7 (Marks: 1) - Please choose one
We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

► True
► **False**

Question No: 8 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class
For which we are overloading these operators.

► **True (Page 453)**
► False

Question No: 9 (Marks: 1) - Please choose one
Which of the following is correct way to initialize a variable x of int type with value 10?

► int x ; x = 10 ;
► **int x = 10 ; (sure)**
► int x, x = 10;
► x = 10 ;

Question No: 10 (Marks: 1) - Please choose one
Default mechanism of function calling in case of array is _____ and in case of variable is ___.

► Call by value, call by reference
► Call by referene, call by reference
► **Call by reference, call by value (Page 119)**
► Call by value, call by value

Question No: 11 (Marks: 1) - Please choose one
What does STL stand for?
► Source template library

► **Standard template library (Page 519)**
► Stream template library
► Standard temporary library

Question No: 12 (Marks: 1) - Please choose one
Skill(s) that is/are needed by programmer's _____.

► Paying attention to detail
► Think about the reusability
► Think about user interface
► **All of the given options (Page 4 & 5)**

Question No: 13 (Marks: 1) - Please choose one
For which array, the size of the array should be one more than the number of elements in an array?

► int
► double
► float
► **char (Page 113)**

Question No: 14 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Question No: 15 (Marks: 1) - Please choose one
Friend functions are _____ of a class.

► Member functions
► Public member functions
► Private member functions
► **Non-member functions (Page 346)**

Question No: 16 (Marks: 1) - Please choose one
The prototype of friend functions must be written ____ the class and its definition must be written ____

► inside, inside the class
► **inside, outside the class (Page 346)**
► **outside, inside the class**
► outside, outside the class

Question No: 17 (Marks: 1) - Please choose one
If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?
obj3 = obj1 + obj2 ;
► obj2 will be passed as an argument to + operator whereas obj2 will

drive the + operator
► obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator
► **Both objects (obj1, obj2) will be passed as arguments to the + operator (not sure)**
► Any of the objects (obj1, obj2) can drive the + operator

Question No: 18 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

► Class-name operator +() ;
► Class-name operator +(int) ;
► **Class-name operator ++() ;** **(page 389)**
► **Class-name operator ++(int) ;**

Question No: 19 (Marks: 1) - Please choose one
For cin, the source is normally a _____ and destination can be _____.

► File, native data type
► Disk, user-define type
► **Keyboard, variable (Page 424)**
For *cin*, the source is normally keyboard and the destination can be an ordinary variable

► File, user-define type

Question No: 20 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

► **True (Page 272)**
► False

Question No: 21 (Marks: 1) - Please choose one
The programs, in which we allocate static memory, run essentially on _____

► Heap
► System Cache
► None of the given options
► **Stack (Page 280)**
The programs, in which we allocate static memory, run essentially on stack.

Question No: 22 (Marks: 1) - Please choose one
The default value of a parameter can be provided inside the _____

► function prototype
► **function definition (Page 249)**
► both function prototype or function definition
► none of the given options.

The default value of a parameter is provided inside the function prototype or function definition.

Question No: 23 (Marks: 1) - Please choose one

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

While calling function, the arguments are assigned to the parameters from _____.

► **left to right. (page 295)**
► right to left
► **no specific order is followed**
► none of the given options.

Question No: 24 (Marks: 1) - Please choose one
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

► Zero
► One
► **Two (Page 371)**
► **N arguments**

Question No: 25 (Marks: 1) - Please choose one
With user-defined data type variables (Objects), self assignment can produce _____.

► Syntax error
► **Logical error**
► Link error
► **Non of the given options (not sure)**

Question No: 26 (Marks: 1) - Please choose one
Assignment operator is used to initialize a newly declared object from existing object.

► **True (Page 479)**
► False

Question No: 27 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

► **Constructor of enclosing class will be called first (Page 496)**
► Constructor of inner object will be called first
► Constructor and Destructor will be called simultaneously
► None of the given options

Question No: 28 (Marks: 1) - Please choose one
In the member initializer list, the data members are initialized,

► From left to right
► From right to left
► **In the order in which they are defined within class**
► None of the given options

Question No: 29 (Marks: 1) - Please choose one
new operator allocates memory from free store and return _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► **A pointer**
► A reference
► An integer
► A float

Question No: 30 (Marks: 1) - Please choose one
"new" and "delete" keywords are _____ in C++ language.

► Built-in- Function
► Operators
► **Memory Allocation Function (Page 332)**
► None of the given options

Spring 2010
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
&& is ---------- operator.

► An arithmetic
► **Logical (Page 40)**
► Relational
► Unary

Question No: 2 (Marks: 1) - Please choose one
Which of the following function(s) is/are included in **ctype.h** header file?

► isdigit(int c)
► isxdigit(int c)
► tolower(int c)
► **All of the above (Page 188)**

Question No: 3 (Marks: 1) - Please choose one
Disks are _____ devices having access time of _____ milliseconds.

► Electro-physical, 6
► Electro-Mechanical, 4
► Electro-physical, 5
► **Electro-Mechanical, 7 (Page 427)**
http://www.tns.com/taocs_dp.asp

Question No: 4 (Marks: 1) - Please choose one
All preprocessor directives are started with the symbol_____.

► *

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► ++
► @
► # (Page 269)

Question No: 5 (Marks: 1) - Please choose one
When we use manipulators in our program then which header file should be included?

► iostream.h
► stdlib.h
► stdio.h
► **iomanip.h (Page 433)**

Question No: 6 (Marks: 1) - Please choose one
setprecision is a parameter less manipulator.

► True
► **False (Page 439)**

Question No: 7 (Marks: 1) - Please choose one
Which of the following is NOT a preprocessor directive?

► #error
► #define
► #line
► **#ndefine (Page 272)**

Question No: 8 (Marks: 1) - Please choose one
We can do arithmetic with references like pointers.

► True
► **False (Page 369)**

Question No: 9 (Marks: 1) - Please choose one
What does **(*this)** represents?

► The current function of the class
► The current pointer of the class
► **The current object of the class (Page 390)**
► A value of the data member

Question No: 10 (Marks: 1) - Please choose one
Friend function of a class is _____ .

► Member function
► **Non-member function (Page 348)**
► Private function
► Public function

Question No: 11 (Marks: 1) - Please choose one
A function can declare itself a friend of a class.

► True
► **False (Page 346)**

Question No: 12 (Marks: 1) - Please choose one
Overloaded delete operator function takes the same parameter as an argument returned by new operator function.

► **True (Page 414)**
► False

Question No: 13 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

► **True (Page 453)**
► False
The second parameter to *operator* << is an object of the class that we are overloading the operator for. Similar is the case for *operator* >>

Question No: 14 (Marks: 1) - Please choose one
A template function must have at least ---------- generic data type

► Zero
► **One (Page 499)**
► Two
► Three

Question No: 15 (Marks: 1) - Please choose one
A template function must have only generic data types.

► True
► **False (Page 501)**

Question No: 16 (Marks: 1) - Please choose one
We can not make a member function of a class as template function.

► True
► **False (Page 506)**

Question No: 17 (Marks: 1) - Please choose one
Which character is inserted at the end of string to indicate the end of string?

► new line
► tab
► **null (Page 113)**
► carriage return

Question No: 18 (Marks: 1) - Please choose one
is used for _____.

► calculation
► reading
► **assigning value to variables (Page 372)**
► None of the given options.

Question No: 19 (Marks: 1) - Please choose one vuzs
The object _____may be used both for file input and file output

► **fstream, (Page 199)**
► ifstream,
► ofstream,
► none of the given options.

Question No: 20 (Marks: 1) - Please choose one
Which of the following function calling mechanism is true for the function prototype given below?

float func(float &);
► Call by value
► Call by reference using pointer
► **Call by reference using reference variable**
► None of the given options

Question No: 21 (Marks: 1) - Please choose one
The programs, in which we allocate static memory, run essentially on _____

► Heap
► System Cache
► None of the given options
► **Stack (Page 280)**

Question No: 22 (Marks: 1) - Please choose one
Overloaded delete operator function takes parameter of void pointer and returns
_____.

► **void**
► void pointer
► pointer to an object
► pointer of type int

Question No: 23 (Marks: 1) - Please choose one
What should be the return type of the constructor?

► void pointer
► int
► same as object type
► **constructors do not return any thing (Page 313)**

Question No: 24 (Marks: 1) - Please choose one
It is a way of reusing the code when we contain objects of our already written classes into a new class,

► **True**

► False

Question No: 25 (Marks: 1) - Please choose one vuzs
Templates are not type safe.

► true
► **false (Page 518)**

Question No: 26 (Marks: 1) - Please choose one
The functions used for dynamic memory allocation return pointer of type _____

► int
► float
► **void**
► double

FINALTERM EXAMINATION
Fall 2008
CS201- Introduction to Programming

Question No: 1 (Marks: 1) - Please choose one
There are mainly ---------- types of software

► Two (Page 9)
► Three
► Four
► Five

Question No: 2 (Marks: 1) - Please choose one
seekg() and write() are functionally _____ .

► Different
► Identical
► Two names of same function
► None of the above

Question No: 3 (Marks: 1) - Please choose one

When a pointer is incremented, it actually jumps the number of memory addresses

► **According to data type (Page 160)**
► 1 byte exactly
► 1 bit exactly
► A pointer variable can not be incremented

Question No: 4 (Marks: 1) - Please choose one
setw is a parameterized manipulator.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► **True (Page 437)**
► False

Question No: 5 (Marks: 1) - Please choose one
eof( ), bad( ), good( ), clear( ) all are manipulators.

► **True (Page 433)**
► False

Question No: 6 (Marks: 1) - Please choose one
In functions that return reference, use _____variables.

► Local
► Global
► **Global or static (Page 369)**
► None of the given option

Question No: 7 (Marks: 1) - Please choose one
The declarator of Plus (+) member operator function is

► **Class-Name operator + (Class-Name rhs) (Page 371)**
► operator Class-Name + ( )
► operator Class-Name + (rhs)
► Class-Name operator + ( )

Question No: 8 (Marks: 1) - Please choose one
The compiler does not provide a copy constructor if we do not provide it.

► True
► **False (Page 476)**

Question No: 9 (Marks: 1) - Please choose one
What is the functionality of the following syntax to delete an array of 5 objects named ***arr*** allocated using new operator?
delete arr ;

► Deletes all the objects of array
► **Deletes one object of array (Page 411)**
► Do not delete any object
► Results into syntax error
This statement will call the destructor only for the object pointed by the ***arr*** and deallocate the space allocated to this object

Question No: 10 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

► Only block of memory is allocated for objects
► Only constructor is called for objects
► **Memory is allocated first before calling constructor (Page 414)**
► Constructor is called first before allocating memory

Question No: 11 (Marks: 1) - Please choose one
What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects
► Only destructor is called for objects
► Memory is deallocated first before calling destructor
► **Destructor is called first before deallocating memory (Page 414)**

Question No: 12 (Marks: 1) - Please choose one
new and **delete** operators cannot be overloaded as member functions.

► True
► **False (Page 414)**

Question No: 13 (Marks: 1) - Please choose one
The operator function of << and >> operators are always the member function of a class.

► True
► **False (Page 446)**

Question No: 14 (Marks: 1) - Please choose one
A template function must have at least ---------- generic data type

► Zero
► **One (Page 499)**
► Two
► Three

Question No: 15 (Marks: 1) - Please choose one
If we do not mention any *return_value_type* with a function, it will return an _____ value.

► **int (Page 79)**
► void
► double
► float

Question No: 16 (Marks: 1) - Please choose one
Suppose a program contains an array declared as **int arr[100];** what will be the size of array?

► 0
► 99
► **100 (Page 103)**
► 101

Question No: 17 (Marks: 1) - Please choose one
The name of an array represents address of first location of array element.

► **True**
► False
The name of the array is a constant pointer which contains the memory is the address of first element of the array

Question No: 18 (Marks: 1) - Please choose one
Reusing the variables in program helps to save the memory

► **True**
► False

Question No: 19 (Marks: 1) - Please choose one
Which of the following option is true about new operator to dynamically allocate memory to an object?

► The new operator determines the size of an object
► Allocates memory to object and returns pointer of valid type
► Creates an object and calls the constructor to initialize the object
► **All of the given options**

Question No: 20 (Marks: 1) - Please choose one
New and delete are _____ whereas malloc and free are _____.

► Functions, operators
► Classes, operators
► **Operators, functions (Page 342-285)**
► Operators, classes

Question No: 21 (Marks: 1) - Please choose one
Like member functions, _____ can also access the private data members of a class.

► Non-member functions
► **Friend functions (Page 346)**
► Any function outside class
► None of the given options

Question No: 22 (Marks: 1) - Please choose one
Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword.
► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► **It can be declared anywhere in class as these are not affected by the public and private keywords. (Page 346)**

Question No: 23 (Marks: 1) - Please choose one
The operator function overloaded for an Assignment operator (=) must be

- ► Non-member function of class
- ► **Member function of class (Page 372)**
- ► Friend function of class
- ► None of the given options

Question No: 24 (Marks: 1) - Please choose one
For non-member operator function, object on left side of the operator may be

- ► Object of operator class
- ► Object of different class
- ► Built-in data type
- ► **All of the given options (Page 375)**

Question No: 25 (Marks: 1) - Please choose one
The operator function will be implemented as _____, if obj1 drive the - operator whereas obj2 is passed as arguments to - operator in the statement given below.
obj3 = obj1 - obj2;

- ► **Member function (Page 373)**
- ► Non-member function
- ► Friend function
- ► None of the given options

Question No: 26 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

- ► Class-name operator +() ;
- ► Class-name operator +(int) ;
- ► **Class-name operator ++() ; (Page 389)**
- ► Class-name operator ++(int) ;

Question No: 27 (Marks: 1) - Please choose one
The static data members of a class are initialized _____

- ► **at file scope (Page 466)**
- ► within class definition
- ► within member function
- ► within main function

Question No: 28 (Marks: 1) - Please choose one
Class is a user defined_____.

- ► **data type (Page 317)**

- memory referee
- value
- none of the given options.

Question No: 29 (Marks: 1) - Please choose one
We can also define a user-defines manipulators.

- **True**
- False

Parameterized manipulators require one or more arguments. setfill (near the bottom of the iomanip.h header file) is an example of a parameterized manipulator. You can create your own parameterized manipulators and your own simple manipulators.

Question No: 30 (Marks: 1) - Please choose one
Automatic variable are created on _____.

- Heap
- Free store
- static storage
- **stack (Page 461)**

FINALTERM EXAMINATION
fall 2008
CS201- Introduction to Programming (Session - 2)

Question No: 1 (Marks: 1) - Please choose one
If Num is an integer variable then Num++ means,

- Add 1 two times with Num
- **Add 1 with Num**
- Add 2 with Num
- Subtract 2 from Num

Question No: 2 (Marks: 1) - Please choose one
If the return type of a function is void then it means that it will,

- Return any type of data
- Return some specific type of data
- **Return no data**
- Return just character data

Question No: 3 (Marks: 1) - Please choose one
Which of the following is a valid class declaration?

- **class A { int x; };**
- class B { }
- public class A { }
- object A { int x; };

Question No: 4 (Marks: 1) - Please choose one

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

When we use manipulators in our program then which header file should be included?

- iostream.h
- stdlib.h
- stdio.h
- **iomanip.h (Page 433)**

Question No: 5 (Marks: 1) - Please choose one
We can also create an array of user define data type.

- **True (Page 408)**
- False

Question No: 6 (Marks: 1) - Please choose one
The normal source of **cin** object is,

- File
- Disk
- **Keyboard (Page 424)**
- RAM

Question No: 7 (Marks: 1) - Please choose one
A stream is an ordered sequence of bytes.

- **True (Page 423)**
- False

Question No: 8 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?

- Only block of memory is allocated for objects
- Only constructor is called for objects
- **Memory is allocated first before calling constructor (Page 414)**
- Constructor is called first before allocating memory

Question No: 9 (Marks: 1) - Please choose one
We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

- **True (Page 412)**
- False

Question No: 10 (Marks: 1) - Please choose one
The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

- **True (Page 453)**
- False

Question No: 11 (Marks: 1) - Please choose one
Which looping process checks the test condition at the end of the loop?

- for
- while
- **do while**
- no looping process checks the test condition at the end

http://mathbits.com/MathBits/CompSci/looping/MCLooping.htm

Question No: 12 (Marks: 1) - Please choose one
In a group of nested loops, which loop is executed the most number of times?

- the outermost loop
- **the innermost loop**
- all loops are executed the same number of times
- cannot be determined without knowing the size of the loops

http://mathbits.com/mathbits/java/Looping/MCLooping.htm

Question No: 13 (Marks: 1) - Please choose one
Template class can not have static variables.

- True
- **False (Page 513)**

Question No: 14 (Marks: 1) - Please choose one
Consider the following statements to initialize a two-dimensional array.

- int arr[2][3] = {4, 8, 9, 2, 1, 6} ;
- int arr[3][2] = {4, 8, 9, 2, 1, 6} ;
- int arr[][2] = {{4,8},{9, 2},{1, 6}} ;

Which of the following option(s) are correct to initialize a two-dimensional array with 3 rows and 2 columns?

- (ii) only
- (iii) only
- **(ii) and (iii)**
- and (iii)

Question No: 15 (Marks: 1) - Please choose one
There is a pointer variable named ***ptr*** of type int then address of which type of variable the ***ptr*** will store in it?

- variable of type char
- variable of type short
- **variable of type int**
- variable of type double

Question No: 16 (Marks: 1) - Please choose one
Let suppose
Union intorDouble{
Int ival;
Double charvar;
};
main(){

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

intorDouble VAZ;
int size ;
size = sizeof(VAZ);
}
What will be the value of variable "size", if int occupies 4 bytes and double occupies 8 bytes?

- 2
- 4
- 8
- 12

Question No: 17 (Marks: 1) - Please choose one
new and delete are _____ whereas malloc and free are _____.

- Functions, operators
- Classes, operators
- **Operators, functions (Page 342-285)**
- Operators, classes

Question No: 18 (Marks: 1) - Please choose one
The member functions of a class occupy _____ region in memory for ____ object(s) of class.

- separate, each
- **common, all (Page 334)**
- different, each
- different, all

Question No: 19 (Marks: 1) - Please choose one
Friend functions are _____ of a class.

- Member functions
- Public member functions
- Private member functions
- **Non-member functions (Page 346)**

Question No: 20 (Marks: 1) - Please choose one
Which of the following is true while overloading operators?

- Precedence of an operator can be changed
- The arity (number of operands) can be changed
- No new operators can be created
- Associativity of an operator can be changed

Question No: 21 (Marks: 1) - Please choose one
Which of the following option will be true to overload the -= operator?

- **only - operator needs to be overloaded (not sure)**
- Minus (-) and = operators need to be overloaded
- the -= operator need to be overloaded explicitly
- the - and = operators need to be overloaded implicitly

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

Question No: 22 (Marks: 1) - Please choose one
The input/output streams; cin and cout are ____

- Operators
- **Functions**
- Objects
- Structures

Question No: 23 (Marks: 1) - Please choose one
dec, hex, oct are all _____

- Member functions
- Objects of input/output streams
- Parameterized manipulators
- **Non-parameterized manipulators (Page 435)**

Question No: 24 (Marks: 1) - Please choose one
What will be the output of the following statement?
cout << setbase(16) << 52 ;

- 74
- 52
- **34**
- 64

Question No: 25 (Marks: 1) - Please choose one
The first parameter of overloaded stream insertion operator is _____ where second parameter is _____

- input stream, object of class
- object of class, output stream
- **output stream, object of class**
- object of class, input stream

Question No: 26 (Marks: 1) - Please choose one
We can also do conditional compilation with preprocessor directives.

- **True (Page 272)**
- False

Question No: 27 (Marks: 1) - Please choose one
With user-defined data type variables (Objects), self assignment can produce _____.

- Syntax error
- **Logical error**
- Link error
- Non of the given options

Question No: 28 (Marks: 1) - Please choose one
The return type of the operator function for >> operator is _____.

Mc100401285
moaaz.pk@gmail.com
Bc100400662
bc100400662asad@gmail.com

- **class for which we overload this operator**
- reference of ostream class (ostream&)
- reference of istream class (istream&)
- void

Question No: 29 (Marks: 1) - Please choose one
When an object of a class is defined inside an other class then,

- **Constructor of enclosing class will be called first (Page 496)**
- Constructor of inner object will be called first
- Constructor and Destructor will be called simultaneously
- None of the given options

Question No: 30 (Marks: 1) - Please choose one
Where we can include a header file in the program?

- **any where (Page 270)**
- in start
- at the end
- none of the given options.

Moaaz.pk@gmail.com

bc100400662asad@gmail.com

Quiz No 2 will be opened from
15th February,2022 to 17th February, 2022.
Syllabus of Quiz No 2 is from Lecture 23 to 33.
ORANGE MONKEY TEAM

1. **What will be the output of the given code? #include<iostream> using namespace std; #define MAX (A, B)((A)>(B)) int main () { int I, x, y; x=23; y=45; I = MAX(x++, y++); Side effect; ?? larger value incremented twice cout << x =<<x<< y=<<\n}**

   X= 24, y=46

2. **if (day == daysOfMonth (*this)) { // this is the last day of the month // process accordingly }**

   In the above condition, we have checked that day is equal to the number of days in the month or not. If the condition returns true it means loop will make its _____ iteration with _____ day of the month.

   Last, Last

3. **Default constructor generated by _____ does _____ for us.**

   Compiler initialization…..confirm from net

4. **Friend function are used in cases where one class is_____to another class**

   Independent

5. **For casting, we normally declare a pointer of type_____.**

   We are going to use…….confirm

6. **Constructor is special type of function**

   Which has no return type…..confirm from net

7. **Class can be defined as**

   A class includes both objects and structures

8. **The object code of our program is combined with the_____.**

   Object code of the library function….confirm

9. **I+=2 is equivalent to_____.**

   I= i+2….confirm

10. **For accessing data members we use_____operator.**

(Dot.)….confirm

11. The_____is called automatically when an object destroys
destructor

12. The function overloading requires_____.
The argument list to be the same….confirm

13. With user data type variables (objects) self assignment can produce
Logical error……confirm from net

14. Constructor is a special type of function
Which has no return type……confirm from net

15. Look at the statement given below int & a; and tell what will happen
Compiler will generate an error: a declared as reference but not initialized….confirm from net

16. Macros are categorized into_____type(s)
Two…..confrim

17. If the memory in the free store is not sufficient_____.
Malloc function returns NULL pointer…confirm from net

18. A reference cannot be Null it has to point a data type
True…..confirm

19. The friend function of a class can have access _____.
To the private data members….confirm

20. #define CIRCUMFERENCE(X) (2*PI*R) is a.
Definition of a macro…..confirm from net

21. Which of the following is unary operator
i--, i++, ++i…..all options…confirm

22. For overloading a void type pointer ptr into integer type, the correct syntax is
(int*)ptr……confirm

23. Ternary operator is shown as_____.
?;

24. If we do not write our own assignment operator then which of the following problem may occur?
Dangling pointer

25. Getche() is a__ function and defined in_____header file.
Built in function, conio.h……confirm from net

26. Core must be taken about the correct_____ of operator while overloading.
Both semantic and complexity

27. Which function is used to delete the allocated memory space?
Free()……confirm from net

28. Friend functions are_____.
Private……confirm

29. Which one of the following is the correct statement about operator overloading?

Arithmetic operators can be overloaded only

30. **Once we have defined a symbolic constant value using #define, that value _____ during program execution.**
    Cannot be changed……confirm from net

31. **Consider the following code segment. Which of the following will be called while executing code at line 2? String s1, s2; s1 = s2**
    Assignment Operator

32. **The concept of_____allows us to separate the interface from the implementation of the class.**
    Encapsulation…….confirm

33. **In overloading the assignment (=) operator, which object will be passed as an argument(s) in the operator function?**
    Left object of the assignment operator

34. **In overloading the assignment (=) operator, which object(s) will call operator function?**
    Left object of the assignment……confirm from net

35. **Class can be defined as:**
    A class includes both data members as well as functions to manipulate that data

36. **Constructor has_____.**
    The same name as of class…….confirm

37. **We can_____pointer**
    Reassign, decrement, increment…….All options correct

38. **The constructor contains_____.**
    No return type

39. **_____will return the number of bytes reserved for a variable or data type**

    Sizeof operator…….confirm from net

40. **In C++ the region of available memory is called_____.**
    Free store

41. **The only operator that the compiler overloads for user define data type by default is**

Assignment (=) operator….confirm from net

42. **Functions declared with the_____specifier in a class member list are called friend functions of that class.**

    friend

43. **Public or private keywords can be _____**

    written multiple times in the class or structure declaration

44. **The friend keyword provides access_____.**
    in one direction only

    References cannot be uninitialized. Because it is impossible to _____

    reinitialize a reference

45. **new operator can be used for_____.**

    integer , float, char and double data types……confirm

10. **The destructor is used to_____.**

    deallocate memory

43. **Reference is not really an address it is_____.**

    a synonym

44. **If we want to allocate memory to an array of 5 integers dynamically, the syntax will be_____.**
    int *iptr ; iptr = new int[5] ;

45. **Memory allocated from heap or free store_____.**

    cannot be returned back unless freed explicitly using free and delete operators

46. **Operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the_____.**
    Operands
47. **The operator to free the allocated memory using new operator is_____.**
    delete

48. Public or private keywords can be_____.

written multiple times in the class or structure declaration …….confirm from net

49. A class can be declared as a_____of other class

Friend class…..confirm

50. In C++ operators, which of the following operator cannot be overloaded _____.

?;

51. We can_____references.

None of the given options…..confirm from net

52. Being a concise language, C needs something for its_____.

Enhancement…..confirm

53. The compiler gets the modified _____

source code file…..confirm

54. When we write_____this somefile is ordinary text file of C code.

#include….confirm

55. The line where we write the_____statement is replaced by the text of that file.

#include…..confirm

56. All of the preprocessor directives start with_____.

sign……confirm

57. There are_____ways to use #include.

Two……confirm

58. **'h' stands for _____**

header files…..confirm

59. We can include files anywhere in the code but it needs to be_____and at the _____.

Logical, proper position…….confirm

60. The_____of function must be declared before its usage

Prototype…..confirm

61. The_____of compilation will be successful.

first parse…..confirm

62. After the_____of the compiler, it converts the source code into object code

First phase…..confirm

63. _____is machine code but is not re-locateable executable.

Object code……confirm

64. The_____of our program is combined with the object code of the library functions.

object code……confirm

65. The_____performs this task while the compiler includes the name and arguments.

Linker…..confirm

66. For checking the_____of the functions, the compiler needs to know the definition of the function or at least the prototype of the function.

Validity…….confirm

67. The preprocessor will search for the file "myHeaderFile.h" in the current working directory. It will be written as

#include "myHeaderFile.h……confirm

68. _____is a universal constant and has a value of_____.

Pi, 3.1415926…….confirm

69. Be sure that the value of_____can not be changed.

Pi……confirm

70. All the preprocessor directives start with the sharp sign.

(#)…….confirm

71. Macros are classified into_____categories.

Two……confirm

72. The first type of macros can be written using_____.

#define…….confirm

73. The second type of macros takes arguments. It is also called a_____.

parameterized macros…….confirm

74. Being a_____, it does not require any semicolon at the end.

non-C code…..confirm

75. The_____will be replaced by the actual macro definition including the entire parenthesis in the code before compilation.

CIRCLEAREA…..... confirm

76. A symbol cannot be redefined without_____it first.

Undefining… ....confirm

77. The_____directives help in debugging the program

Conditional compilation……confirm

78. _____variable names starting with underscore.

Do not declare……confirm

79. Always use_____while defining macros that takes arguments.

Parenthesis……confirm

80. So an ordinary_____is actually a C code.

English poem .....confirm

81. Earlier, whenever we declared arrays, the size of the arrays was_____.

Predefined..... confirm

82. It is better to compare both the_____and_____allocation methods to understand the benefits of the usage of dynamic memory allocation.

Static , dynamic.....confirm

83. Static allocation is also called _____

Compile time allocation..... confirm

84. A pointer ptr of type void is declared as under

void *ptr ;… confirm

85. The syntax of the calloc function is as follows.

void *calloc (size_t n, size_t el_size)……confirm

86. The_____function takes one argument i.e. the number of bytes to be allocated

Malloc……confirm

87. The syntax of the malloc function is as follows

void * malloc (size_t size) ; .....confirm

88. #undef is used for

Making a symbol undefined…..confirm

89. Let suppose a = b = c. In such situation, it is necessary to return a reference of an object from assignment operator function.

True

90. Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

Class-name operator ++()…..confirm

91. A class is a user defined data type it takes_____.

No space in memory unless we create an object from it..... confirm

92. The heap memory structure_____.

Constantly changes in size..... confirm

93. Which of the following permits function overloading in C++

Both type and arguments……confirm from net

94. The data members of the class are initialized_____.

At runtime…..confirm

95. _____operators are the ones that require two operands on both sides of the operator.

Binary……confirm

96. _____operators are the ones that require only one operator to work to the left of the operand

Unary…..... confirm

97. Which one of the following is mandatory preprocessor directive for c++?

#include <iostream>……confirm from net

98. Functions declared with the_____specifier in a class member list are called friend functions of that class

Friend

99. Default constructor takes_____.

No parameters… confirm

100. The members of a class declared without any keyword are_____by default

Private .....confirm from net

101. In a class we can have_____constructor(s).

Many… confirm

102. _____for parameters is also done for inline functions.

Automatic type checking… .confirm

103. An address is a_____, while a pointer is a_____.

Constant, Variable …… confirm from net

104. The compiler generates_____automatically

Constructors

105. When an operator function is defined as member function for a binary Plus (+) operator then the number of extra arguments it takes is/are

Two….confirm from net

106. Date+=1 is equivalent to_____.

Date + 1 ……confirm

107. Windows operating system may itself takes memory from_____.

Heap….confirm from net

108. Identify the correct method of adding two strings s1 and s2

Strcat (s1, s2)

109. Name the function whose definition can be substituted at a place where its function call is made_____.

Inline function……confirm from net

110. If class A is a friend of class b, and class B is a friend of class C, if class A wants class C to be a friend_____.

It has to declare, class C as a friend….confirm

111. Separate the interface and_____.

Implementation…..confirm

112. Constructor has the same name as of_____.

Class…..confirm

113. Initializing the data members in the definition of the class is a_____.

Syntax error…..confirm

114. The new operator automatically determines the size of memory required to store that object, so it does not need_____.

Sizeof operator…..confirm

115. Object code is machine code but it is not_____and_____.

Relocateable, executable…….confirm

116. While using_____operator we do not need to supply number of bytes allocated

New……confirm

117. The operator to free the allocated memory using_____operator is _____. So whenever we use new to allocate memory.

New, delete…..confirm

118. The_____operator frees the allocated memory that is returned back to free store for usage ahead.

Delete……confirm

119. The memory allocation functions return a chunk of memory with a pointer of type__.

Void……confirm

120. The syntax of declaration of a function that returns the reference to an integer is_____.

Int & myfunc();……confirm

121. For console input and output we use_____.

Conia.h header file…..confirm

122. A pointer is_____.

A variable for storing address…….confirm

123. For the joining of two strings in string class, we may use '+' operator, can we use '-' operator the same way for subtracting strings?

No

124. A friend function of a class is a function defined_____.

Outside that class and that has the right access protected members of the class only

125. The reference data types are used as_____variables without any _____ operator

Ordinary , deference……confirm

126. Operator overloading is to allow the same operator to be_____to more than one implementation, depending on the types of the operands.

Bound…….confirm

127. The concept of friend function negates the concept of_____.

Encapsulation……confirm

128. Overloaded assignment operator must be

Member function of class

129. Header files provide_____so the program running on one operating system can run without an error on the other system.

Portability……confirm

130. Once an object is declared as a friend_____.

It has access to all non-public members as if they were public….confirm from net

131. If text is a pointer of type string then what will be the functionality of following statement?

Creates array of 5 objects dynamically

132. A_____function of a class is defined outside that class scope, but it can access all private and protected members of the class.

Friend

133. For binary member operators operands on the_____drives (calls) the operation

Left

134. A class is a user defined data type it takes_____.

No space in memory unless we create an object from it

135. We cannot increment_____.

Reference…….confirm from net

136. To prevent dangling reference the functions returning reference should be used with_____.

Static and global variables…..confirm from net

137. free function is available in_____header file.

Stdlib.h

138. Assignment operator is used to initialize a newly declared object from existing object

True……confirm

139. The dynamic memory allocation uses_____whereas static memory allocation uses_____.

Heap, stack….confirm from net

140. C++ offers_____levels of data access control inside a class

Three

141. The members of a class declared with the keyboard struct are _____ by default.

Public…..confirm

142. _____is a special type of pointer we have to cost it before we use it.

Void

143. Encapsulation means_____.

That the data of a class cannot be accessed from outside……confirm

144. The friend keyword provides access_____.

In one direction only .....confirm

145. Reference variables must_____.

Be initialized after they are declared……confirm

146. **Reference is not really an address it is_____.**

A synonym……confirm from net

147. **C++ was developed by _____**

Bjarne Stroustrup……confirm

148. **Which of the following syntax is best used to delete an array of 5 objects named 'string' allocated using new operator.**

Delete []string;

149. **if we have a program that writes the output data(numbers) to the disc, and if we collect the output data and write it on the disc in one write operation in the above situation the area where we will gather the number is called.**

Buffer…..confirm

150. **Consider the following code, the printed value will be converted into**

```
int n = 10;
cout<<oct<<n;
```

base 8

151. **When new operator is overloaded at global level then corresponding built-in new operator will also be visible to whole of the program.**

False…..confirm

152. **When new operator is overloaded at global level then corresponding built-in new operator will not be visible to whole of the program.**

True….confirm

153. **To avoid dangling reference, don't return_____.**

The reference of a local variable from the function…….confirm

154. **Reference is a thing by which we can create_____of any data type**

Synonym……confirm from net

155. **Reference cannot be uninitialized. Because it is impossible to_____.**

Reinitialize a reference……confirm from net

156. _____must be included to use stream manipulator in your code

Iostream

157. Let suppose int a, b, c, d, e; a=b=c=d=e=42; This can be interpreted by the compiler as

a = (b=(c=(d=(e=42))));

158. What is meant by the following statement? String str[5] = {string("Programming"), string("CS201")};

Parameterized constructor will call for first 2 objects and default constructor for remaining objects…….confirm from net

159. Overloaded member operator function is always called by _____.

Compiler…….confirm from net

160. The default visibility for the data members of the class is

Private……confirm

161. The operator to free the allocated memory using new operator is _____.

delete……confirm

162. Bugs can occur due to_____.

Uninitialized data…..confirm

163. When the compiler overloads the assignment (=) operator by default then

Compiler does member wise assignment ..... confirm from net

164. In functions, that returns reference use_____.

Global or static variables

165. Look at the program code and identify the error. #include<iostream> using namespace std; #define PI 3.1415926; main() { int radius =5; cout<< "Area of circle with radius"<< radius<< ' =' <<PI * radius * radius;}

Error exist in line number 2. Semi colon is not allowed with define directive

166. Destructor_____.

Cannot be overloaded and have no return type……confirm

167.

_____data isn't accessible by non-member functions or outside classes

Private…..confirm

168. If the request of new operator is not fulfilled due to insufficient memory in the heap_____.

The operator returns 0…….confirm

169. Symbolic constant PI can be defined as;

#define PI 3.14…….confirm from net

170. Data+=1 is equivalent to_____.

Date+1…….confirm

171. _____operators are the ones that require only one operator to work.

Unary……confirm

172. If B is designated as friend of A, B can access A's non-public members.

A can access non-public members of B……confirm

173. Operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the_____.

Operands…..confirm

174. What will be the output of the following c++ code?

```
#include<iostream.h>
#define max 100
Main()
```

{

#ifdef max

cout<< "Hellow";

}

Error……confirm from net

175. While using_____operator we do not need to supply number of bytes allocated.

New

176. The_____data type always represents an empty set of values in C++.

Void……confirm

CS201 For final term

UPDATED QUIZ 2 2021

PROVIDE BY ORANGE MONKEY

Remember Own Prayer

1. **To avoid dangling reference don't return**

   The reference of a local variable from the function……confirm

2. **Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?**

   Class name operator ++()….confirm from net

3. **Reference is a thing by which we can create_____of any data type**

   Synonym…..confirm

4. **Reference cannot be uninitialized because it is impossible to_____.**

   Initialize a Null pointer…..confirm

5. **The syntax of declaration of a function that returns the reference to an integer is _____.**

   Int & myfunc()……confirm

6. **Friend function declaration can go_____the class**

   Anywhere in……confirm

7. **To prevent dangling reference the functions returning reference should be used with _____.**

   Static and global variables .....confirm

8. **The only operator that the compiler overloads for user define data type be default in**

   Arrangement (=) operator..... confirm from net

9. **If operator function is non-member function then object on left side of operator cannot be_____?**

Object of member function…….confirm

10. Which of the following functionality can be achieved through overloading?

New operators cannot be defined through operator overloading

11. The return type of the overloading new operator must be?

Void*….confirm from net

12. For binary member operators, operands on the_____derives (calls) the operation.

Left…….confirm

13. Identify the correct method of adding two strings, s1 and s2.

Strcat(s1,s2)

14. A friend function of a class is a function defined_____.

Outside that class and that has the right to access all members of the class

15. Care must be taken about the correct_____of operator while overloading.

Both Semantic and Complexity……confirm from net

16. We can_____reference.

None of the given….confirm

17. If text is a pointer of type string then what will be the functionality of following statement?

Text = new String {5};

Creates array of 5 objects dynamically…..confirm from net

18. Look at the statement given below
Int & a; and tell what will happen?

Compiler will generate an error 'a' declared as reference but not initialized……confirm

Remember Own Prayer

19. The syntax of declaration of a function that returns the reference to an integer is _____.

Int & myfunc();….confirm

20. Which of the following syntax is best used to delete an array of 5 objects named 'string' allocated using new operator.

Delete []string;….confirm from net

21. If we do not write our own assignment operator then which of the following problem may occur?

22. Identify the correct syntax for making a class friend of other class

```
friend ClassOne
{
OtherClass;
private;
//here we write the data members of ClassOne
};
```

23. Once an object is declared as a friend _____

It has access to all non-public members as if they were public….confirm from net

24. Reference is a thing by which we can create_____of any data type.

alias or synonym…both……confirm

25. A_____function of a class is defined outside that class scope. But it can access all private and protected members of the class.

Firend…..confirm

26. Once the_____are created they exist for the life time of the program

Static variables…..confirm

27. A class can be declared as a_____of other class.

Friend….confirm

Remember Own Prayer

28. Reference variables must_____.

Be initialized after they are declared….confirm from net

29. We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

True

30. Operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the_____.

Operands…….confirm

31. The friend keyword provides access_____.

In one direction only……confirm

32. The friend functions are_____.

Not member of a class….confirm

33. Ternary operator is shown as_____.

?;……confirm

34. Reference is not really an address it is_____.

A synonym

35. In C++ operators, which of the following operator cannot be overloaded_____.

?;…..confrim

36. Overloaded assignment operator must be

Member function of class

37. An address is a_____, while a pointer is a_____.

Variable, constant…confirm from net

38. When new operator is overloaded at global level then corresponding built-in new operator will also be visible to whole of the program.

False….confirm

Remember Own Prayer

39. When new or delete operator is overloaded at global level then corresponding built-in new or delete operator will not be visible to whole of the program.

True….confirm

40. When an operator function is defined as member function for a binary Plus (+) operator then the number of extra arguments it takes is/are

One….confirmF

41. i+=2 is equivalent to_____.

i= i + 2;

42. Assignment operator is used to initialize a newly declared object from existing object.

True….confirm from net

43. When an array of object is created dynamically then there is no way to provide parameterized constructors for array of objects.

True…confirm from net

44. In overloading the assignment (=) operator, which object will be passed as an arguments in the operator function?

Left object of the assignment operator

45. Friend functions are _____

Private

46. We can_____pointer.

Decrement increment reassign…all options…… confirm

47. The function will return a reference to the global variable that exists throughout the program and thus there will be no danger of_____.

Dangling reference……confirm

48. The reference data types are used as_____ variables without any _____ operator.

Ordinary , deference…….confirm

Remember Own Prayer

49. What is the function of the following statement to delete an array of 5 objects named 'arr' allocated using new operator?
Delete arr;

Do not delete any object

50. If B is designated as friend of A, B can access A's non-public members

A cannot access B….confirm from net

51. What is the sequence of event(s) when deallocating memory of an object using delete operator?

Memory is dealloacted first before calling destructor…confirm from net

52. _____operators are the ones that require two operands on both sides of the operator.

Binary…..confirm

53. Int & I; it means that i is a_____to an integer.

Reference…..confirm

54. When operator function is implemented as member function then return type of function_____.

Can be any data type….confirm from net

55. The concept of_____allows us to separate the interface from the implementation of the class.

Encapsulation….confirm

56. With user data type variables (objects) self assignment can produce

Logical error….confirm

57. When the compiler overloads the assignment (=) operator by default then

Compiler does member wise assignment..confirm from net

58. Which of the following is unary operator?

+i, i++, ==I …all options…..confirm

59. Reference is not really an address it is_____.

Remember Own Prayer

A synonym

60. What is meant by the following statement?
String str[5] = {String("Programming''), String("CS201")};

Parameterized constructor will be called for all objects of array

61. When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

Two…confirm

62. Friend classes are used in cases where one class is _____ to another class

Independent

63. The difference between pointers and references is that _____

We can do arithmetic with pointers….confirm

64. When an operator function is defined as member function for a Unary operator then the number of extra arguments it takes is/are.

One……confirm

65. *This is a pointer that always points to

Current pointer of the class

Remember Own Prayer

Question # 1:-

Care must be taken about the correct_____of operator while overloading. (Choose the most appropriate).

1. Semantic
2. Complexity
3. Both Semantic and Complexity
4. None of the given options

Question # 2:-

In functions, that returns reference use_____.

1. global or local variables
2. global or static variables
3. ordinary variables

Question # 3:-

In overloading the assignment (=) operator, which object(s) will call the operator function?

1. Left object of the assignment operator
2. Right object of the assignment operator
3. Both objects will call the operator function
4. No object will call the operator function

Question # 4:-

Friend functions are_____.

1. Unidirectional **(not confirm)**
2. bidirectional
3. like inline functions
4. private

Question # 5:-

We cannot increment_____.

1. pointers
2. arrays
3. references
4. Vaiables

Question # 6:-

Remember Own Prayer

A pointer is_____.

1. the address of a variable
2. an indication of the variable to be accessed next.
3. the data type of an address variable

Question # 7:-

Overloaded assignment operator must be

1. Member function of class
2. Non-member function of class **(not confirm)**
3. Friend function of class
4. Global function

Question # 8:-

The concept of friend function negates the concept of_____.

1. inheritance
2. polymorphism
3. persistence
4. encapsulation

Question # 9:-

If class A is a friend of class B, and class B is a friend of class C. If class A wants class C to be a friend, _____

1. it has to declare, class C as a friend
2. it has to declare, class B as a friend
3. it has to declare , class A as a friend
4. it has to declare, class B and class A as friend classes

Question # 10:-

The difference between pointers and references is that_____.

1. we cannot do arithmetic with pointers

Remember Own Prayer

2. we can do arithmetic with pointers
3. we cannot reassign pointers
4. we can assign references

Question # 11:-

A reference cannot be_____.

1. 1
2. zero
3. NULL
1. 4 integer

Question # 12:-

An address is a_____, while a pointer is a _____.

1. constant , variable
2. variable , constant
3. global , variable
4. non static variable , constant

Question # 13:-

The reference data types are used as_____variables without any_____operator

1. ordinary , deference
2. global , dot
3. static , deference
4. local , &

Question # 14:-

The syntax of declaration of a function that returns the reference to an integer is_____.

Remember Own Prayer

1. int & myfunc() ;
2. int myfunc();
3. int myfunc() &;
4. integer & myfunc();

Question # 15:-

References cannot be uninitialized. Because it is impossible to

_____

1. reinitialize a pointer
2. reinitialize a reference
3. initialize a NULL pointer

cast a pointer

mber Own Prayer

If we write a statement like s2 = s1;**___will be the calling object and____will be passed to the = operator as an argument.**

- **s1, s1**
- **s1, s2**
- **s2, s1**
- **s2, s2**

If we write a statement like *s2 = s1*; *s2* **will be the calling object and** *s1* **will be passed to the = operator as an argument. P# 397**

www.vuzs.info

what will be out put of cout <<setfill('0')<<setw(7)<<128;

Overloaded new operator function takes parameter of type *size_t* and returns

- **void (nothing)**

- ► void pointer
- ► object pointer
- ► int pointer

Also note that the *new* operator returns a *void* pointer. Any *new* operator we write must have this parameter and return type.

Which of the following is the correct way to declare a variable x of integer type?

- ► x int ;
- ► integer x ;
- ► int x;
- ► x integer

Reserve words cannot be used as a variable name.

- ► True
- ► False

There are few data types in C language. These data types are reserved words of C language. The reserve words can not be used as a variable manes. P# 17

A template function must have at least---------- generic data type

- ► Zero
- ► One
- ► Two
- ► Three

The function arguments must contain at least one generic data type. P# 499

Template functions can also be overloaded

- ► True
- ► False

We can write overloaded template functions as long as there is use of different number or type of arguments.. P # 503

We can not make a member function of a class as template function.

- ► True
- ► False not sure

When break statement is encountered in switch statement, it

- ► Stops the entire program
- ► Stops the execution of current statement
- ► Exits from switch statement
- ► None of the given options

We can also define a variable of user define data type (object) as static.

- True
- False

The declarator of Plus (+) member operator function is

- Class-Name operator + (Class-Name rhs)
- operator Class-Name + ( )
- operator Class-Name + (rhs)
- Class-Name operator + ( )

Let suppose

```
int a, b, c, d, e;
a = b = c = d = e = 42;
```

This can be interpreted by the complier as:

- a = (b = (c = (d = (e = 42))));
- (a = b = (c = (d = (e = 42))));
- a = b = (c = (d = (e = 42)));
- (a = b) = (c = d) = (e = 42);

a = (b = (c = (d = (e = 42))));

What will be the range of numbers generated by function rand () % 9?

- 0 to 9
- 1 to 9
- 0 to 8
- 1 to 8

When 6 divides any number, the remainder will always be less than 6. Ther result will be between therefore we will add 1. 1 + rand ( ) % 6;

Which of the following is the correct function call having array named *student* of 10 elements as a parameter.

- addRecord(student[]) ;
- addRecord(student) ;
- addRecord(student[10]) ;
- addRecord(*student) ;

when we pass array we don't give limit of array

Example:

Pass array to function

```
#include
  #include
  void read(int *,int);
  void dis(int *,int);

  void main()
  {
   int a[5],b[5],c[5],i;

   printf("Enter the elements of first list \n");
   read(a,5);
   printf("The elements of first list are \n");
   dis(a,5);
  }

  void read(int c[],int i)
  {
   int j;
   for(j=0;j      scanf("%d",&c[j]);
   fflush(stdin);
  }

  void dis(int d[],int i)
  {
   int j;
   for(j=0;j     printf("%d ",d[j]);
   printf("\n");
  }
```

Declaring structures does not mean that memory is allocated.
Example:

- ► True
- ► False

structures do not occupy any memory until it is associated with the structure variable

Identifier is a name that can be given to variables, labels and functions.

- ► True
- ► False

An 'Identifier' means any name that the user creates in his/her program. These names can be of variables, functions and labels

If a class A declares itself a friend of class B and a class B declares itself a friend of class C then

- ► Class A is also a friend of class C.
- ► Class B is also a friend of class A.
- ► Class A is also a friend of class C if A declares C as its friend.
- ► Class A is also a friend of class C if C declares A as its friend.

If we want a two-way relationship, OtherClass will have to declare ClassOne as a friend class, resulting in a complete two-way relationship

Which of the following statement is best regarding declaration of friend function?

- Friend function must be declared after public keyword.
- Friend function must be declared after private keyword.
- Friend function must be declared at the top within class definition.
- It can be declared anywhere in class as these are not affected by the public and private keywords.

Friend is a very strong statement. It is too strong to be affected by public or private we can put it anywhere in the class

A pointer is a special type of variable that contain _____

- Memory Address
- Data values
- Both Values and Memory
- None of given of options

Pointer is a special type of variable that contains a memory address.

When memory for a program is allocated at run time then it is called _____

- static memory allocation
- dynamic memory allocation
- stack memory allocation
- virtual memory allocation

When we create an object of the class at run time, it will allocate memory according to our requirement. So there is no waste of memory and the situations in which we want to store large data in small memory or vice versa are prevented. So we do dynamic memory allocation inside these classes.

What purpose do classes serve?

- Data encapsulation
- Providing a convenient way of modeling real-world objects
- Simplifying code reuse
- All of the given options

Which of the following function cannot be overloaded?

- Member functions
- Utility functions
- Constructor

► Destructor

The destructors can be summarized as The destructors cannot be overloaded. The destructors take no arguments. The destructors don't return a value

The following prototype of unary operator function indicates that it is_____.
Date operator++(int)

► Member functions of post increment operator
► Member functions of pre increment operator
► Non-member functions of post increment operator
► Non-member functions of pre increment operator

Overloading Unary Operators

```
//Preincrement operator overloaded as a member function.
Date Date::operator++()
{
  helpIncrement();
  return *this; // value return; not a reference return
}

// Postincrement operator overloaded as a member function.
// Note that the dummy integer parameter does not have a
// parameter name.
Date Date::operator++(int)
{
  Date temp = *this;
  helpIncrement();

  //return non-incremented, saved, temporary object
  return temp;  // value return; not a reference return
} // This paper was solved by vuzs Team and meant for hosting
at vuzs otherwise its stolen contents
```

Static variable which is defined in a function is initialized_____.

► Only once during its life time
► Every time the function call
► Compile time of the program
► None of the above

Once the static variables are created, they exist for the life of the program. They do not die.

In the member initialize list, the data members are initialized,

► From left to right
► From right to left
► In the order in which they are defined within class
► None of the given options

If we do not indent the code properly it will _____

- ► Be a syntax error
- ► Be a logical error
- ► Not be an error at all
- ► None of the given options

we Indent the code for better readability and understanding

Truth tables are used for analyzing_____.

- ► logical expressions
- ► arithmetic expressions
- ► both logical and arithmetic expressions
- ► none of the given options.

The truth tables are very important. These are still a tool available for analyzing logical expressions.

Static memory allocation is also known as _____

- ► Dynamic allocation
- ► Compile time allocation
- ► Run time allocation
- ► None of the given options

This type of memory static allocation. It is also known as compile time allocation.

Question No: 1 (Marks: 1) - Please choose one
When we define an array of objects then,
· Destructor will call once for whole array
· **Destructor will call for each object of the array**
· Destructor will never call
· Depends on the size of array

Question No: 2 (Marks: 1) - Please choose one
We can also create an array of user define data type
· **True**
· False

Question No: 3 (Marks: 1) - Please choose one
What is the sequence of event(s) when allocating memory using new operator?
· Only block of memory is allocated for objects
· Only constructor is called for objects
· **Memory is allocated first before calling constructor**
· Constructor is called first before allocating memory
If a single object is allocated, *operator new* is called to allocate memory, and then the constructor is called to initialize the object.

· If an array of objects is allocated, *operator new[]* is called to allocate memory for the whole array, and then the constructor is called for each element of the array.
· When a single object is deleted, the destructor for the object is called first, and then *operator delete* is called to free the memory occupied by the object.
· When an array of objects is deleted, the destructor for each element of the array object is called first, and then *operator delete[]* is called to free the memory occupied by the array.
http://www.vuzs.info/

Question No: 4 (Marks: 1) - Please choose one

We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally

· **True**
· False

Although, this is good to deallocate an array of objects without specifying array operator ([]) as there is no dynamic memory allocation occurring from inside the Date class. But this is a bad practice.

Question No: 5 (Marks: 1) - Please choose one

The declarator of Plus (+) member operator function is

· **Class-Name operator + (Class-Name rhs)**
· Operator Class-Name + ( )
· Operator Class-Name + (rhs)
· Class-Name operator + ( )

Page 371,373 example are here
Complex operator + (Complex &);
Complex operator + (parameter-list);
The syntax of the prototype of the overloaded operator function is:
return-type operator operator-symbol (parameter-list);
operator is the keyword here. An example of this will be as follows:
Complex operator + (Complex &);

Question No: 6 (Marks: 1) - Please choose one

The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators

· True (not sure)
· False

http://www.vuzs.info/

Question No: 7 (Marks: 1) - Please choose one

Which of the following is correct way to initialize a variable x of int type with value 10?

· int x ; x = 10 ;
· int x = 10 ;
· int x, x = 10;
· **x = 10 ;**

variable already created in question only it is asking for initialization.

Question No: 8 (Marks: 1) - Please choose one

Default mechanism of function calling in case of array is_____and in case of variable is _____

· Call by value, call by reference
· Call by referene, call by reference
· **Call by reference, call by value**
· Call by value, call by value

Question No: 9 (Marks: 1) - Please choose one

What does STL stand for?

· Source template library
· **Standard template library**
· Stream template library
· Standard temporary library

STL stands for Standard Template Library

Question No: 10 (Marks: 1) - Please choose one

Skill(s) that is/are needed by programmers _____

· Paying attention to detail
· Think about the reusability
· Think about user interface

· **All of the given options**

Programming is an important activity as people life and living depends on the programs one make. Hence while programming one should

o Paying attention to detail
o Think about the reusability.
o Think about user interface
o Understand the fact the computers are stupid
o Comment the code liberally

http://www.vuzs.info/

Question No: 11 (Marks: 1) - Please choose one

For which array, the size of the array should be one more than the number of elements in an array?

· int
· double
· float
· **char**

Question No: 12 (Marks: 1) - Please choose one

new and delete are_____whereas malloc and free are _____

· Functions, operators
· Classes, operators
· **Operators, functions**
· Operators, classes

new and delete are operators in c++

C functions like **malloc()** and **free()** functions can also be used from within C++ code

Question No: 13 (Marks: 1) - Please choose one

The prototype of friend functions must be written_____the class and its definition must be written _____

· inside, inside the class
· **inside, outside the class**
· outside, inside the class
· outside, outside the class

Question No: 14 (Marks: 1) - Please choose one

Friend function of a class are_____of a class.

· **Non-member functions** **not sure**
· Friend functions
· Any function outside class
· None of the given options

http://www.vuzs.info/

Question No: 15 (Marks: 1) - Please choose one

If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?

obj3 = obj1 + obj2 ;

· **obj2 will be passed as an argument to + operator whereas obj2 will drive the + operator**
· obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator
· Both objects (obj1, obj2) will be passed as arguments to the + operator
· Any of the objects (obj1, obj2) can drive the + operator

c3 = c1 + c2 ; In the above statement (c3 = c1 + c2;), c1 is the object that is calling or driving the **+** operator. **c2** object is being passed as an argument to the **+** operator. So c1 and c2 objects are added by the **+** operator and resultant

Question No: 16 (Marks: 1) - Please choose one

Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

· Class-name operator +() ;
· Class-name operator +(int) ;
· **Class-name operator ++()** ;
· Class-name operator ++(int) ;

Overloading Unary Operators

```
// Preincrement operator overloaded as a member function.
Date Date::operator++()
{
   helpIncrement();
   return *this; // value return; not a reference return
}

// Postincrement operator overloaded as a member function.
// Note that the dummy integer parameter does not have a
// parameter name.
Date Date::operator++(int)
{
   Date temp = *this;
   helpIncrement();

   // return non-incremented, saved, temporary object
   return temp; // value return; not a reference return
}
```

Question No: 17 (Marks: 1) - Please choose one

For cin, the source is normally a_____and destination can be _____

· File, native data type
· Disk, user-define type
· **Keyboard, variable**
· File, user-define type

For cin, the source is normally keyboard and the destination can be an ordinary variable i.e. native-data type variable. It could be some area of memory or our own data type, i.e. object for which we h

Question No: 18 (Marks: 1) - Please choose one

We can do condition compilation with pre processor directives.

· **True**
· False

All the preprocessor directives start with the sharp sign (#). We can also do conditional compilation with it.

Question No: 19 (Marks: 1) - Please choose one

The programs, in which we allocate static memory, run essentially on _____

· Heap
· System Cache
· None of the given options
· **Stack**

The programs, in which we allocate static memory, run essentially on stack

Question No: 20 (Marks: 1) - Please choose one

A template function must have at least---------- or more arguments

· Zero
· **One**
· Two
· Three

The function arguments must contain at least one generic data type. Normal function declaration is: return_type function_name(argument_list)

Question No: 21 (Marks: 1) - Please choose one

The default value of a parameter can be provided inside the _____

· function prototype
· function definition
· **both function prototype or function definition**
· none of the given options

The default value of a parameter is provided inside the function prototype or function definition.

Question No: 22 (Marks: 1) - Please choose one

While calling function, the arguments are assigned to the parameters from _____

· **left to right**
· right to left

· no specific order is followed
· none of the given options

While calling function, the arguments are assigned to the parameters from left to right.

Question No: 23 (Marks: 1) - Please choose one
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are

· Zero
· One
· Two
· N arguments

Operators as member functions
Aside from the operators which must be members, operators may be overloaded as member or non-member functions. The choice of whether or not to overload as a member is up to the programmer. Operators are generally overloaded as members when they:
change the left-hand operand, or
1. require direct access to the non-public parts of an object.
When an operator is defined as a member, the number of explicit parameters is reduced by one, as the calling object is implicitly supplied as an operand. Thus, binary operators take one explicit parameter and unary operators none. In the case of binary operators, the left hand operand is the calling object, and no type coercion will be done upon it.

Question No: 24 (Marks: 1) - Please choose one
new operator allocates memory from free store and return _____
· A pointer
· A reference
· An integer
· A float

Question No: 25 (Marks: 1) - Please choose one
With user-defined data type variables (Objects), self assignment can produce_____

· Syntax error **not sure**
· Logical error
· Link error
· Non of the given options

Question No: 26 (Marks: 1) – Write Simple Program
Assignment operator is used to initialize a newly declared object from existing object
· **True**
· False

Question No: 27 (Marks: 1) – Briefly define/Justify
When an object of a class is defined inside an other class then,
· Constructor of enclosing class will be called first
· Constructor of inner object will be called first
· Constructor and Destructor will be called simultaneously
· None of the given options
• ***A class can contain instances of other classes as its data members.*** • It is a way of reusing the code when we contain objects of our already written classes into a new class.
• The inner data members of the object are constructed and then the object itself.
The order of destruction of an object is reverse to this construction order, where the outer object is destroyed first before the inner data members.
• Initializer list is used to initialize the inner objects at the construction time.
• In C++, we can have structures or classes defined inside classes. Classes defined within other classes are called nested classes.

Question No: 28 (Marks: 1) – Brief answer required
In the member initializer list, the data members are initialized,
· From left to right
· From right to left
· In the order in which they are defined within class
· None of the given options

Question No: 29 (Marks: 1) - Brief answer required

"new" and "delete" keywords are_____in C++ language

· Built-in- Function
· **Operators**
· Memory Allocation Function
· None of the given options

Remember Own Prayer

In C/C++ if we define an array of size eight (8) i.e. int Arr [8]; then the last element of this array will be stored at,

► Arr[0]

- Arr[8]
- **Arr[7]**
- Arr[-1]

Question No: 2 (M - 1) .
When an array is passed to a function then default way of passing this array is,

- By data
- **By reference**
- By value
- By data type

Question No: 3 (M - 1) .
Array is a data structure which store

- Memory addresses
- Variables
- Data Type
- **Data**

Question No: 4 (M - 1) .
We can also create an array of user define data type.

- **True**
- False

Question No: 5 (M - 1) .
When we define an array of objects then,

- Destructor will call once for whole array
- **Destructor will call for each object of the array**
- Destructor will never call
- Depends on the size of array

Question No: 6 (M - 1) .
What is the sequence of event(s) when allocating memory using new operator?

- Only block of memory is allocated for objects
- Only constructor is called for objects
- **Memory is allocated first before calling constructor**
- Constructor is called first before allocating memory

Question No: 7 (M - 1) .
We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

- **True**
- False

Question No: 8 (M - 1) .
The second parameter of operator functions for are objects of the class for which we are overloading these operators.

- **True**
- False

Question No: 9 (M - 1) .
Which of the following is correct way to initialize a variable x of int type with value 10?

- int x ; x = 10 ;
- int x = 10 ;
- int x, x = 10;
- **x = 10 ;**

Question No: 10 (M - 1) .
Default mechanism of function calling in case of array is_____and in case of variable is___.

- Call by value, call by reference
- Call by referene, call by reference
- **Call by reference, call by value**
- Call by value, call by value

Question No: 11 (M - 1) .
What does STL stand for?

- ► Source template library
- ► **Standard template library**
- ► Stream template library
- ► Standard temporary library

Question No: 12 (M - 1) .
Skill(s) that is/are needed by programmers_____.

- ► Paying attention to detail
- ► Think about the reusability
- ► Think about user interface
- ► **All of the given options**

Question No: 13 (M - 1) .
For which array, the size of the array should be one more than the number of elements in an array?

- ► int
- ► double
- ► float
- ► **char**

Question No: 14 (M - 1) .
new and delete are_____whereas malloc and free are_____.

- ► Functions, operators
- ► Classes, operators
- ► **Operators, functions**
- ► Operators, classes

Question No: 15 (M - 1) .
Friend functions are_____of a class.

- ► Member functions
- ► Public member functions
- ► Private member functions
- ► **Non-member functions**

Question No: 16 (M - 1) .
The prototype of friend functions must be written_____the class and its definition must be written _____

- ► inside, inside the class
- ► **inside, outside the class**
- ► outside, inside the class
- ► outside, outside the class

Question No: 17 (M - 1) .
If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?
obj3 = obj1 + obj2 ;

- ► obj2 will be passed as an argument to + operator whereas obj2 will drive the + operator
- ► **obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator**
- ► Both objects (obj1, obj2) will be passed as arguments to the + operator
- ► Any of the objects (obj1, obj2) can drive the + operator

Question No: 18 (M - 1) .
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

- ► Class-name operator +() ;
- ► Class-name operator +(int) ;
- ► **Class-name operator ++() ;**
- ► Class-name operator ++(int) ;

Question No: 19 (M - 1) .
For cin, the source is normally a_____and destination can be_____.

- ► File, native data type
- ► Disk, user-define type

► **Keyboard, variable**
► File, user-define type

Question No: 20 (M - 1) .
We can also do conditional compilation with preprocessor directives.

► **True**
► False

Question No: 21 (M - 1) .
The programs, in which we allocate static memory, run essentially on _____

► Heap
► System Cache
► None of the given options
► **Stack**

Question No: 22 (M - 1) .
The default value of a parameter can be provided inside the _____

► function prototype
► function definition
► **both function prototype or function definition**
► none of the given options.

Question No: 23 (M - 1) .
While calling function, the arguments are assigned to the parameters from_____.

► **left to right.**
► right to left
► no specific order is followed
► none of the given options.

Question No: 24 (M - 1) .
When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

► Zero
► **One**
► Two
► N arguments

Question No: 25 (M - 1) .
With user-defined data type variables (Objects), self assignment can produce_____.

► Syntax error
► **Logical error**
► Link error
► Non of the given options

Question No: 26 (M - 1) .
Assignment operator is used to initialize a newly declared object from existing object.

► **True**
► False

Question No: 27 (M - 1) .
When an object of a class is defined inside an other class then,

► Constructor of enclosing class will be called first
► **Constructor of inner object will be called first**
► Constructor and Destructor will be called simultaneously
► None of the given options

Question No: 28 (M - 1) .
In the member initializer list, the data members are initialized,

► From left to right
► From right to left
► **In the order in which they are defined within class**
► None of the given options

Question No: 29 (M - 1) .
new operator allocates memory from free store and return_____.

- ► **A pointer**
- ► A reference
- ► An integer
- ► A float

Question No: 30 (M - 1) .
"new" and "delete" keywords are_____in C++ language.

- ► Built-in- Function
- ► **Operators**
- ► Memory Allocation Function
- ► None of the given options

Remember Own Praye

Operator overloading can be performed through_____.

- ► Classes
- ► **Functions**
- ► Operators
- ► Reference

Question No: 2 (M - 1) .
When a value is referred by a normal variable then it is known as,

- ► **Direct Reference**
- ► Indirect Reference
- ► Partial Reference
- ► Proper Reference

When a value is referred by a normal variable is known as direct reference

Question No: 3 (M - 1) .
Which of the following function is used to increase the size of already allocated memory chunk?

- ► malloc
- ► calloc
- ► **realloc**
- ► free

(FQ, vuzs, 2010)

Question No: 4 (M - 1) .
Which of the following is NOT a preprocessor directive?

- ► #error

► #define
► #line
► **#ndefine**

list of preprocessors
• #include • #include "filename" • #define • #undef • #ifdef • #ifndef • #if • #else • #elif • #endif • #error • #line • #pragma • #assert

Question No: 5 (M - 1) .
The stream objects **cin** and **cout** are included in which header file?
► **iostream.h**
► fstream.h
► istream.h
► ostream.h
http://www.vuzs.info/
Question No: 6 (M - 1) .
Overloaded delete operator function takes the same parameter as an argument returned by new operator function.
► **True**
► False
The same pointer that is returned by the new operator, is passed as an argument to the delete operator. These rules apply to both, if operators (new and delete) are overloaded as member or non-member operators (as global operators).

Question No: 7 (M - 1) .
When an array of object is created dynamically then there is no way to provide parameterized constructors for array of objects.
► **True**
► False
if we are allocating an array of objects, there is no way to pass arguments to objects' constructors. Therefore it is required that the objects that are stored in such an array have a no-argument constructor.

Question No: 8 (M - 1) .
C is widely known as development language of_____operating system.
► Linux
► Windows
► **Unix**
► Mac OS
In the start C became widely known as the development language of the UNIX operating system, and the UNIX operating system was written by using this C language. The C language is so powerful that the compiler of C and other various operating systems are written in C.
http://vuzs.ne
Question No: 9 (M - 1) .
Computer can understand only machine language code.
► **True**
► False
Question No: 10 (M - 1) .
We can not define a function as a friend of a Template class.
► True
► **False**

Class templates can have friends. A class or class template, function, or function template can be a friend to a template class. Friends can also be specializations of a class template or function template, but not partial specializations.

Question No: 11 (M - 1) .

What will be the value of 'a' and 'b' after executing the following statements?

```
a = 3;
b = a++;
```

- ► 3, 4
- ► 4, 4
- ► 3, 3
- ► **4, 3**

Question No: 12 (M - 1) .

Consider the following code segment. What will be the output of following code?

```
int addValue (int *a){
int b = (*a) + 2;
return b ;
}
main () {
int x =6 ;
cout << x << ",” ;
cout << addValue(&x) << ",” ;
cout << x ;
}
```

http://vuzs.ne

- ► **6,8,6**
- ► 6,6,8
- ► 6,8,8
- ► 6,6,6

http://www.vuzs.info/

Question No: 13 (M - 1) .

_____is used to trace the logic of the program and correct the logical errors.

- ► Compiler
- ► Editor
- ► Linker
- ► **Debugger**

Question No: 14 (M - 1) .

new and delete are_____whereas malloc and free are_____.

- ► Functions, operators
- ► Classes, operators
- ► **Operators, functions**
- ► Operators, classes

Hence, we can call new and delete operators, P# 342

we have allocated a memory space for our use by malloc function. P# 285

Question No: 15 (M - 1) .

Like member functions,_____can also access the private data members of a class.

- ► Non-member functions
- ► **Friend functions**
- ► Any function outside class
- ► None of the given options

Question No: 16 (M - 1) .

Which situation would require the use of a non-member overloaded operator?

- ► The overloaded operator is an Assignment operator.

http://vuzs.ne

► The left most operand is an object of a class.

► **The left operand is built-in data type.**

► The operator returns a reference.

(100% confirmed by Rainbowbright)

Question No: 17 (M - 1) .

The stream insertion and stream extraction operators are already overloaded for_.

► User-defined data types

► **Built-in data types**

► User-defined and built-in data types

► None of the given options

Question No: 18 (M - 1) .

If we define an identifier with the statement **#define PI 3.1415926** then during the execution of the program the value of PI_____.

► can not be replaced

► None of the given options

► **Remain constant.**

► can be changed by some operation

Question No: 19 (M - 1) . vuzs

Assignment operator is ----------associative.

► **right**

► left

► binary

► unary

You can assign values to several variables in a single statement. For example, the following code sets the contents of apples and oranges to the same value:
apples = oranges = 10;
The assignment operator is right associative, so this statement executes by first storing the value 10 in oranges and then storing the value in oranges in apples, so it is effectively
apples = (oranges = 10);

http://www.vuzs.info/

Question No: 20 (M - 1) .

When ever dynamic memory allocation is made in C/C++, it is freed_____.

► **Explicitly**

► Implicitly

► Both explicitly and implicitly

► None of the given options

(Rainbowbright, vuzs. jul2011)

Question No: 21 (M - 1) .

The appropriate data type to store the number of rows and colums of the matrix is_____.

► float

► **int**

► char

► none of the given options.

Question No: 22 (M - 1) .

Which of the following function do NOT initialize the chunk of memory to all zero?

► calloc() function

► Both malloc() and calloc()

► None of the above

► **malloc() function**

The **malloc** function differs from **calloc** in the way that the space allocated by **malloc** is not initialized and contains any values initially.

Question No: 23 (M - 1) .

The function free() returns back the allocated memory got thorough calloc and malloc to_____.

- ► stack
- ► **heap**
- ► stack and heap
- ► None of the given options

http://vuzs.ne

Question No: 24 (M - 1) .

width() is member function of _____

- ► cin object
- ► cout object
- ► **Both cin and cout object**
- ► None of the given option

Question No: 25 (M - 1) .

Templates are not type safe.

- ► true
- ► **false**

Templates are type-safe. This is because the types that templates act upon are known at compile time, so the compiler can perform type checking before errors occur.

Question No: 26 (M - 1) .

A Matrix can be composed of ints, floats or doubles as their elements. Best way is to handle this , _____

- ► Write a separate class to handle each
- ► **Use templates**
- ► Use strings to store all types
- ► None of the given options

1. For binary member operators, operands on the _____ drives (calls) the operation.

Left

2. We cannot increment_____.

references

3. We can_____pointer.

all of the given

4. We can_____references.

None of the given

Remember Own Pray

5. What will be the correct syntax for the following function call?
float add (int &);

add(x);

6. An instance of a class is called_____.

object

7. The_____is called automatically when an object destroys

destructor

8. The destructor is used to_____.

1. deallocate memory

9. _____data isn't accessible by non-member functions or outside classes.

private

10. Member functions of the class_____main program.

1. are accessible from

11. Overloading means :

1. Using the same name to perform multiple tasks or different tasks depending on the situation.

12. The main advantage of function overloading is _____.

1. The program becomes more readable

13. You cannot overload the_____operator.

1. ? :

Remember Own Prayer

14. In C++, a variable can be declared anywhere in the program this will increase_____.

1. efficiency

15. Memory allocated from heap or free store_____.

1. cannot be returned back unless freed explicitly using free and delete operators

16. We cannot use_____pointer for storing and reading data from it.

1. 'NULL

17. The dynamic memory allocation uses_____whereas static memory allocation uses_____.

1. heap , stack

18. What will be the output of the given code?

```
#include #define MAX( A, B ) ((A) > (B) ? (A) : (B))
void main() {
int i, x, y;
x = 23;
y = 45;
i = MAX( x++, y++ );
// Side-effect: // larger value incremented twice
cout<< "x = " << x << " y = " << y << '\n';
}
```

1. x=24 y=47

19. NULL has been defined in_____header file.

1. Stdlib.h

20. Symbolic constant PI can be defined as:

1. #define PI 3.14

21. The friend function of a class can have access _____.

Remember Own Prayer

1. to the private data members

22. C++ was developed by_____.

1. BejarneStroustrup

23. Once the_____are created, they exist for the life time of the program.

1. static variables

24. Encapsulation means_____.

1. that the data of a class can be accessed from outside

25. An address is a_____, while a pointer is a _____.

1. variable, constant

26. The syntax of declaration of a function that returns the reference to an integer is_____.

1. int &myfunc();
2. int myfunc();
3. int myfunc() &;
4. integer &myfunc();

27. Which one of the following is mandatory preprocessor directive for c++?

1. #undef
2. #include
3. #undef
4. All of the given

28. The members of a class declared with the keyword struct are_____by default.

1. static
2. Private
3. protected
4. public

Remember Own Prayer

29. getche() is a_____function and defined in _____header file.

1. user-define function ,conio.h
2. built-in function ,conio.h
3. built-in function, stlib.h
4. built -in function, iostream.h

30. _____operators are the ones that require two operands on both sides of the operator.

1. Double
2. Tow sided
3. Binary
4. None of the given

31. _____will return the number of bytes reserved for a variable or data type.

1. sizeof operator
2. free operator
3. void pointer
4. new operator

32. _____are not available in C language.

1. User defned functions
2. Built in functions
3. Library functions
4. Inline functions

33. The members of a class declared without any keyword are_____by default.

1. protected
2. private
3. public
4. constant

34. For console input and output we use_____.

Remember Own Prayer

1. conio.h header file
2. stdlib.h header file
3. process.h header file
4. getch.h header file

35. The name of the destructor is the same as that of a class proceeding with a_____.

1. &sign
2. # sign
3. @ sign
4. ~ sign

36. A reference cannot be NULL it has to point a data type.

1. True
2. False

37. A pointer is_____.

1. the address of a variable
2. an indication of the variable to be accessed next
3. a variable for storing address
4. the data type of an address variable

38. Constructor is a special function, called whenever we _____.

1. create a function
2. instantiate an object of a class
3. destroy an object
4. create a class

39. Symbolic constant PI can be defined as:

1. #define PI 3.14;
2. #define PI 3.14
3. #define PI=3.14
4. # include pi=3.14

40. Object code is machine code but it is not_____and _____.

Rehmat's Own Prayer

1. relocatable, executable
2. faster, efficient
3. compiled, debugged
4. tested, compiled

41. The default visibility for the data members of the class is

1. private
2. protected
3. public
4. accessible outside the class

42. The_____is called automatically when an object destroys.

1. destructor
2. constructor
3. main program
4. default constructor

43. Constructor is special type of function :

1. which has no return type
2. which returns NULL pointer
3. which returns zero
4. which returns integer type data

44. _____variables are those that are defined outside of main.

1. Local
2. Dynamic
3. Global
4. Static

45. Within the statement obj1=obj2; obj1 will call the assignment operator function and obj2 will be passed as an argument to function.

1. True

Ramdhan Own Prayer

2. False

46. When the compiler overload the assignment (=) operator by default then

1. Class members are not assigned properly
2. Compiler does not allow default assignment operator
3. Compiler does member wise assignment.
4. None of the given

47. It is possible to return an object from function using this pointer.

1. True
2. False

48. Overloaded assignment operator must be

1. Member function of class
2. Non-member function of class
3. Friend function of class
4. Global function

49. Let suppose
int a, b, c, d, e;
a = b = c = d = e = 42;
This can be interpreted by the compiler as

1. (a = b = (c = (d = (e = 42))));
2. a = (b = (c = (d = (e = 42))));
3. a = b = (c = (d = (e = 42)));
4. (a = b) = (c = d) = (e = 42);

50. In statement a+b+c, at first

1. a+b is executed first
2. b+c is executed first
3. All executed at the same time
4. None of the given

51. Suppose int i = 10; then what is the output of cout<<oct<<i;

Remember Own Prayer

1. 10
2. 11
3. 12
4. 13

52. ostream is a_____operator.

1. dependent
2. member
3. standalone
4. None of the given

53. _____must be included to use stream manipulation in your code.

1. conio.h
2. iostream
3. stdlib.h
4. iomanip

54. _____operators are the ones that require only one operator to work.

1. Unit
2. Unary
3. Single
4. None of the given

55. The endl and flush are_____.

1. Functions
2. Operators
3. Manipulators
4. Objects

56. When operator function is implemented as member function then return type of function_____.

1. Must be an object of same class
2. Must be user-defined data type
3. Must be built-in data type

Remember Own Prayer

4. Can be any data type

57. When a variable is defined as static in a class then _____.

1. Separate copy of this variable is created for each object
2. Only one copy is created for all objects of this class
3. A copy of this variable is created for only static objects
4. None of the given

58. Automatic variables are created on_____.

1. Heap
2. Free store
3. Static storage
4. stack

59.
```
cout<<i<< " ";
cout<< d <<" ";
cout<< f;
```

Above statements can be written within statement of one line as:

1. cout<<i<<" "<< d " "<< f << ;
2. cout<<i<<<< d <<<< f <<;
3. cout<<i<<" "<< d <<" "<< f;
4. cout<<i<< " "<< d <<" " f<<;

60. dec, hex, oct are all_____.

1. Member functions
2. Objects of input/output streams
3. Parameterized manipulators
4. Non-parameterized manipulators

61. What will be the output of following statement?
```
cout<<setfill('0')<<setw(7)<< 128;
```

1. 0128128
2. 0000128

Remember Own Prayer

3. 1280000
4. 0012800

62. Which of the following syntax is best used to delete an array of 5 objects named 'string' allocated using new operator.

1. delete string;
2. delete []string;
3. delete string[];
4. delete string[5];

63. If we have a program that writes the output data(numbers) to the disc, and if we collect the output data and write it on the disc in one write operation instead of writing the numbers one by one.
In the above situation the area where we will gather the number is called

1. Heap
2. Stack
3. Buffer
4. Cache

64. The first parameter of operator function for << operator _____.

1. Must be passed by value
2. Must be passed by reference
3. Can be passed by value or reference
4. Must be object of class

65. The second parameter of operator function for >> operator must always be passed

1. By reference
2. Function takes no argument
3. By value
4. None of the given

Remember Own Prayer

66. The only operator that the compiler overloads for user define data type by default is

1. Plus (+) operator
2. MInus (-) operator
3. Assignment (=) operator
4. Equal (==) operator

67. Consider the following code, the printed value will be converted into:

```
int n=10;
cout<<oct<<n;
```

1. Base 8
2. Base 2
3. Base 10
4. Decimal number system

68. _____variables are defined in the main.

1. Global
2. Dynamic
3. Local
4. All

69. ostream class is_____and not under our control.

1. user-defined
2. built-in
3. both user-defined and built-in
4. None of the given

70. The memory allocation in C++ is carried out with the help of_____.

1. NULL pointer
2. new operator
3. dot operator
4. + operator

Remember Own Prayer

71. If B is designated as friend of A, B can access A's non-public members.

1. B cannot access private member of A
2. B cannot access protected member of A
3. A can access non-public members of B
4. A cannot access B

72. If the request of new operator is not fulfilled due to insufficient memory in the heap_____.

1. the new operator returns 2
2. the new operator returns 1
3. the operator returns 0
4. free operator returns nothing

73. We should not use such variable names that are starting with_____because in C++, there are lots of internal constants and symbolic names that start with it.

1. upper case alphabets
2. lower case alphabets
3. double underscore
4. None of the given

74. The friend keyword provides access_____.

1. in one direction only
2. in two directions
3. to all classes
4. to the data members of the friend class only

75. The malloc function takes_____argument(s).

1. two
2. three
3. four
4. one

76. The constructor contains_____.

Remember Own Prayer

1. return type
2. no return type
3. objects
4. classes

77. What will be the output of the following c++ code?

```
#include<iostream.h>
#define max 100
main()
{
#ifdef max
Cout<<"Hellow;
}
```

1. Hello
2. "Hellow"
3. Max is 100
4. Error

78. Once we have defined a symbolic constant value using #define, that value_____during program execution

1. can be changed
2. cannot be changed
3. varies
4. becomes zero

79. The memory allocation functions return a chunk of memory with a pointer of type_____.

1. integer
2. float
3. ptr
4. void

80. A class can be declared as a_____of other class.

1. member
2. member function
3. friend

Remember Own Prayer

4. part

81. To avoid dangling reference, don't return_____.

1. the reference of a local variable from the function
2. the reference of a global variable from the function
3. the reference of a static variable from the function
4. the reference of a private data member from the function

82. Constructor is itself a_____of C++ and_____.

1. class, can be overloaded
2. function, cannot be overloaded
3. function, can be overloaded
4. object, can not be initialized

83. The parameter passed to isdigit() function is _____ variable.

1. Character
2. Boolean
3. Integer
4. Float

84. char **argv can be read as_____.

1. pointer to pointer
2. pointer to char
3. pointer to pointer to char
4. None of the given

85. To read command-line arguments, the main() function itself must be given_____arguments.

1. 1
2. 2
3. 3
4. 4

86. How many bytes an integer type pointer intPtr will jump in memory if the statement below is executed?
intPtr += 2 ;

Remember Own Prayer

1. 2
2. 4
3. 8
4. 12

87. The increment of a pointer depends on its_____.

1. variable
2. value
3. data type
4. None of the given

88. The statement cout<<yptr will show the _____ theyptr points to.

1. Value
2. memory address
3. variable
4. None of the given

89. _____is used as a dereferencing operator.

1. *
2. +
3. -
4. None of the above

90. Transpose of a matrix means that when we interchange rows and columns_____.

1. the first row becomes the Last column
2. the first row becomes the first column
3. the Last row becomes the first column
4. the first column becomes the first row

91. Individual characters in a string stored in an array can be accessed directly using array_____.

1. superscript
2. script
3. subscript

Remember Own Prayer

4. value

92. We can define a matrix as_____array.

1. Sorted
2. Unsorted
3. Single dimensional
4. Multi dimensional

93. A_____is an array of characters that can store number of character specified.

1. Char
2. String
3. Multidimensional array
4. Data type

94. Given a two dimensional array of integers, what would be the correct way of assigning the value 6 to the element at third row and fourth column?

1. array[3][4] = 6 ;
2. array[2][4] = 6 ;
3. array[4][3] = 6 ;
4. array[2][3] = 6 ;

95. _____of a variable means the locations within a program from where it can be accessed.

1. Data type
2. Visibility
3. Value
4. Reference

96. Which of the following function call is "call by reference" for the following function prototype?
int add (int *);

1. add(&x);
2. add(int x);
3. add(x);

Remember Own Prayer

4. add(*x);

97. Which of the following function call is "call by reference" for the following function prototype?
float add (float *);

1. add(&x);
2. add(float x);
3. add(x);
4. add(*x);

98. Which of the function call is call by value for the following function prototype?
float add(float);

1. add(&x);
2. add(x);
3. add(float x);
4. add(*x);

99. Which of the function call is "call by value" for the following function prototype?
float add(int);

1. add(&x);
2. add(x);
3. add(int x);
4. add(*x);

100. Return type of a function that does not return any value must be_____.

1. char
2. int
3. void
4. double

101. _____will be used for enclosing function statements into a block.

1. " "

Remember Own Prayer

2. ()
3. []
4. {}

102. What is the output of the following code if the 2nd case is true

```
switch (var) {
case 'a': cout<<"apple"<<endl;
case 'b':cout<<"banana"<<endl;
case 'm':cout<<"mango"<<endl;
default: cout<<"any fruit"<<endl;
}
```

1. banana
2. banana
   any fruit
3. banana
   mango
   any fruit
4. None of the given

103. When the break statement is encountered in a loop's body, it transfers the control_____from the current loop.

1. Inside
2. Outside
3. To break statement
4. To continue statement

104. What is the output of the following code if the 3rd case is true

```
switch (var) {
case 'a':cout<<"apple"<<endl;
case 'b':cout<<"banana"<<endl;
case 'm':cout<<"mango"<<endl;
default: cout<<"any fruit"<<endl;
}
```

Remember Own Prayer

1. mango
2. mango
   any fruit
3. apple
4. None of the given

105. What is the output of the following code, if the first case is true

```
switch (var) {
case 'a':cout<<"apple"<<endl;
case 'b':cout<<"banana"<<endl;
case 'm':cout<<"mango"<<endl;
default: cout<<"any fruit"<<endl;
}
```

1. apple
2. apple
   any fruit
3. apple
   banana
   mango
   any fruit
4. none of above

106. What will be the output of following code segment?

```
for (int i = 2; i<10; i++) {
if ( i == 5) continue;
cout<<i<< "," ;
}
```

1. 2,3,7,8,9
2. 2,3,4,6,7,8,9
3. 2,3,4
4. 4,6,7,8,9

107. _____statement is used to terminate the processing of a particular case and exit from switch structure.

Remember Own Prayer

1. if
2. goto
3. break
4. continue

108. What will be the result of the expression j = i++; if initially j = 0 and i = 5?

1. 0
2. 5
3. 6
4. 4

109. What will be the result of the expression k = ++m; if initially k = 0 and m = 4?

1. 0
2. 5
3. 6
4. 4

110. What will be the result of the expression k = ++m; if initially k = 0 and m = 5?

1. 0
2. 5
3. 6
4. 4

111. How many times the following do-while loop will execute?
int k = 10; do { cout<< "Statements" <<endl; k -= 2; } while(k>0);

1. 4
2. 5
3. 6
4. 7

112. Which of the following loops checks the test condition at the end of the loop?

Remember Own Prayer

1. While
2. Do-While
3. For
4. Nested Loop

113. The operators ++ and -- are used to increment or decrement the value of a variable by_____.

1. 3
2. 2
3. 1
4. 4

114. How many times the following loop will execute? int j = 3; while(j > 0) { cout<< "Statements" <<endl; j -= 2; }

1. 0
2. 1
3. 2
4. 3

115. A_____structure specifies that an action is to be repeated while some condition remains true.

1. Control
2. Logical
3. Repetition
4. Relational

116. !(x> 3) means in C++ that

1. x is greater than 3
2. x is less than or equal to 3
3. x is less than 3
4. x is equal to 3

117. When the logical operator && combines two expressions then the result will be true only when the both expressions are_____.

Remember Own Prayer

1. Logical
2. Arithmetic
3. true
4. false

118. < and > both are_____operators.

1. Arithmetic
2. Relational
3. Logical
4. Mathematical

119. What will be the value of variable "input" if the initial value of input is 67?

```
if(input >= 50)
input = input + 1;
if(input <= 75)
input = input + 2;
else
input = input - 1;
```

1. 68
2. 69
3. 70
4. 66

120. !(x< 3) means in C++ that

1. x is less than 3
2. x is greater than or equal to 3
3. x is greater than 3
4. x is equal to 3

121. != operator is used to check whether the operand on the left-hand-side is_____to the operand on the right-hand-side.

1. Less than or equal
2. Greater than or equal
3. Not equal

Remember Own Prayer

4. Approximately equal to

122. When the if statement consists more than one statement then enclosing these statement in curly braces is,

1. Not required
2. Good programming
3. Relevant
4. Must

123. The most suitable data type for number 325.25 is _____.

1. char
2. int
3. short
4. float

124. What will be the result of arithmetic expression 6+48/4*3?

1. 10
2. 40.5
3. 42
4. 41

125. Which of the following will be the most appropriate data type to store the value 63.547?

1. Integer
2. Character
3. Short
4. Float

126. In the given expression which operator will be evaluated first? 10 + (6 / 2) - 2 * 3?

1. +
2. -
3. /

Remember Own Prayer

4. *

127. What will be the value of the variable output in the given piece of code?

```
double output = 0;
output = (2 + 2) * 4 + 2 / (4 - 2);
```

1. 15
2. 17
3. 12
4. 11

128. It is the job of_____to transfer the executable code from hard disk to main memory.

1. interpreter
2. Debugger
3. Linker
4. Loader

129. In computer systems there are mainly_____type of softwares.

1. 1
2. 2
3. 3
4. 4

130. _____will explain the function of a program.

1. Comments
2. Debugger
3. Compiler
4. Linker

131. if (a>b && a>c) then the condition will be true only if

1. Both a>b and a>c are true
2. a>b is false and a>c is true
3. a>b is true and a>c is false

Remember Own Prayer

4. Both a>b and a>c are false

132. A variable of character data type occupies _____ byte(s) in memory.

1. 1
2. 2
3. 4
4. 8

133. We must include the header file_____to convert the value of one type into another type using built-in functions.

1. conio.h
2. stdlib.h
3. string.h
4. iostream.h

134. A function is a block of statements that can be defined once and used_____in the program.

1. One time
2. Two times
3. Three times
4. As many times as user wants

135. Select the correct way to assign the address of first element of array to pointer?

1. int *ptr = &data[1];
2. int *ptr = &data;
3. int *ptr = data;
4. int *ptr = data[0];

136. Consider the following code segment. What will be the output of following code?

```
int addValue (int *a){
int b = (*a) + 2;
return b;
}
```

Remember Own Prayer

```
main() {
int x = 6;
cout<<addValue(&x)<<",";
cout<<x;
}
```

1. 6,8,6
2. 6,6,8
3. 6,8,8
4. 6,6,6

137. Here the code is given below. You have to identify the problem in the code.

```
while(i< 10) && (i> 24))
```

1. the logical operator && cannot be used in test condition
2. the while loop is an exit-condition loop
3. the test condition is always true
4. the test condition is always false

138. The correct syntax of do-while loop is_____.

1. (condition) while; do {statements;};
2. {statements;} do-while();
3. while(condition); do {statements;};
4. do {statements;} while (condition);

139. Matrix is defined as_____.

1. Single dimensional array
2. Multi-dimensional array
3. Vector product
4. Scalar product

140. In programming, comments are used to explain the functioning of the_____.

1. Debugger
2. Editor
3. Program

Remember Own Prayer

4. Linker

141. Operating System is a type of a/an_____.

1. application software
2. system software
3. computer language
4. interpreter

142. From the options given, you need to choose the option which is true for the given code.

```
for (int i = 1; i>0; i++) {
/*loop code*/
}
```

1. the logical operator && cannot be used in a test condition
2. the while loop is an exit-condition loop
3. the test condition is always false
4. the test condition is always true

143. Which of the following values are used in C/C++ to represent true and false?

1. 1 and 0
2. 1 and -1
3. 11 and 00
4. any numerical value

144. 'While' loop may execute_____or more times.

1. three
2. zero
3. two
4. one

145. Body of any function is enclosed within_____.

1. { }
2. ( )
3. [ ]

Remember Own Prayer

4. " "

146. What will be the correct syntax for initialization of a pointer ptr with string "programming"?

1. char ptr = 'programming';
2. char *ptr = "programming";
3. char *ptr = 'programming';
4. *ptr = "programming";

147. Which one of the given option is not a mode for reading/writing the data from a file?

1. in
2. out
3. trunc
4. get

148. Which of the following operators is used to access the value of variable pointed by a pointer?

1. * operator
2. -> operator
3. && operator
4. &operator

149. In case of single dereferencing, the value of the _____is the address of the_____.

1. pointer, variable
2. pointer, constant
3. variable, pointer
4. constant, pointer

150. The remainder (%) operator is a_____operator.

1. Logical
2. Arithmetic
3. Relational
4. Conditional

Remember Own Prayer

151. What will be the output of following code?

```
int x = 10;
cout<<"x="<<x;
```

1. 10
2. "x=10"
3. x=10
4. 10=x

152. The purpose of using cout<< is to_____.

1. Display information on the screen
2. Read the data from keyboard
3. Read the data from a file
4. Write into a file

153. Which of the following data types will be assumed if no data type is specified with constant?

1. short
2. float
3. int
4. double

154. When an array element is passed to a function, it is passed by_____.

1. reference
2. data type
3. value
4. data

155. While programming, it is good to provide an easy to understand and easy to use interface; this programming skill is called_____.

1. scalability
2. usability
3. reliability
4. sustainability

Remember Own Prayer

156. _____executes all the lines before error and stops at the line which contains the error.

1. Intrepreter
2. Compiler
3. Linker
4. Debugger

157. Which of the following is the correct syntax to access the value of first element of an array using pointer ptr?

1. ptr[0]
2. *(ptr+1)
3. ptr[1]
4. *ptr[0]

158. C is a/an_____language.

1. low level
2. object based
3. object oriented
4. function oriented

159. _____of a function is also known as signature of a function.

1. Definition
2. Declaration
3. Calling
4. Invoking

160. _____are very good tools for code reuse.

1. operators
2. loops
3. functions
4. variables

161. If any break statement is missed in switch statement then_____.

Remember Own Prayer

1. compiler will give error
2. this may cause a logical error
3. no effect on program
4. program stops its execution

162. A 2D array multi[5][10] can be accessed using the array name as **multi, this technique is called_____.

1. Single referencing
2. Single dereferencing
3. Double referencing
4. Double dereferencing

163. In C/C++, the default command line arguments passed to the main function are_____.

1. float argc, char **argv
2. int argc, char **argv
3. int *argc, char *argv
4. int argc, float **argv

164. A record is a group of related_____.

1. Data
2. Fields
3. Bytes
4. Files

165. The microsoft word document (.doc) is a kind of _____.

1. Sequential File
2. Random AccessFile
3. Binary Access File
4. Executable File

166. NULL character is used to indicate the_____of string.

1. Start
2. End

Remember Own Prayer

3. Begin
4. Middle

167. How many dimensions does n-dimensional array has?

1. n dimensions
2. 2n dimensions
3. (n+1) dimensions
4. (n-1) dimensions

168. Which of the following function call is "call by reference" for the following function prototype?

1. func(int &num);
2. func(&num);
3. func(*num);
4. func(num);

169. The loop which is most suitable to be used when the number of iterations is known is called_____.

1. for
2. while
3. do-while
4. all looping processes require that the iterations be known.

170. In C/C++, the string constant is enclosed in_____.

1. curly braces { }
2. parentheses( )
3. single quotes ' '
4. double quotes " "

171. In order to get the right most digit of a number, we divide this number by 10 and take_____.

1. Its remainder
2. Its quotient
3. Its divisor
4. The number

Remember Own Prayer

172. What is the correct syntax to declare an array of size 10 of int data type?

1. int [10] name;
2. name[10] int;
3. int name[10];
4. int name[];

173. How many bytes of memory are occupied by array 'str'?

```
char str[] = "programming";
```

1. 10
2. 11
3. 12
4. 13

174. Suppose that an integer type pointer contains a memory address 0x22f230. What will be the new memory address if we increment this pointer by one?

1. 0x22f231
2. 0x22f234
3. 0x22f226
4. 0x22f238

175. Which of the following if missing would result in infinite recursion in case of recursive function?

1. Recursive call
2. Base case
3. Function parameters
4. Local variables

176. Whenever we use a library function or a predefined object or macro, we need to use a_____.

1. source file
2. object file
3. header file
4. exe file

Remember Own Prayer

177. Switch statement deals with_____type of data.

1. Integer
2. Float
3. Character
4. Both Integer and Character

178. Both compiler and_____are used to translate program into machine language code.

1. debugger
2. linker
3. loader
4. interpreter

179. TWAIN stands for_____.

1. Technology With An Interesting Name
2. Technology Without An Informative Name
3. Technology Without An Interesting Name
4. Technology With An Informative Name

180. The parameter passed to isdigit() function is_____.

1. a character variable
2. a boolean variable
3. an integer variable
4. a character string

181. C++ views each file as a sequential stream of _____.

1. Bits
2. Bytes
3. Numbers
4. Words

182. Structure is a collection of_____under a single name.

1. only functions

Remember Own Prayer

2. only variables
3. both functions and variables
4. only data types

183. The default mode for writing into a file using ofstream object is_____.

1. out
2. bin
3. app
4. ate

184. The memory address of the first element of an array is called_____.

1. floor address
2. foundation address
3. first address
4. base address

185. We want to access array in random order which of the following approach is better?

1. Pointer
2. Array index
3. Both pointers and array index are better
4. Matrix

186. The_____structure is a multiple-selection construct which makes the code more efficient and easy to read and understand.

1. multiple-if
2. switch
3. if-else
4. else-if

187. Which of the following is not a reserved word in C/C++?

1. int
2. float

Remember Own Prayer

3. double
4. sum

188. To access rand(), which library is required to be included in program?

1. conio.h
2. stdio.h
3. stdlib.h
4. iostream.h

189. What is the highest legal index for the following array? int arr[4]

1. 4
2. 3
3. 2
4. 1

190. Word processor is a type of a/an_____.

1. operating system
2. application software
3. device driver
4. utility software

191. Identify the correct option which is used for calling the function float area (int).

1. area(&num);
2. area(num);
3. area(int num);
4. area(*num);

192. The_____statement allows us to select from multiple choices based on a set of fixed values for a given expression.

1. switch
2. break
3. continue

Remember Own Prayer

4. goto

193. C is widely known as development language of _____operating system.

1. Windows
2. Unix
3. Mac OS
4. Linux

194. To convert the value of one type into another type using built-in functions, we include_____header file.

1. conio.h
2. stdlib.h
3. iostream.h
4. string.h

195. The keyword_____is used to get some value back from a function.

1. return
2. break
3. continue
4. goto

196. The function seekg() takes_____parameter(s).

1. 0
2. 1
3. 2
4. 3

197. The function write() takes_____as parameter(s).

1. String of pointer type
2. String of variable lengths, no. of bytes to be read and flags
3. Poimter array of characters and a delimiter
4. String and no. of bytes to be written

Remember Own Prayer

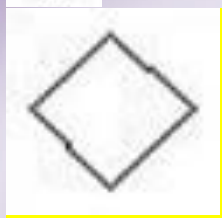
198. When the logical operator AND (&&) combines two expressions exp1 and exp2 then the result will be true only _____.

1. When both exp1 and exp2 are true
2. When both exp1 and exp2 are false
3. When exp1 is true and exp2 is false
4. When exp1 is false and exp2 is true

199. Syntax of a union is identical to_____.

1. structure
2. class
3. function
4. loop

200. Which one of the symbol is used to represent a decision in a flow chart?

1. 
2.  **correct**
3. 
4. 

201. In Flow Chart, flow of control is represented by _____.

1. Rectangle
2. Circle
3. Diamomd
4. Arrow

202. There can be_____'default' statement(s) in any switch structure.

Remember Your Own Prayer

1. 1
2. 2
3. 3
4. n

203. The condition in loop should be a(n)_____.

1. Constant Expression
2. Boolean Expression
3. Primary Expression
4. Arithmetic Expression

204. How many nested loops would be required to manipulate n-dimensional array?

1. n
2. n + 1
3. n - 1
4. 2n

205. Which of the following is not an example of int data type?

1. 0
2. -32
3. 65531
4. -4

206. We should use_____for clarity and to force the order of evaluation in an expression.

1. brackets []
2. parenthesis ()
3. curly braces {}
4. quotation marks " "

207. Which of the following is the starting index of an array in C++?

1. 0
2. 1

Remember Own Prayer

3. -1
4. 2

208. The statement x += y can be interpreted as_____.

1. Adding the value of the x to the value of the y and storing the result in x
2. Adding the value of the y to the value of x, store the result in y
3. Adding the value of the x to the value of x, store the result in x
4. Adding the value of the y to the value of y, store the result in x

209. Given a 2D array of integers, what would be the correct way of assigning the value 5 to the element at second row and third column?

1. m[2][3] = 5;
2. m[3][2] = 5;
3. m[1][2] = 5;
4. m[2][3] = '5';

210. Array is a data structure that stores_____.

1. Memory addresses
2. Variables
3. Data type
4. Data

211. A program statement that invokes a function is called _____.

1. function declaration
2. function call
3. function definition
4. function prototype

212. If a function has been declared but not defined before its function call then it is termed as_____.

Remember Own Prayer

1. logical error
2. syntax error
3. run time error
4. program time error

213. The compiler of C language is written in _____ language.

1. JAVA
2. BASIC
3. FORTRAN
4. C

214. Which one of the below functions is not included in ctype.h header file?

1. isdigit(int c)
2. isxdigit(int c)
3. tolower(int c)
4. getdigit(int c)

215. Which function is used to locate the first occurance of a character in any string?

1. strchr()
2. strstr()
3. strtok()
4. strlen()

216. To access the data members of structure,_____is used.

1. Logical operator
2. Dereference operator
3. Dot operator
4. Address operator

217. In the following nested For Loop, which loop will run most number of times?

```
for( inti = 0; i< 5; i++)
{
```

Remember Own Prayer

```
for(int k = 0; k < 5; k++)
{
. . . . .
}
}
```

1. Outer loop
2. Inner loop
3. Both loops run equal number of times
4. Depends upon the statements in the inner loop's body

218. Structure use_____allocation.

1. Queue
2. Heap
3. Cache
4. Stack

219. _____function give the position of the next character to be read from that file.

1. tellp()
2. tellg()
3. seekg()
4. seekp()

220. What will be the size of the following character array?
char name[] = "Adeel";

1. 5
2. 6
3. 4
4. 7

221. Function prototype is written,

1. Within main function
2. After the return statement in main
3. Before the return statement in main
4. Before call of that function

Remember Own Prayer

222. Which one of the following languages has been used to write the compiler of "C" language?

1. Java
2. Fortran
3. Basic
4. C

223. A hierarchy of classes which are used to deal with console and disk files are called_____.

1. Stream classes
2. Simple classes
3. Binary classes
4. IO classes

224. _____stops execution at the line that contains error(s) in the code.

1. Compiler
2. Debugger
3. Interpreter
4. Linker

225. C++ is a_____language.

1. High level
2. Low level
3. Machine
4. Assembly language

226. How many elements are stored in the following?
int matrix [4][5];

1. 9
2. 20
3. 25
4. 10

227. _____is a substitute of multiple if statement.

Remember Own Prayer

1. if. . .elseif statement
2. Continue statement
3. Break statement
4. Default statement

228. if
int sum = 54;
Then the value of the following statement is
sum = sum - 3;

1. 52
2. 50
3. 51
4. 57

229. What will be the correct syntax for declaration of the following statement?
"ptr is a constant pointer to an integer"

1. const * int myptr;
2. const int *myptr;
3. int const *ptr;
4. int *const ptr;

230. _____operator is used to pass the address of a variable in call by reference method.

1. %
2. +
3. @
4. &

231. _____data type can operate on modulus operator.

1. int
2. float
3. char
4. double

232. Whenever some number is added in an array name, it will jump as many_____as the added number.

Remember Own Prayer

1. rows
2. value
3. column
4. bytes

233. Suppose that an integer type pointer contains a memory address 0x22f220. What will be the new memory address if we increment this pointer by one?

1. 0x22f221
2. 0x22f222
3. 0x22f223
4. 0x22f224

234. _____is the pointer which determines the position in a file from where the next read operation occurs.

1. put
2. seek
3. get
4. tell

Remember Own Prayer

To access the data members of structure_____is used.

► **dot operator (.)**

► * operator

► à operator

► None of given.

Question No: 2 (M - 1) .

What does 5 ^ 6 , evaluate to in decimal where '^' is Exclusive OR operator?

► 1

► 2

► **3**

► 4

Question No: 3 (M - 1) .

If constructor contains a return statement in its body then compiler will give _____

► **No error**

► Syntax error

► Logical error

► Run time error

Question No: 4 (M - 1) .

We can use New keyword inside of Class Constructor.

► True

► **False**

Question No: 5 (M - 1) .

When an operator function is define as member function for a Unary operator then the number of argument it take is/are,

► Zero

► **One**

► Two

► N arguments

The declarator of Plus (+) member operator function is

► **Class-Name operator + (Class-Name rhs)**
► operator Class-Name + ( )
► operator Class-Name + (rhs)
► Class-Name operator + ( )

Question No: 7 (M - 1) .

Friend function of a class is_____.
► Member function
► **Non-member function**
► Private function
► Public function

Question No: 8 (M - 1) .

We can also create an array of user define data type.

► **True**

► False

Question No: 9 (M - 1) .

What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects

► Only destructor is called for objects
► **Memory is deallocated first before calling destructor**
► Destructor is called first before deallocating memory

Question No: 10 (M - 1) .

Deleting an array of objects without specifying [] brackets may lead to memory leak

► **True**

► False

Question No: 11 (M - 1) .

Which of the following data type(s) can operate on modulus operator '%'?

► float, int

► float, double

► **int**

► char

Question No: 12 (M - 1) .

Array is passed by value to a function by default.

► True

► **False**

Question No: 13 (M - 1) .

With template function, the compiler automatically detects the passed data and generates a new copy of function using passed data.

► **True**
► False

Question No: 14 (M - 1) .

What will be the correct syntax to initialize all elements of two-dimensional array to value 0?

► **int arr[2][3] = {0,0} ;**
► int arr[2][3] = {{0},{0}} ;
► int arr[2][3] = {0},{0} ;
► int arr[2][3] = {0} ;

Question No: 15 (M - 1) .

When an operator function is define as member function then operand on the left side of operator must be an object.

► **True**

► False

Question No: 16 (M - 1) .

break statement can be used outside a loop or switch statement.

► **True**

► False

Question No: 17 (M - 1) .

The keyword_____is used to return some value from a function.

► **return**

►break

► continue

► goto

Question No: 18 (M - 1) .

Every data member or function inside the structure is____by default whereas everything declared inside a class is_____by default.

► private, public

► **public, private**

► private, protected

► public, protected

Question No: 19 (M - 1) .

Which of the following is true for the C++ statement given below?

int &ref = val ;

► It creates a synonym for variable 'val'

► It creates an alias for variable 'val'

► It's a new name for variable 'val'

► **All of the given options**

Question No: 20 (M - 1) .

If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?

obj3 = obj1 + obj2 ;

► obj2 will be passed as an argument to+ operator whereas obj2 will drive the + operator

► obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator

► **Both objects (obj1, obj2) will be passed as arguments to the + operator**

► Any of the objects (obj1, obj2) can drive the + operator

Question No: 21 (M - 1) .

Unary operator implemented as member function takes_____arguments whereas non-member function takes_____arguments.

►One, zero

► **Zero, one**

► One, two

► Two, one

Question No: 22 (M - 1) .

The input/output streams cin and cout are_____therefore have_____.

► Structures, function

► Objects, member functions

► Functions, objects

► **None of the given options**

Question No: 23 (M - 1) .

If a symbolic constant has been defined, it will be an error to define it again.

► **True**

► False

Question No: 24 (M - 1) .

Every class contains_____.

► **Constructor**

► Destructor

► Both a constructor and a destructor

► None of the given options

Question No: 25 (M - 1) .

new and delete keywords are_____in C++ language.

► Built-in- Function

► Operators

► **Memory Allocation Function**

► None of the given options

Question No: 26 (M - 1) .

Consider the following code segment.

```
class M {

public:

  M &operator+(const M &);

...

};
```

p + q //code of line implies that p.operator+(q)

...

Let assume if p and q are class objects then function is implemented as _____

► **Member function**

►Non-member function

► Friend function

►None of the given options

Question No: 27 (M - 1) .

Assignment operator is ---------- associative.

► **right**

► left

► binary

►unary

Assignment operators in imperative programming languages are usually defined to be right-associative.

(Refrenced by: **asad ali (asad_call2002@yahoo.com)**

Question No: 28 (M - 1) .

Static variable which is defined in a function is initialized_____.

► **Only once during its life time**

► Every time the function call

► Compile time of the program

► None of the above

www.vuzs.info

Question No: 29 (M - 1) .

Wecan not define a variable of user-defined data type in the class.

► True

► **False**

Question No: 30 (M - 1) .

A constructor that will create a new object with a full copy of the other object, is copy is known as _____

► **deep copy**

► shallow copy

► constructor copy

► none of the options

Remember Own Prayer

http://vustudents.ning.com

Quiz Start Time: 09:34 PM Time Left 82 sec(s)

Question # 1 of 10 (Start time: 09:34:54 PM) Total Marks: 1
While developing a program; should we think about the user interface? //handouts main reusability hay..us ki yahan option hi nahi hay
Select correct option:

Yes

No

Quiz Start Time: 09:34 PM Time Left 75 sec(s)

Question # 2 of 10 (Start time: 09:35:20 PM) Total Marks: 1
A character is stored in the memory in _____
Select correct option:

byte

integer

string

None of the given

Quiz Start Time: 09:34 PM

Question # 3 of 10 (Start time: 09:36:21 PM) Total Marks: 1
These functions seekg() and seekp() requires an argument of type_____to let them how many bytes to move forward or backward.
Select correct option:

int

short

long

double

Quiz Start Time: 09:34 PM Time Left 85 sec(s)

Question # 4 of 10 (Start time: 09:37:30 PM) Total Marks: 1
dereferencing operator is represented by _____
Select correct option:

*

+

–

None of the given

Quiz Start Time: 09:34 PM Time Left 86 sec(s)

Question # 5 of 10 (Start time: 09:38:55 PM) Total Marks: 1

_____ transfers the executable code from main memory to hard disk.

Select correct option:

Loader

Debugger

Compiler

Linker

Quiz Start Time: 09:34 PM Time Left 85 sec(s)

Question # 6 of 10 (Start time: 09:40:21 PM) Total Marks: 1

When the logical operator OR (||) combine two expressions exp1 and exp2 then the result will be false only, //bitwise “or” applies here….

Select correct option:

When both exp1 and exp2 are true

When both exp1 and exp2 are false

When exp1 is true and exp2 is false

When exp1 is false and exp2 is true

Quiz Start Time: 09:34 PM Time Left 79 sec(s)

Question # 7 of 10 (Start time: 09:41:43 PM) Total Marks: 1
suppose we have int y[10]; To access the 4th element of the array we write_____
Select correct option:

y[4];

y[3];

y[2];

none of given

Quiz Start Time: 09:34 PM Time Left 82 sec(s)

Question # 8 of 10 (Start time: 09:43:10 PM) Total Marks: 1
we have opened a file stream myfile for reading (getting), myfile.tellg () gives us the current get position of the file pointer. It returns a whole number of type_____
Select correct option:

long

int

short

double

Quiz Start Time: 09:34 PM Time Left 80 sec(s)

Question # 9 of 10 (Start time: 09:44:42 PM) Total Marks: 1
ofstream is used for_____
Select correct option:

Input file stream

Output file stream

Input and output file stream

All of the given

Quiz Start Time: 09:34 PM Time Left 80 sec(s)
http://vustudents.ning.com

Question # 10 of 10 (Start time: 09:46:11 PM) Total Marks: 1
A structure is a collection of_____under a single name.
Select correct option:

values

variables

data

None of the given

Question No: 1 (Marks: 1) - Please choose one

In C/C++ the string constant is enclosed .....**corrected r red one**

► In curly braces
► In small braces
► In single quotes
► **In double quotes**

Question No: 2 (Marks: 1) - Please choose one

The size of int data type is

► 1 bytes
► 2 bytes
► 3 bytes
► 4 bytes

Question No: 3 (Marks: 1) - Please choose one

In Flow chart process is represented by

► Rectangle
► Arrow symbol
► Oval
► Circle

Question No: 4 (Marks: 1) - Please choose one

If the break statement is missed in switch statement then,**// a break statement with no label always completes abruptly, the reason being a break with no label. If no switch, while, do, or for statement in the immediately enclosing method, constructor or initializer encloses the break statement, a compile-time error occurs.**

► **The compiler will give error**
► This may cause a logical error
► No effect on program
► Program stops its execution

Question No: 5 (Marks: 1) - Please choose one

When we are using const keyword with a variable x then initializing it at the time of declaration is,

► **Must**
► Optional
► Not necessary
► A syntax error

Question No: 6 (Marks: 1) - Please choose one

Which of the following is the correct way to assign an integer value 5 to element of a matrix say 'm' at second row and third column?

► m[2][3] = 5 ;

► m[3][2] = 5 ;

► m[1][2] = 5 ;
► m[2][3] = '5';

Question No: 7 (Marks: 1) - Please choose one

How many total elements must be in two-dimensional array of 3 rows and 2 columns?

► 4
► 5
► 6
► 7

Question No: 8 (Marks: 1) - Please choose one

Which of the following is the correct statement for the following declaration?
const int *ptr.

► ptr is a constant pointer

► ptr is constant integer pointer

► ptr is a constant pointer to int

► ptr is a pointer to const int

Question No: 9 (Marks: 1) - Please choose one

Consider the following code segment. What will be the output of this code segment?As arrays starts from 0 index……at 1st position is 3……n at position 4th is 5……5-3=2

```
int arr[6] = {2, 3, 7, 4, 5, 6} ;
int *ptr1 =&arr[1] ;
int *ptr2 = &arr[4] ;
cout << (ptr2-ptr1) ;
```

► 3

► 9
► 12
► **2**

Question No: 10 (Marks: 1) - Please choose one

What will be the correct syntax to assign an array named *arr* of 5 elements to a pointer *ptr*?

□□□□□□□►□*ptr = arr ;

□□□□□□□►□ptr = arr ;

□□□□□□□►□***ptr = arr[5] ;**

□□□□□□□►□ptr = arr[5] ;

Question No: 11 (Marks: 1) - Please choose one

C is a/an_____language
□□□□□□□►□low level
□□□□□□□►□object based
□□□□□□□►□**object oriented**
□□□□□□□►□function oriented

Question No: 13 (Marks: 1) - Please choose one

The variables having a name, type and size are just like empty boxes.
► □False
□□□□□□□►□True

Question No: 14 (Marks: 1) - Please choose one

What's wrong with this for loop?
for (int k = 2, k <=12, k++)

► the increment should always be ++k
► the variable must always be the letter i when using a for loop
► there should be a semicolon at the end of the statement
► the commas should be semicolons

Question No: 15 (Marks: 1) - Please choose one

Most efficient method of dealing with structure variables is to define the structure globally

□□□□□□□►□True
□□□□□□□ ►□False

Question No: 16 (Marks: 1) - Please choose one

Syntax of union is identical to _____

□□□□□□□▶□Structure

□□□□□□□▶□Class

□□□□□□□▶□Function

□□□□□□□▶□None of the given options

FINALTERM EXAMINATION

Fall 2008

CS201- Introduction to Programming

Time: 120 min

Marks: 75

Question No: 1 (Marks: 1) - Please choose one

There are mainly ----------types of software.//

- ▶ Two
- ▶ Three
- ▶ Four
- ▶ Five

Question No: 2 (Marks: 1) - Please choose one

seekg() and write() are functionally_____.//

Write() Writes a specified number of bytes from a memory location to the file.
seekg() Moves the file position indicator to a specific location in the file.
The Position property of TFileStream simplifies seeking in a file and performs the same function as the ifstream methods tellg() and seekg(). You can read Position to determine the current file position, or you can write to Position to move the file position.

- ▶ Different
- ▶ Identical
- ▶ **Two names of same function**
- ▶ None of the above

Question No: 3 (Marks: 1) - Please choose one

When a pointer is incremented, it actually jumps the number of memory addresses

- ▶ According to data type
- ▶ 1 byte exactly
- ▶ 1 bit exactly
- ▶ A pointer variable can not be incremented

Question No: 4 (Marks: 1) - Please choose one

setw is a parameterized manipulator.

- ► True
- ► False

Question No: 5 (Marks: 1) - Please choose one

eof( ), bad( ), good( ), clear( ) all are manipulators.

- ► True
- ► False

Question No: 6 (Marks: 1) - Please choose one

In functions that return reference, use_____variables.

- ► Local
- ► Global
- ► Global or static
- ► None of the given option

Question No: 7 (Marks: 1) - Please choose one

The declarator of Plus (+) member operator function is
//not confirmed

- ► Class-Name operator + (Class-Name rhs)
- ► operator Class-Name + ( )
- ► operator Class-Name + (rhs)
- ► Class-Name operator + ( )

Question No: 8 (Marks: 1) - Please choose one

The compiler does not provide a copy constructor if we do not provide it…….

//Normally it provides but in some cases of classes it is good practice to provide user define copy constructor…

- ► True
- ► False

Question No: 9 (Marks: 1) - Please choose one

What is the functionality of the following syntax to delete an array of 5 objects named ***arr*** allocated using new operator?
delete arr ;

- ► Deletes all the objects of array
- ► Deletes one object of array
- ► Do not delete any object
- ► Results into syntax error

Question No: 10 (Marks: 1) - Please choose one

What is the sequence of event(s) when allocating memory using new operator?

- ► Only block of memory is allocated for objects
- ► Only constructor is called for objects
- ► Memory is allocated first before calling constructor
- ► Constructor is called first before allocating memory

Question No: 11 (Marks: 1) - Please choose one
What is the sequence of event(s) when deallocating memory using delete operator?

- ► Only block of memory is deallocated for objects
- ► Only destructor is called for objects
- ► Memory is deallocated first before calling destructor
- ► Destructor is called first before deallocating memory

Question No: 12 (Marks: 1) - Please choose one

new and **delete** operators cannot be overloaded as member functions.

- ► True
- ► **False**

Question No: 13 (Marks: 1) - Please choose one

The operator function of << and >> operators are always the member function of a class.

- ► True
- ► False

Question No: 14 (Marks: 1) - Please choose one

A template function must have at least ---------- generic data type

- Zero
- One
- Two
- Three

Question No: 15 (Marks: 1) - Please choose one

If we do not mention any *return_value_type* with a function, it will return an _____ value.

- int
- void
- double
- float

Question No: 16 (Marks: 1) - Please choose one

Suppose a program contains an array declared as **int arr[100];** what will be the size of array?

- 0
- 99
- 100
- 101

Question No: 17 (Marks: 1) - Please choose one

The name of an array represents address of first location of array element.

- True
- False

Question No: 18 (Marks: 1) - Please choose one

Reusing the variables in program helps to save the memory

- True
- False

Question No: 19 (Marks: 1) - Please choose one

Which of the following option is true about new operator to dynamically allocate memory to an object?

// **new** is an operator that allows dynamic memory allocation on the heap. Except for a form called the "placement new", new attempts to allocate enough memory on the heap for the new data and, if successful, returns the address to the newly allocated memory. However if new can not allocate memory on the heap it will throw an exception of type std::bad_alloc. This removes the need to explicitly check the result of an allocation.

- The new operator determines the size of an object
- Allocates memory to object and returns pointer of valid type
- Creates an object and calls the constructor to initialize the object
- **All of the given options**

Question No: 20 (Marks: 1) - Please choose one
new and delete are_____whereas malloc and free are_____.

- Functions, operators
- Classes, operators
- Operators, functions
- Operators, classes

Question No: 21 (Marks: 1) - Please choose one

Like member functions,_____can also access the private data members of a class.

- Non-member functions
- Friend functions
- Any function outside class
- None of the given options

Question No: 22 (Marks: 1) - Please choose one

Which of the following statement is best regarding declaration of friend function?

- Friend function must be declared after public keyword.
- Friend function must be declared after private keyword.
- Friend function must be declared at the top within class definition.
- It can be declared anywhere in class as these are not affected by the public and private keywords.

Question No: 23 (Marks: 1) - Please choose one
The operator function overloaded for an Assignment operator (=) must be

- Non-member function of class
- **Member function of class**
- Friend function of class
- None of the given options

Question No: 24 (Marks: 1) - Please choose one
For non-member operator function, object on left side of the operator may be

- Object of operator class
- Object of different class
- Built-in data type
- All of the given options

Question No: 25 (Marks: 1) - Please choose one
The operator function will be implemented as_____, if obj1 drive the - operator whereas obj2 is passed as arguments to - operator in the statement given below.
obj3 = obj1 - obj2;

- Member function
- Non-member function
- **Friend function**
- None of the given options

Question No: 26 (Marks: 1) - Please choose one
Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

- Class-name operator +() ;
- Class-name operator +(int) ;
- Class-name operator ++() ;
- Class-name operator ++(int) ;

Question No: 27 (Marks: 1) - Please choose one

The static data members of a class are initialized _____

- at file scope
- **within class definition**
- within member function
- within main function

Question No: 28 (Marks: 1) - Please choose one

Class is a user defined_____.

- data type
- memory referee
- value
- none of the given options.

Question No: 29 (Marks: 1) - Please choose one
We can also define a user-defines manipulators.

- **True**
- False

Question No: 30 (Marks: 1) - Please choose one

Automatic variable are created on_____.

- Heap
- Free store
- static storage
- stack

Current

Question No: 1 (Marks: 1) - Please choose one

Compiler is a

- System software
- Application Software
- Driver
- Editor

Question No: 2 (Marks: 1) - Please choose one

Operating System is

- An application software
- System Software
- Computer Language
- Interpreter

Question No: 3 (Marks: 1) - Please choose one

Which one is the correct syntax for defining an identifier **PI** with preprocessor directive?

- #define PI 3.1415926;
- #define PI 3.1415926
- #define PI = 3.1415926 ;
- #define PI = 3.1415926

Question No: 4 (Marks: 1) - Please choose one

Character strings are arrays of characters that can store

- Only one character
- Limited number of characters
- **Number of characters specified.**
- None of the given option

Question No: 5 (Marks: 1) - Please choose one
Which of the following is NOT a preprocessor directive?

- #error
- #define
- #line
- **#ndefine**

Question No: 6 (Marks: 1) - Please choose one

Which kind of functions can access private member variables of a class?

- Friend functions of the class
- Private member functions of the class
- Public member functions of the class
- Friend, private and public functions

Question No: 7 (Marks: 1) - Please choose one
Let suppose int a, b, c, d, e;, a = b = c = d = e = 42;
This can be interpreted by the complier as:// **assignment is right-associative.**

- **a = (b = (c = (d = (e = 42))));**
- (a = b = (c = (d = (e = 42))));
- a = b = (c = (d = (e = 42)));
- (a = b) = (c = d) = (e = 42);

Question No: 8 (Marks: 1) - Please choose one

Friend function of a class is_____. http://vustudents.ning.com

- **Member function**
- Non-member function
- Private function
- Public function

Question No: 9 (Marks: 1) - Please choose one

Function implementation of friend function must be defined outside the class.

- **True**
- False

Question No: 10 (Marks: 1) - Please choose one
When we define an array of objects then,

- ► Constructor will be called once for whole array
- ► **Constructor will be called for each object of the array**
- ► Constructor will never call
- ► Depends on the size of array

Question No: 11 (Marks: 1) - Please choose one

The stream objects **cin** and **cout** are included in which header file?

- ► **iostream.h**
- ► fstream.h
- ► istream.h
- ► ostream.h

Question No: 12 (Marks: 1) - Please choose one

What is the sequence of event(s) when allocating memory using new operator?

- ► Only block of memory is allocated for objects
- ► Only constructor is called for objects
- ► **Memory is allocated first before calling constructor**
- ► Constructor is called first before allocating memory

Question No: 13 (Marks: 1) - Please choose one
Missing semicolon ';' at the end of C++ statement is

- ► Logical error
- ► **Syntax error**
- ► Runtime error
- ► None of the given options

Question No: 14 (Marks: 1) - Please choose one

An array stores the numbers into consecutive memory locations.

- ► **True**
- ► False

Question No: 15 (Marks: 1) - Please choose one

The template functions do NOT promote the code reuse

- ► True
- ► **False**

Question No: 16 (Marks: 1) - Please choose one

What will be the correct syntax for initialization of pointer ***ptr*** of type int with variable ***x***?

- ► int ptr = &x ;
- ► int ptr = x ;
- ► **int *ptr = &x ;**
- ► int ptr* = &x ;

Question No: 17 (Marks: 1) - Please choose one

Which of the following function calling mechanism is true for the function prototype given below?
float func(float &);

- ► **Call by value**
- ► Call by reference using pointer
- ► Call by reference using reference variable
- ► None of the given options

Question No: 18 (Marks: 1) - Please choose one

If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?
obj3 = obj1 + obj2 ;

- ► **obj2 will be passed as an argument to + operator whereas obj2 will drive the + operator**
- ► obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator
- ► Both objects (obj1, obj2) will be passed as arguments to the + operator
- ► Any of the objects (obj1, obj2) can drive the + operator

Question No: 19 (Marks: 1) - Please choose one
Which of the following object(s) will call the member operator function within the statement given below?
obj1=obj2+obj3;

- Object obj1
- Object obj2
- **Object obj3**
- Any of the object

Question No: 20 (Marks: 1) - Please choose one

For cin, the source is normally a_____and destination can be_____.

- File, native data type
- Disk, user-define type
- **Keyboard, variable**
- File, user-define type

Question No: 21 (Marks: 1) - Please choose one

The static data members of a class will be _____

- shared by objects
- created for each object
- initialized within class
- **initialized within main function**

Question No: 22 (Marks: 1) - Please choose one

Which of the following function cannot be overloaded?

- Member functions
- Utility functions
- Constructor
- **Destructor**

Question No: 23 (Marks: 1) - Please choose one

The region of memory, available for allocation at run time in C language is called _____ memory whereas in C++ language is called as_____.

- Heap, Stack
- Stack, Free Store
- Heap, Free Store
- **None of above**

Question No: 24 (Marks: 1) - Please choose one

A variable which is defined inside a function is called

- **Automatic variable**
- Global variable
- Functional variable
- None of the given option

Question No: 25 (Marks: 1) - Please choose one

The default copy constructor provides member wise assignment.

- **True**
- False

Question No: 26 (Marks: 1) - Please choose one

The code is written to_____the program.

- **implement**
- design
- analysis
- none of the given options.

Question No: 27 (Marks: 1) - Please choose one

When a call to a user-defined function finishes, the variable defined inside the function is still in existence.

- True
- **False**

Question No: 28 (Marks: 1) - Please choose one

To avoid dangling reference, always return the reference of a local variable from a function.

- **True**
- False

Question No: 29 (Marks: 1) - Please choose one

Analysis is the----------step in designing a program.

- Last
- Middle
- Post Design

► **First**

Question No: 30 (Marks: 1) - Please choose one

"delete" operator is used to return memory to free store, which is allocated by the "**new**" operator.

► **True**
► False

Question No: 1 (Marks: 1) - Please choose one

In C/C++ if we define an array of size eight (8) i.e. int Arr [8]; then the last element of this array will be stored at,

► Arr[0]
► Arr[8]
► Arr[7]
► Arr[-1]

Question No: 2 (Marks: 1) - Please choose one

When an array is passed to a function then default way of passing this array is,

► By data
► By reference
► By value
► By data type

Question No: 3 (Marks: 1) - Please choose one

Array is a data structure which store

► **Memory addresses**
► Variables
► Data Type
► Data

Question No: 4 (Marks: 1) - Please choose one

We can also create an array of user define data type.

► True
► False

Question No: 5 (Marks: 1) - Please choose one

When we define an array of objects then,

- ► Destructor will call once for whole array
- ► Destructor will call for each object of the array
- ► Destructor will never call
- ► Depends on the size of array

Question No: 6 (Marks: 1) - Please choose one

What is the sequence of event(s) when allocating memory using new operator?

- ► Only block of memory is allocated for objects
- ► Only constructor is called for objects
- ► Memory is allocated first before calling constructor
- ► Constructor is called first before allocating memory

Question No: 7 (Marks: 1) - Please choose one

We can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally.

- ► True
- ► False

Question No: 8 (Marks: 1) - Please choose one

The second parameter of operator functions for << and >> are objects of the class for which we are overloading these operators.

- ► **True**
- ► False

Question No: 9 (Marks: 1) - Please choose one

Which of the following is correct way to initialize a variable x of int type with value 10?

- ► int x ; x = 10 ;
- ► **int x = 10 ;**
- ► int x, x = 10;
- ► x = 10 ;

Question No: 10 (Marks: 1) - Please choose one

Default mechanism of function calling in case of array is_____and in case of variable is ___. http://vustudents.ning.com

- ► Call by value, call by reference
- ► Call by referene, call by reference
- ► **Call by reference, call by value**
- ► Call by value, call by value

Question No: 11 (Marks: 1) - Please choose one

What does STL stand for?

- ► Source template library
- ► **Standard template library**
- ► Stream template library
- ► Standard temporary library

Question No: 12 (Marks: 1) - Please choose one

Skill(s) that is/are needed by programmers_____.

- ► Paying attention to detail
- ► Think about the reusability
- ► Think about user interface
- ► **All of the given options**

Question No: 13 (Marks: 1) - Please choose one

For which array, the size of the array should be one more than the number of elements in an array?

- ► int
- ► double
- ► float
- ► **char**

Question No: 14 (Marks: 1) - Please choose one

new and delete are_____whereas malloc and free are_____.

- ► Functions, operators
- ► Classes, operators
- ► **Operators, functions**
- ► Operators, classes

Question No: 15 (Marks: 1) - Please choose one

Friend functions are_____of a class.

- ► Member functions
- ► Public member functions
- ► Private member functions
- ► **Non-member functions**

Question No: 16 (Marks: 1) - Please choose one

The prototype of friend functions must be written____the class and its definition must be written ____

- ► inside, inside the class
- ► inside, outside the class
- ► **outside, inside the class**
- ► outside, outside the class

Question No: 17 (Marks: 1) - Please choose one

If overloaded plus operator is implemented as non-member function then which of the following statement will be true for the statement given below?
obj3 = obj1 + obj2 ;

► **obj2 will be passed as an argument to + operator whereas obj2 will drive the + operator**

► obj1 will drive the + operator whereas obj2 will be passed as an argument to + operator

► Both objects (obj1, obj2) will be passed as arguments to the + operator

► Any of the objects (obj1, obj2) can drive the + operator

Question No: 18 (Marks: 1) - Please choose one

Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?

- Class-name operator +() ;
- Class-name operator +(int) ;
- Class-name operator ++() ;
- **Class-name operator ++(int) ;**

Question No: 19 (Marks: 1) - Please choose one

For cin, the source is normally a_____and destination can be_____.

- File, native data type
- Disk, user-define type
- **Keyboard, variable**
- File, user-define type

Question No: 20 (Marks: 1) - Please choose one

We can also do conditional compilation with preprocessor directives.

- **True**
- False

Question No: 21 (Marks: 1) - Please choose one

The programs, in which we allocate static memory, run essentially on _____

- Heap
- System Cache
- None of the given options
- **Stack**

Question No: 22 (Marks: 1) - Please choose one

The default value of a parameter can be provided inside the _____

- function prototype
- **function definition**
- both function prototype or function definition
- none of the given options.

Question No: 23 (Marks: 1) - Please choose one

While calling function, the arguments are assigned to the parameters from_____.

- ▶ left to right.
- ▶ right to left
- ▶ **no specific order is followed**
- ▶ none of the given options.

Question No: 24 (Marks: 1) - Please choose one

When an operator function is defined as member function for a binary Plus (+) operator then the number of argument it take is/are.

- ▶ Zero
- ▶ One
- ▶ Two
- ▶ **N arguments**

Question No: 25 (Marks: 1) - Please choose one

With user-defined data type variables (Objects), self assignment can produce _____.

- ▶ Syntax error
- ▶ **Logical error**
- ▶ Link error
- ▶ Non of the given options

Question No: 26 (Marks: 1) - Please choose one

Assignment operator is used to initialize a newly declared object from existing object.

- ▶ **True**
- ▶ False

Question No: 27 (Marks: 1) - Please choose one

When an object of a class is defined inside an other class then,

- ▶ Constructor of enclosing class will be called first
- ▶ **Constructor of inner object will be called first**
- ▶ Constructor and Destructor will be called simultaneously
- ▶ None of the given options

Question No: 28 (Marks: 1) - Please choose one

In the member initializer list, the data members are initialized,

- From left to right
- From right to left
- **In the order in which they are defined within class**
- None of the given options

Question No: 29 (Marks: 1) - Please choose one

new operator allocates memory from free store and return_____.

- **A pointer**
- A reference
- An integer
- A float

Question No: 30 (Marks: 1) - Please choose one

"new" and "delete" keywords are_____in C++ language.

- Built-in- Function
- Operators
- **Memory Allocation Function**
- None of the given options

Cs201 quiz today made by Abdullah

1. which software testing refers to tests that do not rely on knowledge of software's interior composition

(white-box testing)

2. The language understood by a computer without translation is known as:

(Machine language)

3. which one is a unary operator?

(NOT)

4. In the case of highly standardized language, compiler designers often provide feature ,sometimes called

(Language extension)

5. Under_____paradigm a program is viewed as an entity that accept inputs and produces outputs

(functional)

6. in c++,symbolic representation of "Assignment" operator is _____

(=)

7. A translator which takes assembly language program as input & produce machine language code as output is know as:

(Assembler)

8. Which of the following statement is used in c++.

(cout<<)

9. We used_____to reduce unnecessary dependencies or effects on other module

(information hiding)

10. The memory allocation functions return a chunk of memory with a pointer of type_____

(void)

Abdullah

11. default constructor takes_____

(no parameters)

12. In a class we have_____constructor(s).

(many)

13. which one of the following in mandatory preprocessor directive for c++?

(#include <iostream>)

14. Header files provide_____so the program running on the operating system can run without an error on the other system

(accessibility)

15. With the use of dynamic allocation of memory,the system resources can be_____

(Used efficiently)

16. _____for parameters is also done for inline functions.

(automatic type checking)

17. We should not use such variable names that are starting with because in c++. There are lots of internal constants and symbolic names that start with it

(double underscore)

18. Which of the following code segment represents a left a left shift operator?

(Cout<<1)

19. c++ was developed by _____

(bejama stoustrup)

20. special name which is substituted in code by its definition and as a result we get an expanded code is called

(macro)

21. a=a+1; can be written as_____

(a + = 1;)

22. The compilar generates_____automatically

(constuctors)

23. The data numbers of the class are initialized_____

(At runtime)

24. _____for parameters is also done for infine functions

(automatic type checking)

25. AN instance of a class is called_____

(object)

26. with the use of dynamic allocation of memory. The system resources can be _____

(used efficiently)

27. When we include header file in the angle brackets the compiler search for it in_____

(specific directory)

28. _____is a special type of pointer we have to castit before we use it.

(void)

29. Multiplicative, additive and assignment operators are_____operators.

(binary)

30. a =a+1; can be written as_____

(a + = 1;)

31. The compiler generates_____automatically

(constructors)

32. The data members of the class are initialized_____

(at runtime)

33. _____for parameters is also done for inline functions

(automatic type checking)

34. AN instance of class of a class is called_____

(object)

35. with the use of dynamic allocation of memory ,
the system resources can be_____

(used efficiently)

36. for converting a void type pointer ptr, the correct syntax is:

(Int* ptr)

37. header files provide___so the program running on one operating system can run without an error on the other system

(reliability)

38. a pointer with value null is defined in the header files___and____

(stdlib.h,stdef.h)

39. molic function returns____to the____from the available memory

(void pointer,starting of chunk of memory...from the array)

40. constructor is a special function, called whenever we_____

(Instantiate an object of a class)

41. we should not use such variable names that are starting with_____because in c++there are lots of internal constant....with it

(double underscope)

42. CONStructor is itself a_____of c++ and_____

(function can be overloaded)

43what is the other name of variable

(Identifier)

44. what will be the output of the following c++ code?

#include<iostream.h>

#define max 100

Main()

{#ifdef max

Cout<"hello world"

(error)

45. In shifting operations if zero is inserted at the left most bit,the negative number will become a___number

(positive)

46. The compilar generates____automatically

Abdullah

(constructors

47.A class is_____

(a member function)

48. bugs can occur due to_____

(uninitialized data)

49. the heap memory structure_____

(constantly change in size

50 how many bits form a byte

(8)

51.the function calloc takes two arguments. First argument is the_____ and the second argument is the_____

(require space in terms of number ,size of space)

52the main advantage of function overloading is that___

(the program become more redable)

53.A pointer with value null is defined in the header files_____and_____

(stdlib.h.....)

54. the_____is the increment operator in c++

(++)

55. what will be the output of the given code)

X=32; y=45;

(x=24 y=46)

56. the constructor constains_____

(no returntype)

57. multiplicative,additive and assignment operators are_____operators

(Binary)

58 for accessing data members we use____operator

(dot)

59. in a class we can have_____constructor

(Only one)

60. object code is machine code but it is not____and_____

(relocatable ,executable)

61. shifting the binary number Is similar to shifting the__number

(decimal)

62. The____is called automatically when an object destroys

(destructor)

63. an integer uses four bytes and the integer calculation occur in___bytes

(2byte)

64. which of the following code segment represents a left shift operator?

(cout<<1)

65. Raid stand for___

(redundant array of inexperience device)

66. windows operating system may itself takes memory from_____

(list)

67what is the storage space of double data type

(8)

68. dynamic memory allocation uses____Whereas static memory allocation uses_____

(Heap , stack)

69.which one is the correct example of a unary operator

(-)

70.which is the output of following code

(b correct answr)

71. array indexes start from_____

0

72.which one of the following is used to perform bit-wise exclusive or operation

(^)

73.getche() is a_____function and defined in_____header file

(build-in function,iostream.h)

74.In c++, a variable can be declared anywhere in the program, this will increase____

(Portability)

75the compilar use a name mangling technique to_____

Compile the function

76.macros are categorized into type(s)

Two

77constructor is a special function called whenever we____

Instantiate an object of a clas

Abdullah

Cs201 today quiz

1.Once the_____are created, they exist for the life time of the program

(static variable)

2.A class can be declared as a_____of other class.

(friend)

3.Reference variabales must_____

(be initialized after they are declared)

4.we can delete an array of objects without specifying [] brackets if a class is not doing dynamic memory allocation internally

(False)

5.operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the

(operands)

6.the friend keyword provides access_____

(in one direction only)

7.the friend functions are_____

(not member of a class)

8.temary operator is shown as_____

(?:)

9.Reference is not really an address it is_____

(a synonym)

10. Reference is a thing by which we can create_____of any data type

(synonym)

11. In c++ operators. Which of the following operator cannot

Abdullah cheater

19. to prevent dangling reference the functions returning reference should be used with_____

(static and global variables)

20. the only operator that the compiler overloads for user define data type by default is

(assignment(=) operator

21. if operator function is non-member function that object on left side of operator cannot be_____?

(object of operator class)

22. which of the following functionality can be achieved through overloading?

(new operators cannot be defined through operator overloading)

23. oveloaded member operator function is always called by_____

(class)

24 Overloaded member operator function is always called by _____

► **Class**
► Object
► Compiler
► Primitive data type

25 Loader loads the executable code from hard disk to main memory.

► **True**
► False

26 Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

► new int(10) ;
► **new int[10] ;**
► int new(10) ;
► int new[10];

27. The prototype of friend functions must be written_____the class and its definition must be written _____

► inside, inside the class
► **inside, outside the class**
► outside, inside the class
► outside, outside the class

28. Like member functions,_____can also access the private data members of a class.

► Non-member functions
► Friend functions
► Any function outside class
► None of the given options

29. To perform manipulation with input/output, we have to include_____header file.

► **iostream.h**
► stdlib.h
► iomanip.h
► fstream.h

30. The **endl** and **flush** are _____

► Functions
► Operators
► **Manipulators**
► Objects

31 If we want to use stream insertion and extraction operators with_____then we have to overload these operators.

► int, float, double
► objects of class
► int, float, object
► **int, char, float**

32. The static data members of a class can be accessed by _____

► only class
► only objects
► **both class and objects**
► none of given options

33. Classes defined inside other classes are called_____classes

► looped
► **nested**
► overloaded
► none of the given options.

34. Which value is returned by the destructor of a class?

► A pointer to the class.
► An object of the class.
► A status code determining whether the class was destructed correctly
► **Destructors do not return a value.**

35. Consider the following code segment
class M {

friend int operator!(const M &);

…

};

!s // code of line implies that operator!(s)
…
Let assume if s is an object of the class then function is implemented as _____

► **Member function**
► Non-member function
► Binary operator function
► None of the given options

36. If **text** is a pointer of class **String** then what is meant by the following statement?
text = new String [5];
► **Creates an array of 5 string objects statically**
► Creates an array of 5 string objects dynamically
► Creates an array of pointers to string
► Creates a string Object

37. Static variable which is defined in a function is initialized_____.
► **Only once during its life time**
► Every time the function call

► Compile time of the program
► None of the above

38. In functions that return reference, use_____variables.
► Local
► Global
► **Global or static**
► None of the given option

39. What is the functionality of the following syntax to delete an array of 5 objects named ***arr*** allocated using new operator?
delete arr ;
► Deletes all the objects of array
► Deletes one object of array
► Do not delete any object
► **Results into syntax error**

40. What is the sequence of event(s) when allocating memory using new operator?
► Only block of memory is allocated for objects
► Only constructor is called for objects
► Memory is allocated first before calling constructor
► **Constructor is called first before allocating memory**

41. What is the sequence of event(s) when deallocating memory using delete operator?

► Only block of memory is deallocated for objects
► Only destructor is called for objects
► Memory is deallocated first before calling destructor
► **Destructor is called first before deallocating memory**

42. **new** and **delete** operators cannot be overloaded as member functions.
► True
► **False**

43. The operator function of << and >> operators are always the member function of a class.
► True
► **False**

44. Which of the following option is true about new operator to dynamically allocate memory to an object?

► **The new operator determines the size of an object**
► Allocates memory to object and returns pointer of valid type
► Creates an object and calls the constructor to initialize the object
► All of the given options

45. new and delete are_____whereas malloc and free are_____.

► Functions, operators
► Classes, operators
► **Operators, functions**
► Operators, classes

46. Which of the following statement is best regarding declaration of friend function?

► Friend function must be declared after public keyword.
► Friend function must be declared after private keyword.
► Friend function must be declared at the top within class definition.
► **It can be declared anywhere in class as these are not affected by the public and private keywords.**

47. The operator function will be implemented as_____, if obj1 drive the – operator whereas obj2 is passed as arguments to – operator in the statement given below.
obj3 = obj1 – obj2;

► Member function
► **Non-member function**
► Friend function
► None of the given options

48. Which one of the following is the declaration of overloaded pre-increment operator implemented as member function?
► Class-name operator +() ;
► Class-name operator +(int) ;

► Class-name operator ++() ;
► **Class-name operator ++(int) ;**

49. Class is a user defined_____.

► **data type**
► memory referee
► value
► none of the given options.

50. The static data members of a class are initialized _____

► at file scope
► within class definition
► within member function
► **within main function**

51. There are mainly----------types of software

► **Two**
► Three
► Four
► Five

52. When x = 7; then the expression x%= 2; will calculate the value of x as,

► **1**
► 3
► 7
► 2

53. A pointer variable can be,

► Decremented only
► Incremented only
► Multiplied only
► **Both 1 and 2**

54. setprecision is a parameter less manipulator.

► True
►**False**

55. We can change a Unary operator to Binary operator through operator overloading.

►**False**
► True

56. **delete** operator is used to return memory to free store which is allocated by the **new** operator

► **True**
►False

57. When we do dynamic memory allocation in the constructor of a class, then it is necessary to provide a destructor.

► **True**
►False

58. What is the functionality of the following statement?

String str[5] = {String("Programming"), String("CS201")};
► Default constructor will call for all objects of array
► Parameterized constructor will call for all objects of array
► **Parameterized constructor will call for first 2 objects and default constructor for remaining objects**
► Default constructor will call for first 3 objects and Parameterized constructor for remaining objects vuzs

59. What is the sequence of event(s) when allocating memory using new operator?

► Only block of memory is allocated for objects
► Only constructor is called for objects
► **Memory is allocated first before calling constructor**
► Constructor is called first before allocating memory

60. Deleting an array of objects without specifying [] brackets may lead to memory leak

► **True**
►**False**

61. Which of the following data type will be assumed if no data type is specified with constant?

► short
► float
► **int**
► double

62. There is an array of characters having name 'course' that has to be initialized by string 'programming' which of the following is the correct way to do this,

i. course[] = {'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g'};
ii.course[] = 'programming' ;
iii. course[12] = "programming" ;
iv. course = "programming" ;
Choose the correct options.
► (i) and (ii) only
► (i) and (iv) only
► **(i) and (iii) only**
► (ii) and (iii) only

63. What will be the correct syntax of the following statement?

***ptr is a constant pointer to integer*.**
► const int *ptr ;
► const *int ptr ;
► int const *ptr ;
► **int *const ptr ;**

The keyword **const** for pointers can appear before the type, after the type, or in both places. The following are legal declarations:

```
const int * ptr1;       /* A pointer to a constant integer:
                           the value pointed to cannot be changed */
int * const ptr2;       /* A constant pointer to integer:
                           the integer can be changed, but ptr2
                           cannot point to anything else          */
const int * const ptr3; /* A constant pointer to a constant integer:
                           neither the value pointed to
                           nor the pointer itself can be changed    */
```

Declaring an object to be const means that the this pointer is a pointer to a const object. A const this pointer can by used only with const member functions vuzs.info

64. Overloaded member operator function is always called by _____

- ► Class
- ► Object
- ► **Compiler**
- ► Primitive data type

65. Loader loads the executable code from hard disk to main memory.

- ► **True**
- ► **False**

66. Which of the following is the correct C++ syntax to allocate space dynamically for an array of 10 int?

- ► new int(10) ;
- ► **new int[10] ;**
- ► int new(10) ;
- ► int new[10];

67. The prototype of friend functions must be written_____the class and its definition must be written _____

- ► inside, inside the class
- ► **inside, outside the class**
- ► outside, inside the class
- ► outside, outside the class

68. Like member functions,_____can also access the private data members of a class.

- ► Non-member functions
- ► **Friend functions**
- ► Any function outside class
- ► None of the given options

69. To perform manipulation with input/output, we have to include_____header file.

- ► iostream.h
- ► stdlib.h
- ► iomanip.h
- ► fstream.h

70. The **endl** and **flush** are _____

- ► Functions
- ► Operators
- ► **Manipulators**
- ► Objects

71. If we want to use stream insertion and extraction operators with_____then we have to overload these operators.

- ► int, float, double
- ► **objects of class**
- ► int, float, object
- ► int, char, float

72. If we want to use stream insertion and extraction operators with_____then we have to overload these operators.

- ► int, float, double
- ► **objects of class**
- ► int, float, object
- ► int, char, float

73. Classes defined inside other classes are called_____classes

- ► looped
- ► **nested**
- ► overloaded
- ► none of the given options.

74. Which value is returned by the destructor of a class?

- ► A pointer to the class.
- ► An object of the class.
- ► A status code determining whether the class was destructed correctly
- ► **Destructors do not return a value**

75. Consider the following code segment

```
class M {
 friend int operator!(const M &);
...
};
```

!s // code of line implies that operator!(s)

...

Let assume if s is an object of the class then function is implemented as _____

- ► Member function
- ► Non-member function
- ► Binary operator function
- ► None of the given options

None of the given options

76. When the compiler overloads the assignment (=) operator by default then _____

► **compiler does member wise assignment.**

► compiler does not allow default overload of assignment (=) operator
► member of the class are not assigned properly
► None of the given options

77. If **text** is a pointer of class **String** then what is meant by the following statement?

text = new String [5];
► Creates an array of 5 string objects statically
► **Creates an array of 5 string objects dynamically**
► Creates an array of pointers to string
► Creates a string Object

78. Static variable which is defined in a function is initialized_____.

► **Only once during its life time**
► Every time the function call
► Compile time of the program
► None of the above

79. The appropriate data type to store the number of rows and is_____.

► floatcolums of the matrix
► **int**
► char
► none of the given options.

80. Copy constructor becomes necessary while dealing with_____allocation in the class.

► **Dynamic memory**
► Static memory
► Both Dynamic and Static memory
► None of the given options

81.

Abdullah Ch

Lecture No. 23

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 17

Summary

- Pre-processor
- include directive
- define directive
- Other Preprocessor Directives
- Macros
- Example
- Tips

Preprocessor

Being a concise language, C needs something for its enhancement. So a preprocessor is used to enhance it. It comes with every C compiler. It makes some changes in the code before the compilation. The compiler gets the modified source code file. Normally we can't see what the preprocessor has included. We have so far been using *#include* preprocessor directive like *#include<iostream.h>*. What actually *#include* does? When we write *#include<somefile>*, this *somefile* is ordinary text file of C code. The line where we write the *#include* statement is replaced by the text of that file. We can't see that file included in our source code. However, when the compiler starts its work, it sees all the things in the file. Almost all of the preprocessor directives start with # sign. There are two ways to use *#include*. We have so far been including the file names enclosing the angle brackets i.e. *#include <somefile>*. This way of referring a file tells the compiler that this file exists in some particular folder (directory) and should be included from there. So we have included *iostream.h, stdlib.h, fstream.h, string.h* and some other files and used angle brackets for all of these files. These files are located in a specific directory. While using the *Dev-Cpp* compiler, you should have a look at the directory structure. Open the *Dev-Cpp* folder in the windows explorer, you will see many subfolders on the right side. One of these folders is '*include'*. On expansion of the folder '*include'*, you will see a lot of files in this directory. Usually the extension of these files is 'h'. Here 'h' stands for header files. Normally we add these files at the start of the program. Therefore these are known as header files. We can include files anywhere in the code but it needs to be logical and at the proper position.

© Copyright Virtual University of Pakistan

include directive

As you know, we have been using functions in the programs. If we have to refer a function (call a function) in our program, the prototype of function must be declared before its usage. The compiler should know the name of the function, the arguments it is expecting and the return type. The first parse of compilation will be successful. If we are using some library function, it will be included in our program at the time of linking. Library functions are available in the compiled form, which the linker links with our program. After the first parse of the compiler, it converts the source code into object code. Object code is machine code but is not re-locateable executable. The object code of our program is combined with the object code of the library functions, which the program is using. Later, some memory location information is included and we get the executable file. The linker performs this task while the compiler includes the name and arguments of the function in the object code. For checking the validity of the functions, the compiler needs to know the definition of the function or at least the prototype of the function. We have both the options for our functions. Define the function in the start of the program and use it in the main program. In this case, the definition of the function serves as both prototype and definition for the function. The compiler compiles the function and the main program. Then we can link and execute it. As the program gets big, it becomes difficult to write the definitions of all the functions at the beginning of the program. Sometimes, we write the functions in a different file and make the object file. We can include the prototypes of these functions in our program in different manners. One way is to write the prototype of all these functions in the start before writing the program. The better way is to make a header file (say myheaderfile.h) and write the prototypes of all the functions and save it as ordinary text file. Now we need to include it in our program using the *#include* directive. As this file is located at the place where our source code is located, it is not included in the angle brackets in *#include* directive. It is written in quotation marks as under:

```
#include “myHeaderFile.h”
```

The preprocessor will search for the file *“myHeaderFile.h”* in the current working directory. Let’s see the difference between the process of the including the file in brackets and quotation marks. When we include the file in angle brackets, the compiler looks in a specific directory. But it will look into the current working directory when the file is included in quotation marks. In the Dev-Cpp IDE, under the *tools* menu option, select *compiler options*. In this dialogue box, we can specify the directories for libraries and include files. When we use angle brackets with *#include*, the compiler will look in the directories specified in include directories option. If we want to write our own header file and save it in ‘My Document’ folder, the header file should be included with the quotation marks.

When we compile our source code, the compiler at first looks for the include directives and processes them one by one. If the first directive is *#include<iostream.h>*, the compiler will search this file in the include directory. Then it will include the complete header file in our source code at the same position where the ‘include directive’ is written. If the 2nd include directive contains another file, this file will also be included in the source code after the *iostream.h* and so on. The

© Copyright Virtual University of Pakistan

compiler will get this expanded source code file for compilation. As this expanded source code is not available to us and we will get the executable file in the end.
Can we include the header file at the point other than start of the program? Yes. There is no restriction. We can include wherever we want. Normally we do this at the start of the program as these are header files. We do not write a portion of code in a different file and include this file somewhere in the code. This is legal but not a practice. We have so far discussed include directive. Now we will discuss another important directive i.e. define directive.

define directive

We can define macros with the *#define* directive. Macro is a special name, which is substituted in the code by its definition, and as a result, we get an expanded code. For example, we are writing a program, using the constant Pi. Pi is a universal constant and has a value of 3.1415926. We have to write this value 3.1415926 wherever needed in the program. It will be better to define Pi somewhere and use Pi instead of the actual value. We can do the same thing with the variable Pi as *double Pi = 3.1415926* while employing Pi as variable in the program. As this is a variable, one can re-assign it some new value. We want that wherever we write Pi, its natural value should be replaced. Be sure that the value of Pi can not be changed. With the define directive, we can define Pi as:

```
#define PI 3.1415926
```

We need to write the name of the symbolic constant and its value, separated by space. Normally, we write these symbolic constants in capitals as it can be easily identifiable in the code. When we request the compiler to compile this file, the preprocessor looks for the define directives and replaces all the names in the code, defined with the define directives by their values. So compiler does not see PI wherever we have used PI is replaced with 3.1415926 before the compiler compiles the file.

A small program showing the usage of #define.

```
/* Program to show the usage of define */

#include <iostream.h>

#define PI 3.1415926            // Defining PI

main()
{
   int radius = 5;
   cout << "Area of circle with radius " << radius << " = " << PI * radius * radius;
}
```

What is the benefit of using it? Suppose we have written a program and are using the value of PI as 3.14 i.e. up to two decimal places. After verifying the accuracy of the result, we need to have the value of PI as 3.1415926. In case of not using PI as define, we have to search 3.14 and replace it with 3.1415926 each and every place in the source code. There may be a problem in performing this 'search and replace' task. We

© Copyright Virtual University of Pakistan

can miss some place or replace something else. Suppose at some place, 3.14 is representing something else like tax rate. We may change this value too accidentally, considering it the value for PI. So we can't conduct a blind search and replace and expect that it will work fine. It will be nicer to define PI at the start of the program. We will be using PI instead of its value i.e. 3.1415926. Now if we want to change the value of PI, it will be changed only at one place. The complete program will get the new value. When we define something with the *#define* directive, it is substituted with the value before the compiler compiles the file. This gives us a very nice control needed to change the value only at one place. Thus the complete program is updated.

We can also put this definition of PI in the header file. The benefit of doing this is, every program which is using the value of PI from this header file, will get the updated value when the value in header file is changed. For example, we have five functions, using the PI and these functions are defined in five different files. So we need to define PI (i.e. *#define PI 3.1415926*) in all the five source files. We can define it in one header file and include this header file in all the source code files. Each function is getting the value of PI from the header file by changing the value of PI in the header file, all the functions will be updated with this new value. As these preprocessor directives are not C statements, so we do not put semicolon in the end of the line. If we put the semicolon with the *#include* or *#define*, it will result in a syntax error.

Other Preprocessor Directives

There are some other preprocessor directives. Here is the list of preprocessor directives.

- #include <filename>
- #include "filename"
- #define
- #undef
- #ifdef
- #ifndef
- #if
- #else
- #elif
- #endif
- #error
- #line
- #pragma
- #assert

All the preprocessor directives start with the sharp sign (#). We can also do conditional compilation with it. We have *#if*, *#else*, *#endif* and for else if *#elif* is used. It can also be checked whether the symbol which we have defined with *#define*, is available or not. For this purpose, *#ifdef* is used. If we have defined PI, we can always say:

```
#ifdef PI
  … Then do something
```


© Copyright Virtual University of Pakistan

```
#endif
```

This is an example of conditional compilation. If a symbolic constant is defined, it will be error to define it again. It is better to check whether it is already defined or not. If it is already defined and we want to give it some other value, it should be undefined first. The directive for undefine is *#undef*. At first, we will undefine it and define it again with new value. Another advantage of conditional compilation is 'while debugging'. The common technique is to put output statements at various points in the program. These statements are used in the code to check the value of different variables and to verify that the program is working fine. It is extremely tedious to remove all these output statements which we have written for the debugging. To overcome this problem, we can go for conditional compilation. We can define a symbol at the start of the program as:

```
#define DEBUG
```

Here we have defined a symbol *DEBUG* with no value in front of it. The value is optional with the define directive. The output statements for debugging will be written as:

```
#ifdef DEBUG
        cout << "Control is in the while loop of calculating average";
#endif
```

Now this statement will execute if the *DEBUG* symbol is defined. Otherwise, it will not be executed.

Here is an example using the debug output statements:

```
// Program that shows the use of Define for debugging
// Comment the #define DEBUG and see the change in the output

#include <iostream.h>
#include <stdlib.h>

#define DEBUG

main()
{
        int z ;
        int arraySize = 100;
        int a[100] ;
        int i;

        // Initializing the array.
        for ( i = 0; i < arraySize; i++ )
        {
                a[i] = i;
        }

        // If the symbol DEBUG is defined then this code will execute
```


© Copyright Virtual University of Pakistan

```
        #ifdef DEBUG
                for ( i = 0 ; i < arraySize ; i ++ )
                    cout << "\t " << a[i];
    #endif

        cout << " Please enter a positive integer " ;
        cin >> z ;
        int found = 0 ;

        // loop to search the number.
        for ( i = 0 ; i < arraySize ; i ++ )
        {
                if ( z == a[i] )
                {
                        found = 1 ;
                        break ;
                }
        }
        if ( found == 1 )
                cout << " We found the integer at position " << i ;
        else
                cout << " The number was not found " ;
}
```

With preprocessor directives, we can carry out conditional compilation, a macro translation that is replacement of a symbol by the value in front of it. We can not redefine a symbol without undefining it first. For undefining a symbol, *#undef* is used. e.g. the symbol PI can be undefined as:

```
#undef PI
```

Now from this point onward in the program, the symbol PI will not be available. The compiler will not be able to view this symbol and give error if we have used it in the program after undefining.

As an exercise, open some header files and read them. e.g. we have used a header file *conio.h* (i.e. *#define<conio.h>*) for consol input output in our programs. This is legacy library for non-graphical systems. We have two variants of *conio* in Dev-Cpp i.e. *conio.h* and *conio.c* (folder is 'Dev-Cpp\include'). Open and read it. Do not try to change anything, as it may cause some problems. Now you have enough knowledge to read it line by line. You will see different symbols in it starting with underscore (_). There are lots of internal constants and symbolic names starting with double underscore. Therefore we should not use such variable names that are starting with underscore. You can find the declaration of different functions in it e.g. the function *getche()* (i.e. get character with echo) is declared in *conio.h* file. If we try to use the function *getche()* without including the *conio.h* file, the compiler will give error like 'the function *getche()* undeclared'. There is another interesting construct in *conio.h* i.e.

```
#ifdef __cplusplus
```


© Copyright Virtual University of Pakistan

```
        extern "C" {
#endif
```

If the symbol *cplusplus* is defined, the statement '*extern "C" {* ' will be included in the code. We have an opening brace here. Look where the closing brace is. Go to the end of the same file. You will find the following:

```
#ifdef __cplusplus
        }
#endif
```

This is an example of conditional compilation i.e. if the symbol is defined, it includes these lines in the code before compiling. Go through all the header files, we have been using in our programs so that you can see how professional programmers write code. If you have the linux operating system, it is free with a source code. The source code of linux is written in C language. You can see the functions written by the C programming Gurus. There may be the code of string manipulation function like string copy, string compare etc.

Macros

Macros are classified into two categories. The first type of macros can be written using *#define.* The value of PI can be defined as:

```
#define PI 3.1415926
```

Here the symbol PI will be replaced with the actual value (i.e. 3.1415926) in the program. These are simple macros like symbolic names mapped to constants.

In contrast, the second type of macros takes arguments. It is also called a parameterized macros. Consider the following:

```
#define square(x) x * x
```

Being a non-C code, it does not require any semicolon at the end. Before the compiler gets the file, the macro replaces all the occurrences of square (x) (that may be square (i), square (3) etc) with (x * x) (that is for square (i) is replaced by i * i, square(3) is replaced by 3 * 3). The compiler will not see square(x). Rather, it will see x * x, and make an executable file. There is a problem with this macro definition as seen in the following statement.

```
square (i + j);
```

Here we have *i+j* as *x* in the definition of macro. When this is replaced with the macro definition, we will get the statement as:

```
i + j * i + j
```

This is certainly not the square of *i + j*. It is evaluated as *(i + (j * i) + j* due to the precedence of the operators. How can we overcome this problem? Whenever you

© Copyright Virtual University of Pakistan

write a parameterized macro, it is necessary to put the parenthesis in the definition of macro. At first, write the complete definition in the parenthesis, and then put the *x* also in parenthesis. The correct definition of the macro will be as:

```
#define square(x) ((x) * (x))
```

This macro will work fine. When this macro definition is replaced in the code, parenthesis will also be copied making the computation correct.

Here is a sample program showing the use of a simple square macro:

```
/* Program to show the use of macro */

#include <iostream.h>

// Definition of macro square
#define square(x) ((x) * (x))

main()
{
   int x;

   cout << endl;
   cout << " Please enter the value of x to calculate its square ";
   cin >> x;
   cout << " Square of x = " << square(x) << endl;
   cout << " Square of x+2 = " << square(x+2) << endl;
   cout << " Square of 7 = " << square(7);
}
```

We can also write a function to *square(x)* to calculate the square of a number. What is the difference between using this *square(x)* macro and the *square(x)* function? Whenever we call a function, a lot of work has to be done during the execution of the program. The memory in machine is used as stack for the program. The state of a program (i.e. the value of all the variables of the program), the line no which is currently executing etc is on the stack. Before calling the function, we write the arguments on the stack. In a way, we stop at the function calling point and the code jumps to the function definition code. The function picks up the values of arguments from the stack. Do some computation and return the control to the main program which starts executing next line. So there is lot of overhead in function calling. Whenever we call a function, there is some work that needed to be done. Whenever we do a function call, like if we are calling a function in a loop, this overhead is involved with every iteration. The overhead is equal number of times the loop executed. So computer time and resources are wasted. Obviously there are a number of times when we need to call functions but in this simple example of calculating square, if we use square function and the program is calling this function 1000 times, a considerable time is wasted. On the other hand, if we define square macro and use it. The code written in front of macro name is substituted at all the places in the code where we are using square macro. Therefore the code is expanded before compilation and compiler see ordinary multiplication statements. There is no function call

© Copyright Virtual University of Pakistan

involved, thus making the program run faster. We can write complex parameterized macros. The advantage of using macros is that there is no overhead of function calls and the program runs faster. If we are using lot of macros in our program, it is replaced by the macro definition at every place in the code making the program bloat. Therefore our source code file becomes a large file, resulting in the enlargement of the executable file too. Sometimes it is better to write functions and define things in it. For simple things like taking a square, it is nice to write macros that are only one line code substitution by the preprocessor.

Take care of few things while defining macros. There is no space between the macro name and the starting parenthesis. If we put a space there, it will be considered as simple macro without parameters. We can use more than one argument in the macros using comma-separated list. The naming convention of the arguments follows the same rules as used in case of simple variable name. After writing the arguments, enclosing parenthesis is used. There is always a space before starting the definition of the macro.

Example

Suppose we have a program, which is using the area of circle many times in it. Therefore we will write a macro for the calculation of the area of circle. We know that the formula for area of circle is $PI*r^2$. Now this formula is substituted wherever we will be referring to this macro. We know that the PI is also a natural constant. So we will define it first. Then we will define the macro for the area of the circle. From the perspective of visibility, it is good to write the name of the macro in capital as CIRCLEAREA. We don't need to pass the PI as argument to it. The only thing, needed to be passed as argument, is radius. So the name of the macro will be as CIRCLEAREA (X).We will write the formula for the calculation of the area of the circle as:

```
#define CIRCLEAREA(X) (PI * (X) * (X))
```

Here is the complete code of the program:

```
/* A simple program using the area of circle formula as macro */

#include <iostream.h>

// Defining the macros
#define PI 3.14159
#define CIRCLEAREA(X) ( PI * X * X)

main()
{
        float radius;
        cout << " Enter radius of the circle: ";
        cin >> radius;
        cout << " Area of circle is " << CIRCLEAREA (radius);
}
```


© Copyright Virtual University of Pakistan

The CIRCLEAREA will be replaced by the actual macro definition including the entire parenthesis in the code before compilation. As we have used the parenthesis in the definition of the CIRCLEAREA macro. The statement for ascertaining the area of circle with double radius will be as under:

```
CIRCLEAREA(2 * radius);
```

The above statement will work fine in calculating the correct area. As we are using multiplication, so it may work without the use of parenthesis. But if there is some addition or subtraction like CIRCLEAREA(radius + 2) and the macro definition does not contain the parenthesis, the correct area will not be calculated. Therefore always use the parenthesis while writing the macros that takes arguments.

There are some other things about header files. As a proficient programmer writing your own operating systems, you will be using these things. There are many operating systems, which are currently in use. Windows is a popular operating system, DOS is another operating system for PC's, Linux, and different variety of Unix, Sun Solaris and main frame operating systems. The majority of these operating systems have a C compiler available. C is a very elegant operating systems language. It is very popular and available on every platform. By and large the source code which we write in our programs does not change from machine to machine. The things, which are changed, are system header files. These files belong to the machine. The header files, which we have written for our program, will be with the source code. But the *iostream, stdlib, stdio, string* header files have certain variations from machine to machine. Over the years as the C language has evolved, the names of these header files have become standard. Some of you may have been using some other compiler. But you have noted that in those compilers, the header files are same, as *iostream.h, conio.h* etc are available. It applies to operating systems. While changing operating systems, we come up with the local version of C/C++ compiler. The name of the header files remains same. Therefore, if we port our code from one operating system to another, there is no need to change anything in it. It will automatically include the header files of that compiler. Compile it and run it. It will run up to 99 % without any error. There may be some behavioral change like function *getche()* sometimes read a character without the enter and sometimes you have to type the character and press enter. So there may be such behavioral change from one operating system to other. Nonetheless these header files lead to a lot of portability. You can write program at one operating system and need not to take the system header file with the code to the operating system.

On the other hand, the header files of our program also assist in the portability in the sense that we have all the function prototypes, symbolic definitions, conditional compilations and macros at one place. While writing a lot of codes, we start writing header files for ourselves because of the style in which we work. We have defined some common functions in our header files. Now when we are changing the operating system, this header file is ported with the source code. Similarly, on staring some program, we include this header file because it contains utility function which we have written.

Here is an interesting example with the *#define*. If you think you are sharp here is a challenge for you. Define you own vocabulary with the #define and write C code in

© Copyright Virtual University of Pakistan

front of it. One can write a poem using this vocabulary which will be replaced by the preprocessor with the C code. What we need is to include one header file that contains this vocabulary. So an ordinary English poem is actually a C code. Interesting things can be done using these techniques.

Tips

- All the preprocessor directives start with the # sign
- A symbol can not be redefined without undefining it first
- The conditional compilation directives help in debugging the program
- Do not declare variable names starting with underscore
- Always use parenthesis while defining macros that takes arguments

Lecture No. 24

Reading Material

Deitel & Deitel - C++ How to Program Chapter 15, 18

15.3, 18.10

Summary

1) Memory Allocation
2) Dynamic Memory Allocation
3) calloc Function
4) malloc Function
5) free ()
6) realloc Function
7) Memory Leak
8) Dangling Pointers
9) Examples
10) Exercise
11) Tips

© Copyright Virtual University of Pakistan

Memory Allocation

After having a thorough discussion on static memory allocation in the previous lectures, we will now talk about dynamic memory allocation. In this lecture, the topics being dilated upon include- advantages and disadvantages of these both types of memory allocation and the common errors, which usually take place while programming with dynamic memory allocation. Let's first talk about the dynamic memory allocation.

Dynamic Memory Allocation

Earlier, whenever we declared arrays, the size of the arrays was predefined. For example we declared an array of size 100 to store ages of students. Besides, we need 20, 25 or 50 number of students to store their ages. The compiler reserves the memory to store 100 integers (ages). If there are 50 integers to be stored, the memory for remaining 50 integers (that has been reserved) remains useless. This was not an important matter when the programs were of small sizes. But now when the programs grow larger and use more resources of the system, it has become necessary to manage the memory in a better way. The dynamic memory allocation method can be helpful in the optimal utilization of the system.

It is better to compare both the static and dynamic allocation methods to understand the benefits of the usage of dynamic memory allocation. In static memory, when we write the things like int i, j, k ; these reserve a space for three integers in memory. Similarly the typing of char s[20] will result in the allocation of space for 20 characters in the memory. This type of memory allocation is static allocation. It is also known as compile time allocation. This memory allocation is defined at the time when we write the program while exacting knowing how much memory is required.

Whenever, we do not know in advance how much memory space would be required, it is better to use dynamic memory allocation. For example if we want to calculate the average age of students of a class. Instead of declaring an array of large number to allocate static memory, we can ask number of students in the class and can allocate memory dynamically for that number. The C language provides different functions to allocate the memory dynamically.

The programs, in which we allocate static memory, run essentially on stack. There is another part of memory, called heap. The dynamic memory allocation uses memory from the heap. All the programs executing on the computer are taking memory from it for their use according to the requirement. Thus heap is constantly changing in size. Windows system may itself use memory from this heap to run its processes like word processor etc. So this much memory has been allocated from heap and the remaining is available for our programs. The program that will allocate the memory dynamically, will allocate it from the heap.

Let's have a look on the functions that can be used to allocate memory from the heap.

Before actually allocating memory, it is necessary to understand few concepts. We have already studied these concepts in the lectures on 'pointers'. Whenever we allocate a memory what will we get? We need to be careful about that. When we say **int i,** a space is reserved for an integer and it is labeled as **i**. Here in dynamic memory, the situation is that the memory will be allocated during the execution of the program. It is difficult to determine whether the memory allocated is an array, an integer, 20 integers or how much space is it? To over this uncertainty, we have to use pointers.

© Copyright Virtual University of Pakistan

Whenever we allocate any memory from the heap, the starting position of the block of the memory allocated is returned as an address that is kept in a pointer. Then we manipulate the memory with the help of this pointer. We have to introduce a new type of a pointer, called '**void'**. We have used the pointers of type- int, char, float etc. For these, we write like **int *i ;** which means **i** is a pointer to an integer. In this case, the compiler automatically knows that **i** has the address of the memory, occupied by an integer. Same thing applies when we write **char *s .** It means **s** is a pointer to a character data type. So every pointer we have used so far pointed to a specific data type.

The functions used for dynamic memory allocation, provide a chunk of memory from heap. The function does not know for what data type this chunk of memory will be used? It returns a pointer of type void. A pointer ptr of type void is declared as under.

void *ptr ;

The 'void' is a special type of pointers. We have to cast it before its use. The cast means the conversion of 'void' into a type of pointer that can be used for native data type like int, char, float etc. The operator used for casting, in C, is standard cast operator. We write the name of the type in parentheses. Suppose we have a pointer ptr defined as a void pointer like

void *ptr ;

Before using this pointer to point to a set of integers, we will at first cast it. It means that it will be converted into a type of a pointer to an integer. The syntax of doing this casting is simple and is given below.

(int *) ptr ;

Here both int and * are written in parentheses. The **int** is the data type into which we are converting a **void** pointer **ptr**. Now **ptr** is a pointer to an integer. Similarly, we can write char, float and double instead of 'int', to convert **ptr** into a pointer to char, float and double respectively.

Casting is very useful in dynamic memory allocation. The memory allocation functions return a chunk of memory with a pointer of type void. While storing some type of data , we at first, cast the pointer to that type of data before its usage. It is an error to try to use the void pointer and dereference it. In case, we write *ptr and use it in an expression, there will be an error. So we have to cast a void pointer before its use.

Another interesting aspect of pointer is the NULL value. Whenever we define a pointer or declare a pointer, normally, it is initialized to a NULL value. NULL has been defined in the header files stdlib.h and stddef.h. So at least one of these files must be included in the program's header to use the NULL. A NULL pointer is a special type of pointer with all zeros value. All zeros is an invalid memory address. We can't use it to store data or to read data from it. It is a good way to ascertain whether a pointer is pointing to a valid address or has a NULL value.

calloc Function

The syntax of the calloc function is as follows.

void *calloc (size_t n, size_t el_size)

This function takes two arguments. The first argument is the required space in terms of numbers while the second one is the size of the space. So we can say that we require **n** elements of type **int**. We have read a function **sizeof.** This is useful in the cases where we want to write a code that is independent of the particular machines that we are running on. So if we write like

void calloc(1000, sizeof(int))

© Copyright Virtual University of Pakistan

It will return a memory chunk from the heap of 1000 integers. By using **sizeof (int)** we are not concerned with the size of the integer on our machine whether it is of 4 bytes or 8 bytes. We will get automatically a chunk that can hold 1000 integers. The said memory will be returned if a chunk of similar size is available on the heap. Secondly, this memory should be available on heap in continuous space. It should not be in split blocks. The function returns a pointer to the starting point of the allocated memory. It means that if starting point of the chunk is gotten, then the remaining memory is available in a sequence from end to end. There cannot be gaps and holes between them. It should be a single block. Now we have to see what happens when either we ask for too much memory at a time of non-availability of enough memory on the heap or we ask for memory that is available on the heap , but not available as a single chunk?. In this case, the call to calloc will fail. When a call to memory allocation functions fails, it returns a NULL pointer. It is important to understand that whenever we call a memory allocation function, it is necessary to check whether the value of the pointer returned by the function is NULL or not. If it is not NULL, we have the said memory. If it is NULL, it will mean that either we have asked for too much memory or a single chunk of that size is not available on the heap.

Suppose, we want to use the memory got through calloc function as an integer block We have to cast it before using. It will be written as the following statement.

(int *) calloc (1000, sizeof (int)) ;

Another advantage of calloc is that whenever we allocate memory by using it. The memory is automatically initialized to zeros. In other words it is set to zeros. For casting we normally declare a pointer of type which we are going to use. For example, if we are going to use the memory for integers. We declare an integer pointer like **int *iptr;** Then when we allocate memory through calloc, we write it as

iptr = (int *) calloc (1000, sizeof(int)) ;

(int *) means cast the pointer returned by calloc to an integer pointer and we hold it in the declared integer pointer **iptr**. Now **iptr** is a pointer to an integer that can be used to manipulate all the integers in that memory space. You should keep in mind that after the above statement, a NULL check of memory allocation is necessary. An 'if statement' can be used to check the success of the memory allocation. It can be written as under

```
if (iptr == NULL)
    any error message or code to handle error ;
```

If a NULL is returned by the calloc, it should be treated according to the logic so that the program can exit safely and it should not be crashed.

The next function used for allocating memory is malloc.

malloc Function

The malloc function takes one argument i.e. the number of bytes to be allocated. The syntax of the function is

void * malloc (size_t size) ;

It returns a void pointer to the starting of the chunk of the memory allocated from the heap in case of the availability of that memory. If the memory is not available or is fragmented (not in a sequence), malloc will return a NULL pointer. While using malloc, we normally make use sizeof operator and a call to malloc function is written in the following way.

malloc (1000 * sizeof(int)) ;

Here * is multiplication operator and not a dereference operator of a pointer.

© Copyright Virtual University of Pakistan

In the above call, we request for 1000 spaces in the memory each of the size, which can accommodate an integer. The 'sizeof(int)' means the number of bytes, occupied by an integer in the memory. Thus the above statement will allocate memory in bytes for 1000 integers. If on our machine, an integer occupies 4 bytes. A 1000 * 4 (4000) bytes of memory will be allocated. Similarly if we want memory for 1000 characters or 1000 floats, the malloc function will be written as

malloc (1000 * sizeof(char)) ;

and malloc (1000 * sizeof(float)) ;

respectively for characters and floats.

So in general, the syntax of malloc will be.

malloc (n * sizeof (*datatype*)) ;

where 'n' represents the numbers of required data type. The **malloc** function differs from **calloc** in the way that the space allocated by **malloc** is not initialized and contains any values initially.

Let's say we have a problem that states 'Calculate the average age of the students in your class.' The program prompts the user to enter the number of students in the class and also allows the user to enter the ages of the students. Afterwards, it calculates the average age. Now in the program, we will use dynamic memory. At first, we will ask the user 'How many students are in the class? The user enters the number of students. Let's suppose, the number is 35. This number is stored in a variable say '**numStuds'**. We will get the age of students in whole numbers so the data type to store age will be **int**. Now we require a memory space where we can store a number of integers equal to the value stored in **numStuds**. We will use a pointer to a memory area instead of an array. So we declare a pointer to an integer. Suppose we call it **iptr**. Now we make a call to calloc or malloc function. Both of them are valid. So we write the following statement

iptr = (int *) malloc (numStuds * sizeof (int)) ;

Now we immediately check **iptr** whether it has NULL value. If the value of **iptr** is not NULL, it will mean that we have allocated the memory successfully. Now we write a loop to get the ages of the students and store these to the memory, got through malloc function. We write these values of ages to the memory by using the pointer **iptr** with pointer arithmetic. A second pointer say **sptr** can be used for pointer arithmetic so that the original pointer **iptr** should remain pointing to the starting position of the memory. Now simply by incrementing the pointer **sptr**, we get the ages of students and store them in the memory. Later, we perform other calculations and display the average age on the screen. The advantage of this (using malloc) is that there is no memory wastage as there is no need of declaring an array of 50 or 100 students first and keep the ages of 30 or 35 students in that array. By using dynamic memory, we accurately use the memory that is required.

free ()

Whenever we get a benefit, there is always a cost. The dynamic memory allocation has also a cost. Here the cost is incurred in terms of memory management. The programmer itself has to manage the memory. It is the programmer's responsibility that when the memory allocated is no longer in use, it should be freed to make it a part of heap again. This will help make it available for the other programs. As long as the memory is allocated for a program, it is not available to other programs for use. So it is programmer's responsibility to free the memory when the program has done with it. To ensure it, we use a function **free**. This function returns the allocated memory,

© Copyright Virtual University of Pakistan

got through **calloc** or **malloc,** back to the heap. The argument that is passed to this function is the pointer through which we have allocated the memory earlier. In our program, we write

free (iptr) ;

By this function, we call the memory allocated by **malloc** and pointed by the pointer **iptr** is freed. It goes back to the heap and becomes available for use by other programs. It is very important to note that whenever we allocate memory from the heap by using **calloc** or **malloc,** it is our responsibility to free the memory when we have done with it.

Following is the code of the program discussed above.

```
//This program calculates the average age of a class of students
//using dynamic memory allocation

#include <iostream.h>
#include <stdlib.h>
#include <string.h>

int main( )
{

        int numStuds, i, totalAge, *iptr, *sptr;
        cout <<"How many students are in the class ?    " ;
        cin >> numStuds;
        // get the starting address of the allocated memory in pointer iptr
        iptr = (int *) malloc(numStuds * sizeof(int));
        //check for the success of memory allocation
        if (iptr == NULL)
        {
                cout << "Unable to allocat space for " << numStuds << " students\n";
        return 1;
        // A nonzero return is usually used to indicate an error
        }
        sptr = iptr ; //sptr will be used for pointer arithmetic/manipulation
        i = 1 ;
        totalAge = 0 ;
        //use a loop to get the ages of students
        for (i = 1 ; i <= numStuds ; i ++)
        {
                cout << "Enter the age of student  " << i << " = " ;
        cin >> *sptr ;
        totalAge = totalAge + *sptr ;
        sptr ++ ;
        }
        cout << "The average age of the class is " << totalAge / numStuds << endl;
        //now free the allocated memory, that was pointed by iptr
        free (iptr) ;
        sptr = NULL ;
 }
```


© Copyright Virtual University of Pakistan

Following is a sample out put of the program.

```
How many students are in the class ?  3
Enter the age of student  1 = 12
Enter the age of student 2 = 13
Enter the age of student 3 = 14
The average age of the class is 13
```

realloc Function

Sometimes, we have allocated a memory space for our use by **malloc** function. But we see later that some additional memory is required. For example, in the previous example, where (for example) after allocating a memory for 35 students, we wanted to add one more student. So we need same type of memory to store the new entry. Now the question arises 'Is there a way to increase the size of already allocated memory chunk ? Can the same chunk be increased or not? The answer is yes. In such situations, we can reallocate the same memory with a new size according to our requirement. The function that reallocates the memory is **realloc**. The syntax of **realloc** is given below.

void realloc (void * ptr, size_t size) ;

This function enlarges the space allocated to ptr (in some previous call of calloc or malloc) to a (new) size in bytes. This function receives two arguments. First is the pointer that is pointing to the original memory allocated already by using calloc or malloc. The second is the size of the memory which is a new size other than the previous size. Suppose we have allocated a memory for 20 integers by the following call of malloc and a pointer iptr points to the allocated memory.

(iptr *) malloc (20 * sizeof(int)) ;

Now we want to reallocate the memory so that we can store 25 integers. We can reallocate the same memory by the following call of realloc.

realloc (iptr, 25 * sizeof(int)) ;

There are two scenarios to ascertain the success of 'realloc'. The first is that it extends the current location if possible. It is possible only if there is a memory space available contiguous to the previously allocated memory. In this way the value of the pointer iptr is the same that means it is pointing to the same starting position, but now the memory is more than the previous one. The second way is that if such contiguous memory is not available in the current location, realloc goes back to the heap and looks for a contiguous block of memory for the requested size. Thus it will allocate a new memory and copy the contents of the previous memory in this new allocated memory. Moreover it will set the value of the pointer iptr to the starting position of this memory. Thus iptr is now pointing to a new memory location. The original memory is returned to the heap. In a way, we are handling dynamic arrays. The size of the array can be increased during the execution. There is another side of the picture. It may happen that we have stored the original value of iptr in some other pointer say sptr. Afterwards, we are manipulating the data through both the pointers. Then ,we use realloc for the pointer iptr. The realloc does not find contiguous memory with the original and allocates a new block of memory and points it by the pointer iptr. The original memory no longer exists now. The pointer iptr is valid now as it is pointing to the starting position of the new memory. But the other pointer sptr is no longer valid. It is pointing to an invalid memory that has been freed and may be is being used some other program. If we manipulate this pointer, very strange things can

© Copyright Virtual University of Pakistan

happen. The program may crash or the computer may halt. We don't know what can happen. Now it becomes the programmer's responsibility again to make it sure that after realloc, the pointer(s) that have the value of the original pointer have been updated. It is also important to check the pointer returned by realloc for NULL value. If realloc fails, that means that it cannot allocate the memory. In this case, it returns a NULL value. After checking NULL value, (if realloc is successful), we should update the pointer that was referencing the same area of the memory.

We have noticed while getting powers of dynamic memory allocation, we face some dangerous things along with it. These are real problems. Now we will talk about the common errors that can happen with the memory allocation.

Memory Leak

The first problem may be the unreferenced memory. To understand this phenomenon, suppose, we allocate memory from heap and there is a pointer pointing to this memory. However, it is found that this pointer does not exist any more in our program. What will happen to the memory we had allocated. That chunk of memory is now unreferenced. Nothing is pointing to that memory. As there is no pointer to this memory, our program can't use it. Moreover, no other program can use it. Thus, this memory goes waste. In other words, the heap size is decreased as we had allocated memory from it despite the fact that it was never utilized. If this step of allocating memory and then destroy the pointer to this memory carries on then the size of the heap will going on to decrease. It may become of zero size. When there is no memory on heap, the computer will stop running and there may be a system crash. This situation is called a memory leak. The problem with memory leak is that you may be unaware of the memory leak caused by the program. Suppose there is 128 MB memory available on heap. We run our program that allocates 64 KB memory and terminates without freeing this memory. It does not effect but when if the memory is being allocated in a loop, that, suppose runs 1000 times and in each loop it allocates 64 KB of memory with out freeing the previous one. Then this program will try to allocate 64 * 1000 KB memory and at a certain point there will be no memory available and the program will crash. The same thing (no memory available) happens to other programs and the whole system locks up. So memory leak is a very serious issue.

This bug of memory leak was very common in the operating systems. This was a common thing, that the system was running well and fine for 4-5 hours and then it halted suddenly. Then the user had to reboot the system. When we reboot a system all the memory is refreshed and is available on the heap. People could not understand what was happening. Then there come the very sophisticated debugging techniques by which this was found that memory is being allocated continuously without freeing and thus the heap size becomes to zero. Thus memory is leaking out and it is no longer useable.

Let us see how does this happen and what we can do to prevent it. A simple way in which memory leak can happen is that suppose our main program calls a function. There, in the function, a pointer iptr is declared as a pointer to an integer. Then we call calloc or malloc in the function and allocate some memory. We use this memory and goes back to the main function without freeing this memory. Now as the pointer iptr has the function scope it is destroyed when the function exits. It is no longer there but the memory allocated remains allocated and is not being referenced as the pointer

© Copyright Virtual University of Pakistan

pointing to it no longer exists. Now this memory is unreferenced which means it is leaked. This is a memory leak. Now if this function is being called repeatedly it means a chunk of memory is being allocated and is left unreferenced each time. Thus, each time a memory chunk from heap will be allocated and will become useless(as this will be unreferenced) and the heap size may become zero. As a programmer, it is our responsibility and a good rule of thumb will be that in which function the memory is allocated, it should be freed in the same function.

Sometimes the logic of the program is that the memory is being allocated somewhere and is being used somewhere else. It means we allocate memory in a function and use it in another function. In such situations, we should keep in mind that this all scenario is memory management and we have to take care of it. We allocate memory in a function and cannot free it here because it is being used in some other function. So we should have a sophisticated programming to make it sure that whenever we allocate a memory it should be freed somewhere or the other. Now it is not to do just with function calls. It also has to do when the program ends. Let consider, our program is running and we allocate memory somewhere and somewhere else there is a condition on which the program exits. If we exit without freeing the memory then there is a memory leak. The memory leakage is at operating system level. The operating system does not know that this memory is not being used by anyone now. From its aspect, some program is using this memory. So whenever we write program we should free the allocated memory wherever it is allocated. But at the program exit points we should do some task. This task is make it sure that when we allocated memory in the program this memory should be freed at exit points. The second necessary thing is that after freeing the memory, explicitly assign NULL to the pointer. Its benefit is that this pointer can be checked if it is pointing to some memory.

Whereas we do get this considerable flexibility in doing dynamic memory management, it is also our responsibility for freeing all the memory that we allocated from the heap. The other side of the coin is also that if we are using dynamic memory allocation in our program then we should check immediately if we have got memory. If we did not get (allocated) memory then exit the program in a good and safe way rather than to crash the program.

Dangling Pointers

Memory leak is one subtle type of error that can happen. There is another one. This other one is even more dangerous. This is dangling pointer. It has the inverse effect of the memory leak. Suppose, there was a pointer that was pointing to a chunk of memory, now by some reason that memory has deallocated and has gone back to heap. The pointer still has the starting address of that chunk. Now what will happen if we try to write something in the memory using this pointer? Some very strange thing can happen. This can happen that when we have put that memory back to heap some other program starts to use that memory. Operating system itself might have started using that memory. Now our program, by using that pointer try to write something in the memory that is being used by some other program. This may halt the machine as the position that is being tried to written may be a critical memory position. How does this situation arise? Lets consider a case. We have two pointers ptr1 and ptr2. These are pointers to integers. We allocate some memory from the heap by using calloc or malloc. The pointer ptr1 is pointing to the starting point of this allocated memory. To use this memory through a variable pointer we use the pointer ptr2. At start, we put the address of ptr1 in ptr2 and then do our processing with the help of ptr2. In the meantime, we go to exit the function. To free the allocated memory we use the pointer

© Copyright Virtual University of Pakistan

ptr1. Thus the memory allocated goes back to heap and some other program may use it. The pointer ptr2 has the address of the same memory that it got from the ptr1. Now ptr2 points in a way to the memory that no longer belongs to the program. It has gone back to the heap. We can read the data residing at that memory location. But now if we try to write something in that location everything might break loose. We have to be very careful. The pointer ptr2 points to no location it is called dangling pointer. We have to be very careful about memory leak and dangling pointer.

The dynamic memory allocation is a very useful technique. In it what memory we require we take from the heap and use it and when it is no longer required we send it back to the heap. All the programs running on our machine (which are running on modern operating systems which are multitasking) work efficiently. They take memory of their requirement from the memory resources and return it back after using.

The sharing is not limited to memory resources this also include printers attached with the computer. The printer resource is being used by different programs like MS WORD, EXCEL and even may be by our program if we want to print something. We are also sharing the other resources like keyboard, monitor, and hard disk etc. But in terms of dynamic usage we are also sharing the memory. Our program in a way has to be a good neighbor to use the memory. It should use memory as long as it required and then after use it should give back this memory to the heap so that other programs can use this resource. So remember to free the memory it is as important as the allocation of memory.

So what interesting things we can do with memory allocation. A common thing in file handling is to copy a file. Our hard disks being electro mechanical devices are very slow. It is very expensive to access them. So while reading from them or writing to them we try that a big chunk should be written or read from them so that fewest disk writes and disk reads should occur. In order to do that, think combining dynamic memory allocation with disk read and write. Suppose we have to copy a file. We can easily find out the size of the file in bytes. Now we allocate this number of bytes from heap. If this size of memory is successfully allocated, we can say for a single file read of this allocated size. This means the entire file will be read to memory. This way we read a whole file with one command. Similarly, we can use a command to write the whole file. In this way we can be assured that we are doing the more efficient disk access.

Examples

Following are the examples, which demonstrate the use of dynamic memory allocation.

Example 1

In the following simple example we allocate a memory which is pointing by a character pointer. We copy an array of characters to that location and display it. After that we free that memory before exiting the program.

```
//This program allocates memory dynamically and then frees it after use.

#include                                    <iostream.h>
#include                                    <stdlib.h>
```


© Copyright Virtual University of Pakistan

```
#include <string.h>

int main()
{
        char s1[] = "This is a sentence";
        char *s2;
        s2 = (char *) malloc(strlen(s1) + 1);
      /* Remember that stings are terminated by the null terminator, "\0',
          and the strlen returns the length of a string not including the terminator */
        if (s2 == NULL)
        {
                cout << "Error on malloc";
        return 1;
        /* Use a nonzero return to indicate an error has occurred */
        }

        strcpy(s2,s1);

        cout << "s1: “ << s1 << endl;
        cout << "s2: “ << s2 << endl;
        free(s2);
        return 0;
}
```

The output of the program is given below.

```
S1: This is a sentence
S2: This is a sentence
```

Example 2

Following is another example that allocates a memory dynamically according to the requirement and displays a message for the failure or success of the memory allocation.

```
// This program shows the dynamic allocation of memory according to the
requirement to //store a certain number of a structure.

#include <iostream.h>
#include <stdlib.h>
#include <string.h>

struct Employee
{
        char name[40];
        int id;
};

int main()
{
```


© Copyright Virtual University of Pakistan

```
        Employee *workers, *wpt;
        int num;
        cout <<"How many employees do you want\n“ ;
        cin >> num;
        // the pointer workers gets the starting address of the memory if allocated
successfully
        workers = (Employee *) malloc(num * sizeof(Employee));
        if (workers == NULL)
        {
                cout << "Unable to allocate space for employees\n";
        return 1;
        // A nonzero return is usually used to indicate an error
        }
        cout << “Memory for “ << num << “ employees has allocated successfully” ;
        //now free the allocated memory
        free(workers) ;
}
```

A sample output of the program is as below.

```
How many employees do you want
235
Memory for 235 employees has allocated successfully
```

Exercise

As an exercise, you can find the maximum available memory from the heap on your computer. You can do this by using a loop in which first time you allocate a certain number of bytes(say 10000). If it is successfully allocated then free it and in the next iteration allocate twice of the previous size of memory. Thus we can find the maximum amount of memory available. Suppose you find that 2MB memory is available. Then run some other applications like MS WORD, MS EXCEL etc. Now again run your program and find out the size of the memory available now. Is there any difference in the size of the memory allocated? Yes, you will see that the size has decreased. It proves that the heap is being shared between all of the programs running on that machine at that time.

Dynamic memory allocation is a very efficient usage of computer resources as oppose to static memory allocation. The benefit of static memory is that its usage is very neat and clean, there are no errors. But disadvantage is that there are chances of wastage of resources.

The dynamic memory allocation is very efficient in terms of resources but added baggage is that freeing the memory is necessary, pointers management is necessary. You should avoid the situations that create memory leakage and dangling pointers.

Tips

- Using dynamic memory is more efficient then the static memory.
- Immediately after a memory allocation call, check whether the memory has allocated successfully.
- Whenever possible free the allocated memory in the same function.
- Be careful about memory management to prevent memory leakage and

© Copyright Virtual University of Pakistan

dangling pointers.

- Before exiting the program, make sure that the allocated memory has freed.



© Copyright Virtual University of Pakistan

Lecture No. 25

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 3***

3.16, 3.18, 3.20

Summary

- Lecture Overview
- History of C/C++
- Structured Programming
- Limitations of Structured Programming
- Default Function Arguments
- Example of Default Function Arguments
- Placement of Variable Declarations
- Example of Placement of Variable Declarations
- Inline Functions
- Example of Inline Functions versus Macros
- Function Overloading
- Example of Function Overloading

Lecture Overview

From this lecture we are starting exciting topics, which we have been talking about many times in previous lectures. Until now, we have been discussing about the traditional programming following top down approach using C/C++. By and large we have been using C language, although, we also used few C++ functions like C++ I/O using **cin** and **cout** instead of standard functions of C i.e., **printf()** and **scanf**(). Today and in subsequent lectures, we will talk about C++ and its features. Note that we are not covering Object Oriented Programming here as it is a separate subject.

© Copyright Virtual University of Pakistan

History of C/C++

C language was developed by scientists of Bell Labs in 1970s. It is very lean and mean language, very concise but with lot of power. C conquered the programming world and took it by storm. Major operating systems e.g., Unix was written in C language.

Going briefly into the history of languages, after the Machine Language (language of 0s and 1s), the Assembly Language was developed. Using Assembly language, programmers could use some symbolic codes, which were easier to understand by novice people. After that high-level languages like COBOL, FORTRAN were developed. These languages were more English like and as a result easier to understand for us as human beings. This was the age of spaghetti code where programs were not properly structured and their branches were growing in every direction. As a result, it is difficult o read, understand and manage. These problems lead to the innovation of structured programming where a problem was broken into smaller parts. But this approach also had limits. In order to understand those limits, we will see what is structured programming first before going into its limitations detail.

Structured Programming

We have learned so far, C is a language where programs are composed of functions. Basically, a problem is broken into small pieces or modules and each small piece corresponds to a function. This was the **top-down structured programming** approach. We have already discussed few rules of structured programming, which are still valid and will remain valid in the future. Let's reiterate those:

- **Divide and Conquer**; one should not write very long functions. If a function is getting longer than two or three pages or screens then it is divided into smaller, concise and well-defined tasks. Later each task becomes a function.
- Inside the functions, **Single Entry Single Exit** rule should be tried to obey as much as possible. This rule is very important for readability and useful in managing programs. Even if the developer itself tries to use the same function after sometime, it would be easier for him to read his own code if he has followed the rules properly. We try to reuse our code as much as possible. It is likely that we may reuse our code or functions. That reuse might happen quite after sometime. Never think that your written code will not change or will not be used again.
- You should **comment your programs** well. Your comments are only not used by other people but by yourself also, therefore, you should write useful and lots of comments. At least comment, what the function does, what are its parameters and what does it return back. The comments should be meaningful and useful about the processing of the function.

You should use the principles of structured programming as the basis of your programs.

Limitations of Structured Programming

When we design a functional program, the data it requires to process, is an entity that lies outside of the program. We take care of the function rather than the data it is

© Copyright Virtual University of Pakistan

going to process. When the problems became complex, we came to know that we can't leave the data outside. Somehow the data processed by the program should be present inside it as a part of it. As a result, a new thought process became prevalent that instead of the program driven by functions, a program should be driven by data. As an example, while working with our Word processors when we want a text to be bold, firstly that text is selected and then we ask Word to make it bold. Notice in this example the data became first and then the function to make it bold. This is programming driven by data. This approach originated the Object Oriented Programming.

In the early 1980s a scientist in Bell Labs Bejarne Stroustrup started working in enhancing C language to overcome the shortcomings of structured approach. This evolution of C language firstly known to be **C with Classes,** eventually called **C++**. Then the follow-up version of C++ is the Java language. Some people call Java as C plus plus minus. This is not exactly true but the evolution has been the same way.

C++ does not contain the concept of Classes only but some other features were also introduced. We will talk about those features before we talk about the classes.

Default Function Arguments

While writing and calling functions, you might have noticed that sometimes the parameter values remain the same for most of the calls and others keep on changing. For example, we have a function:

```
power( long x, int n )
```

Where x is the number to take power of and n is the power to which x is required to be raised.

Suppose while using this function you came to know that 90% of the calls are for squaring the number x in your problem domain. Then this is the case where default function arguments can play their role. When we find that there are some parameters of a function that by and large are passed the same value. Then we start using default function arguments for those parameters.

The default value of a parameter is provided inside the function prototype or function definition. For example, we could declare the default function arguments for a function while declaring or defining it. Below is the definition of a very simple function **f()** that is called most of the times with parameters values of i as 1 and x as 10.5 most of the times then by we can give default values to the parameters as:

```
void f ( int  i = 1, double  x = 10.5 )
{
        cout << "The value of i is: " << i;
        cout  <<  "The value of x is: "  <<  x;
}
```

Now this function can be called 0, 1 or 2 arguments.

Suppose we call this function as:

```
f();
```

See we have called the function **f()** without any parameters, although, it has two parameters. It is perfectly all right and this is the utility of default function arguments. What do you think about the output. Think about it and then see the output below:

© Copyright Virtual University of Pakistan

```
The value of i is: 1
The value of x is: 10.5
```

In the above call, no argument is passed, therefore, both the parameters will use their default values.
Now if we call this function as:

```
f(2);
```

In this case, the first passed in argument is assigned to the first variable (left most variable) **i** and the variable **x** takes its default value. In this case the output of the function will be as under:

```
The value of i is: 2
The value of x is: 10.5
```

The important point here is that your passed in argument is passed to the first parameter (the left most parameter). The first passed in value is assigned to the first parameter, second passed in value is assigned to the second parameter and so on. The value **2** cannot be assigned to the variable x unless a value is explicitly passed to the variable **i**. See the call below:

```
f(1, 2);
```

The output of the function will be as under:

```
The value of i is: 1
The value of x is: 2
```

Note that even the passed in value to the variable **i** is the same as its default value, still to pass some value to the variable **x**, variable **i** is explicitly assigned a value.

While calling function, the arguments are assigned to the parameters from left to right. There is no luxury or feature to use the default value for the first parameter and passed in value for the second parameter. Therefore, it is important to keep in mind that the parameters with default values on left cannot be left out but it is possible for the parameter with default values on right side.
Because of this rule of assignment of values to the parameters, while writing functions, the default values are written from right to left. For example, in the above example of function **f()**, if the default value is to be provided to the variable **x** only then it should be on the left side as under:

```
void f( int  i, double  x = 10.5 )
{
        // Display statements
}
```

If we switch the parameters that the variable **x** with default value becomes the first parameter as under:

```
void f( double  x = 10.5, int  i )
{
        // Display statements
}
```


© Copyright Virtual University of Pakistan

Now we cannot use the default value of the variable **x**, instead we will have to supply both of the arguments. Remember, whenever you want to use default values inside a function, the parameters with default values should be on the extreme right of the parameter list.

Example of Default Function Arguments

```
// A program with default arguments in a function prototype

#include <iostream.h>
void show( int = 1, float = 2.3, long = 4 );
void main()
{
  show();              // All three arguments default
  show( 5 );           // Provide 1st argument
  show( 6, 7.8 );      // Provide 1st and 2nd
  show( 9, 10.11, 12L ); // Provide all three argument
}
void show( int first, float second, long third )
{
  cout << "\nfirst = "  << first;
  cout << ", second = " << second;
  cout << ", third = "  << third;
}
```

The output of the program is:

```
first = 1, second = 2.3, third = 4
first = 5, second = 2.3, third = 4
first = 6, second = 7.8, third = 4
first = 9, second = 10.11, third = 12
```

Placement of Variable Declarations

This has to do with the declaration of the variables inside the code. In C language, all the variables are declared at the top of the function or code block and then we can use them later on in the code. We have already relaxed this rule, now, we will discuss it explicitly.

One of the enhancements in C++ over C is that a variable can be declared anywhere in the function. The philosophy of this enhancement is that a variables is declared just before it is actually used in the code. That will increase readability of the code.

It is not hard and fast direction but it is a tip of good programming practice. One can still declare variables at the start of the program, function or code block. It is a matter of style and convenience. One should be consistent in his/her style.

We should be clear about implications of declaring variables at different locations. For example, we declare a variable **i** as under:

```
{       // code block
        int  i;
```


© Copyright Virtual University of Pakistan

```
        . . .
        . . .
    }
```

The variable **i** is declared inside the code block in the beginning of it. **i** is visible inside the code block but after the closing brace of this code block, **i** cannot be used. Be aware of this, whenever you declare a variable inside a block, the variable **i** is alive inside that code block. Outside of that code block, it is no more there and it can not referenced any further. Compiler will report an error if it is tried to access outside that code block.

You must have seen in your books many times, a for loop is written in the following manner:

```
for (int  i = 0; condition; increment/decrement statements )
{
        . . .
}
i = 500;          // Valid statement and there is no error
```

The variable **i** is declared with the for loop statement and it is used immediately. We should be clear about two points here. Firstly, the variable **i** is declared outside of the for loop opening brace, therefore, it is also visible after the closing brace of the for loop.
So the above declaration of i can also be made as under:

```
int  i;
for ( i = 0; condition; increment/decrement statements)
{
        . . .
}
```

This approach is bit more clear and readable as it clearly declares the variable **i** outside the for statement. But again, it is a matter of style and personal preference, both approaches are correct.

Example of Placement of Variables Declarations

```
// Variable declaration placement
#include  <iostream.h>

void  main()
{
//  int  lineno;
   for( int lineno = 0; lineno < 3; lineno++ )
   {
      int  temp = 22;
      cout << "\nThis is line number " << lineno
          << " and temp is " << temp;
   }
```


© Copyright Virtual University of Pakistan

```
  if( lineno == 4 ) // lineno still accessible
     cout << "\nOops";
  // Cannot access temp
}
```

The output of the program is:

```
This is line number 0 and temp is 22
This is line number 1 and temp is 22
This is line number 2 and temp is 22
```

Inline Functions

This is also one of the facilities provided by C++ over C. In our previous lectures, we discussed and wrote macros few macros like **max** and **circlearea**.
While using macros, we use the name of the macro in our program. Before the compilation process starts the macro names are replaced by the preprocessor with their definitions (defined with #define).
Inline functions also work more or less in the same manner as macros. The functions are declared inline by writing **inline** keyword before the name of the function. This is a directive to the compiler and it causes the full definition of the function to be inserted in each place the function is called. Inserting individual copies of functions eliminates the overhead of calling a function (such as loading parameters onto the stack).

We see what are the advantages and disadvantages of it:

We'll discuss the disadvantages first. Let's suppose the inline function is called 100 times inside your program and that function itself is of 10 lines in length. Then at 100 places inside your program this 10 lines function definition is written, causes the program size to increase by 1000 lines. Therefore, the size of the program increases significantly. The increase in size of program may not be an issue if you have lots of resources of memory and disk space available but preferably, we try not to increase the size of the program without any benefit.

Also the inline directive is a request to the compiler to treat the function as inline. The compiler is on its own to accept or reject the request of inlining. To get to know whether the compiler has accepted the request to make it inline or not, is possible through the program's debugging. But this is bit tedious at this level of our programming expertise.

Now we'll see what are the advantages of this feature of C++. While writing macros, we knew that it is important to enclose the arguments of macros within parenthesis. For example, we wrote square macro as:

```
#define  square(x)      (x)  *  (x)
```

when this macro is called by the following statement in our code:

© Copyright Virtual University of Pakistan

```
square( i + j );
```

then it is replaced with the definition of the square macro as:

```
( i + j ) * ( i + j );
```

Just consider, we have not used parenthesis and written our macro as under:

```
#define  square(x)      x  *  x
```

then the substitution of the macro definition will be as:

```
i + j  *  i + j;
```

But the above definition has incorrect result. Because the precedence of the multiplication operator (*) is higher than the addition operator (+), therefore, the above statement is executed semantically as:

```
i + (j * i) + j;
```

Hence, the usage of brackets is necessary to make sure that the macros work as expected.

Secondly, because the macros are replaced with preprocessors and not by compiler, therefore, they are not aware of the data types. They just replace the macro definition and there is no type checking on the parameters of the macro. Same macro can be used for multiple data types. For instance, the above square macro can be used for **long**, **float**, **double** and **char** data types.

Inline functions behave as expected like a function and they don't have any side effects. Secondly, the automatic type checking for parameters is also done for inline functions. If there is a difference between data types provided and expected, the compiler will report an error unlike a macro.

Now, we see a program code to differentiate between macros and inline functions:

Example of Inline Functions versus Macros

```
// A macro vs. an inline function

#include  <iostream.h>

#define  MAX( A, B )  ((A) > (B) ? (A) : (B))
inline  int  max( int a, int b )
{
   if ( a > b )
         return a;
   return b;
}

void main()
{
  int  i, x, y;
  x = 23; y = 45;
```


© Copyright Virtual University of Pakistan

```
  i = MAX( x++, y++ );  // Side-effect:
                        // larger value incremented twice
  cout << "x = " << x << "    y = " << y << '\n';

  x = 23; y = 45;
  i = max( x++, y++ ); // Works as expected
  cout << "x = " << x << "    y = " << y << '\n';
}
```

The output of this program is:

```
x = 24   y = 47
x = 24   y = 46
```

You can see that the output from the inline function is correct while the macro has produced incorrect result by incrementing variable y two times. Why is this so?
The definition of the macro contains the parameters A and B two times in its body and keeping in mind that macros just replace the argument values inside the definition, it looks like the following after replacement.

```
( (x++) > (y++)  ?  (x++)  :  (y++) );
```

Clearly, the resultant variable either **x** or **y**, whichever is greater (**y** in this case) will be incremented twice instead of once.

Now, the interesting point is why this problem is not there in inline functions. Inside the code, the call to the inline function **max** is made by writing the following statement:

```
i  =  max( x++, y++ );
```

While calling the inline function, compiler does the type checking and passes the parameters in the same way as in normal function calls. The arguments are incremented once after their values are replaced inside the body of the function **max** and this is our required behavior.

Hence, by and large it is better to use inline functions rather than macros. Still macros can be utilized for small definitions.

The inline keyword is only a suggestion to the compiler. Functions larger than a few lines are not expanded inline even if they are declared with the inline keyword.

If the inline function is called many times inside the program and from multiple source files (until now, usually we have been using only one source file) then the inline function is put in a header file. That header file can be used (by using #include) by multiple source files later.

Also keep in mind that after multiple files include the header file that contains the inline function, all of those files must be recompiled after the inline function in the header file is changed.

Now, we are going to cover exciting part of this lecture i.e., Function Overloading.

© Copyright Virtual University of Pakistan

Function Overloading

You have already seen overloading many times. For example, when we used **cout** to print our string and then used it for **int**, **long**, **double** and **float** etc.

```
cout << "This is my string";
cout << myInt ;
```

This magic of **cout** that it can print variables of different data types is possible because of overloading. The operator of cout (<<) that is stream insertion operator is overloaded for many data types. Header file **iostream.h** contains prototypes for all those functions. So what actually is overloading?

"Using the same name to perform multiple tasks or different tasks depending on the situation."

cout is doing exactly same thing that depending on the variable type passed to it, it prints an **int** or a **double** or a **float** or a **string**. That means the behavior is changing but the function **cout** << looks identical.

As we all know that computers are dumb machines and they cannot decide anything on their own. Therefore, if it is printing variables of different types, we have to tell it clearly and separately for each type like **int** or **double** etc. In this separately telling process, the operator used is the same <<. So in a way that operator of << is being overloaded. For this lecture, we will not go into the detail of operator overloading but we will limit our discussion to function overloading.

Function overloading has the same concept that the name of the function will remain same but its behavior may change. For example, if we want to take square root of a number. That number can be an integer, float or a double and depending on the type of the argument, we may need to do different calculation. If we want to cater to the two data types **int** and **double**, we will write separate functions for **int** and **double**.

```
double intsqrt ( int i );
double doublesqrt ( double d );
```

We can use the function **intsqrt()** where integer square root is required and **doublesqrt()** where square root of double variable is required. But this is an overhead in the sense that we have to remember multiple function names, even if the behavior of the functions is of similar type as in this case of square root. We should also be careful about auto-widening that if we pass an **int** to **doublesqrt()** function, compiler will automatically convert it to **double** and then call the funtion **doublesqrt()**. That may not be what we wanted to achieve and there is no way of checking that we have used the correct function. The solution to this problem is function overloading.

While overloading functions, we will write separate functions for separate data types but the function name will remain same. Return type can be different if we want to change, for example in the above case we might want to return an **int** for square root function for **int**s and **double** for a square root of a **double** typed variable. Now, we will declare them as under:

```
int     sqrt ( int i );
```


© Copyright Virtual University of Pakistan

```
double  sqrt ( double d );
```

Now, we have two functions with the same name. How will they be differentiated inside the program?

The differentiation comes from the parameters, which are passed to these functions. If somewhere in your program you wrote: **sqrt(10.5)**, the compiler will automatically determine that **10.5** is not an integer, it is either **float** or a **double**. The compiler will look for the **sqrt()** with parameter of type **float** or a parameter with type as **double**. It will find the function **sqrt()** with double parameter and call it. Suppose in the subsequent code, there is a call to **sqrt()** function as under:

```
int  i;
sqrt ( i );
```

Now, the compiler will automatically match the prototype and will call the **sqrt()** with **int** as parameter type.

What is the advantage of this function overloading?
Our program is more readable after using function overloading. Instead of having lot of functions doing the same kind of work but with different names. How does the compiler differentiate, we have already discussed that compiler looks at the type and number of arguments. Suppose there are two overloaded functions as given below:

```
int  f( int x, int y );
int  f( int x, int y, int z );
```

One function **f()** takes two **int** parameter and other one takes three **int** type parameters. Now if there is call as the following:

```
int x = 10;
int y = 20;
f( x, y );
```

The function **f()** with two int parameters is called.
In case the function call is made in the following way:

```
int x = 10;
int y = 20;
int z = 30;
f( x, y, z );
```

The function **f()** with three **int** parameters is called.

We have not talked about the return type because it is not a distinguishing feature while overloading functions. Be careful about it, you cannot write:

```
int     f ( int  );
double  f ( int );
```

The compiler will produce error of ambiguous declarations.

© Copyright Virtual University of Pakistan

So the overloaded functions are differentiated using type and number of arguments passed to the function and not by the return type. Let's take a loop of some useful example. We want to write functions to print values of different data types and we will use function overloading for that.

```
/* Overload functions to print variables of different types */

#include <iostream.h>

void print (int i)
{
        cout << "\nThe value of the integer is: " << i;
}
void print (double d)
{
        cout << "\nThe value of the double is: " << d;
}

void print (char* s)
{
        cout << "\nThe value of the string is: " << s;
}


main (void)
{
        int i = 100;
        double d = 123.12;
        char *s = "This is a test string";

        print ( i );
        print ( d );
        print ( s );
}
```

The output of the program is:

```
The value of the integer is: 100
The value of the double is: 123.12
The value of the string is: This is a test string
```

You must have noticed that automtically the **int** version of **print()** function is called for **i**, **double** version is called for **d** and **string** version is called for **s**.
Internally, the compiler uses the **name mangling** technique to generate a unique token that is assigned to each function. It processes the function name and its parameters within a logical machine to generate this unique number for each function.

© Copyright Virtual University of Pakistan

Example of Function Overloading

```
/* The following example replaces strcpy and strncpy with the single function name
      stringCopy. */

// An overloaded function
#include <iostream.h>
#include <string.h>

inline void stringCopy( char *dest, const char *src )
{
   strcpy( dest, src );          // Calls the standard C library function
}
inline void stringCopy( char *dest, const char *src, int len )
{
   strncpy( dest, src, len );  // // Calls another standard C library function
}

static char stringa[20], stringb[20];  // Declared two arrays of characters of size 20

void main()
{
   stringCopy( stringa, "That" );        // Copy the string 'That' into the array stringa
   stringCopy( stringb, "This is a string", 4 );  // Copy first 4 characters to stringb array
   cout << stringb << " and " << stringa;  // Display the contents on the screen
}
```


© Copyright Virtual University of Pakistan

Lecture No. 26

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 6

6.5, 6.7, 6.8, 6.10, 6.11, 6.14

Summary

- Classes and Objects
- Definition of a class
- Separation of Interface from the Implementation
- Structure of a class
- Sample program
- Constructor
- Default arguments with constructors
- Tips

Classes and Objects

In today's lecture, we will try to learn about the concepts of 'classes' and 'objects'. However, we are not going to formally cover the object-oriented programming but only the ways to manipulate the classes and objects.

We had talked about structures in our previous lectures. In structures, some data variables are gathered, grouped and named as a single entity. Class and structure are very closely related. In classes, we group some data variables and functions. These functions normally manipulate these variables.
Before going ahead, it is better to understand what a class is:

"A class includes both data members as well as functions to manipulate that data"

These functions are called 'member functions'. We also call them methods. So a class has data (the variables) and functions to manipulate that data. A class is a 'user defined' data type. This way, we expand the language by creating a new data type. When we create variables of a class, a special name is used for them i.e. Objects.

"Instances of a class are called objects"

With the definition of class, we have a new data type like int, char etc. Here *int i;* means 'i' is an instance of data type *int*. When we take a variable of a class, it becomes the instance of that class, called object.

© Copyright Virtual University of Pakistan

Definition of a class

Let's have a look on the structure of a class. It is very similar to the struct keyword. Keyword class is used and the braces enclose the definition of the class i.e.

```
class name_of_class{

        // definition of class
}
```

The new data type i.e. classes helps us to have grouped data members and member functions to manipulate the data. Consider a structure of Date having data members i.e. year, month and day. Now we can declare a variable of structure Date and use dot operator to access its members i.e.

```
Date myDate;
myDate.month=3;
```

We have to use the name of the object, a dot operator and the data member of structure to be accessed. The data members are of normal data types like int, float, char etc. Other data types can also be used.

Let's consider an example of Date Class shown in the following statement.

```
class Date{

        int Day;
        int month;
        int year;
};
```

Now we will take its object in the fashion given below:

```
Date myDate;
```

Separation of Interface from the Implementation

To access the data members of the class, we will again use dot operator. Before going ahead, we will see what is the difference between struct and class. It is the visibility of the data members that differentiates between struct and class. What does the word 'visibility' mean? Consider an example of payroll system. We have stored the tax rate i.e. 5% in a variable *i* of type int. Later, we used the same *i* in a loop and changed the value of tax rate unintentionally. Now the calculation of the pay in the end will not provide the correct results. To avoid this problem, we can tag the tax rate variable as *int tax_rate*;. But this variable again is visible in the whole program and anyone can change its value. The data is open and visible to every part of the program, creating a big problem.

In normal programming, we will like to see the data encapsulated. It means that data is hidden somewhere. However, it can be used. Let's consider a real world problem to understand it. Most of us have wrist-watches. To have accuracy, it is necessary to adjust the time. How can we do that? We can change the time by using the button that is provided on one side of the watch. This is a kind of encapsulation. We can see the

© Copyright Virtual University of Pakistan

hands of the watch but cannot touch them. To change their position we used the button. Whenever we talk about the class, we have to think of this concept that data is available somewhere. We don't need to know about the exact structure i.e. what is inside the watch. All we know is that its internal structure is defined somewhere that cannot be seen or touched. We can only see its interface. If we need to adjust the time, a button may be used. It is a nice separation of implementation and interface. Classes allow us to do that.

Structure of a class

Let's have a look inside a class. Consider the example of class Date. Can we set the values of the data members of the object 'myDate' i.e. day, month or year. We cannot say like *myDate.month = 11*;. Try to do this. The compiler will give error and stop compiling the program. It will not recognize the variable 'month'. In other words, it cannot see 'month'. The default visibility for the data members of the class is called 'private'. These can only be used within the class and are not visible outside.

"The default visibility of all the data members and member function of a class is
hidden and private"

'private' is also a keyword. What will be the opposite of the private? What we will have to do to use the data members and manipulate them. The keyword for this purpose is public. In the class definition, if you do not mention the visibility and start defining the data and functions, these will be by default private. As a good programmer, we should always write the keyword private with a colon as:

```
private:
```

Now all the data and functions following this statement will have the private visibility. To define the public data, we need to write the keyword public with a colon as:

```
public:
```

Now all the data and functions following the public keyword will have the public visibility. These will be visible from outside the class. We can have multiple public and private parts in the class definition but it becomes confusing. So normally we have only one public and one private part. Again consider the Date example. By making the data members as private, we will write functions to set and get the date. As this is needed to be visible from outside the class, these functions will be defined as public.

```
class Date
{
   private:
                  // private data and functions
   public:
                  // public data and functions
};
```


© Copyright Virtual University of Pakistan

Normally, the data in a class is kept private. If we make the data public, it is same as structure and anyone can access this data. On the other hand, the functions which we have written to manipulate this data, are kept as public. These methods can be called from outside the class i.e. from the main program. These are the member functions of the class. The difference between these and the ordinary functions is that they are part of class. Moreover, they can see the private data members of the class and also manipulate them.

We have made the data members private in the Date class. In the program, we take an object of Date class as *Data myDate*;. *myDate* is a variable of type Date. Now if we say *myDate.month = 3;* this statement will be illegal as the *month* is a private data member of the Date class. Now try to understand this concept. You can think class as a box having different things in it. How can we touch inside the box? We have a window and can see only those things that are visible through this window. Those things which we cannot see from the window, can not be accessed from outside. Day, month and year are somewhere inside the box and are not visible through the window. Now we want to assign some values to these data members. For this purpose, we will define a member function in the class in the public section. Being present in public section, it will be visible through the window. As this is the member function, it can see and manipulate the private data of the class. Now it's a two-step process. We can see the public functions and public functions can view the private data members of the class. We will write a function to set the value to the month. We cannot write it as *myDate.month = 10*;. So our function prototype will be as:

```
void setMonth(int month)
```

and we may call this function as:

```
myDate.setMonth(10);
```

Now the function *setMonth* will assign the value 10 to *month* data member of the object *myDate*. The same thing will be applicable if we want to print the date. We can write a public function print and can access it as:

```
myDate.print();
```

The function print can see the private data members. So it will format the date and print it. In structures, the data members are public by default. It means that these are visible to all and anyone can change them. Is there any disadvantage of this? Think about the date. What may be the valid values of the day? Can we have a day less than zero or greater than 32. So the minimum and maximum values of the day are 1 and 31 respectively. Similarly, in case of month, the minimum and maximum values may be 1 and 12. We can assign different values to year like 1900, 2002 etc. If we are using Date structure instead of a class, we can write in the program as *myDate.month=13;* and the month will be set to 13. So the date will become invalid. We may want that other programmers also use this structure. But other programmers may put invalid values to the data-member as these are publicly accessible. Similarly in structures, everything is visible i.e. what are the names of the data members. How are these manipulated?

© Copyright Virtual University of Pakistan

Now we want that only those things should be visible which we want to show and those things which we want to hide should not be visible We can get this by using the private and public in the classes. Public becomes the interface of the class, what we want to show to others. With the use of public interface, the objects can be manipulated. Private becomes the inside of the class i.e. the data members, the implementation. We don't want to show the implementation of our classes to others. This is the concept of separation of interface from implementation. It is a crucially important concept in modern programming. We have separated the interface from the implementation. As long as the interface remains the same, the implementation can be changed. Let's think about it in real world. The example from the automobiles sector can help us understand further. The production of cars in the world started in the late 18th century and early 19th century. Let's compare these early or prototype cars with today's modern ones. There is a big difference between the old and new cars. Technology has changed. Now what is still common in both the types. Steering, clutch, brakes and accelerator pads are still the basic components of a car. So the interface is same. The internal functionality can be changed. To turn the car, old cars used rod mechanisms and modern cars have the microprocessor to do this job. Our physical action is same in both the cases. The interface i.e. steering is same and also the effect that wheels have turned to right is the same too. The internal implementation has completely changed. The old combustion engine cannot be compared with the state-of-the technology based modern engines. But the interface is the same i.e. we turn the key to start an engine. This concept of separation of implementation from interface comes into our programming. We have written a program today to calculate the orbital time of moon around the earth. In today's physics, we have formula to calculate this. We have defined the interface *calculateOrbitalTime()*. This is a function that will calculate the orbital time of moon around earth. This formula may prove wrong after some time. Now what can we do? Despite the change in the implementation, interface remains the same i.e. the name of the function is same. Now when the program will use this function, it gets the correct result as we have implemented the new formula inside the function. Moreover, the main program does not need to be changed at all. Being a very neat concept, it can be used while dealing with objects and classes.

Sample program

Let's see the example of Date class in detail.

```
class Date
{
    public:
                    void display();
                    Date(int, int, int);
    private:
                    int day, month, year;
};
```

Date is the name of new user defined data type. After the braces, we have written the keyword public. In this section, we will define the interface of the class. We have declared a function *display()* which will print the date on the screen. Another function *Date(int day, int month, int year)* is declared. The name of this function is same as the name of the class, having no return type. This function is called constructor. Then we

© Copyright Virtual University of Pakistan

write the keyword private and define the implementation of the class. Here we have three variables i.e. day, month and year of type int. In the end closing braces and the semi-colon. This is the definition of user defined data type i.e. class. It will not occupy any memory space as it has no data currently. It is the same as we write in case of 'int'. It does not occupy any memory but when we say *int I*, the memory is reserved for *i*. The class is a 'user defined data' type. Now in our program, when we write *Date myDate*; an instance of the class is created i.e. an object. Object reserves space in the memory. Object will have these data members. What about the 'functions'? For a moment, we can say that functions are also in the memory.

We want to use this class in our program and display the date using the *display()* function. We have written the prototype of the *display()* function in the class without defining the *display()* function yet. A special way is used to define these functions. We will write the name of the class, followed by two colons and the name of the function. The rest is same as we used to do with ordinary functions.

```
Date::display()
{
        // the definition of the function
        cout << "The date is " << day << "-" << month << "-" << year <<
endl;
}
```

You might have noted the difference in the first line. The double colon is called scope resolution operator. It resolves the scope and tells that this function belongs to whom. In this case, the (Date::display())) tells that the *display()* function belongs to the Date class. So the scope resolution is required. In a way, consider it as function is defined inside the class. If you have private function, even then the definition mechanism is same. We will define the function outside of the class. Even then it will not be visible as its visibility is private. The way to define the member functions is, class name, double colon, name of the function including arguments and then the body of the function. Can we define the function inside the class? Yes we can. When we write the function inside the class, the compiler tries to treat that function as inline function. As a good programming practice, we define the functions outside of the class. So to make sure that the function belongs to the class, the scope resolution operator is used.

We have so far tried to discuss Date class at a rudimentary level. That is we can create objects of Date class and display the date using its functions. We can do a lot of other things with this class. When we say *int i;* and ask to print its value. The answer is that we have not assigned any value to it yet and don't know what will be there at that memory location. Similarly, when we declare an object of the Date class as *Date myDate*; an object is created. But we don't know about the values of day, month and year. Now if we call its public function *display()* using the dot operator as *myDate.display()*. It will print whatever the value is in the data members. We need functions to set/change the date. Suppose we want to set the day, the month and the year separately. For this purpose, we need three more public functions. We can name these functions as setDay(int), setMonth(int) and setYear(int). These functions may be called inside the program as *myDate.setDay(15)*, *myDate.setMonth(12)* and *setYear(2002)*. These functions will change the value of day, month and year. As these are member functions, so scope resolution operator is being used.

© Copyright Virtual University of Pakistan

```
void Date::setDay(int i)
{
        day = i;
}

void Date::setMonth(int i)
{
        month = i;
}

void Date::setYear(int i)
{
        year = i;
}
```

The question arises, which objects data members are being set. In the *setDay* function, we are assigning value to the *day*. But this *day* belongs to which object. The answer is, we have just defined the function, it is not called yet. The functions are called by the objects, not by the class. When we say *Date myDate*; it means that we have an object of type Date. Now we can say *myDate.setDay(10)*. The value of day of *myDate* object will be set to 10. When we create objects, these will reserve space in memory. Suppose, the objects are date1, date2, date3. These will be created at different memory locations having there own data members. When we call a member function with the object name, this function will manipulate the data of this object. Let's consider the following code snippet to understand it.

```
Date date1, date2, date3;
// Manipulating date1 object
date1.setDay(10);
date1.setMonth(12);
date1.setYear(2002);
date1.display();

// Manipulating date2 object
date2.setDay(15);
date2.setMonth(1);
date2.setYear(2003);
date2.display();
```

We have declared three objects of type Date. All these objects have data members day, month and year. When we call a function, that is defined in class, with some object name, it uses the data of that object which is calling the function. Suppose, when we write *date1.setMonth(12);* it will manipulate the data of object *date1*. Similarly when we say *date2.display()*, the function is defined inside the class. However, it will use the data of *date2* object. Remember that we will always call these member functions by referring to some specific object. We can call these functions with date1, date2 or date3 respectively. We will never call these functions referring to a class that is we cannot say *Date.display();* It is illegal. The functions of getting data from objects and setting data of objects are standard. So we normally use

© Copyright Virtual University of Pakistan

the word 'set' for setting the data and 'get' for getting the data. It is a matter of style. You can call it whatever you want. But it will be a bad idea to name a function print() and it is setting the value of month. It will work but in a very confused manner. If we want to write a function to set month, the logical choice of the function name is *setMonth(int)*. Similarly, *setDay(int)* and *setYear(int)* will be used to set the day and year respectively. If we want to get the values of these data members, the logical choice will be *getDay(), getMonth()* and *getYear()*. The names are self-explanatory. These functions are defined as member functions of the class. They are put in the public section of the class and constitute the public interface of the class. These will be visible from outside the class. Normally they manipulate the data that is hidden inside the class i.e. in the private section of the class. No need to show the working of the functions only its name, argument and the return type is told to the user. User of the class is our program.

Here is the complete code of the Date class.

```
/* A sample program with the Date class. Set methods are given to set the day, month
and year.The date is also diplayed on the screen using member function. */

#include <iostream.h>

// defining the Date class
class Date{
   // interface of the class
   public:
      void display();       // to display the date on the screen
      void setDay(int i);   // setting the day
      void setMonth(int i); // setting the month
      void setYear(int i);  // setting the year

   // hidden part of the class
   private:
      int day, month, year;
};

// The display function of the class date
void Date::display()
{
        cout << "The date is " << day << "-" << month << "-" << year << endl;
}
// setting the value of the day
void Date::setDay(int i)
{
        day = i;
}

// setting the value of the month
void Date::setMonth(int i)
{
        month = i;
```


© Copyright Virtual University of Pakistan

```
}

// setting the value of the year
void Date::setYear(int i)
{
        year = i;
}
// Main program. We will take two date objects, set day, month, year and display the
date.
int main()
{
   Date date1,date2;  // taking objects of Date class

   // setting the values and displaying
   date1.setDay(1);
   date1.setMonth(1);
   date1.setYear(2000);
   date1.display();

   // setting the values and displaying
   date1.setDay(10);
   date1.setMonth(12);
   date1.setYear(2002);
   date1.display();
}
```

The output of the program is:

```
The date is 1-1-2000
The date is 10-12-2002
```

Constructors

We have written a function named *Date(int, int, int)* in our class. This is in the public section of our class. It has no return type, having the name as that of class. Such functions are called constructors. When we create an object by writing *Date myDate;* A function is invisibly called which does something with this object. This function is constructor. If we do not write a constructor, C++ writes a default constructor for us. By and large, we want that the object should be created in a certain state. When our object *myDate* is created its data members-day, month and year have some value. We can initialize these data members with zero or with some specific date. How can we do that? Native data types can be initialized as:

```
        int i;
        i = 10;
OR
        int i = 10;
```

Generally, a constructor initializes the object into a state that is recognizable and acceptable. The default constructor does not take any parameter. We can have many constructors of a class by overloading them. The constructor for Date class is:

© Copyright Virtual University of Pakistan

```
Date(int, int, int);
```

This is the prototype of the constructor that is defined in the class. The definition of constructor is same as we used with the member functions.

```
Date::Date(int theDay, int theMonth, int theYear)
{
        day = theDay;
        month = theMonth;
        year = theYear;
}
```

How can we call this constructor? We know that constructor is automatically called when an object is created. To use this constructor, we will take an object as:

```
Date myDate(1, 1 , 2003);
```

Here two things have taken place. 1) An object is created 2) The data members are initialized. This is happening in the memory at run time. Nothing will happen at compile time. The constructor will be called after the object creation and before the control given back to the program. Here the value of day of the *myDate* object is 1, the value of month is 1 and the value of year is 2003. It has created and initialized an object. Now if we call the *display()* function. These values will be displayed. Constructor is used to initialized an object and put it into a consistent and valid state.

Default arguments with constructors

We can also use the default arguments with the constructors. In the case of Date, normally the days and months are changing and the year remains same for one year. So we can give the default value to year.

```
Date::Date(int theDay, int theMonth, int theYear = 2002)
{
        // The body of the constructor
}
```

Now we have different ways of creating objects of class Date.

```
Date myDate;
```

In this case, the default constructor will be called while the data members remain un-initialized.

```
Date myDate(1, 1, 2000);
```

The constructor will be called and initialized. The day will be 1, month will be 1 and the year will be 2000.

```
Date myDate(1, 1);
```


© Copyright Virtual University of Pakistan

The constructor will be called and initialized. The day will be 1, month will be 1 and the year will be initialized to the default value i.e. 2002.

There are some complications. Constructor is itself a function of C++ and can be overloaded. We can have many constructors. Suppose, we are asked to write the date i.e. 1, 1, 2000. Some of us may write it as 1, 1, 2000. Some will write it as 1/1/2000. A considerable number may write as 1-1-2000. One can write date as 1 Jan. 2000. There may have many formats of dates. It will be nice if we can initialize the object using any of these formats. So we may have a constructor which takes a character string. The date format is '01-Jan-2003'. So the constructor should parse the string. The string before the hyphen is day (i.e. 01) convert it into an integer and assign it to day. Again get the strings before the 2nd hyphen (i.e. Jan), check which month is it (i.e. 1) and assign it to month. Rest of the string is year so convert it into integer and assign it to year. We are doing a lot of horizontal integration here. The good thing is that the rules of simple functions overloading applies to constructors also. The rules of default arguments also apply while we are using default arguments with constructors. The idea is to make the class as friendly as possible for the users. We have two constructors. Of these, one takes three ints and the other takes the date as a character string. We may want to add more constructors. But we don't want to add too many constructors in the class as there is a limit of everything. Within limits and the reasons, provision of two to three alternatives to the users of the class for object creation is nice. May be the program that is using our class, is applying months as character strings. We should provide a constructor that deals with this. We will further explain this subject in the coming lectures. A constructor is a special kind of function having same name as that of a class. It has no return type. Declare it without return type. Constructor can take arguments. The default constructor takes no argument.
Here is the code of the Date class using the different constructors.

```
/*
A sample program with the Date class. Use of constructors is shown here.
*/

#include <iostream.h>
//#include <stdlib.h>

// defining the Date class
class Date{
   // interface of the class
   public:
      void display();       // to display the date on the screen
      void setDay(int i);    // setting the day
      void setMonth(int i);  // setting the month
      void setYear(int i);  // setting the year
      int getDay();         // getting the value of day
      int getMonth();       // getting the value of month
      int getYear();        // getting the value of year

      // Constructors of the class
      Date();
      Date(int, int, int);
```


© Copyright Virtual University of Pakistan

```
    // hidden part of the class
    private:
        int day, month, year;
};

// defining the constructor
// default constructor. setting the date to a default date

Date::Date()
{
    day = 1;
    month = 1;
    year = 1900;
}

// Constructors with default arguments

Date::Date(int theDay, int theMonth, int theYear = 2002)
{
        day = theDay;
        month = theMonth;
        year = theYear;
}

// The display function of the class date
void Date::display()
{
        cout << "The date is " << getDay() << "-" << getMonth() << "-" << getYear()
<< endl;
}
// setting the value of the day
void Date::setDay(int i)
{
        day = i;
}

// setting the value of the month
void Date::setMonth(int i)
{
        month = i;
}

// setting the value of the year
void Date::setYear(int i)
{
        year = i;
}
// getting the value of the day
int Date::getDay()
```


© Copyright Virtual University of Pakistan

```
{
        return day;
}

// getting the value of the month
int Date::getMonth()
{
        return month;
}

// getting the value of the year
int Date::getYear()
{
        return  year;
}


// Main program. We will take three date objects using constructors, and display the
date.
int main()
{
   Date date1, date2(1, 1, 2000), date3(10,12);  // taking objects of Date class

   // displaying the dates on the screen
   date1.display();
   date2.display();
   date3.display();

}
```

The output of the program is:

```
The date is 1-1-1900
The date is 1-1-2000
The date is 10-12-2002
```

Summary

A class is a user defined data type. It has data members and member functions. Normally member functions are called methods. Data members are generally kept as private. The member functions, used to manipulate the data members, are kept public so that these are visible from outside the class. The public part of the class is known as the interface of the class. It may contain data members and functions but normally we put functions as public. The member functions can manipulate the data members (public and private) of the class. Non-member functions can not see or access the private part of the class. We try to separate the implementation of the class from its interface.

Tips

- Explicitly write keyword private in the class definition

© Copyright Virtual University of Pakistan

- Separate the interface and implementation
- The default constructor has no arguments
- Constructor has the same name as of class
- The data members of the class are initialized at runtime
- Initializing the data members in the definition of the class is a syntax error



© Copyright Virtual University of Pakistan

Lecture No. 27

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 6

6.9, 6.10, 6.11, 6.12, 6.13, 6.14

Summary

1) Classes And Objects
2) Constructors
3) Types of Constructors
4) Utility Functions
5) Destructors

Classes and Objects

This lecture is a sequel of the previous discussion on 'Classes' and 'Objects'. The use of 'classes and objects' has changed our way of thinking. Instead of having function-oriented programs i.e. getting data and performing functions with it, we have now data that knows how to manipulate itself. This way, now the programming is object-oriented. It means that our programs revolve around data and the objects. Therefore, it would be nice to have some building blocks for programs so that these can be combined to write a program. We have so far talked about simple variables like integer, double and char, followed by strings of characters and arrays of different data types. But now, in the form of an object, we have a block which knows itself about contents and the behavior. The upcoming discussion will further explain it. We have used *cout* for displaying many things like integers, doubles and strings. Here integer did not know how it is going to display itself. However, *cout* knows how to display an integer on the screen. Now we want to see that an integer should know how to display itself. So it will be a different process. Now the question arises whether it will be good to see an integer knowing how to display itself? For this purpose, we will have to expand the scope of thinking.

While engaged in the process of programming, we try to solve a real-world problem. The real world is not only of integers, floats, doubles and chars, but there are other things like cycles, cars, buildings, schools and people. We perceive all these things as objects. Each object has a behavior associated with it. Consider the example of a man who can talk, walk, sit and stand etc. Similarly, we can think of a vehicle that has many functions. These objects also have attributes. For example, a man has height, weight, color of eyes and hair and so on. These all are his attributes. His actions will be referred as functions or methods. This principle may be applicable to vehicles, aeroplanes and all other real-world things. An aeroplane has attributes like its height, width, number of seats and number of engines etc. These are attributes called data members. Its actions include takeoff, flying and landing. These actions are its functions or methods.

© Copyright Virtual University of Pakistan

In terms of language, the attributes and actions may be equated with nouns and verbs. The verbs are actions which are also called methods in the programming terminology. These methods should be included in the object in such a way that the object itself knows how to achieve this function or method. Now consider this all in terms of data. Is it always in terms of salary, payroll, amount and numbers? Actually, data comes in different varieties. Now a day, a computer is a multimedia equipment. We see that there are not only alphabets and digits displayed on the screen, but also pictures, images, windows, dialogue boxes, color and so many other things. These are numbers, letters and graphics. Other than this, we can see videos on our computer. It is just another type of media. Similarly, we find audio material, which can be played on the computer. Thus in the expanded terms of data, we come across numbers, pictures, audio and video while dealing with multimedia.

Now we think about the concept that an integer should know how to display itself. With the enhanced scope of data, we can also have an audio, which knows how to play itself. The same applies to video.

Class

A class is a way of defining a user-defined data type. In a class, one may find data members and functions that can manipulate that data. In the previous lectures, we have talked about the concept of data hiding i.e. encapsulation that means that the data of a class cannot be accessed from outside. However, it can be done through some defined functions (methods). These are the member functions of the class. To hide the data, we declare it *private*. If a data is private, it will be available only to member functions of the class. No other function outside the class (except friend functions) can access the private data. Normally in a class we divide the private part which is normally what we called implementation of the class, from the functions that manipulate that private data which is called the interface (which is the front end).

The example of a room can help us understand private and public parts of a class. A class is a room having a curtain in its middle. The things behind the curtain (private) are visible to the residents (insiders) of the room. They know about every thing present in the room. When the door opens, the outsiders see only the things in front of the curtain. This is the public interface of the class while behind the curtain is the private interface. A function inside the class (i.e. a member function) can access and manipulate all things in the class. A function outside the class can only access and manipulate its public interface part. A constructor has to be in the public section of the class. There should also be a public interface so that it can be called from outside.

Constructors

Constructor is a special function, called whenever we instantiate an object of a class. If we do not define a constructor function in a class, the C++ provides a default constructor. It is executed at the time of instantiating an object.

To understand the basic function of constructor, we have to go back. While writing c++ Stroustrup noticed that the majority of programming problems, which we call bugs, occur due to the use of uninitialized data. That is, we declare variables and use them without providing them any value. For example, we declare an integer as *int i ;* And it is not initialized with a value like *i= 0; or i = 5;* And then somewhere in the

© Copyright Virtual University of Pakistan

program, we write, say, *j = 2 * i ;*. This is the usage of an uninitialized data variable. This technique has a demerit that despite having no syntax error, it may cause a logical error, which is difficult to find. Thus, initialization of data is a very critical activity. The constructors give us an opportunity to initialize the data members of an object in such a way that when our program gets an object, the data part of the object is in a known state. Being in a valid state, it can be used. The constructors are used to initialize the data members of an object. A class is a user defined data type it does not take space in the memory unless we create an object from it. The constructors create space for data members and put values in them. We want these values to be there when an object is instantiated. Thus initialization is a good reason for using constructors.

Types of Constructors

Compiler Generated Constructor

If a constructor is not defined by the use the compiler generates it automatically. This constructor has no parameter. It does nothing. Although the compiler will create a default constructor for us, the behavior of the compiler-synthesized constructor is rarely what we want. Thus the default constructor provided by the compiler does no initialization for us.

Simple Constructor

We have earlier discussed that we can write a constructor that takes no argument. The user defined constructor, that takes no argument is called a simple constructor. We know that when a compiler generated default constructor is called, it does no initialization. It does not know whether to put a value in data members like day, month in our previous class **Date**. We can avoid this problem by not writing a class without having its constructor.

A simple constructor can do initialization without any need to take any argument. So we can write a constructor of **Date** class like **Date ();**. When we write such a constructor, it automatically assumes the roll of the default-constructor. The compiler will not call the default constructor. Rather, the constructor written by the programmer will be called whenever an object will be instantiated. It is also a good programming practice to provide a default constructor (i.e. a constructor wit no argument).

Parameterized constructors

We may define a constructor, which takes arguments as well. This constructor will be automatically called when the required number of arguments are passed to it. Through this, we can easily assign the passed values to our class data members for that particular object.

In our previous example of class **Date,** we have written a constructor as follows

Date (int, int, int);

This is a parameterized constructor which takes three arguments of type int.

Constructor Overloading

We can provide more than one constructors by using function overloading. The rules for function overloading are that the name of the function remains the same. However, its argument list may be different. There are two ways to change the

© Copyright Virtual University of Pakistan

argument list. It can either vary in the number or type of the arguments. We cannot have two functions with the same number and type of arguments. In such a case, these will be identical. So it will not be function overloading. The function overloading requires the argument list to be different. The same concept of function overloading applies to constructors. If we supply more than one constructor for a class, it can be called one or the other depending on the way of calling.

As the constructor does not return any thing, so it has no return type. It means that the body of the construct function cannot have any return statement. Otherwise, the compiler will give a syntax error.

Explanation of Constructors

The main purpose of the constructor is to initialize the object in such a manner that it is in a known valid state. Consider the example of **Date** class again. In that example, there were three data members i.e. day, month and year of type int. What values will we give to these data variables by default if we create an object of **Date**? There may be any valid date. We can give a value 01 to day, month and 1901 to year or what ever we want. It will be a known state despite being meaningless. We can write a constructor of class **Date** which takes three arguments int day, int month and int year, and puts values in the data members of the object, being created. Now the question arises when does this happen? It happens when we instanciate an object of class **Date** by writing **Date myDate;** When this line executes in the program, some space for 'myDate' is reserved in the memory. This space contains the space for day, month and year variables. Then control goes to the constructor that assigns values to day, month and year. Being a member of the class, the constructor can write values to the data members of the class that is private. .

In C++ language, we can provide default arguments to functions. As a function, the constructor can take default arguments. Suppose we have written a constructor of class date with the arguments by providing default values to its arguments. We can write a constructor as **Date (int day=1, int month=1, int year=1);**
and create an object of class **Date** as

Date myDate;

This creates an object of type Date. Which constructor will be called? A constructor with no arguments or the parameterized constructor in which each argument has given a value? If we provide a constructor which has default values for all the arguments, it will become the default constructor for the class. Two constructors cannot be considered same. So it will be better not to write a constructor Date (); (constructor with no argument) in case of providing a fully qualified constructor (with default values for all arguments).

Now suppose, we want to initialize the **Date** object properly by passing a character string to its constructor. Is it possible to write such a constructor? Yes, we can write such a constructor. This constructor will take date as a string, say, 01-jan-1999. And in the constructor, we can split up this string with 'string manipulation functions' and assign respective values to day, month and year.

Now we recapture the concept of constructors with special reference to their characteristics.

A constructor is a function which has the same name as the class.
It has no return type, so it contains no return statement.

© Copyright Virtual University of Pakistan

Whenever an instance of a class comes into scope, the constructor is executed.
The constructors can be overloaded. We can write as many constructors as we require. At one time, the compiler will call the correct version of the constructor".

Utility Functions

The second issue, we usually come across while dealing with the concepts of 'classes and objects' is that a class has a data on one side (normally private part) and functions on the other (normally public part). The functions (methods) are normally written in public part of the class. Are there functions which are private to a class? Answer is yes. The functions of a class may be of two categories. One category contains the member functions which manipulate the data or extract the data and display it. Through these, we can set and get values to manipulate data. These are the functions which are in public interface of the class and manipulate the data in the object. But sometimes, we need such functions that is the requirement of these member functions. Suppose we write a **setDate** function. This function is given an argument and it does the same thing as done by the constructor. In other words, it sets a value of date. Now that function can be public so that it can be called from outside the class. Now we want that the member functions of the class can call this function. But it should not be called from outside. In this case, we put this function in private section of the class. These functions are called utility functions. These are a utility used by other methods of the class. However, they are not functions, supposed to be accessed from outside the class. So they are kept private.

Destructors

The name of the destructor is the same as that of a class with a preceding tilde sign (~). The ~ and name of the class is written as a single word without any space between them. So the name of the destructor of class **Date** will be **~Date**. The destructor can not be overloaded. This means that there will be only one destructor for a class.
A destructor is automatically called when an object is destroyed. When does an object gets destroyed? When we create an object in a function, this is local to that function. When the function exits the life of the object also comes to end. It means that the object is also destroyed. What happens if we declare an object in the main program? When the main program ends, its objects also comes to end and the destructor will be called.
The destructor is normally used for memory manipulation purposes. Suppose we have such a class that when we create an object of it then its constructor has allocated some memory. As we know that we have to free the allocated memory to ensure its utilization for some other program. The destructor is used normally for this purpose to make sure that any allocated memory is de-allocated and returned to free store (heap).

The destructors can be summarized as the following.

The destructors cannot be overloaded.
The destructors take no arguments.
The destructors don't return a value. So they don't have a return type and no return statement in the body.

© Copyright Virtual University of Pakistan

Now we come to the previously defined class **Date.** Let's see what can we further put in it. We have put in it constructors. We have provided a parameterized constructor without default arguments. So the constructor with no arguments will become default one. We have another constructor with three parameters so that we can pass it the values for day, month and year. There is also provided a destructor. We have written some methods to set day, month and year. These were **setDay, setMonth** and **setYear** respectively. Each one of them takes one parameter, a simple integer. Then we have get functions. The functions **getDay**, **getMonth** and **getYear** return a simple integer. There is also a function **setDate,** which takes three parameters (i.e. day, month and year) and sets them. In set function, we do not simply assign the values to the data members. This can be done through a constructor. Whenever we put data into an object, it is necessary to make it sure that valid values should be stored. For example, if we say **Date myDate ;** and give it values like 35 for day, 13 for month and 2000 for year. The constructor will set these values. But these are invalid values for a date. Here we want that these values should be validated before being assigned to data members. So we write some code for error checking of the values and store only valid values in data members i.e. day, month and year. We do the same thing in set function. Then what is the advantage of using set functions. The set functions are public part of the class and can be called from outside the class and also by the constructor. So write all the code for error checking and to validate the data in set function and call this set function in the constructor. Thus when we create an object of class date, it is written as the following

Date myDate (12,10,2000);

Then an object of **Date** class is created and the constructor of the class that takes three arguments, is executed by passing these three values. In the constructor, we call the set function which sets the values of the data members properly. Thus we get a fine initialization of the data members.

What an Object is ? An object is an instance of a class. When we say an instance that means that this object exists and takes space in the memory. What happens when we create an object i.e. take an instance of the class. A class contains data and methods. Are these methods reproduced for every object? Every object has data of its own as every object is distinct from the other. For example, in case of the date class, there may be objects date1, date2 and date3. These are three different objects having their own value of date. Being distinct objects, they must have distinct space in memory. What about functions inside the class?

Whenever we create an object of a class, the functions of the class take a space in the memory and remain there. There is only one copy of these functions in the memory. The data part of the class takes individual locations in the memory. So if we create three objects of a class, say date, there will be one copy of the functions in the memory at the time of execution of the program. The data will have allocated three spaces in the memory with different values. Now suppose, we want to change the data of date1, there is need of setting month of date1 to 3. So we call *setMonth* function for the object date1. We use dot operator (.) to call the function of an object. We write this as **date1.setMonth(3);** The *setMonth* function is called from the copy of the functions that is in the memory. The object name with dot operator makes sure that the function will operate on the data of that object. Thus only the value of the month of date1 will be set to 3. The values of date2 and date3 will remain untouched. Similarly if we say **date2.setDay(23);** the *setDay* function will be called for object date2 and day of date2 will be set to 23. Thus it is clear that which object calls the function the data of that object is visible to the function and it manipulates only that

© Copyright Virtual University of Pakistan

data. Thus we have not wasted the memory by making separate copy of the functions for each object. All objects of one class share the common functions. On the other hand, every object has its own data space. The overloaded functions and constructors are also found in this single copy and called whenever needed. In the overloaded functions, the appropriate function to be called is resolved by the parameter list (type and number of the arguments to be passed).

In our class **Date,** we need no functionality for the destructor. We write the destructor **~Date** and a cout statement in it. That displays the message 'The object has destroyed' just to demonstrate the execution of the destructor. Similarly we can display a message like 'Date object created' in our constructor function. By this, we can see when the constructor is called. By seeing these messages on the screen we know that the object is being created and destroyed properly. If the constructor function is overloaded, we can put appropriate message in each constructor to know which constructor is called while creating an object. For example in default constructor, we can display a message 'Default constructor is called'.

The following program demonstrates the execution of constructors and destructors. It is the previous example of Date class. It displays appropriate messages according to the constructor called. You will see that the constructor is called depending upon the parameter list provided when the object is being created.

```
/*
A sample program with the Date class. Use of constructors and destructor is shown
here.
A message is displayed to show which one constructor is called
*/

#include <iostream.h>
//#include <stdlib.h>

// defining the Date class
class Date{
   // interface of the class
   public:
      void display();       // to display the date on the screen
      void setDay(int i);    // setting the day
      void setMonth(int i); // setting the month
      void setYear(int i);  // setting the year
      int getDay();          // getting the value of day
      int getMonth();        // getting the value of month
      int getYear();         // getting the value of year

      // Constructors of the class
      Date();
      Date (int, int);
      Date(int, int, int);
      // Destructor of the class
      ~Date ();
   // hidden part of the class
   private:
      int day, month, year;
};
```


© Copyright Virtual University of Pakistan

```
// defining the constructor
// default constructor. setting the date to a default date

Date::Date()
{
   day = 1;
   month = 1;
   year = 1900;
   cout << "The default constructor is called" << endl;
}

// Constructors with two arguments

Date::Date(int theDay, int theMonth)
{
        day = theDay;
        month = theMonth;
        year = 2002;
        cout << "The constructor with two arguments is called" << endl ;
}
// Constructors with three arguments

Date::Date(int theDay, int theMonth, int theYear)
{
        day = theDay;
        month = theMonth;
        year = theYear;
        cout << "The constructor with three arguments is called" << endl;
}
//Destructor
Date::~Date()
{
cout << "The object has destroyed" << endl;
}
// The display function of the class date
void Date::display()
{
        cout << "The date is " << getDay() << "-" << getMonth() << "-" << getYear()
<< endl;
}
// setting the value of the day
void Date::setDay(int i)
{
        day = i;
}

// setting the value of the month
void Date::setMonth(int i)
{
```


© Copyright Virtual University of Pakistan

```
        month = i;
}

// setting the value of the year
void Date::setYear(int i)
{
        year = i;
}
// getting the value of the day
int Date::getDay()
{
        return day;
}

// getting the value of the month
int Date::getMonth()
{
        return month;
}

// getting the value of the year
int Date::getYear()
{
        return year;
}


/* Main program. We will take three date objects using the three constructors
(default, two arguments and three arguments and display the date.
*/
int main()
{
   Date date1, date2(12,12), date3(25,12,2002);  // taking objects of Date class

   // displaying the dates on the screen
   date1.display();
   date2.display();
   date3.display();
}
```

Following is the output of the above program.

```
The default constructor is called
The constructor with two arguments is called
The constructor with three arguments is called
The date is 1-1-1900
The date is 12-12-2002
The date is 25-12-2002
The object has destroyed
```


© Copyright Virtual University of Pakistan

```
The object has destroyed
The object has destroyed
```



© Copyright Virtual University of Pakistan

Lecture No. 28

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 7***
7.6, 7.8

Summary

- Lecture Overview
- Memory Allocation in C
- Memory Allocation in C++
- new Operator and Classes
- Example Program 1
- Classes and Structures in C++
- new Operator and Constructors
- delete Operator and Classes
- Example Program 2
- new, delete outside Constructors and Destructors
- main() Function and Classes
- Class Abstraction
- Messages and Methods
- Classes to Extend the Language
- Tips

Lecture Overview

In the previous lectures, we have been discussing about Classes, Objects, Constructors and Destructors. In this lecture we will take them further while discussing Memory Allocation.

- We'll see how the memory allocation is done in C++, while discussing memory allocation in C?
- How C++ style is different from the C-style of allocation discussed earlier?
- What are the advantages of C++ approach as compared to that of C?

© Copyright Virtual University of Pakistan

Memory Allocation in C

Before further proceeding with the concept of memory, it is better to know what else we can create with classes besides objects.

Recapturing of the concept of 'structures' can help us to move forward. Consider the following statement.

```
struct abc
{
        int integer;
        float floatingpoint;
};
```

We could have declared a structure object as:

```
struct abc xyz;      // Declared an object of structure type
```

and access data members inside structure by using dot operator (".") as:

```
xyz.integer = 2134;
xyz.floatingpoint = 234.34;
```

Similarly, we could have a pointer to a structure object as:

```
struct abc* abcPtr; // Declared a pointer of a structure type
abcPtr = xyz;          // Pointer is pointing to xyz object now
```

We can access the individual data member as:

```
abcPtr->integer = 2134;
abcPtr->floatingpoint = 234.34;
```

We can have pointers to different data structures, similarly, pointer to a class object. Here we are going to discuss about Pointers, Classes and Objects.

Let's start by talking about memory allocation. We introduced few functions of memory allocation in C: **malloc()**, **calloc()** and **realloc()**. Using these functions, memory is allocated while the program is running. This means while writing your program or at compile time, you don't need to know the size of the memory required. You can allocate memory at runtime (dynamically) that has many benefits. The classic example will be of an array declared to store a string. If the length of the actual string is lesser than the size of the array, then the part that remains unoccupied will be wasted. Suppose we declare an array of length **6** to contain student name. It is alright if the student name is let's say **Jamil** but what will happen for the student named **Abdul Razzaq**. This is a case where dynamic memory allocation is required.

In C language, the region of memory allocated at runtime is called **heap**. However, in C++, the region of available memory is called **free store**. We have different functions to manipulate memory in both C and C++.

You know that while using **malloc()**, we have to tell the number of bytes required from memory like:

```
malloc(number of bytes required to be allocated);
```


© Copyright Virtual University of Pakistan

Sometimes, we also do a little manipulation while calculating the number of bytes required to be allocated: i.e.

```
malloc( 10 * ( sizeof(int) ) );
```

The **malloc()** returns a void pointer (**void ***). A pointer that points to a void type of memory. So in order to use this memory, we have to cast it to our required type. Suppose, we want to use it for **int**s. For this purpose, you will cast this returned void pointer to **int *** and then assign it to an **int *** before making its further use. The following code is an example of **malloc()** usage.

```
class Date
{
    public:
        Date( ) ;
        Date(int month, int day, int year);
        ~Date ( ) ;
        setMonth( int month ) ;
        setDay( int day ) ;
        setYear( int year ) ;
        int getDay ( ) ;
        int getMonth ( ) ;
        int getYear ( ) ;
        setDate(int day, int month, int year);
    private:
        int month, day, year;
};

    Date  *datePtr;                                    // Declared a pointer of
Date type.
 int  i;
    datePtr  =  (Date *) malloc( sizeof( Date ) );     //  Used  malloc()  to
allocate memory
 i =  datePtr->getMonth();                          // Returns undefined month value
```

So there is some house-keeping involved during the use of this function. We have to determine the number of bytes required to be allocated and cast the returned void pointer to our required type and then assign it to a variable pointer. Lastly, the memory returned from this function is un-initialized and it may contain garbage.

The contrasting function used to free the allocated memory using **malloc()** is **free()** function. As a programmer, if you have allocated some memory using **malloc()**, it is your responsibility to free it. This responsibility of de-allocation will be there while using C++ functions. But these new functions are far easier to use and more self-explanatory.

© Copyright Virtual University of Pakistan

Memory Allocation in C++

The memory allocation in C++ is carried out with the use of an operator called **new**. Notice that new is an operator while the **malloc()** was a function. Let's see the syntax of **new** operator through the following example.

```
new  int;
```

In the above statement, the new operator is allocating memory for an **int** and returns a pointer of **int** type pointing to this region of memory. So this operator not only allocated required memory but also spontaneously returned a pointer of required type without applying a cast.

In our program, we can write it as:

```
int  *iptr;
iptr  =  new  int;
```

So while using **new** operator, we don't need to supply the number of bytes allocated. There is no need to use the **sizeof** operator and cast the pointer to the required type. Everything is done by the **new** operator for us. Similarly, new operator can be used for other data types like **char**, **float** and **double** etc.

The operator to free the allocated memory using **new** operator is **delete**. So whenever, we use **new** to allocate memory, it will be necessary to make use of 'delete' to de-allocate the allocated memory.

```
delete  iptr;
```

The **delete** operator frees the allocated memory that is returned back to free store for usage ahead.

What if we want to allocate space for any array? It is very simple. Following is the syntax:

```
new data_type [number_of_locations];
```

For example, we want to allocate an array of 10 **int**s dynamically. Then the statement will be like this:

```
int  *iptr;
iptr  =  new  int[10];
```

What it does is, it tries to occupy memory space for 10 **int**s in memory. If the memory is occupied successfully, it returns **int *** that is assigned to **iptr**.

Whenever we allocate memory dynamically, it is allocated from free store. Now we will see what happens if the memory in the free store is not sufficient enough to fulfill the request. **malloc()** function returns **NULL** pointer if the memory is not enough. In C++, **0** is returned instead of **NULL** pointer. Therefore, whenever we use **new** to allocate memory, it is good to check the returned value against **0** for failure of the **new** operator.

© Copyright Virtual University of Pakistan

Remember, **new** is an operator,it is not a function. Whenever we use **new**, we don't use parenthesis with it, no number of bytes or **sizeof** operator is required and no cast is applied to convert the pointer to the required type.

delete operator is used to free the memory when the allocation is done by using **new** as shown below:

```
int  *iptr;
iptr = new int [10]; // Memory for 10 ints is allocated dynamically.
delete   iptr;              // Allocated is freed and returned to the free store.
```

Can we apply the concept of dynamic memory allocation/deallocation while using new/delete with classes and objects? The answer is obviously yes.

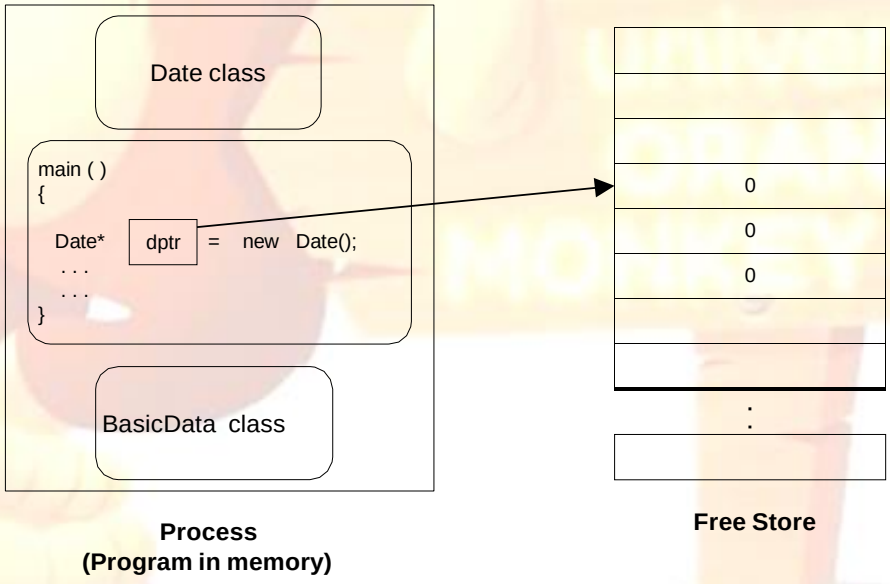
new Operator and Classes

As we declare a pointer to a primitive datatype, similarly, we can have a pointer to a class object.

```
Date   *dptr;    // dptr is a pointer to an object of type Date.
```

Now, we create the object using the **new** operator. Remember, the basic definition of a class i.e. it is a user-defined data type. In other words, the language has been extended to a programmer to have user defined data types. When we use them in our programs, these are used in the same manner as the primitive data types.

```
dptr  =  new  Date;
```



Whatever amount of memory is required for a Date object, is allocated from the free store. A pointer to of Date type is returned back and assigned to the **dptr** pointer variable. Is this all what new is doing? If it is so, can we use **malloc()** function by

© Copyright Virtual University of Pakistan

providing number of bytes required for **Date** object with the help of **sizeof** operator. The answer to this question lies in the further discussion.

```
Date mydate;
cout << sizeof (mydate);
```

As discussed in the last lecture, whenever we instantiate an object of a class, the data members are allocated for each object. However, the member functions occupy a common region in memory for all objects of a class. Therefore, **sizeof** operator returns the size of the data-members storage excluding the member functions part. In the above statement, the **sizeof** operator returns the sum of the sizes of three integers **day**, **month** and **year**, declared in the **Date** class.

The amount of memory allocated in the above statement using **new** (dptr = new Date;) is same as reflected in the following statement:

```
dptr = (Date *) malloc( sizeof(Date) );
```

The **new** operator in the above statement (dptr = new Date;) has automatically determined the size of the **Date** object and allocated memory before returning a pointer of **Date *** type. Is this all what new is doing? Actually, it is doing more than this. It is also creating an object of type **Date**. C functions like **malloc()** do nothing for object creation. Rather these C functions allocate the required number of bytes and return a **void *** pointing to the allocated memory where the memory might contain garbage. But the **new** operator not only allocates the memory after automatically determining the size of the object but also creates an object before returning a pointer of object's class type.

Additionally, within the call to the **new** operator, the memory assigned to the created object with the use of **new** operator can be initialized with meaningful values instead of garbage (think of C functions like **malloc()**).

How the data members are initialized with meaningful values? Actually, a **constructor** is called whenever an object is created. Inside the constructor, individual data members can be initialized. The C++ compiler generates a default constructor for a class if the programmer does not provide it. But the default constructor does not perform any data members initialization. Therefore, it is good practice that whenever you write a class, use a constructor function to initialize the data members to some meaningful values.

Whenever **new** operator is used to create an object, following actions are performed by it:

- It automatically determines the size of the memory required to store that object, leaving no need for the use of **sizeof** operator.
- Calls the constructor of the Class, where the programmers normally write initialization code.
- Returns pointer of the class type that means no casting is required.

Hence, **new** operator is extremely useful, powerful and a good way of allocating memory.

Let's suppose, we want to allocate space for 10 **int**s as under:

© Copyright Virtual University of Pakistan

```
int  * iptr;
iptr  =   new   int [10];
```

This **new** statement allocates contiguous space for an array of 10 **int**s and returns back pointer to the first **int**. Can we do this operation for objects of a class? The answer to this question is yes. The syntax in this case will be identical. To create an array of 10 objects of Date type, following code is written:

```
Date  * dptr;
dptr  =   new   Date [10];
int  day   =   dptr->getDay();
```

Here the **new** operator allocates memory for 10 **Date** objects. It calls the default or parameter-less constructors of the **Date** class and returns the pointer to the first object, assigned to the **dptr** variable. Arrow operators (->) is used while accessing functions or data members from the pointer variable.

Example Program 1

```
/* Following program demonstrates the new operator. This program has the problem of
memory leak because delete operator is not called for the allocated memory. */

#include <iostream.h>

class  MyDate
{
   public:     // public members are below

      /* Parameterless constructor of MyDate class */
      MyDate( )
       {
            cout  <<   "\n Parameterless constructor called ...";
            month   =   day   =   year   =   0;       //  all data member initialized to 0
       }

     /* Parameterized constructor of MyDate class. It assigns the parameter values  to the
……..data members of the class */
      MyDate(int month, int day, int year)
       {
            cout << "\n Constructor with three int parameters called ...";
            this->month  =  month;  // Notice the use of arrow operator ( -> )
            this->day  =  day;
            this->year  =   year;
       }

      /* Destructor of the MyDate class */
      ~MyDate ( )
      {
           cout  <<   "\n Destructor called ...";
```


© Copyright Virtual University of Pakistan

```
}

  /* Setter function for the month data member. It assigns the parameter value to
  the month data member */
void setMonth ( int month )
{
      this->month =  month;
}

 /* Setter function for the day data member. It assigns the parameter value to the
    day data member */
void setDay ( int day )
{
      this->day =  day;
}

 /* Setter function for the year data member. It assigns the parameter value to the
    year data member */
void setYear ( int year )
{
      this->year =  year;
}

/* Getter function for the day data member. It returns the value of the day data
   member */
int getDay ( )
{
      return this->day;
}

/* Getter function for the month data member. It returns the value of the
    month data member */
int getMonth ( )
{
      return this->month;
}

/* Getter function for the year data member. It returns the value of the year data
   member */
int getYear ( )
{
      return this->year;
}

/* A function to set all the attributes (data members) of the Date object */
void setDate ( int day, int month, int year )
{
      this->day =  day;
      this->month = month;
      this->year = year;
```


© Copyright Virtual University of Pakistan

```
    }

  private:              // private members are below
    int month, day,  year;
};

main(void)
{
   MyDate  *dptr;                  // Declared a pointer dptr to MyPointer class object
   dptr  =  new  MyDate [10];      // Created 10 objects of MyDate and assigned the
                                   //  pointer to the first object to dptr pointer variable.

// delete should have been called here before the program terminates.
}
```

The output of this example program is as follows:

```
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
```

Notice that the constructor is called 10 times with 10 **new** calls but there is no call to destructor. What is the reason? The objects are created with the **new** operator on free store, they will not be destroyed and memory will not be de-allocated unless we call **delete** operator to destroy the objects and de-allocate memory. So memory allocated on free store is not de-allocated in this program and that results in memory leak. There is another point to be noted in this example program, which is not relevant to our topics of discussion today that all the functions are requested to be **inline** automatically as the functions are defined within the class body.

Classes and Structures in C++

Structures and classes in C++ are quite similar. C++ structure is declared with the same keyword **struct** as in C. Unlike C structure, C++ structure can have data and member functions. The difference between class and structure is of visibility. Every data member or function written inside the structure is **public** (visible from outside) by default unless declared otherwise. Similarly, everything declared inside a class is **private** (not visible from outside) by default unless declared as **public**.

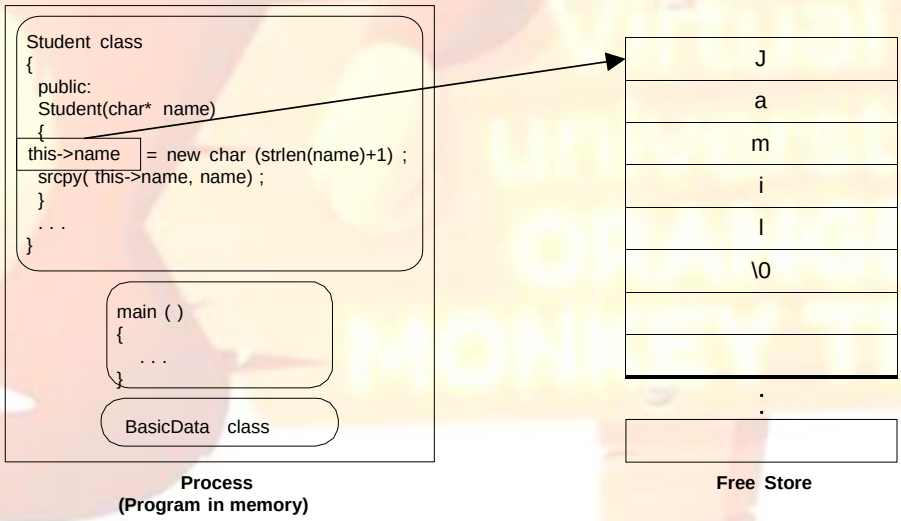
While writing classes, good programming practice is to write **private** keyword explicitly, despite the fact that this is the default behavior. Similarly, while writing structures, it is good to write the **public** keyword explicitly. This averts confusion and increases readability.

© Copyright Virtual University of Pakistan

Another good practice is to write **public** or **private** keywords only once in the class or structure declaration, though there is no syntactical or logical problem in writing them multiple times.
Also remember while writing a class or a structure that once a keyword is written, say **public,** the declarations falling below this keyword will be **public** until the **private** keyword is mentioned.
There is another keyword protected. We are not using this keyword in this course because that deals with inheritance that is a part of Object Oriented Programming, a separate course.

new Operator and Constructors

It is clear that whenever **new** operator is called to create an object, the constructor is also called for that object. What will happen if we have to call new from inside a constructor function. Can we do that? The answer is definitely yes. There are times when we have to do dynamic memory allocation or create new objects from inside a constructor. For example, we have a **Student** class with attributes i.e. **roll number**, **age**, **height** and **name**. The attributes like **roll number**, **age** and **height** can be contained in **int**s or **float**s but the **name** attribute will require a **string**. Because of the nature of this attribute (as it can have different lengths for different students), it is better to use dynamic memory allocation for this. So we will use **new** operator from within the constructor of **Student** class to allocate memory for the **name** of the



student.

We know whenever we use **new** to allocate memory, it is our responsibility to de-allocate the memory using the **delete** operator. Failing which, a **memory leak** will happen. Remember, the memory allocated from **free store** or **heap** is a system resource and is not returned back to the system (even if the allocating program terminates) unless explicitly freed using **delete** or **free** operators.

Now, we will see how the **delete** works for objects and what is the syntax.

© Copyright Virtual University of Pakistan

delete Operator and Classes

As in our **Student** class, as we will be allocating memory from within the constructor of it. Therefore, there is a need to call **delete** to de-allocate memory. What is the appropriate location inside the class **Student** to call **delete** operator to de-allocate memory? In normal circumstances, the location is the destructor of a class (**Student** class's destructor in this case). The destructor is used to de-allocate memory because it is called when the object is no more needed or going to be destroyed from the program's memory. So this is the real usefulness of destructors that these are used to release the system resources including memory occupied by the objects.

As a thumb rule , whenever there is a pointer data member inside our class and pointer is being used by allocating memory at runtime. It is required to provide a destructor for that class to release the allocated memory. A constructor can be overloaded but not a destructor. So there is only one destructor for a class. That one destructor of a class must do house keeping before the object is destroyed. Normal data members **int**, **char**, **float** and **double**, not allocated using **malloc()** or **new** operator, don't need to be de-allocated using **free()** or **delete**. These are automatically destroyed.

Let's be sure that **free()** is used with **malloc()** function while **delete** operator with **new** operator. Normally, **new** will be called in a constructor. However, **delete** will be called in the destructor.

Example Program 2

```
/* Following program demonstrates the new and delete operators. It deallocates the
memory properly before terminating. */

#include <iostream.h>


class  MyDate
{
   public:    //public members are below

     /* Parameterless constructor of MyDate class */
     MyDate( )
      {
            cout  <<  "\n Parameterless constructor called ...";
            month  =  day  =  year  =  0;      //  all data member initialized to 0
      }

    /* Parameterized constructor of MyDate class. It assigns the parameter values  to the
……..data members of the class */
     MyDate(int month, int day, int year)
      {
            cout << "\n Constructor with three int parameters called ...";
            this->month  =  month;  // Notice the use of arrow operator ( -> )
            this->day  =  day;
```


© Copyright Virtual University of Pakistan

```
        this->year = year;
 }

/* Destructor of the MyDate class */
~MyDate ( )
{
      cout << "\n Destructor called ...";
}

  /* Setter function for the month data member. It assigns the parameter value to
  the month data member */
void setMonth ( int month )
{
      this->month = month;
}

 /* Setter function for the day data member. It assigns the parameter value to the
    day data member */
void setDay ( int day )
{
      this->day = day;
}

 /* Setter function for the year data member. It assigns the parameter value to the
    year data member */
void setYear ( int year )
{
      this->year = year;
}

/* Getter function for the day data member. It returns the value of the day data
   member */
int getDay ( )
{
      return this->day;
}

/* Getter function for the month data member. It returns the value of the
    month data member */
int getMonth ( )
{
      return this->month;
}

/* Getter function for the year data member. It returns the value of the year data
   member */
int getYear ( )
{
      return this->year;
}
```


© Copyright Virtual University of Pakistan

```
    /* A function to set all the attributes (data members) of the Date object */
    void setDate ( int day, int month, int year )
    {
        this->day =  day;
        this->month = month;
        this->year =  year;
    }

  private:              // private members are below
    int  month,  day,   year;
};

main(void)
{
   MyDate   *dptr;                  // Declared a pointer dptr to MyPointer class object
   dptr   =   new   MyDate [10];    // Created 10 objects of MyDate and assigned the
                                    //  pointer to the first object to dptr pointer variable.

   delete   [] dptr;                // Deleted (freed) the assigned memory to the objects
}
```

The output of this example program is as follows:

```
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Parameterless constructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
Destructor called ...
```

It is very clear from the output that the destructor for all the objects is called to avert any memory leak. The memory allocated using new operator is being de-allocated using the delete operator. Notice the syntax of **delete** while de-allocating an array, the brackets ([]) precedes the name of the array after the **delete** operator.

© Copyright Virtual University of Pakistan

new, delete outside Constructors and Destructors

Can **new** be called from some location other than constructor? The answer is yes and we usually need to do that. Suppose, we have an object of **Student** class. The name of the student is: **Abdul Khaliq**. So for the **name** attribute, the space is allocated dynamically to store the string **Abdul Khaliq**. When our program is running and we have already allocated space for the **Abdul Khaliq** string using the **new** operator, after sometime, we are required to increase the size of the string. Let's say we want to change the string to **Abdul Khaliq Khan** now.
So what we can do, without destroying this student object:
De-allocate the **name** previously occupied string using the **delete** operator, determine the size of memory required with the help of **strlen()** function, allocate the memory required for the new string **Abdul Khaliq Khan** using the **new** operator and finally assign the returned pointer to the **name** data member.

Hence, we can call **new** and **delete** operators, not only outside the class to create objects but also within the class. The objects of the same class can have different sizes of memory space like in case of objects of **Student** class, student 1 object can be of one size and student 2 object can be of an another size, primarily varying because of string **name**. But independent of this object size, the destructor of the object remains the same and de-allocates memory for different objects regardless of their different sizes. **delete** operator is used from within the destructor to deallocate the memory. We call **delete** operator to determine the size of the memory required to be de-allocated and only provide it a pointer pointing to it.

Please note that C functions like **malloc()** and **free()** functions can also be used from within C++ code. But while writing classes inside C++ code, we prefer to use **new** and **delete** operators as they are designed to work with classes and objects.

main() Function and Classes

We used to discuss about main() function a lot while writing our programs in C. You might have noticed that while discussing about classes and objects, we are not talking about the **main()** function. This does not mean that **main()** function is not there in C++. It is there but it does not contain as much code in C++ . But as you go along and write your own classess, you will realize that almost 90% of your program's code lies inside the class definitions. So firstly we write our classes and **main()** function is written after classes have been defined. That is why the **main()** function is very small. Our example programs clearly depict this fact.

Class Abstraction

Whenever we write a class, we think about its users. Who are the ones going to use this class? The users are not only the **main()** function of the program but also our colleagues around us. Remember, we only expose interface to our users and not the class implementation. All what users need to know is provided in the interface, the

© Copyright Virtual University of Pakistan

methods signatures and what can be achieved by calling that method. The users do not need to know how the functions or interfaces are implemented, what are the variables, how is the data inside and how is it being manipulated, it is abstract to the users.

Messages and Methods

When we create an object, we ask that object to do something by calling a function. This way of asking objects in Windows operating system is called Messaging or in other words function calling is sending a message to the object. Sending a message is a synonym of calling a method of an object. The word 'method' is from the fact that it is a way of doing something. So the whole program is sending messages and getting responses back. It is a different way of looking at things.

Notice lot of things have been repeated in this lecture many times, the reason is that now, you are required to think differently, more in terms of classes and objects. There are lots of exciting things coming up to be covered later.

Classes to Extend the Language

We know that in C, there is no data type for complex numbers. Therefore, we needed to define our own class for complex numbers. We might use **double** data type for **real** and **imaginary** parts. From basic Mathematics, we also know that whenever two complex numbers are added, real part of one complex number is added into the real part of other complex number and imaginary part of one complex number is added into the imaginary part of other complex number. We might write a function for this operation and might call this as **cadd()**. We might also write other functions for multiplication and division. In C++, the operators like '+', '*' and '/' can be overloaded, therefore, we could **overload** these operators for complex numbers, so that we could easily use these ordinary addition, multiplication, and division operators for complex numbers. Actually, we don't need to write this class on our own because this is already been provided in many C++ libraries.

Remember, there is no primitive data type in C++ for complex numbers but a class has been written as part of the many C++ libraries. Moral of the above paragraph is; by using user defined data types i.e., classes, we can now really extend the language.

Tips

- Classes are one way of extending the C++ language.
- Whenever **new** operator is used, no number of bytes or **sizeof** operator is required and no cast is applied to convert the pointer to the required type.
- Whenever **new** operator is called to create an object, the constructor is also called for that object. It is a good practice that whenever you write a class, use a

© Copyright Virtual University of Pakistan

constructor function to initialize the data members to some meaningful values.

- The usual practice is to use constructor to allocate memory or system resources and destructors to de-allocate or return the resources back to the system.

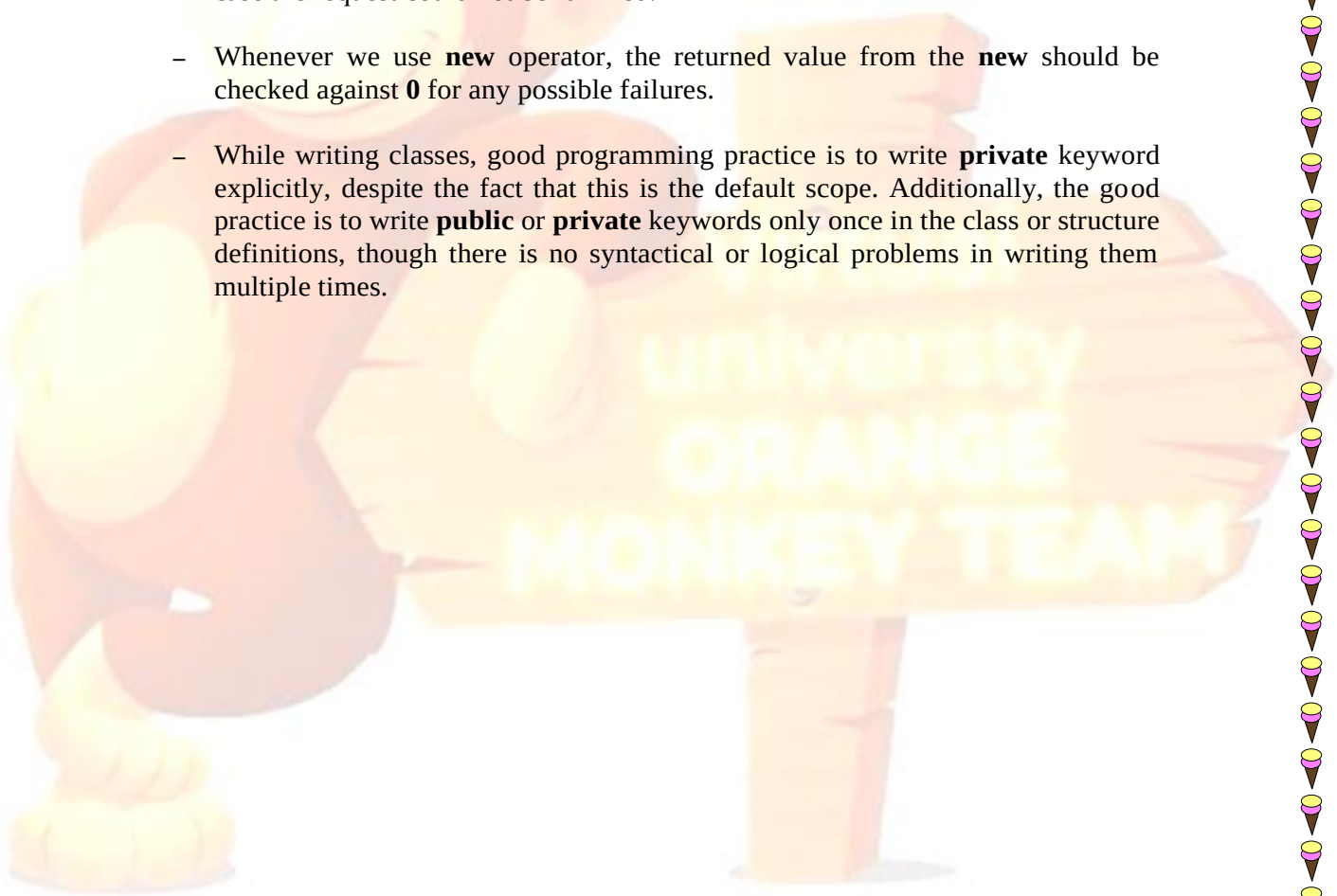
- In C language, the region of memory allocated at runtime is called **heap**. However, in C++, the region of available memory is called **free store**. There are different functions in C and C++ to manipulate memory at runtime. However, all C functions are useable in C++ code.

- The memory allocated from **free store** or **heap** is a system resource and is not returned back to the system unless explicitly freed using **delete** or **free** operators.

- If the memory in the free store is not sufficient enough to fulfill the request, **malloc()** function returns **NULL** pointer. Similarly, the new function returns **0** in case the request could not be fulfilled.

- Whenever we use **new** operator, the returned value from the **new** should be checked against **0** for any possible failures.

- While writing classes, good programming practice is to write **private** keyword explicitly, despite the fact that this is the default scope. Additionally, the good practice is to write **public** or **private** keywords only once in the class or structure definitions, though there is no syntactical or logical problems in writing them multiple times.



© Copyright Virtual University of Pakistan

Lecture No. 29

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 7

7.4

Summary

6) Friend functions
7) Declaration of Friend Functions
8) Sample Program 1
9) Sample Program 2
10) Sample Program 3
11) Friend Classes
12) Summary

Friend functions

Today, we are going to discuss a very interesting subject i.e. Friend Functions. We will see what is the relationship of friendship with our object-based programming. Before going into details of the subject, it is better to have a fresh look on the definition of 'class'. 'Class is a user defined data type'. The 'class' provides encapsulation facility to the programmer. We can gather data at some place and some function that manipulates that data. In the previous lecture, two keywords, 'private' and 'public' were introduced. We define data members as 'private' that are visible only from inside the class and hidden from the outside. However, 'public data member functions' is the interface of the class available for outside world. Objects are accessed by these functions that can manipulate the private data of the class. We cannot access the private data of the class directly. This concept of data encapsulation and data hiding is very important concept in software engineering. It allows us to separate the interface from the implementation of the class i.e. we can hide how we have done the task and make visible what to do. It is critically important for large and complex systems. Sometimes, a need may arise to access the private data of the class from outside.

Let's talk about the concept of friendship. What you see on the screen during the lecture is the picture of the instructor. This is the public interface. That is all you know. What is inside his mind you never know. It is all 'private'. The instructor has access to his own mind and feelings. But you do not have access to that. Do you know any human being who has access to your mind and feelings? What we call that human

© Copyright Virtual University of Pakistan

being. He is known as friend. Normally other people don't know about our thoughts. Only friends know about it. Friends have access to the inner thoughts and have inner knowledge of a friend. Can we apply this definition to objects?

The friend functions of a class have access to the private data members of class. Despite being a good thing, there is possibility of vulnerability. We are opening our thoughts, inside view for somebody else. Without having 100% trust, it will be risky to make our thoughts and feelings public. We want that our private data is accessible to someone outside, not public for everybody. Otherwise, the data encapsulation and data-hiding concept will be violated. We keep the data members private and declare some specific functions that are not member of the class but friend of the class. As friends, they have access to the inside data structure of the class despite not being members.

Declaration of Friend functions

To declare a friend function, we can put it anywhere in the class. According to the definition of the friend functions, they have access to the private data members of the class. These can also access the private utility functions of the class. The question arises where we should put the friend function whether in the private or public part of the class. Be sure that friend is a very strong statement. It is too strong to be affected by public or private. We can put it anywhere in the class. But remember that friend functions are not member of the class. So their definition will be always outside the class. However, the prototype of the function will be written in the class. We use the keyword 'friend' before the prototype of the function.

```
friend    return_type    friend_function_name(int, char);
```

If we have a class, suppose 'Date' and want to declare a friend function of this class. In the definition of the class, we will write the friend function's prototype with the keyword 'friend'. To access the private data, friend function will need the object. Therefore, usually in the parameter list of friend function, we provide the object of that class. Normally, the programmers work this way. As the friend function is not affected by the private or public keyword, so we can declare it anywhere inside the class definition. Programmers generally declare the friend functions at the top of the class definition. So, the friend functions are declared at the start of the class definition, followed by the private data and public data. This is a guideline. You can develop your own style. We normally make a header file of the class definition and implementation in the other file. The member functions are defined in the implementation file and compiled to get an object file. We declare the friend function in the class definition that is in the header file.

Let's go back to the definition of the friendship. I can declare you my friend and tell you about my inner thoughts and feelings. But it does not work both ways. In other words, friendship is granted, never taken. So, a class can declare a friend function and someone from outside the class cannot declare itself friend of a class. This is also an important concept. If someone from outside the class can declare itself friend of the class, then by definition that external function would have access to the private data member of the class. But this will negate the concept of the encapsulation and data hiding. It does not work this way. A function cannot declare itself friend of a class. Rather, a class has to declare itself that a function is friend of the class or not. So the

© Copyright Virtual University of Pakistan

class declares a friend function. These functions can not declare themselves friend of a class from outside. Once, the friend functions are declared and the class is compiled, no one from outside cannot make his function friend of your class. Outside functions can only view the interface of the class.

Let's summaries this concept. Friend functions are not member functions of the class. The class itself declares the friend functions. The prototype of friend functions is written in the definition of the class with the keyword 'friend'. These functions have access to the private data member of the class, which means they have access to everything in the class. Normally we pass an object of the class to these functions in the argument list so that it can manipulate the data of the object. Style is up to you but normally we write friend functions at the top of the class definition.

Sample Program 1

We have a class with a single private data member of type *int*. We have declared a friend function that accepts an object of that class as argument. We call that friend function *increment*. This friend function will increment the private integer data member of the class. We will give another integer argument to that function that will be added to the data member. The name of the private data member is, for example, *topSecret*. Let's call the class as *myClass*. In the interface, we write *display()* function that will print the value of the *topSecret*. The constructor of the class will initialize the *topSecret* with 100. The definition of the friend function will be outside the class. We do not write the keyword 'friend' with the function definition. It will be a void function, having two arguments as:

```
void increment(myClass *a, int i)
{
        a->topSecret += i;
}
```

Now the increment function has added the value of *i* to the private data member i.e. *topSecret* of the passed object. In the main function, we declare an object of type *myClass* as *myClass x;* On the execution of this statement, an object will be created in the memory. A copy of its data members and functions will also be created besides calling a constructor. The place for *topSecret* will be reserved in the memory while the constructor will assign the value 100 to the variable *topSecret*. Now if we say *x.display();* it will display the value of the *topSecret* i.e.100. After this, we call the increment friend function and pass it *&x* and 10 as arguments. Again we call the display function of *myClass* as *x.display();* Now the value of the *topSecret* will be 110. That means the '*topSecret*' which was the private data member of the class has been changed by the *increment* friend function. Be sure that the increment function is not the member function of the class. It is an ordinary function sitting outside the class but class itself has declared it as friend. So now the friend function has access to the private data member and has the ability to change it. Try to write an ordinary function (not friend function) '*increment2*' which tries to manipulate the *topSecret*. See what will happen? The compiler will give an error that a non- member function can not access the private data of the class.

Here is the complete code of the program.

```
/*
```


© Copyright Virtual University of Pakistan

```
A sample program showing the use of friend function,
which access the private data member of the class.
*/

#include <iostream.h>

class myClass
{
        friend void increment(myClass *, int);

        private:
                        int topSecret;

        public:
                        void display() { cout << "\n The value of the topSecret is " <<
topSecret; }
                        myClass();
};
// constructor of the class
myClass::myClass()
{
   topSecret = 100;
}

// Friend function definition
void increment(myClass *a, int i)
{
   a->topSecret += i;    // Modify private data
}

// showing the use of the friend function
void main()
{
        myClass x;
        x.display();
        increment(&x, 10);
        x.display();
}
```

The output of the program is:

```
The value of the topSecret is 100
The value of the topSecret is 110
```

Sample Program 2

Let's consider some complex example. We have two classes-*myClass1* and *myClass2*. Both classes have one private data member of type *int* i.e. *int topSecret;* Now we want to add the values of private data members of both the classes and display it on the screen. *topSecret* is a private data member of both the classes. One class can not see inside the other class. *myClass1* and *myClass2* are both separate classes. We need a

© Copyright Virtual University of Pakistan

function sitting outside the classes but can access the private data members of both the classes. Let's call the function as *addBoth*. This function will add the value of *topSecret* of *myClass1* to *topSecret* of *myClass2* and display the result on the screen. We need a function that can look inside both classes i.e. friend of both classes. We know that classes have to declare a function as friend.

The arguments of *addBoth* function will contain *myClass1* and *myClass2*. In the definition of the *myClass1*, we will write the prototype of *addBoth* function as:

```
friend void addBoth(myClass1, myClass2);
```

Can we write this line in the definition of the *myClass1*? We know that if we refer some function as *f(x)* and the function *f()* is not defined or declared before this, the compiler will give an error that function *f()* is not defined. So we at least declare the function before *main()* so that compiler successfully compile the program. So there was declaration of the function before its being called. Now same problem is in our friend function prototype. We are referring both classes in it and our program does not know anything about *myClass2*. We can tackle this problem by writing a line before the definition of the class *myClass1* as:

```
class myClass2;
```

It will declare that *myClass2* is a class having its definition somewhere else. It is the same as we declare functions before *main*. After writing that statement, we can refer *myClass2* in our code. The definition of the class *myClass1* will be as:

```
class  myClass1
{
        private:
                int topSecret;

        public:
                void display() { cout << "\nThe value of the topSecret is " <<
  topSecret; }
                myClass1();

        friend void addBoth(myClass1, myClass2);
};

myClass1::myClass1()
{
        topSecret = 100;
}
```

The definition of *myClass2* is also similar to *myClass1*.

```
class  myClass2
{
        private:
                int topSecret;
```


© Copyright Virtual University of Pakistan

```
        public:
                void display() { cout << "\nThe value of the topSecret is " <<
  topSecret; }
                myClass2();

                friend void addBoth(myClass1, myClass2);
};

myClass2::myClass2()
{
        topSecret = 200;
}
```

You must have noted that we have used the *topSecret* data member in both the classes. Is it legal? Yes it is. There is no problem as one *topSecret* is part of *myClass1* and other is part of *myClass2*. Will there be same problem while declaring the friend function in *myClass2*, i.e. *myClass1* is not known? No. We have already defined the *myClass1*. We have to declare a class only at a time when we are referring to it and it is not defined yet.

In the main program, we will take the object of *myClass1* i.e. *myClass1 a;* The object will be created in the memory and constructor is called to initialize the data members. The value of *topSecret* will be 100. In the next line, we will take the object of *myClass2* as *myClass2 b;* Now *b* is an object of class *myClass2*. The memory will be reserved for it. It has its own data members and the value of *topSecret* will be 200, initialized by the constructor. Now we will display the values of both data members, using *display()* function.

Now we will call the *addBoth(a, b);* As this function is friend of both classes, so it has access to both the classes and their private data members. The definition of *addBoth* function will be as under:

```
        void addBoth(myClass1 a, myClass2 b)
        {
                cout << "\nThe value of topSecret in the myClass1 object is "
<< a.topSecret;
                cout << "\nThe value of topSecret in the myClass2 object is "
<< b.topSecret;
                cout << "\nThe sum of values of topSecret in myClass1 and
myClass2 is " <<                        a.topSecret + b.topSecret;
        }
```

This is an interesting function. Despite not being the member of any class, it can access the data of both the classes. This function is friend of both the classes.

Here is the complete code of the program.

```
/*
A sample program showing the use of friend function,
which access the private data members of two classes.
```


© Copyright Virtual University of Pakistan

```
*/

#include <iostream.h>

class myClass2; // declaring the class for the friend function in myClass1

// definition of the myClass1
class myClass1
{
   // private data members. Hidden
        private:
         int topSecret;

        // interface of the class
        public:
         void display() { cout << "\nThe value of the topSecret is " << topSecret; }
         myClass1();

        // friend function
        friend void addBoth(myClass1, myClass2);
};
// definition of the constructor.
myClass1::myClass1()
{
        topSecret = 100;
}

// Definition of the myClass2
class myClass2
{
   // private data members. Hidden
        private:
         int topSecret;

        // interface of the class
        public:
         void display() { cout << "\nThe value of the topSecret is " << topSecret; }
         myClass2();

        // friend function
        friend void addBoth(myClass1, myClass2);
};

// definition of the constructor.
myClass2::myClass2()
{
        topSecret = 200;
}

// The definition of the friend function which is adding the topSecret data member of both
```


© Copyright Virtual University of Pakistan

```
the classes.

void addBoth(myClass1 a, myClass2 b)
        {
                cout << "\nThe value of topSecret in the myClass1 object is " <<
a.topSecret;
                cout << "\nThe value of topSecret in the myClass2 object is " <<
b.topSecret;
                cout << "\nThe sum of values of topSecret in myClass1 and
myClass2 is " << a.topSecret + b.topSecret;
        }

// main program
void main()
{
        // declaring the objects and displaying the values
        myClass1 a;
        myClass2 b;
        a.display();
        b.display();
        // calling friend function and passing the  objects of both the classes
        addBoth(a, b);
}
```

The output of the program is;

```
The value of the topSecret is 100
The value of the topSecret is 200
The value of topSecret in the myClass1 object is 100
The value of topSecret in the myClass2 object is 200
The sum of values of topSecret in myClass1 and myClass2 is 300
```

The classes have defined and declared this function *addBoth* to be a friend. In each class, we have declared it as a friend function. This function cannot declare itself a friend function for these classes from outside. So be careful about this as a class declares its friend functions. A function out side the class cannot declare itself a friend of the class. The friend functions are not used very often.

Sample Program 3

Now we can expand our previous example. We can define functions *subBoth*, *mulBoth* and *divBoth* as friend functions of the class, in addition of *addBoth* function. These friend functions can manipulate the data members of the class.

Following is the code of the example that shows the usage of friend functions.

```
/* The following program demonstrate the declaration and uses of friend functions of
a class
We set values in the constructors of the classes. The program prompts the user to
enter a choice of addition, subtraction, multiplication or division. And then performs
the appropriate
```


© Copyright Virtual University of Pakistan

```
operation by using the friend functions.
*/

#include <iostream.h>
#include <stdlib.h>

class myClass2;          // declaration of the myClass2 for the friend functions

class myClass1
{
    private:
        float value ;

    public:
        myClass1 ( )
        {
                value = 200 ;
        }

        // friend functions
        friend float addBoth ( myClass1, myClass2 ) ;
        friend float subBoth ( myClass1, myClass2 ) ;
        friend float mulBoth ( myClass1, myClass2 ) ;
        friend float divBoth  ( myClass1, myClass2 ) ;
};

class myClass2
{
    private:
        float value ;

    public:
        myClass2 ( )
        {
           value = 100 ;
        }

        // friend functions
        friend float addBoth ( myClass1 , myClass2 ) ;
        friend float subBoth ( myClass1 , myClass2 ) ;
        friend float mulBoth ( myClass1 , myClass2 ) ;
        friend float divBoth ( myClass1 , myClass2 ) ;
};

void main ( )
{
    myClass1  myClass1Obj ;          //create an object of class myClass1
    myClass2  myClass2Obj ;          //create an object of class myClass2
    char choice;
    cout << "Please enter one of the operator +, -, /, *  " << "followed by Enter " <<
```


© Copyright Virtual University of Pakistan

```
endl;
    cin >> choice;

    if ( choice == '+' )
    {
       cout << "The sum is : " << addBoth(myClass1Obj , myClass2Obj) << endl;
    }
    else if ( choice == '-' )
    {
       cout << "The difference is : " << subBoth(myClass1Obj , myClass2Obj) <<
endl;
    }
    else if ( choice == '*' )
    {
       cout << "The multiplication is : " << mulBoth(myClass1Obj , myClass2Obj) <<
endl;
    }
    else if ( choice == '/' )
    {
      cout << "The division is : " << divBoth(myClass1Obj , myClass2Obj) << endl;
    }
    else
    {
       cout << "Enter a valid choice next time. The program is terminating" << endl;
    }

    system ( "PAUSE" ) ;
}

float addBoth ( myClass1 object1 , myClass2 object2 )
{
    return ( object1.value + object2.value ) ;
}

float subBoth ( myClass1 object1 , myClass2 object2 )
{
    return ( object1.value - object2.value ) ;
}

float mulBoth ( myClass1 object1 , myClass2 object2 )
{
    return ( object1.value * object2.value ) ;
}

float divBoth ( myClass1 object1 , myClass2 object2 )
{
    return ( object1.value / object2.value ) ;
}
```

Following is the output of the program.

© Copyright Virtual University of Pakistan

```
Please enter one of the operator +, -, /, *  followed by Enter
*
The multiplication is : 20000
```

Friend Classes

We have seen that a class can define friend functions for itself. Similarly a class can be declared as a friend class of the other class. In that case, the function of a class gets complete access to the data members and functions of the other class. So it is an interesting expansion of the definition that not only the functions but also a class can be a friend of the other class. The syntax of declaring a friend class is that within the class definition, we write the keyword *friend* with the name of the class. It is going to be a friend class. i.e. *friend class-name*;
We can also write the word class after the keyword friend and before the class name as

friend class class-name ;

Now let's take another example of a class. Suppose, we have classes *ClassOne* and *OtherClass*. We want to make *OtherClass* a friend class of the *ClassOne***.** So we declare *OtherClass* a friend class in the definition of the *ClassOne* as following.

```
class ClassOne
    {
        friend OtherClass ;
        private:
        //here we write the data members of ClassOne
    };
```

The line

```
friend OtherClass ;
```

can also be written as

```
friend class OtherClass ;
```

The line *friend OtherCalss;* explains that *OtherClass* is a friend of *ClassOne*. If *OtherClass* is the friend of *ClassOne*, all the functions of *OtherClass* will have access to all the inside part of *ClassOne*.
The following code segment shows the declaration of friend class. It shows that *OtherClass* is a friend of *ClassOne* so it has access to the private data of *ClassOne*.

```
class ClassOne
{
        friend class OtherClass;
        private:
        int topSecret;
};

class OtherClass
{
        public:
        void change( ClassOne co )
};
```


© Copyright Virtual University of Pakistan

```
void OtherClass::change( ClassOne co )
{
   co.topSecret++;   // Can access private data of class one
}
```

The *friend* keyword provides access in one direction only. This means that while *OtherClass* is a friend of *ClassOne*, the reverse is not true. Here *ClassOne* declares that *OtherClass* is my friend. But it does not work the other way. It does not mean that *ClassOne* has access to the inside data members and methods of *OtherClass*. Thus, it is a one way relationship i.e. the *OtherClass* can look into *ClassOne*, but *ClassOne* cannot look inside *OtherClass*. If we want a two-way relationship, *OtherClass* will have to declare *ClassOne* as a friend class, resulting in a complete two-way relationship.

Like functions, a class cannot declare itself a friend of some other class. A class can declare its friend classes in its declaration and cannot be a friend of other classes by declaring itself their friend. In the above example, *ClassOne* declares that *OtherClass* is my friend class. So *otherClass* can access all the data members and methods (private, public or utility functions) of *ClassOne*. It does not (and cannot) declare that I (*ClassOne*) am a friend class of *OtherClass*. So *ClassOne* has no access to private data members and methods of *OtherClass*. It can access these only if *OtherClass* declares *ClassOne* as its friend. This means that by using the keyword friend, a class gives rights of accessing its data members and methods to other classes and does not get the rights to access other classes.

By declaring friend functions and classes, we negate the concept of data hiding and data encapsulation and show the internal structure of the class to the friends. But the good thing in it is that a class declares its friends while the other functions or classes cannot look inside the class. The disadvantage of friend classes is that if we declare such a relationship of friendship for two classes, this will become a pair of classes. To explain it we go back to the concept of separating the interface and implementation. In case of change in the implementation of *ClassOne*, the private data structure will also change. For example, at first we have an integer variable *int i*; and later, we need two more variables and we write it as *int j, k, l*; As the implementation of *ClassOne* has now changed, the functions of *OtherClass* that wanted to manipulate the members of *ClassOne* will not work now. It is critically important that friend classes should be declared very carefully. When is it necessary? This can be understood by an example from mathematics. We have straight line in math. The equation of straight line is: y = mx + c. Here *m* is the slope of line i.e. the angle which the line makes with x-axis. And *c* is the intercept at y-axis. So if we have to define a straight line, there is need of two numbers i.e. *m* and *c*. Now if we have to define a class *StraightLine*, the private data of it will be *double m, c;* or let's use the names which are self explanatory like *double slope, intercept*; And then in the class, there will be the methods of the class. We can write it as

```
calss StraightLine
{
        //some methods
        private:
                double slope, intercept ;
```


© Copyright Virtual University of Pakistan

```
};
```

Now we can also have another class quadratic that also belongs to mathematics. Suppose, we have a parabola, the equation of which is $y= ax^2 + bx + c$. Where a, b and c, for the time being, are real constants. To define this quadratic equation as class, we have to define the three coefficients a, b and c. The statement will be as under:

```
class Quadratic
{
        //some methods
        private:
                double a, b, c ;
};
```

Now we have two classes i.e. StraightLine and Quadratic. In a mathematical problem, when we have given a parabola (a quadratic equation) and a straight line (straight line equation) and are asked to find the point at which the straight line intersects the parabola. To solve it, we setup equations and solve them simultaneously and find out the result, which may be in three forms. Firstly, there is the line that does not intersect the parabola. The second is that it intersects the parabola at one point (i.e. it is a tangential line) and third may be that the line intersects the parabola at two points.

When we setup these equations, we come to know that here the constants *m, c*(of straight line), *a, b* and *c* of quadratic equation are being used. So from a programming perspective if we had an object l1 of type StraighLine and an object q1 of type quadratic. And wanted to find the intersection of l1 with q1. Now here is a situation where we need either a friend function of both classes, so that it can manipulate the data of both classes, or need to declare both classes as friend classes of each other and then write their methods to find the intersection. Similarly we can have many other examples in which a class may need to look into the other class. But it is not some thing to be done all the time. It should be done only when necessary. Use of friend functions is normally a better idea. Using friend classes means that both the classes are linked with each other. If the code in any one of the class is modified i.e. its implementation is changed, we have to recompile both the classes. Due to change in one class, the other class also needs to be changed, necessitating the compilation of both the classes.

So we have lost the principle of separating the interface from the implementation. Now let's talk about the limitations of this friendship business. Firstly, there is no transitive dependency in friend declarations. Suppose I say *student A* is my friend and being a friend he knows my thoughts and ideas. Now the *student A* says "*student B* is my friend" i.e. *student B* knows thoughts and ideas of *student A*. Does it mean that *student B* is also my friend? Does *student B* knows my thoughts and ideas? The answer is no. As I have not declared *student B* a friend of mine, so he (*student B*) does not know about my thoughts and ideas. The same applies to the *friend* definition for classes. The friendship is not transitive. It is not like 'A is a friend of B and B is a friend of C, therefore A is a friend of C'. It does not work. A has to specifically declare 'B is my friend and C is my friend' to make B and C friends of him. There is no transitive dependency in friend declarations.

© Copyright Virtual University of Pakistan

Secondly, I can declare you to be my friend. This means I have unveiled my thoughts and ideas to you. But I cannot get your thoughts and ideas unless you declare me a friend of yours. So there is no association, which means A saying B is my friend does not imply in any way that A is a friend of B. Here B is a friend of A. But B has to declare 'A' its friend. Thus the *friend* keyword produces one-way relationship.

Summary

The concept of classes allows us to separate implementation from interface.
A class is a user defined data type. In a class, we declare private data members and utility functions so that they cannot be access from outside. Similarly, we declare some parts of the class *public* that become the interface for the class and can be accessed from the outside. These interface methods or public methods can manipulate the data of the class. This is the encapsulation and data hiding.

We have the concept of friend functions. By declaring an external function as a friend function, that function gets the complete access to the inner structure of the class including all private data. When classes need to be interactive, these must be declared friends of each other. Thus we have the concept of friend classes. The use of friend function and class is a useful feature that sometimes we need to use. But we should use it very sparingly and carefully as it basically negates the concepts of encapsulation and data hiding.

The principles of friendship of functions and classes are that the friendship is granted, not taken. So a class declares its friend functions and friend classes. If a class declares another class as a friend, it is not always reciprocal. So declaration and granting of a right is one way. The owner of the right grants it. So the class itself grants the privilege of access to outsider functions or to other classes. It is not transitive. It does not go 'A is a friend of B and B is a friend of C therefore A is a friend of C'. It does not work that way. It is restricted to a one-step relationship. If A is a friend of B, and B is a friend of C. If A wants C to be a friend, it has to declare, "C is my friend".

© Copyright Virtual University of Pakistan

Lecture No. 30

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 3

3.17

Summary

13) Reference data type
14) Example 1
15) Difference Between References and Pointers
16) Dangling References
17) Example 2

Reference data type

Out today's topic is about references. This is a very important topic from the C++ prospective. Today we will see what is a reference, how can we use them. C++ defines a thing by which we can create an alias or synonym of any data type. That synonym is called reference. How do we declare a reference? We declare it by using & operator. Now it is little bit confusing. We have used & as address-of operator and here we are using it for referencing. We will write as

```
int &i;
```

It means that *i* is a reference to an integer. Keep that statement very clear in your mind. It is easier to read from right to left. A reference is a synonym. If we want to give two names to same thing then we use reference. Reference has to be initialized when it is declared. Suppose if we have an integer as *i* and we want to give it second name. We will reference it with *j* as:

```
int &j = i;
```

We declared an integer reference and initialized it. Now *j* is another name for *i*. Does it mean that it creates a new variable? No, its not creating a new variable. Its just a new name for the variable which already exists. So if we try to manipulate *i* and *j* individually, we will came to know that we have been manipulating the same number.

© Copyright Virtual University of Pakistan

Lets take a look at a very simple example. In the main function we take an int variable *i* and then we write *int &j = i;* Now we assign some value (say 123) to *i*. Now display the value of *i* using *cout*. It will show its value as 123. Display the value of *j* using *cout*. We will not use & operator to display the value of *j*. We will only use it at the time of declaration and later we don't need it. The & is not reference operator rather it acts as reference declarator. The value of *j* will be same as of *i* i.e. 123.

```
int i;
int &j = i;
i = 123;
cout << "\n The value of i = " << i;
cout << "\n The value of j = " << j;
```

Now what will happen if we increment *i* as *i++;* and print the values of *i* and *j*. You will note that the value of *i* and *j* both have been incremented. We have only incremented *i* but *j* is automatically incremented. The reason is that both are referring to the same location in the memory. *j* is just another name for *i*.

What is the benefit of reference and where can we use it? References are synonyms and they are not restricted to int's, we can have reference of any data type. We can also take reference of a class. We wrote a function to show the use of pointers. That function is used to interchange two numbers. If we have two integers *x* and *y*. We want that *x* should contain the value of *y* and *y* should get the value of *x*. One way of doing this is in the main program i.e.

```
int x = 10;
int y = 20;
int tmp;
tmp =  y;
y = x;
x = tmp;
```

The values of both *x* and *y* have been interchanged. We can also swap two numbers using a function. Suppose we have a swap function as *swap(int x, int y)* and we write the above code in it, what will happen? Nothing will be changed in the calling program. The reason is call by value. So when the main function calls the function *swap(x, y)*. The values of *x* and *y* will be passed to the swap function. The swap function will get the copies of these variables. The changes made by the swap function have no effect on the original variables. Swap function does interchange the values but that change was local to the swap function. It did not effect anything in the main program. The values of *x* and *y* in the main program remains same.

We said that to execute actual swap function we have to call the function by reference. How we did that. We did not send *x* and *y* rather we sent the addresses of *x* and *y*. We used address operator to get the addresses. In the main function we call swap function as *swap(&x, &y);* In this case we passed the addresses of two integers. The prototype of the swap function is *swap(int*, int*)* which means that swap function is expecting pointers of two integers. Then we swap the values of *i* and *j* using the * notations. It works and in the main program, the values are interchanged. This was a clumsy way. We can use reference in this case with lot of ease. Let us see

© Copyright Virtual University of Pakistan

how we can do that. Lets rewrite the swap function using references. The prototype will be as:

```
swap (int &i, int &j);
```

Swap is a function that is expecting *i* as a reference to an integer and the second argument is *j* which is also a reference to an integer. The calling function has to pass references. What will we write in the body of the function? Here comes the elegance of the references. In the body we will treat *i* and *j* as they are ordinary integers. We will take a temporary integer and interchange the values of *i* and *j*.

```
swap (int &i, int &j)
{
        int  temp;
        temp = x;
        x = y;
        y = temp;
}
```

In the main program, you will see that the values of two integers have been interchange. What is the way to call this function? In the main program, we will call this function as *swap(x, y)*. Its an ordinary call but the function is expecting addresses which is automatically done. The memory locations of the integers are passed and the function is interchanging the original numbers. This is one beautiful example in which we avoided all the cumbersome of pointer notation. What is the downfall of this? As nothing comes for free. In this case when you are reading the main function you will see *swap (x, y)* which seems a call by value. This is a rule of C/C++ that when we pass two variables to some function they are passed by values. You will have to look for the definition of swap function to realize that it is not call by value but is call by reference. Second thing is if we have another swap function, which is receiving two integers. Can we define two functions as *swap(int x, int y)* and *swap(int &x, int &y)*? One function is receiving two integers and other is receiving two references of integers. Can we do that? Types are different so we can overload. Unfortunately not, in the main function the way to call both functions is same i.e. *swap(x, y)*. How does the compiler know that which functions is being called? There is no way for the compiler to find out. Therefore there is an ambiguity and that is not allowed. The only thing to realize is the side effect. Side effects are critical to take care of whenever you are doing call by reference. Here in this example we do want that two numbers should be interchanged. There may be some situation where we want to send the references and don't want that original data should be affected. These situations arise when we want to pass a large data structure to a function. To understand this we have to understand how the function call executes. We have discussed it before, now lets recap it. In real world, suppose I am reading a book. While reading I notice a word which I think I have to look up. I stop reading, markup the page and then look that word in dictionary or encyclopedia. While reading the definition of that word I look another special word which I need to lookup. I put a marker here and go for looking the definition of that new word. Eventually I understand all the words I need to look up. Now I want to go to the point at which I left the study. I close the dictionary or encyclopedia and goes back to the original book which I was studying. Now I understand all the words in my study and continue the study from the point where I

© Copyright Virtual University of Pakistan

left. If we think about it, it was a function call. We were doing some work, suddenly we call a function and stop that work and execution went to the function. When the execution of the function came to end, we came back to our calling function and continued with it. Computers do the same work with stack. So when the program comes back from the function it should know the point at which it lefts. We supposed here a word to look up, now consider that it was a paragraph or essay I am going to look up. Now to lookup that essay in other books I have to take the entire paragraph or essay with me to that book. Think about the stack. On the stack, the original condition of the program (state) has saved. Now we put our essay or paragraph on it and then opened the other book and searched the book for this essay. In this way, we want to explain that the thing we passed to the function from the main was itself a huge/large thing (as we resemble it with paragraph or essay). So there was a big overhead in writing that thing out into a temporary space in memory and then picking it up and looking it up.

We can make this process more efficient. The issue is that in this example we do not want to change the paragraph or essay which we are going to look up. We only want to look it up. We want only to use it but don't want to change its words. Unfortunately the baggage that comes with doing this is that first make a copy of this (essay) then go with this copy and when the work with it ends, leave (through away) the copy and start the original work. This is inefficient.

But if we took the reference of that essay and passed the address of it and went to the function to look it up. There is a danger that comes with the address, that is while looking up that essay I underlined different words and when I came back to original book I saw that these line marks were also there. Thus we passed something by value rather we passed something by reference. By passing the reference, we actually pass the original. Think about it in another way. We go to a library and said the librarian to issue us a book, which we want to take home for study. Suppose, that book is the only one copy available in the library (or in the world). The librarian will not issue the book. Because it is the only copy available in the world. He does not want to issue this original book to someone as someone can marks different lines with a pen and thus can damage the original book. The librarian will do that he will take a photocopy of that book and issue it. Making a photocopy of the book and then take the book is a bothersome work.

Here we don't want to damage the book. We just want to read it. But can I somehow take the original book? Put it in a special polythene bag and give it to you in such a way that you can read it without damaging it. By doing this we get efficiency but danger is still there. This is actually a call by reference. We have the reference (the original book). If we do something to the original book, the library book will be damaged. Can we somehow prevent this from happening? And also have the efficiency of not having to make a copy.

Now come back to the computer world. Suppose we have a data structure. There is a string of 1000 characters in it. We want to pass that data structure to a function. If we pass it by value which is sake, the original structure will not be affected. We first will copy that string of 1000 characters at some place, which is normally made on the stack. Then the function will be called. The function will take the copy of these 1000 characters and will manipulate it. Then it will give the control back to the caller

© Copyright Virtual University of Pakistan

program and will destroy that copy of string. For efficiency, we want that instead of making a copy of this string, its reference should be written. We have been doing this with pointers and addresses. So we write there the address and pass it to the function. How we can prevent the side effects? There may be these side effects with references. So be very careful while using references with function calls.

Can we do something to prevent any changes? The way we do it is by using the *const* key word. When we write the *const* key word with the reference, it means that it is a reference to some thing but we cannot change it. Now we have an elegant mechanism. We can get the efficiency of call by reference instead of placing a string of 1000 characters on the stack, we just put the address of the string i.e. reference on the stack. In the prototype of the function, it is mentioned that it takes a *const*. This is a reference that may be to a char, int, double or whatever but it is a *const*. The function cannot change it. The function gets the address, does its work with it but cannot change the original value. Thus, we can have an efficiency of a call by reference and a safety of a call by value. To implement all this we could have used the key word *const* with an address operator or a pointer but we can use a reference that is an elegant way. There is no need in the function to dereference a reference by using * etc, they are used as ordinary variable names.

Example 1

Now let us have an example. Here we defined a structure *bigone* that has a string of 1000 characters. Now we want to call a function by three different ways to manipulate this string. The first way is the call by value, which is a default mechanism, second is the call by reference using pointers and the third way is call by reference using reference variables. We declared the prototypes of these functions. Here we declared three functions. The first function is *valfunc* which uses a call by value. We simply wrote the value of the structure. The function prototype is as under.

```
void valfunc( bigone v1 );
```

The second function is *ptrfunc* in which we used call by reference using pointers. We passed a pointer to the structure to this function. The prototype of it is as follows.

```
void ptrfunc( const bigone *p1 );
```

The third function is *reffunc* which uses the way of calling by reference using references. We wrote its prototype as

```
void reffunc( const bigone &r1 );
```

Note that we wrote & sign with the name of the variable in the prototype of the function, we will not write it in the call of the function.

In the main program, we called these function. The call to the *valfunc* is a simple one we just passed the name of the object of the structure i.e. *v1*. As the function is called by using the call by value the manipulation in the function will not affect the original value. We wrote it as:

```
valfunc ( bo );
```

In this call a copy of *bo* is placed on the stack and the function uses that copy.

Next we called the function *ptrfunc*. We passed the address of the structure to *ptrfunc* by using the & operator. Here we are talking about the function call (not function prototype) and in function call we write *ptrfunc (&bo) ;* which means we passed the

© Copyright Virtual University of Pakistan

address of *bo* (the object of structure) to the function. The efficiency here is that it writes only the address of the object to the stack instead of writing the whole object.

The call to the third function *reffunc* is simple and looks like the call by value. There is no operator used in this call it is simply written as:

```
reffunc ( bo ) ;
```

Here we cannot overload the valfunc and reffunc, their names must be different. Otherwise the calls look same and become ambiguous.

The pointer call and reference call are sending the references to the original structures so these are dangerous. If we want to prevent the function from changing that then we should define the function by *const* keyword with its argument pointer or reference. Then the function can not modify the original value, it can only read it. So by this we get the efficiency of the call by reference and the safety of the call by value.

The complete code of the example is given here.

```
// Reference parameters for reducing overhead
// and eliminating pointer notation

#include <iostream.h>

// A big structure
struct bigone
{
   int serno;
   char text[1000];  // A lot of chars
} bo = {123, "This is a BIG structure"};

// Three functions that have the structure as a parameter
void valfunc( bigone v1 );                    // Call by value
void ptrfunc( const bigone *p1 );      // Call by pointer
void reffunc( const bigone &r1 );      // Call by reference

// main program
void main()
{
   valfunc( bo );                 // Passing the variable itself
   ptrfunc( &bo );                // Passing the address of the variable
   reffunc( bo );                 // Passing a reference to the variable
  }

//Function definitions
// Pass by value
void valfunc( bigone v1 )
{
   cout << '\n' << v1.serno;
   cout << '\n' << v1.text;
}
// Pass by pointer
void ptrfunc( const bigone *p1 )
```


© Copyright Virtual University of Pakistan

```
{
  cout << '\n' << p1->serno;     // Pointer notation
  cout << '\n' << p1->text;
}
// Pass by reference
void reffunc( const bigone &r1 )
{
  cout << '\n' << r1.serno;      // Reference notation
  cout << '\n' << r1.text;
}
```

Following is the output of the above program.

```
123
This is a BIG structure
123
This is a BIG structure
123
This is a BIG structure
```

Difference Between References and Pointers

The reference in a way keeps the address of the data entity. But it is not really an address it is a synonym, it is a different name for the entity. We have to initialize the reference when we declare it. It has to point to some existing data type or data value. In other words, a reference cannot be NULL. So immediately, when we define a reference, we have to declare it. This rule does not apply to functions. When we are writing the argument list of a function and say that it will get a reference argument, here it is not needed to initialize the reference. This reference will be passed by the calling function. But in the main program if we declare a reference then we have to initialize it. When a reference is initialized, we cannot reassign any other value to it. For example, we have *ref* that is a reference to an integer. In the program we write the line *int &ref = j ;*

Here *j* is an integer which has already been declared. So we have declared a reference and initialized it immediately. Suppose we have an other integer *k*. We cannot write in the program ahead as *ref = k;* Once a reference has defined, it always will refer to the same integer location as *j*. So it will always be pointing to the same memory location. We can prove this by printing out the address of the integer variable and the address of the reference that points to it.

In programming, normally we do not have a need to create a reference variable to point to another data member or data variable that exists, because creating synonym that means two names for the same thing, in a way is confusing. We don't want that somewhere in the program we are using *i* (actual name of variable) and somewhere *ref* (reference variable) for manipulating the same data variable. The main usage of it is to implement the call by reference through an elegant and clean interface. So reference variables are mostly used in function calls.

© Copyright Virtual University of Pakistan

The difference between pointers and references is that we can do arithmetic with pointers. We can increment, decrement and reassign a pointer. This cannot be done with references. We cannot increment, decrement or reassign references.

References as Return Values

A function itself can return a reference. The syntax of declaration of such a function will be as under.

datatype& function_name (parameter list)

Suppose we have a function *myfunc* that returns the reference to an integer. The declaration of it will be as:

int & myfunc() ;

Dangling Reference

The functions that return reference have danger with it. The danger is that when we return a value from such a function, that value will be reference to some memory location. Suppose that memory location was a local variable in the function which means we declare a variable like *int x;* in the function and then returns its reference. Now when the function returns, *x* dies (i.e. goes out of scope). It does not exist outside the function. But we have sent the reference of that dead variable to the calling function. In other words, the calling program now has a reference variable that points to nowhere, as the thing (data variable) to which it points does not exist. This is called a dangling reference. So be careful while using a function that returns a reference. To prevent dangling reference the functions returning reference should be used with global variables. The function will return a reference to the global variable that exists throughout the program and thus there will be no danger of dangling reference. It can be used with static variables too. Once the static variables are created, they exist for the life of the program. They do not die. So returning their reference is all right.

So, never return a reference to a local variable otherwise, there will be a dangling reference. Some compilers will catch it but the most will not. The reason is that the function that is returning a reference has defined separately. It does not know whether the reference is to a global or local variable, because we can do many manipulations in it and then return it. But normally compilers will catch this type of error.

Example 2

Let us look at an example of functions returning references. First, we declare a global variable that is an integer called *myNum* and say it is zero. Then we declare a function *num* that returns a reference to an integer. This function returns *myNum*, the global variable, in the form of reference. So now when there will be a function call, the return of the function will be a reference to the global variable called *myNum*. Now we can write the main function. Here we write *myNum = 100* ; This assigns a value 100 to the global variable. Next we write

```
int i ;
i = num () ;
```

Now a reference to *myNum* is returned. We would want to assign a reference to a reference but we can use it as an ordinary variable. Thus that value is assigned to *i*.

© Copyright Virtual University of Pakistan

Now look at the next line which says *num () = 200 ;* We know that the left hand side of the assignment operator can only be a simple variable name, what we called l-value (left hand side value). It cannot be an expression, or a function call. But here in our program the function call is on left hand side of the assignment. Is it valid? In this case it is valid, because this function called *num* is returning a reference to a global variable. If it returns a reference, it means it is a synonym. It is like writing *myNum = 200 ;* The example shows that it can be done but it is confusing and is a bad idea. We can put a reference returning function on the left hand side of an assignment statement but it is confusing and bad idea.

Following is the code of the example.

```
/*Besides passing parameters to a function, references can also be used to return values from a function */

#include <iostream.h>

int myNum = 0;    // Global variable

int& num()
{
   return myNum;
}
void main()
{
   int i;
   i = num();
   cout << " The value of i = " << i << endl;
   cout << " The value of myNum = " << myNum << endl;
   num() = 200;  // mynum set to 200
   cout << " After assignment the value of myNum = " << myNum << endl;
}
```

Following is the output of the program.

```
The value of myNum = 0
 After assignment the value of myNum = 200
```

The references are useful in implementing a call by reference in an efficient fashion and writing the function very elegantly without using dereference operators.

We use & sign for declaring a reference. In the program code, how do we find out that it is a reference or an address is being taken? The simple rule is that if in the declaration line there is reference symbol (& sign) with the variable name then that is a reference declaration. These will be like *int &i, float &f* and *char &c* etc. In the code whenever we have simply &i, it means we are taking address. So it's a simple rule that when, in the code, we see a data type followed by & sign, it's a reference. And when the & sign is being used in the code with a variable name then it is the address of the variable.

In C and C++ every statement itself returns a value. It means a statement itself is a value. Normally the value is the value of left hand side. So when we write *a = b*; the

© Copyright Virtual University of Pakistan

value of *b* is assigned to *a* and the value of *a* becomes the value of the entire statement. Therefore when we write *a = b = c* ; first *b = c* executes and the value of *c* is assigned to *b*. Since *b = c* is a statement and this statement has the value of *b*. Now *a* takes the value of this statement (which happened to be *b*). So *a = b* also works. Similarly *a + b + c* also works in the same way that the value of *c* is added to *b* and then this result is added to *a*.

What happens when we write *cout << "The value of integer is " << i << endl* ;
Here first extreme right part will be executed and then the next one and so on or the other way. On the screen the "The value of integer is" displayed first and then the value of the *i* and in the end new line. So it is moving from left to right. When *cout* gets the first part i.e. "The value of integer is", this is a C statement. When this will be executed, the sentence "The value of integer is" is displayed on the screen. But what will be its value? That has to do something with the next << part and is needed with this << sign. We know that we need *cout* on the left side of << sign. So actually what happened is when the first part of the statement is executed. When the statement *cout << " The value of integer is"* executed *cout* is returned. The next part is *<< i* and it becomes *cout << i;* the value of *i* is printed and as a result of the statement *cout* is returned again which encounters with *<< endl;* and a new line is inserted on the screen and *cout* is returned as a result of the statement execution. The return of the complete statement remains *cout*. The *cout* is stream, it does not have value per se. The reference to the stream is returned. The same reference which we have discussed today. The same thing applies to operators like +, -, *, /. This will also apply to = (assignment operator) and so on. We will be using lot of reference variables there.

Summary

We have learned a new data type i.e. reference data type. We said that reference is synonym or alias for another type of data. Take int's synonym or double's synonym. In other words, it's the second name of a variable. Then we talk about some do's and dont's. Normally we do not use two names for the same variable. It's a bad idea and leads to confusing the programmer. Then we found the most useful part of using a reference. If we have to implement call by reference with function then using the prototype of the function which is expecting references and it leads to clean programming. You use the names of the arguments without using any dereferencing operator like *. The most useful part is implementing the call by reference. Then we looked at the difference of pointers and references. We cannot increment the reference variable. Arithmetic is not allowed with references but most importantly, reference variables must be initialized when they are declared. This is import. We can declare pointers and later can assign it some value. The use of reference with classes will be covered later. We have also seen a preview of the usage of references. In that preview we have learned new things that every statement itself has some value and that value is returned. Use it or not it's a different issue. We call a function on a single line like *f(x);* may be *f(x)* returns some value and we did not use it. Not a problem. Similarly if we say *a = b;* this statement itself have some value whether we use it or not. Then we see how the cout statement is executed. Every part of the statement returns some value which is the reference to *cout* itself. It becomes the reference to the stream.
How these references will be declared and used? We will cover this with operator overloading. Try to write some programs using references and implement call by reference using references instead of pointers.

© Copyright Virtual University of Pakistan

Tips

- The use of reference data type is the implementation of call by reference in an elegant way.
- We cannot do arithmetic with references like pointers.
- Reference variables must be initialized immediately when they are declared.
- To avoid dangling reference, don't return the reference of a local variable from a function.
- In functions that return reference, use global or static variables.
- The reference data types are used as ordinary variables without any dereference operator.

Lecture No. 31

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 8***

8.2, 8.3, 8.4, 8.6, 8.7

Summary

- Lecture Overview
- What is Operator Overloading and Why is it Required
- Where is it Relevant to Apply
- Operators to Overload
- Restrictions on Operator Overloading
- Examples of Operator Overloading
- Non-member Operator Functions
- Example Program 1
- Example Program 2
- Tips

Lecture Overview

The topic of this lecture is **Operator Overloading**. In previous lectures, we discussed about it a bit while discussing about references. So we will see in detail what is operator overloading, how to overload operators, where it is relevant to apply and what are the restrictions on it.

© Copyright Virtual University of Pakistan

What is Operator Overloading and Why is it Required?

Operator overloading is to allow the same operator to be bound to more than one implementation, depending on the types of the operands.

As you know that there are standard arithmetic operators in C/C++ for addition (+), subtraction (-), multiplication (*) and division (/). We should only use these operators for their specific purposes. If we want to add two **int**s, say **i** and **j**, the addition will take place in the following manner i.e. **i + j**. To add two **double** numbers, we use the same operator and write **d1 + d2**. We may add two floats with the help of the same operator as **f1 + f2**. Similarly other operations of **-**, ***** and **/** on the primitive types (sometimes called as native or built-in types) can be employed. In other words, these operators are already overloaded for primitive types in C++. But these C++ operators cannot be used for classes and their objects. We have to write our own operator functions that can work with objects.

Let's take an example of complex numbers. There are two parts of a complex number i.e. **real** and **imaginary**. As complex numbers are part of mathematical vocabulary, so the mathematical manipulations are done on them like addition, subtraction and multiplication. Suppose, we write our own class for complex numbers named **Complex**, but we can't add two complex numbers **c1** and **c2** as **c1 + c2** because until now we don't know how to write it. Although, we are able to write a function say **cadd()** to serve this purpose.

```
Complex cadd ( Complex c1, Complex c2 ) ;
```

It accepts two complex numbers as parameters and returns back the resultant complex number. But the usage of this function to add two complex numbers is generally clumsy. It gets more cumbersome and complex if we want to carry out cascading operations like

i + j + k . It is better to use the standard operators of **+**, **-**, ***** and **/** as they are more readable and elegant.

Where is it Relevant to Apply?

Firstly, the operator overloading gets relevant whenever there is the application of the mathematical functions of addition, subtraction, multiplication and division. Complex number is one example of it. As discussed earlier, in case of **Date** class, the operators can be effectively used to get the future or past dates.

Secondly, the operators are also used sometimes in case of non-mathematical manipulation. The example of **String** class to manipulate strings help us understand it in a better way. The operator **+** can be used to concatenate two strings. Previously, we used **strcat()** function declared inside **string.h** header file to concatenate two strings. As compared to **strcat()**, the use of **+** to concatenate two strings is definitely easier and more readable. But there is a little bit cost associated with this process of operators overloading.

The cost is involved whenever we overload an operator. We have to write a function and make use of the operator semantics correctly while implementing the function.

© Copyright Virtual University of Pakistan

This means that the function written to overload + operator should do addition or concatenation of strings in case of String objects.

Operators to Overload

There are two types of operators to overload:

1. Unary
2. Binary

Unary operators are the ones that require only one operator to work. Unary operators are applied to the left of the operand. For example, ^, **&**, ~ and **!**.

Binary operators require two operands on both sides of the operator. **+**, **-**, *****, **/**, **%**, **=**, **<** and **>** are examples of binary operators.

The complete list of C++ operators that can be overloaded is as follows:

+	-	*	/	%	^	&
|	~	!	=	<	>	+=
-=	*=	/=	%=	^=	&=	|=
<<	>>	>>=	<<=	==	!=	<=
>=	&&	||	++	- -	-> *	,
[ ]	( )	new	new[ ]	delete	delete[ ]	

The following operators can't be overloaded.

.	:	::	.*	?	sizeof

Let's start with operator overloading mechanism. Consider an object **date** of the **Date** class. The data member **day** can be accessed as follows:

date.day = 2;

In this statement, the **day** data member of the **date** object is accessed and assigned value **2**. This expression (**date.day**) is driven by the object name at left.
Similarly, while using operators, the statement like **a + b** is driven by the object at the left. In this case, + operator function for the object **a** will be called and **b** object is passed explicitly to the + operator function as an argument. The rules of function overloading are applied to the operator overloading. We cannot write two + operator functions with exactly identical parameters. Following the overloading rules, the two operator functions have to be different by the type or number of arguments.

The syntax of the prototype of the overloaded operator function is:

return-type operator *operator-symbol* *(parameter-list);*

operator is the keyword here. An example of this will be as follows:

Complex operator + (Complex &);

© Copyright Virtual University of Pakistan

We sometimes write only operator to refer to the operator function in our discussion.

Restrictions on Operator Overloading

There are some restrictions on operator overloading.

- The operator overloading functions for overloading **()**, **[]**, **->** or the assignment (=) Operators must be declared as class members.
- The arity (number of operands) cannot be changed. If you are overloading an operator that requires two operands e.g. *****. It cannot be used as a unary operator that requires one operand.
- No new operators can be created. Like in Fortran language, we have ****** as 'raise to the power (exponent) operator' but this operator does not exist in C++. Therefore, it can't be overloaded. Hence, only existing operators of C++ are used.
- Overloading can't be performed for the built-in (sometimes called primitive or native) data types. For example, we cannot change how two **int**s are added. That means that operators are overloaded to use with defined data types like classes.
- Precedence of an operator cannot be changed. For example, the ***** has higher precedence than **+.** This precedence cannot be changed.
- Associativity of an operator cannot be changed. If some operator is right associative, it cannot be changed to be left associative.

Examples of Operator Overloading

Let's take the complex number's class **Complex** and define a **+** operator function.

We know that when we write the following line:

```
x = y + z ;
```

y and **z** operands are take part in the addition operation but there is no change in them due to this operation. This is the **+** operator's functionality. The resultant is being assigned to the variable **x**. This is assignment operator's functionality.

Now we will discuss a little bit about the assignment operator as well. Let's say we write the following statement for two complex numbers **c1** and **c2**.

```
c1 = c2 ;
```

Here **c2** is being assigned to **c1**. Will this assignment work when we have not written any assignment operator function for complex number? Apparently, it looks that the statement will produce a compilation error (as there is assignment operator defined by us) but this is not true. Whenever, we write our own class and compile it, the compiler automatically generates a default assignment operator. The default assignment operator makes a member to member assignment. This works fine unless there is a pointer data member inside our class and that pointer is pointing to some data inside memory. For that case (when there is a pointer data member) we have to write our own assignment operator otherwise the default assignment operator works fine for us. That will be discussed in the subsequent lectures.

© Copyright Virtual University of Pakistan

By definition of addition of complex numbers, we know that whenever two complex numbers are added, the real part of one number is added into the real part of other number. Similarly, the imaginary part of one number is added to the imaginary part of the other number. We also know that when a complex number is added to another complex number, the resultant is also a complex number consisting of real and imaginary parts. This addition of real, imaginary parts and return of resultant complex number is the functionality of the + operator function we are going to write.
Another thing to decide for this + operator is whether this operator will be a member operator or a **friend** operator. Normally, operators are member operators but there are situations when they cannot be member operators. In case of member operator, following is the syntax of its prototype:

```
Complex operator + (parameter-list);
```

For member operator, the object on the left side of the + operator is driving this + operation. Therefore, the driving object on the left is available by **this** pointer to + operator function. But the object on the right is passed explicitly to the + operator as an argument.
We can define a member operator as under:

```
1.  Complex Complex :: operator + (Complex c)
2.  {
3.      Complex temp ;
4.      temp.real = real + c.real ;
5.      temp.imag = imag + c.imag ;
6.  return temp ;7. }
```

Let's see this code line by line.

Line 1 indicates that the return type is **Complex**, it is an **operator +** function and it is accepting a **Complex** object by value as an argument.
In line 3, a local **Complex** object is declared, called **temp**.
In line 4, **real** part of the calling object (that is the one, driving) on the left of the + operator is being added to the **real** part of the object **c**, where **c** is passed as an argument.
In line 5, **imag** part of the calling object (that is the one, driving) on the left of the + operator is being added to the **imag** part of the object **c**, where **c** is passed as an argument.
In line 6, the **Complex** object **temp** containing the resultant of + operation is being returned by value.

In our code, we can write something as:

```
Complex c1, c2, c3 ;
. . .
. . .
c3 = c1 + c2 ;
```

In the above statement (c3 = c1 + c2;), c1 is the object that is calling or driving the + operator. **c2** object is being passed as an argument to the + operator. So c1 and c2 objects are added by the + operator and resultant **Complex** object containing the

© Copyright Virtual University of Pakistan

addition of these two numbers is returned back. That returned **Complex** object is assigned to the **c3 Complex** object using the default assignment operator (that is created by the C++ compiler automatically).

What happens if we want to add a **double** number to a complex number (a instance of **Complex**)? Like the following:

```
c3 = c1 + d ;
```

This **+** operation is driven by the **c1** object of **Complex** while double number **d** of type **double** is passed as argument. Therefore, our above written **+** operator is not useable for this operation of addition. We need to overload **+** operator for accepting a parameter of type **double**, i.e. we need to write another operator function. The definition of this newly overloaded **+** operator is:

```
Complex Complex :: operator + (double d)
{
        Complex temp ;
        temp.real = real + d ;      // d is added into the real part
        temp.imag = imag ;
        return temp ;
}
```

By now, you should have noticed that operator overloading and function overloading are quite similar.

When we write the following statement:

```
c3 = d + c1;
```

The operand on the left of **+** operator is a **double** number **d.** Therefore, this **+** operation should be driven by (called by) the **double** number. Until now, we have not written such an operator. Our previously written two **+** operators were driven by the **Complex** object. Operator functions, not driven by the class type objects, are kept as **friend**s to the class. **friend** is the keyword used to declare such functions. A **friend** function to a class also has access to the private members of that class.

```
friend Complex operator + (double d, Complex c)
{
        Complex temp;
        temp.real = d + c.real;    // d is added into the real part of c
        temp.imag = c.imag;
        return temp;
}
```

You might have noticed that all the three overloaded **+** operator functions are accepting and returning variables by value. To make these functions better, we can also use references. So our first member **+** operator's prototype can be rewritten as:

```
Complex& operator + (Complex& c);
```


© Copyright Virtual University of Pakistan

Now this operator function is accepting a complex number **Complex** by reference and returning a reference to the resultant complex number.

As discussed above, in case of assignment, the default assignment operator is used because we have not implemented (overloaded) our own assignment operator ('='). But in case, we want to perform the following operation where the two operands are added and the resultant is assigned to one of them as:

```
c1 = c1 + c2;
```

There is one operator (**+=**) that can be used to do both the operations of addition and assignment instead of doing these operations separately within **operator +** and **operator =.** So we can overload this one operator (**+=**) here to make the code more efficient and reduce our work. Therefore, instead of writing:

```
c1 = c1 + c2;
```

We will write:

```
c1 += c2;
```

We will write our **operator +=** as:

```
void Complex :: operator += ( Complex& c )
{
        real += c.real;
        imag += c.imag;

}
```

Non-member Operator Functions

Now we are much clear that when an operator function is implemented as a member function, the leftmost operator must be a class object or reference to a class object of the operator's class.

When an operator function is implemented as a non-member function, the left-most operand may be an object of the operator's class, an object of a different class, or a built-in type. Now we discuss it in a detailed manner.

We can always write our operators as non-member functions. As a non-member functions, the binary operators like **+** gets both the operands as arguments. One thing to take care of while writing non-member functions that they cannot access the private members of classes. Actually, this is just to this reason that we make those non-member functions as **friend**s to the classes whose **private** data members are required to be accessed. But the question arises, can we write a non-member operator function without making it a **friend** of a class. The answer to this question is yes; If there are **public** member functions to access the private data members of the class then they serve the purpose. In this case of **Complex** class, let's say we have two public member functions:

```
double real( );
double imaginary( );
```

to access the **private** data members **real** and **imag** respectively. Then we can write non-member **operator +** function as:

© Copyright Virtual University of Pakistan

```
Complex operator + (Complex& c1, Complex& c2)
{
        Complex temp;
        temp.real ( c1.real() + c2.real() );
        temp.imaginary ( c1.imaginary() + c2.imaginary() );
        return temp;
}
```

But this non-member operation functions without declaring a **friend** of the class is definitely slower than the member function or a **friend** one. The reason for this is obvious from the code that it is making three additional function calls of **real()** and **imaginary()** for each private data member. Also it is not easy to write as compared to member functions. Therefore, it is recommended to write the member functions for operators instead of non-members.

Let's take an example where the operators are performing a non-arithmetical operation. We are writing a class **String** for strings manipulation as:

```
class String
{
  private :

    char string [ 30 ] ;

  public :

    String ( )
    {
        strcpy ( string , "" ) ;
    }

    void getString ( )
    {
       cout << "Enter the String : " ;
       cin >> string ;
    }

    void displayString ( )
    {
      cout << "The String Is : " << string << endl ;
    }

    // Declaration (prototype) of overloaded sum operator

    String operator + ( String & s ) ;
};
```

We want to write + operator to concatenate two strings. Firstly, we will see the operator's behavior in ordinary context (behavior with primitive variables for example) and try to implement the same behavior for this class. We want to

© Copyright Virtual University of Pakistan

concatenate two strings (two **String** objects) and then assign the resultant string to a new **String** object. Here is how we will write + operator function.

```
String String :: operator + ( String &s )
{
   String temp;                        // Declared object temp of String type
   strcpy ( temp.string , "" );        // Initialized the temp with empty string
   strcat ( temp.string , string );    // Concatenated the driving object's string to //
temp object
strcat ( temp.string , s.string );     // Concatenated the argument's string to the
// temp object
   return   temp;                      // Returned the temp object
}
```

As you might have guessed already, the **String** object on the left will be the one to drive this + operation and the second String object on the left of + will be passed as an argument to this function. Note that we are not doing the error checking here, the size of the resultant string **temp** may increase the array size **30** (the array size defined in the class).

Example Program 1

Rudimentary implementation of a class named **Complex** class to cater complex numbers. A + operator function has been implemented to add two complex numbers.

```
/* This program implements the basic class for complex numbers and demonstrates +
operator function */

#include <iostream.h>

class Complex
{
   private :
      double  real ;    // Real Part
      double  imag ;  // Imaginary Part

   public :
      /* Parameterless Constructor */
      Complex ( )
      {
         cout << "\n Parameterless Constructor called ..." ;
      }

      /* Parameterized Constructor */
      Complex ( double  r, double  i )
      {
         cout << "\n Parameterized Constructor called ...";
         real  =  r ;
         imag  =  i ;
      }
```


© Copyright Virtual University of Pakistan

```
    /* Setter of real data member */
    void  real ( double  r)
    {
       real  =   r ;
    }

    /* Getter of the real data member */
    double  real ( )
    {
         return real ;
    }

    /* Setter of the imag data member */
    void  imaginary ( double  i )
    {
       imag = i ;
    }

    /* Getter of the imag data member */
    double  imaginary ( )
    {
       return   imag ;
    }

    /* A Function to display parts of a Complex object */
    void  display ( )
    {
       cout  <<  "\n\n Displaying parts of complex number ...";
       cout  <<  "\n Real Part : " << real << endl ;
       cout  <<   " Imaginary Part : " << imag << endl ;
    }

    /* Declaration (prototype) of overloaded sum operator */
    Complex  operator  + ( Complex & c2 ) ;

};

Complex  Complex  ::  operator + ( Complex  & c1 )
{
   cout  <<  "\n Operator + called ...";
   Complex  temp ;
   temp.real = real + c1.real ;
   temp.imag  =  imag  +  c1.imag ;
   return  temp ;
}

void main ( )
{
```


© Copyright Virtual University of Pakistan

```
    Complex c1 ( 1 , 2 ) ;    // Consturct an object using the parameterized constructor
    Complex c2 ( 2 , 3 ) ;    // Consturct another object using the parameterized
                              // constructor

    Complex result ; // Construct an object using a parameterless constructor

    result = c1 + c2 ;  // Call the Operator + to add two complex numbers (c1 & c2)
                        // and then assign the result to 'result' object

     result.display ( ) ; // Display the result object contents

}
```

The output of the program is as follows:

```
Parameterized Constructor called ...
Parameterized Constructor called ...
Parameterless Constructor called ...
Operator + called ...
Parameterless Constructor called ...

Displaying parts of complex number ...
Real Part : 3
Imaginary Part : 5
```

The + operator function can be enhanced to return reference of Complex object. We can also implement += operator. += operator and the enhanced operator + are implemented as:

```
Complex &  Complex  ::  operator + ( Complex &  c1 )
{
   real =  real  +  c1.real ;
   imag = imag + c1.imag ;
   return  *this;
}

// Declaration (prototype) of overloaded sum assignment operator definition
Complex & Complex :: operator += ( Complex & c2 )
{
   real += c2.real ;
   imag += c2.imag ;
   return  *this;
}
```

Example Program 2

Rudimentary Implementation of String class to manipulate strings. It uses + operator to concatenate strings.

```
/* This program implements the basic class for strings and demonstrates + operator
function to concatenate two strings*/
```


© Copyright Virtual University of Pakistan

```
#include <iostream.h>
#include  <string.h>

class  String
{
   private :

      char    string [ 30 ] ;  // Array to store string

   public :

      /* Parameterless Constructor */
      String ( )
      {
           strcpy ( string , "" ) ;
      }

      /* Getter function of string */
      void  getString ( )
      {
          cout  <<  "Enter the String: " ;
          cin  >>  string ;
      }

      /* Function to display string */
      void  displayString ( )
      {
         cout  <<   "The String is : "   <<   string   <<   endl ;
      }

      // Declaration (prototype) of overloaded sum operator

      String  operator  + ( String & s ) ;
};


String  String  ::  operator + ( String &s )
{
   String  temp ;
   strcpy ( temp.string , "" ) ;
   strcat ( temp.string , string );
   strcat ( temp.string , s.string );
   return  temp;
}

void main ( )
{
     String   string1 , string2 ;          // Declared two String objects
     string1.getString ( ) ;               // Get string for string1 object
     string2.getString ( ) ;               // Get string for string2 object
```


© Copyright Virtual University of Pakistan

```
    String hold  =  string1 + string2 ; // Concatenate string1 and string2 and store the
                                        // result in hold object
    hold.displayString ( ) ;            // Display the string
}
```

The output of the above program is as follows:

```
Enter the String: Operator
Enter the String: Overloading
The String is : OperatorOverloading
```

Tips

Operator Overloading is quite similar to Function Overloading.

There are two types of operators to overload: unary and binary.

C++ built-in operators work for built-in (primitve) types but for user defined data types, user has to write his/her own operators.

There are some restriction while performing Operator Overloading. For example, only existing C++ operators are overloaded without creating a new one in the language. Also, it should not impact the type, semantics (behavior), arity (number of operands required), precedence and associativity of the operator.

For binary member operators, operands on the left drives (calls) the operation.

Operator functions written as non-members but friends of the class, get both the operands as their arguments.

Operators can be written as non-members and even without making them friends. But this is tedious and less efficient way, therefore, it is not recommended.

© Copyright Virtual University of Pakistan

Lecture No. 32

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 8

8.6, 8.7, 8.12

Summary

18) Recap
19) Overloading Minus Operator
20) Operators with Date Class
21) Unary Operators

Recap

Before further discussing the concept of the 'Overloading', we will recapture the things dilated upon in the previous lecture. It is necessary to know that new operators i.e. new symbols cannot be introduced. Only existing symbols can be overloaded. Overloading of operators is exactly like writing functions. However, one should remain close to the original meaning of the operator. Similarly, it is good not to define something in opposite terms e.g. 'plus operator is doing subtraction or multiplication operator carrying out division'. We can do that but it will ultimately a bad thing for a programmer. It makes our program practically unreadable and mis-interpretable. Under operator overloading technique, the binary and unary operators will remain unchanged that is we cannot make unary operator work as binary operator or vice versa. In the previous lectures, we also came across some concepts in terms of driving force behind the operator, e.g. in case of binary operator, the driving force is left hand operand. We have also studied when to use member operators and non-member operators. Today we continue discussion on 'use of operators'.

Overloading Minus Operator

Let's define minus operator (-) with special reference to the complex class. The process of defining the minus operator is quite similar to that of the plus operator. Let's first understand the action of minus operator. It is a binary operator, having two arguments. In this case, both the arguments will be complex numbers. When we subtract two complex numbers, it always return a complex number. Here the subtraction of complex numbers is defined as, 'subtract the real part from real part and subtract the imaginary part from the imaginary one". So a member operator will look like as under:

© Copyright Virtual University of Pakistan

Complex operator – (Complex c)

As we are defining it as a member operator, only one argument will be passed to it. It is going to be on the right hand side of the minus operator. The left-hand-side will call this as it is already available to this function. In the body, we will declare a temporary Complex number. This means that the real part of this temporary complex number is the difference of the calling Complex number and the Complex number passed as argument i.e.:

tmp.real = real – c.real;

In the next line, we calculate the difference of imaginary part as:

tmp.imag = imag – c.image;

and return the tmp Complex number. By defining, the minus operator does not mean that minus equal operator has also been defined. If we want to overload the minus equal operator (-=), it is necessary to define it. Let's see how the defining process is carried out. Minus equal to operator like the plus equal to operator behaves in the way that the value of calling party (i.e. the complex number which is on the left hand side) will also be changed. So now we will see that the number itself changing the value when it takes part in the minus equal to operator. Again, we will make this a member function. So only one argument will be passed to it. The complex number will be on the right hand side of the minus equal to operator. In the body of the function, there is no need of any temporary complex number as we are going to change the number on the left hand side of the minus equal to operator. We can write it as:

real -= c.real;
imag -= c.image;

Here *c* is the complex number which is passed as an argument. Now the minus equal to (-=) operator, used in the above statements, is an ordinary minus equal to operator for the integers defined by the C++. So this is a classic example of overloading i.e. the operator being overloaded is using the original or basic operator of same type. That is the end of this function. The original number has been changed. We can return its reference. It depends on its usage.

Here is the code:

```
// The minus operator definition
Complex Complex::operator - ( Complex c )
{
   Complex tmp; // defining a temporary var
   tmp.real = real - c.real;
   tmp.imag = imag - c.imag;
   return tmp;
}

// The -= operator definition
Complex Complex::operator -= ( Complex c )
{
```


© Copyright Virtual University of Pakistan

```
    real -= c.real ;
    imag -= c.imag ;
}
```

Last time, we discussed the *string* class besides defining the plus operator as joining the two strings. Can we define minus for the *string* class? Is the minus operator relevant to the class *string*? For me it does not. Unless we come with some very artificial definition. Suppose we have a string as "This is a test" and a second string as "test". The subtraction of these two strings means the deletion of a word or words of second string from the first string. It may make some sense in this example. What will happen if the second string contains "My name is xyz". The subtraction of these strings does not make any sense. The thing we need to understand at this point is that every operator does not make sense for every class. Operators should be used only when these make some common sense so that reader can understand it easily. When you add two strings, it makes lot of sense. We can use either cat function or write this plus operator. As subtraction of strings does not make much sense, so it is not advisable to define it. Only define things that are self-explanatory, readable and understandable.

Operators with Date Class

We have so far been using the *Date* class. Let's think what operators make sense for *Date* class. What will be the meaning of plus operator or minus operator? Here we want to remind you a key thing i.e. "Paying attention to detail". Suppose you have some date and want to add some number to it like today's date plus 5. Does that make sense to you? We will get a new date by adding five to today's date i.e. date after five days. Similarly, if we want to subtract, say 10 from today's date, we should get the date of ten days before. Here is the usage of plus and minus which makes some sense. Can we subtract two dates together like subtraction of 1st Jan. 2002 from 15th Oct. 2002. What meaning it will convey? Perhaps nothing.

Let's consider the addition of a number to a date. Adding an integer to some date, according to the definition we will get some date in the future. The *Date* object will be returned from this function. We need a new date after the addition of integer number. We are defining this as a member-function so that the *Date* object that is calling it, will be passed to the function. The integer that is on the right hand side should be passed as an argument. Therefore in the parenthesis, we will have the integer. Now let's discuss it in detail. How can we add an integer to some date? Let's take today's date. Write it in your copy and see how can five be added to it. If you try to add a number to date, there are so many possibilities that can happen. Suppose, today is second day of the current month. After adding five to it, we will get 7th of this month. That was case I. Let's take the case II. Today is 27th of any month. Now what will be the new date after adding five. First thing, which is very obvious, that the month will get changed. But what will be the date of this new month. It depends whether there are 30 days or 31 days in this month. It may be the month of February. Is it the leap year or not? If it is non-leap year, there will be 28 days in February. Otherwise there will be 29 days. What is a leap year? There are rules to determine whether the year is leap year or not. If the year is divisible by four, it will be leap year. Similarly, being a century year, it may be divided by 400. Then again it is a leap year. Now we have seen that there are many cases when we are adding five to 27th of

© Copyright Virtual University of Pakistan

any month. Two things happen. The month is changed and the date changes according to the days in the month. What if it is the 27th of the December? Now you want to add five days. There are 31 days in December, after adding five it will be 1st of next month. We may have declared an array of twelve months. As December is the twelfth month, the last month of the year, so we have to go to first month of the next year. Here the year has also changed. We will also need to increment 1 to year too. It seems very simple that we have to add an integer number of days to some date. It becomes a complex function. Now suppose we have written this complex function and embedded all the rules in it. Then our life will become much easier. Suppose our semester starts from any date. After adding the period of semester, we will get the end date of the semester. We can do date arithmetic. This is a classic example of "paying attention to detail". To use the class for general purposes, we cannot miss even a single case. If you want to publish this class for others to use, you need to pay attention to detail and make sure that your class handles all of the stuff.

Here is the complete code of the program.
File "Date.h"

```
// The Date class is defined here

class Date{
   private:
      int day;
      int month;
      int  year;
      int daysOfMonth(Date d);                  // returns the no of days in a month
      static const int daysInMonth[]; // array containing the 12 month's days
      bool leapYear(int);                       // tells the year is leap year or not

   public:
      Date(int d = 1, int m = 1, int y = 1900); // constructor with default arguments
      void setDate(int, int, int);                          // set the date with given
arguments
      void display();              // Display the date on the screen
         // operators prototypes
      Date operator ++ ();       // pre increment operator used as ++date1
      Date operator + (int);  // Plus operator used as date1 + 5
};

// The implementation of the date class.
// initializing the no of days, take 0 for month zero.
const int Date::daysInMonth[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

// Displaying the function on the screen
void Date::display()
{
   cout <<"\nDate:" << day << "-" << month << "-" << year;
}
//constructor of the date
Date::Date(int d, int m, int y)
{
```


© Copyright Virtual University of Pakistan

```
    setDate(d, m, y);
}

// setting the date as given arguments
void Date::setDate(int d, int m, int y)
{
    year = y;

    // if month is wrong then set it to 1
    if (month < 1 && month > 12)
        month = 1;
    else
        month = m;
    // if day is wrong then set it to 1
    if (month == 2 && leapYear(y))
        if (d >=1 && d <=29)
            day = d;
        else
            day = 1;
    else
        if( d >= 1 && d <= daysInMonth[month])
                day = d;
        else
                day = 1;
}

// This function return the number of days in a month
int Date::daysOfMonth(Date d)
{
    if (d.month == 2 && leapYear(d.year))  // if leap year then Feb is 29
        return 29;
    else
        return  daysInMonth[d.month];
}

// Testing that the year is leap or not.
bool Date::leapYear(int y)
{
    if ( (y%400 == 0) || (y%100 != 0 && y%4 == 0))
        return true;
    else
        return false;
}

// + operator overloaded for the date. Used as date1 + 5
Date Date::operator + (int numberOfDays)
{
    for (int i = 1; i <= numberOfDays; i++)
    {
        ++(*this);    // calling the pre increment operator
```


© Copyright Virtual University of Pakistan

```
    }
    return *this;
}

// Pre increment operator
Date Date::operator ++ ()
{
    if (day == daysOfMonth(*this) && month == 12) // end year
            {
             day = 1;
             month = 1;
             ++year;
            }
            else if(day == daysOfMonth(*this)) // end month
            {
             day = 1;
             ++month;
            }
            else      // not the last day of the month
            {
             day++;
            }
}
```

The main program is:

```
#include  <iostream.h>
#include "date.h"

void main()
{
   Date  d1 (26, 12, 2002), d2(28,2 ,2000), d3;
   d1.display();
   ++d1;
   cout << "\nAfter adding 1 day, the date is ";
   d1.display();
   cout << endl;
   d2.display();
   d2 = d2 + 5;
   cout << "\nAfter adding 5 days to the above date";
   d2.display();
  }
```

Output of the program:

```
Date:26-12-2002
After adding 1 day, the date is
Date:27-12-2002

Date:28-2-2000
After adding 5 days to the above date
```


© Copyright Virtual University of Pakistan

Date:4-3-2000

Similarly we may have a counter-function that subtracts some number from the date. This is the same but of reverse nature. Suppose it is Jan 3rd and we have to subtract ten days from it. The month will be changed to December while the year is going to be decremented by 1. To determine the date of December, we need to know the number of days in December and count backwards. Now we don't need the number of days of current month. Rather the number of days in previous month is important. Suppose it is 3rd of March and subtract seven from it. What will be the date? Now you have to do complex arithmetic and take care of all the cases. It is very complicated but having only one time effort. Date arithmetic is very important and common in business applications. If someone applies for vacations, you just have to enter that this person is going on leave from this date for ten days and you will know his date of re-joining the duty. If someone works on daily wages and paid after a week. Someday, he comes and says that he is going on vacations. We need to calculate the number of days from the day of last payment to to-date. It is simple date arithmetic. Writing a *Date* class with these appropriate operators overloaded will be very useful exercise. It adds to your overall programming vocabulary.

There are two kinds of programming vocabulary. One is the keywords of C/C++ etc while the second is higher-level vocabulary. What sort of vocabulary we have in our tool- box. In the first part of this course, we have learned how to write loops, nested loops etc. We learn to handle matrices and vectors using those rudimentary rules. Now if you think about that we have written a matrix class and a member function *inverseOfMatrix()*. We can use this function again and again. Similarly in the *Date* class, we can put in some rudimentary calculations on date arithmetic. Add or subtract number of days from some date. These are very useful functions. In the daily wages example, you need to subtract a date from a date. Now we need to overload the minus operator again with date minus date. First we overload the minus operator with date minus some integer number. There may be two versions of minus operator. Here, you have to work in detail. Subtracting a date from another date is relatively non-trivial. As a programming idea, you can think that subtracting two dates involves huge calculations. Can we perform some logical tests here? If we want to implement *date1 – date2* while *date1* is smaller than *date2*. The first question is do we want to return a negative number. Let's say we want this, then *date1 – date2* can return a negative number. So it can return a negative number or zero (if the dates are identical) or positive number (the number of days). How we can implement this functionality? One way to do it is with the help of calendar. Under this method, we will start a loop till the other date is got. Then by reading the loop counter, we can tell the difference in days. It is a good idea. But for that, we need a calendar somewhere. If the dates are in different years, we will have to ensure the availability of calendar of next year. Think about it and try to write this function.

Now what about the plus-operator for two dates? Minus operator for strings did not make a lot of sense. Similarly, the plus operator for two dates does not make much sense. We can add some number to date. But how can we add a date to some other date. There is no logical and straight forward answer to this. So we don't define such a function. The meaning of our operator should be obvious. You can write whatever you want in the function. But it is bad idea. The idea of this exercise is to pay attention to detail. Think of all the various things that can happen. Tabulate them,

© Copyright Virtual University of Pakistan

determine the logic and then start programming. Don't start typing your program before your brain has come up to the same point. First analyze the problem, understand it, look at all the cases, draw a flow chart, write pseudo code. Once you are comfortable with this and know what you want to do then start writing your program. The time spending on analyses is arguably the best usage of your time as a programmer. The time you spend on debugging and removing errors from faulty code is huge. Spending time on good design pays off. You should debug for syntax errors like a semi-colon is missing somewhere. You should not face any logical error at debugging stage because logic errors are very hard to track. You may just not worry about the design and start writing code. The program may work for two or three cases. You may declare that you have written the program. When other starts using it on some other case which you did not cater, the program does not work or produces some strange results. There is no syntax error in the program. The compiler compiles it successfully and makes an executable file. Now we have to check the logic. Determining the logic from the code is a million times more difficult than determining code from logic. In this case, analysis will be always followed by design and then code. Please keep this in mind.

Unary Operators

Let's talk about unary operators. Unary operators take one argument like *i*++ or *i*-- (Post Increment or post decrement operators for integers) or ++i, --i (Pre increment or pre decrement operator). You can't make unary operator as binary operator or binary operator as unary. Let's overload unary operator in the *Date* class. We want to overload ++. This operator should add a day in the current date. When we say *++date1 or date1++*, it should get tomorrow's date. This is same as *date1 +=1 or date1 = date1 + 1*. We simply have to change to tomorrow's date. If this is the member function, it will get the date object automatically. The internal structure of the object is available to the function so that it takes no argument. It will return a *Date* object. Its prototype will be as:

```
Date operator ++ ( ); // pre increment operator
```

What will be in the function definition? You have to pay attention to details. The argument that we used in the plus operator, is also applicable here. What will be the next date when we add 1 to the current date. Let's work it out. If it is not the last date of the month, then simply add one to the day. If the date is the last day of the month, then change the month and date to 1st. If the date is the last date of the year, then increment the year too. Suppose we have some function available which returns the days of month given the month number. So if we say *daysOfMonth(6)* it should return 30. The function is intelligent enough that when we say *daysOfMonth(2)* it should return 28 if the year is not leap year. Otherwise, it will be 29. Therefore we have to send it year too along with the month number. We can also pass it the complete *Date* structure as *daysOfMonth(date1);* We will use this function in writing the ++ operator. In a way, the logic is same as we used in the plus operator. Suppose the object *d* is calling this ++ operator as *d*++ where *d* is an object of type *Date*. Therefore the day, month and year data members will be available to this function. In the body of the function, first of all we will check whether this is the last date of the month as:

© Copyright Virtual University of Pakistan

```
if (day == daysOfMonth ( *this ) )
{
        // this is the last day of the month
        // process accordingly
}
```

In the above condition, we have checked that day is equal to the number of days in the month or not. If the condition returns true it means that this is the last day of the month. Here we have used *this* to pass the object (current object) to the function *daysOfMonth*. '*this'* pointer is implicitly available to every member function and *this pointer* points to the current object. As per requirement of the program, we have written *d++* where *d* is the object of type *Date.* We are not using the object *d* in the program. This object is itself available.

Now the data of object *d* is available in the function as day, month or year. The object *d* is itself present either from its member data (day, month, year) or through the '*this* pointer' which points to the current object. We can also expand the definition of the function *daysOfMonth( )* as *daysOfMonth(int day, int month, int year)*. If the given day is the last day of the month, we will increment the month. Before doing this, we need to check whether this is the last month or not. Therefore we have to introduce another nested 'if' condition. The code segment will now be as:

```
if (day == daysOfMonth ( this ) )
{
        // this is the last day of the month
    if (month < 12)
        {
            day = 1;
            month++;
        }
        else  // this is the last month i.e. December
        {
                day = 1;
                month = 1;
                year++;
        }
}
else        // not the last day of the month
{
            day++;
}
```

The ++ operator simply adds one to the date of the calling object. We define it as member function. Therefore, no argument is needed. We can make it non-member but have to pass it a *Date* object.

To distinguish the pre increment operator with post increment operator, an int argument is passed to it. The prototype of post increment operator for Date is:

```
Date operator ++ (int  ); // post  increment operator
```


© Copyright Virtual University of Pakistan

Here we don't need to use this int argument. The implementation is same as pre increment operator as in both cases we want to add 1 to the date.

Can we implement the plus operator using this function? We can write the plus operator in some new fashion. We pass it a positive integer number, which has to be added to the date. We can write a loop in the plus operator. The loop condition will be as *i < number* where *number* is the argument passed to it. So in the program if we have written as *date1+5;* the loop will run for five times and in the body of the loop we have *++date1*. Suddenly our complicated logic has been boiled down to simple as incremented by 1. This is the classic example of code reuse.

We don't know who is going to use this code. Nobody is perfect. But we should think before writing the program about the structure, interface, the setters and getters and the operators to be overloaded. The thumb rule is if the meaning of + and ++ operator is same as in ordinary arithmetic, then + operator can be used in the ++ operator. Keep in mind that we can call a function from another function. This is a good example of code reuse. We can call the + operator as many times as needed. The *daysOfMonth* is a member function and it is used in ++ operator function. '+ operator' is a member function, used in ++ operator. We are building a hierarchy. Suppose there is some small logical error in the code and the *daysOfMonth* is not returning the correct value. This will effect the + operator as well as ++ operator. When we remove that error, then + and ++ operator both will be corrected. Moral of the story is that whenever we write some code, it is better to see whether we are rewriting some code in the same class. If we are calculating the number of months at two places or determining the leap year at two places, then try to combine this in such a way that things should be calculated at one place. That piece of code may become some utility function. This will not be called from outside the class so we will put this function in the private area. But the member functions can call it. We will make the *daysOfMonth()* as a private member function of the class. It will return the days of the month having checked whether this is leap year or not. Using this utility function, we have written + and ++ operator function. Don't repeat code inside a class. Make it a general rule. Make a function for the repeated code and call it where needed. For efficiency and speed, we can repeat the code. For this, we start using macros. It means that if you put all your logic in a single place and then reuse it. You will get lot of safety and security with this. A correction at one place will make the behavior of the whole class correct.

Let's see another interesting function of the *Date* class. Sometimes, we need to compare two dates i.e. whether a date is greater or less than the other date. In other words, the comparison operator is applied. Comparison operators <, >, <=, >=, == can also be overloaded. How do we determine whether *date1* is greater than *date2*? First of all, what will be its return type. Return type has to be either true or false. It says *date1* is greater than *date2* or *date1* is not greater than *date2*. Let's introduce another keyword *bool*. It is a new data type. It is very simple, it only takes two values true or false. So, the return type of greater than operator (>) is *bool*. The prototype of this member function is as:

```
bool operator > (Date d);
```


© Copyright Virtual University of Pakistan

The argument *d* is the *Date* object that is on the right side of the greater than sign. The left hand side *Date* object is available to this as this is the member operator of the class. Before writing the code, think abut the logic. We have to determine that the calling date is greater than the date *d* or not. If the year of current date is greater than date d, will the current date greater than date *d*? Certainly, it will be so. If the year is greater, obviously date is greater. If the years of both the dates are equal, then we have to check whether the month of the current date is greater than date *d* or not. If the month of the current date is greater than the date *d*, current date is greater. If the months are also equal, we will compare the days. It's a very simple hierarchical logic. To be able to write this logic cleanly, you should write case by case on paper. Analyze it thoroughly. The logic can be written in reverse too. If the year of the date *d* is greater than the current date, return false and so on. So we can go either true, true, true or false, false, false logic. You will find the false logic quicker. We can use if, 'else if' structures. Return type of this function is 'boolean'. Suppose that in our calling function we have two *Date* objects as *d1* and *d2*. We will write as *if(d1 > d2).* Why should we write this? As our operator is returning true or false and 'if' also needs true or false, we can write very clean and neat code. This greater than operator is a member operator of *Date* class. In this case, we have the return type as boolean and not returning the *Date* object. Is it the violation of any rule? The answer is no. The return type can be anything. It needs not to be the same as the class. It can be anything. The same applies to difference between two dates. The difference between two dates will be an integer. It is still a member function. It simply tells us the number of days between two days. It could be negative or positive but it is an integer. There is no such rule that the member operators should return the object of the same class.

The code of the greater than operator is as follows:

```
// Definition of the greater than operator

bool Date :: operator > ( Date d )
{
    if ( year > d.year )  // if year is greater date is greater
    {
       return true;
    }
    else if ( year == d.year)     //if years are equal check month
    {
        if ( month > d.month )  // if month is greater date is greater
        {
              return true;
        }
        else if ( month == d.month)      // if months are equal check dates
        {
            if(day > d.day)
              return true;
            else                                          // otherwise return false
              return false;
        }
        else
           return false;
    }
```


© Copyright Virtual University of Pakistan

```
    else
    {
      return false;
    }
}
```

Now you can write all the comparison operator of the *Date* class. The comparison operators are greater than, greater than or equal to, equal to, less than, less than or equal to. If you are writing one, you might want to write all of them. Now we have expanded the *Date* class enough. As an exercise, write your own *Date* class. Keep in mind the principles. What should be in the *Date* class? How should we set its values? How many constructors we need? Do we need a destructor or default destructor is enough. After this, define its public interface that is the member functions that are visible from outside. What operators should be overloaded?

In the program, if we say *date1 + 5,* we know that we will get a date which is five days later. What will happen if we write *5 + date1*? The situation like this may happen. You have published your *Date* class and someone wants to use it in this fashion. Here we have an integer, the plus operator and a *Date* object. It should return an object of type *Date*. To make this work properly, we need to have another operator. You will have to look at the set of operators needed for this class. List them out and write down their behavior. Be very clear what you expect them to do and start writing the class. How can we implement *integer + date*? On the left hand side, we have an integer. If the integer is at the left side, it can't be a member function. Member function is always called by object. Here object is not calling the function. Rather integer is calling. So it has to be a friend function that is sitting outside. It will get two arguments, integer and *Date*. As this is the friend function, the internal structure of the *Date* will be available to it. You will create a new *Date* object based on the given *Date* object and the integer and return it. We have seen that member functions are returning integers or Boolean. Here, a non-member function is returning an object of *Date* class. When we have listed out comprehensively that what will be the interface of our class. Which functions and operators will be visible from outside? When we have written the behavior of our class on paper, it is good to start writing the code. You may have to write a lot of code for this class. Once we have compiled the code and have object file, then anyone can use *Date* object. There will be no problem. We will include the "date.h" file in the program and use and manipulate the *Date* objects. We can use its operators, member functions etc very easily. The effort we put in writing this code does not go waste. It will provide a lot of ease in the main program. The biggest advantage is the encapsulation that has happened. All of the logic that was needed to manipulate the object of class *Date* is now encapsulated in that class. In case of any problem in the behavior of the class, we will need to correct the class, compile it. In the conventional function programming or structured programming, this logic has been split at different locations of the program. It was everywhere. Different things have been embedded at different points. In the function oriented programming, we have written a lot of functions at different locations. Here we have a new data type as *Date*. All the date related functions are at one place. We are encapsulating all the functionalities in the *Date* class that is another reason for doing all of the homework, all the thinking before we write the code. No one can determine all the usage of the *Date* class. If you start determining all the usage of *Date* class and writing the definition of the *Date* class for the last six months, this will be impractical. You

© Copyright Virtual University of Pakistan

would want to keep it within limits but do the homework then you write it. Now you can reuse it as many times as you want.

We need a friend operator when the driving thing is not the object of the class like *integer + date*. The operator is derived by integer. Here, we use a friend function. There are instances where friend operators are used to manipulate two different classes. A classic example in the mathematics is of the multiplication of vector and matrix. If we have to multiply a vector with a matrix, the multiplication operator will be friend of both the classes. It will get both vector and matrix as arguments and manipulate them. Keep in mind that friend operators in a way can also be used to glue two classes. The disadvantage of clubbing them together is that they become interlinked. Write your own overloaded operators and see how they work.



© Copyright Virtual University of Pakistan

Lecture No. 33

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 7, 8

7.5, 8.1, 8.2, 8.7, 8.9

Summary

22) Operator Overloading
23) Assignment Operator
24) Example
25) this Pointer
26) Self Assignment
27) Returning this Pointer from a Function
28) Conversions
29) Sample Program (conversion by constructor)

Operator Overloading

As earlier discussed, overloading of operators is carried out in classes on some occasions to enable ourselves to write a code that looks simple and clean. Suppose, there is a class and its two objects, say *a* and *b*, have been defined. Addition of these objects in the class by writing *a + b* will mean that we are adding two different objects (objects are instance of a class which is a user defined data type). We want our code to be simple and elegant. In object base programming, more effort is made in class definitions, as classes are data types that know how to manipulate themselves. These know how to add objects of their own type together, how to display themselves and do many other manipulations. While discussing *date* class in the previous lecture, we referred to many examples. In an example, we tried to increment the 'date'. The best way is to encapsulate it in the class itself and not in the main program when we come around to use the *date* class.

Assignment Operator

At first, we ascertain whether there is need of an assignment operator or not? It is needed when we are going to assign one object to the other, that means when we want to have expression like *a = b*. C++ provides a default assignment operator. This operator does a member-wise assignment. Let's say, we have in a structure of a class three integers and two floats as data members. Now we take two objects of this class *a* and *b* and write *a = b*. Here the first integer of *a* will have the value of first integer of *b*. The second will have the value of second integer and so on. This means that it is a

© Copyright Virtual University of Pakistan

member-wise copy. The default assignment operator does this. But what is to do if we want to do something more, in some special cases?

Now let's define a *String* class. We will define it our self, without taking the built in *String* class of C. We know that a string is nothing but an array of characters. So we define our *String* class, with a data member buffer, it is a pointer to character and is written as **buf* i.e. a pointer to character (array). There are constructors and destructors of the class. There is a member function *length* that returns the length of the string of the calling object. Now we want an assignment operator for this class. Suppose we have a constructor that allows placing a string into the buffer. It can be written in the main program as under:

String s1 ("This is a test") ;

Thus, an object of *String* has been created and initialized. The string "This is a test" has been placed in its buffer. Obviously, the buffer will be large enough to hold this string as defined in our constructor. We allocate the memory for the buffer by using *new* operator. What happens if we have another *String* object, let's say s2, and want to write *s2 = s1* ; Here we know that the buffer is nothing but a pointer to a memory location. If it is an array of characters, the name of the array is nothing but a pointer to the start of the memory location. If default assignment operator is used here, the value of one pointer i.e. *buf* of one object will be assigned to *buf* of the other object. It means there will be the same address in the both objects. Suppose we delete the object *s1*, the destructor of this object will free the allocated memory while giving it back to the free store. Now the *buf* of *s2* holds the address of memory, which actually has gone to free store, (by the destructor of s1). It is no longer allocated, and thus creates a problem. Such problems are faced often while using default assignment operator. To avoid such problems, we have to write our own assignment operator.

Before going on into the string assignment operator, let's have a look on the addition operator, which we have defined for strings. There is a point in it to discuss. When we defined addition operator for strings, we talked about that what we have to do if we want to add (concatenate) a string into the other string. There we had a simple structure i.e. there is a string defined *char buf* with a space of 30 characters. If we have two string objects and the strings are full in the both objects. Then how these will be added? Now suppose for the moment that we are not doing memory allocation. We have a fixed string buffer in the memory. It is important that the addition operator should perform an error check. It should take length of first string , then the length of second string, before adding them up, and check whether it is greater than the length defined in the String (i.e. 30 in this case). If it is greater than that, we should provide it some logical behavior. If it is not greater, then it should add the strings. Thus, it is important that we should do proper error checking.
Now in the assignment operator, the problem here is that the *buf*, that we have defined, should not point to the same memory location in two different objects of type *String*. Each object should have its own space and value in the memory for its string. So when we write a statement like *s2 = s1*, we want to make sure that at assignment time, the addresses should not be assigned. But there should be proper space and the strings should be copied there. Let's see how we can do this.

© Copyright Virtual University of Pakistan

Now take a look on the code itself. It is quiet straight forward what we want to do is that we are defining an assignment operator (i.e. =) for the String class. Remember that the object on left side will call the = operator. If we write a statement like *s2 = s1*; *s2* will be the calling object and *s1* will be passed to the = operator as an argument. So the data structure of *s2* i.e. *buf* is available without specifying any prefix. We have to access the *buf* of *s1* by writing *s1.buf*.

Suppose *s2* has already a value. It means that if *s2* has a value, its buffer has allocated some space in the memory. Moreover, there is a character string in it. So to make an assignment first empty that memory of *s2* as we want to write something in that memory. We do not know whether the value, we are going to write, is less or greater than the already existing one. So the first statement is:

delete buf ;

Here we write *buf* without any prefix as it is *buf* of the calling object. Now this buffer is free and needs new space. This space should be large enough so that it can hold the string of *s1*. First we find the length of the string of *s1* and then we use the *new* operator and give it a value that is one more than the length of the buffer of *s1*. So we write it as:

buf = new char[length + 1] ;

where *length* is the length of *s1*. Here *buf* is without a prefix so it is the *buf* of object on the left hand side i.e. *s2*. Now, when the *buf* of *s2* has a valid memory address, we copy the *buf* of *s1* into the *buf* of *s2 with the use of string copy function (strcpy).* We write it as:

strcpy (buf, s1.buf) ;.

Now the *buf* of string of *s1* has been copied in the *buf* of *s2 that* are located at different spaces in the memory. The advantage of it is that now if we delete one object *s1* or *s2*, it will not affect the other one. The complete code of this example is given below.

Example

```
/*This program defines the assignment operator. We copy the string of one object
into the string of other object using different spaces for both strings in the memory.
*/

#include <iostream.h>
#include <string.h>
#include <stdlib.h>
// class definition

class String
{
        private :
```


© Copyright Virtual University of Pakistan

```
        char *buf ;
    public:
        // constructors
        String();
        String( const char *s )
        {
            buf = new char [ 30 ];
            strcpy (buf,s);
        }
        // display the string
        void display ( )
    {
            cout << buf << endl ;
    }
        // getting the length of the string
        int length ()const
        {
            return strlen(buf);
        }
        // overloading assignment operator
        void operator = ( const String &other );
};
//------------ Assignment operator
void String::operator = ( const String &other )
{
    int length ;
    length = other.length();
    delete buf;
    buf = new char [length + 1];
    strcpy( buf, other.buf );
}

//the main program that uses the new String class with its assignment operator:
main()
{
    String myString( "here's my string" );
    cout << “My string is = ” ;
    myString.display();
    cout << '\n';

    String yourString( "here's your string" );
    cout << “Your string is = ” ;
    yourString.display();
    cout << '\n';

    yourString = myString;
    cout << “After assignment, your string is = ” ;
    yourString.display();
    cout << '\n';
    system ("pause");
```


© Copyright Virtual University of Pakistan

```
}
```

Following is the output of the program.

```
My string is = here's my string

Your string is = here's your string

After assignment, your string is =  here's my string
```

The above example is regarding the strings. Yet in general, this example pertains to all classes in which we do memory manipulations. Whenever we use objects that allocate memory, it is important that an assignment operator (=) should be defined for it. Otherwise, the default operator will copy the values of addresses and pointers. The actual values and memory allocation will not be done by it.

Let's go on and look what happens when we actually do this assignment? In the assignment of integers, say we have three integers i, j and k with some values. It is quiet legal to write as *i = j* ; By this ,we assign the value of *j* to *i*. After this we write *k = i* ; this assigns the value of *i* to *k*. In C, we can write the above two assignment statements in one line as follows

$$k = i = j ;$$

This line means first the value of *j* is assigned to *i* and that value of *i* is assigned to *k*. The mechanism that makes this work is that in C or C++ every expression itself has a value. This value allows these chained assignment statements to work. For example, when we write *k = i = j* ; then at first *i = j* is executed. The value at the left hand side of this assignment statement is the value of the expression that is returned to the part '*k ='* .This value is later assigned to *k*. Now take another example. Suppose we have the following statement:

$$k = i = ++j ;$$

In this statement, we use the pre-increment operator, as the increment operator (++) is written before *j*. This pre-increment operator will increment the value of *j* by 1 and this new value will be assigned to *i*. It is pertinent to note that this way *++j* returns a value that is used in the statement. After this, the value of *i* is assigned to *k*. For integers, it is ok. Now, how can we make this mechanism work with our *String* objects. We have three *String* objects *s1*, *s2* and *s3*. Let's say there is a value (a string) in the buffer of *s1*. How can we write *s3 = s2 = s1*;. We have written *s2 = s1* in the previous example in assignment operator. We notice that there is no return statement in the code of the assignment operator. It means that operator actually returns nothing as it has a void return type. If this function is not returning any thing then *s3 = s2 = s1* ; cannot work. We make this work with the help of *this* pointer.

this Pointer

Whenever an object calls a member function, the function implicitly gets a pointer from the calling object. That pointer is known as *this* pointer. '*this'* is a key word. We cannot use it as a variable name. '*this'* pointer is present in the function, referring to

© Copyright Virtual University of Pakistan

the calling object. For example, if we have to refer a member, let's say *buf*, of our *String* class, we can write it simply as:

buf ;

That means the *buf* of calling object is being considered. We can also write it as

this->buf ;

i.e. the data member of the object pointed by *this* pointer is being called. These (*buf* and *this->buf*) are exactly the same. We can also write it in a third way as:

*(*this).buf ;.*

So these three statements are exactly equivalent. Normally we do not use the statements written with *this* key word. We write simply *buf* to refer to the calling object.

In the statement *(*this).buf ;* The parentheses are necessary as we know that in *object.buf* the binding of dot operator is stronger than the *. Without parentheses the *object.buf* is resolved first and is dereferenced. So to dereference *this* pointer to get the object, we enforced it by putting it in parentheses.

Self Assignment

Suppose, we have an integer '*i*. In the program, somewhere, we write *i = i ;* It's a do nothing line which does nothing. It is not an error too. Now think about the String object, we have a string *s* that has initialized to a string, say, 'This is a test'. And then we write *s = s ;* The behavior of equal operator that we have defined for the String object is that it, at first deletes the buffer of the calling object. While writing *s = s ;* the assignment operator frees the buffer of *s*. Later, it tries to take the buffer of the object on right hand side, which already has been deleted and trying to allocate space and assign it to *s*. This is known as self-assignment. Normally, self -assignment is not directly used in the programs. But sometimes, it is needed. Suppose we have the address of an object in a pointer. We write:

*String s, *sptr ;*

Now *sptr* is a pointer to a *String* while *s* is a *String* object. In the code of a program we can write

sptr = &s ;

This statement assigns the address of *s* to the pointer *sptr*. Now some where in the program we write *s = *sptr ;* that means we assign to *s* the object being pointed by *sptr*. As *sptr* has the address of *s* , earlier assigned to it. This has the same effect as *s = s ;*. The buffer of *s* will be deleted and the assignment will not be done. Thus, the program will become unpredictable. So self-assignment is very dangerous especially at a time when we have memory manipulation in a class. The *String* class is a classic example of it in which we do memory allocation. To avoid this, in the equal operator (operator=), we should first check whether the calling object (L.H.S.) and the object

© Copyright Virtual University of Pakistan

being gotten (R.H.S.) are the same or not. So we can write the equal operator (operator=) as follows

```
void String::operator=( const String &other )
{
    if( this == &other )
        return;
delete buf;
length = other.length;
buf = new char[length + 1];
strcpy( buf, other.buf );
}
```

Here above, the statement *if (this == &other)* checks that if the calling object (which is referred by *this*) is the same as the object being called then do nothing and return as in this case it is a self assignment. By doing this little change in our assignment operator, it has become safe to use. So it is the first usage of *this* pointer that is a check against self assignment.

Returning *this* Pointer From a Function

Now lets look at the second use of *this* pointer. We want to do the assignment as

```
s3 = s2 = s1 ;
```

In this statement the value of *s2 = s1* is assigned to *s3*. So to do this it is necessary that the assignment operator should return a value. Thus, our assignment operator will expand so that it could return a value. The assignment operator, till by now copies the string on right hand side to the left hand side. This means *s2 = s1* ; can be done by this. Now we want that this *s2* should be assigned to *s3*, which can be done only if *s2 = s1* returns *s2* (an object). So we need to return a *String* object. Here becomes the use of *this* pointer, we will write as

```
return *this ;
```

Here, in the assignment operator code *this* is referring to the calling object (i.e. s2 in this case). So when we write *return *this* ;* it means return the calling object (object on L.H.S.) as a value. Thus *s3* gets the value of *s2* by executing *s3 = s2* where *s2* is the value returned by the assignment operator by *s2 = s1*; Thus the complete assignment operator (i.e. operator= function) that returns a reference to an object will be written as under

```
String &String::operator=( const String &other )
{
    if( &other == this )    //if calling and passed objects are
        return *this;       // same then do nothing and return
    delete buf;
    length = other.length;
buf = new char[length + 1];
strcpy( buf, other.buf );
```


© Copyright Virtual University of Pakistan

```
    return *this;
}
```

Now, here the first line shows that this *operator*= function returns a reference to an object of type *String* and this function takes a reference as an argument. The above version of *operator*= function takes a reference as an argument. First of all it checks it with the calling object to avoid self assignment. Then it deletes the buffer of calling object and creates a new buffer large enough to hold the argument object. And then at the last it returns the reference of the calling object by using *this* pointer.
With this version of the assignment operator (operator= function) we can chain together assignments of *String* objects like *s3 = s2 = s1* ;
Actually, we have been using *this* pointer in chained statements of *cout*. For example, we write

cout << a << b << c ;

Where *a*, *b*, and *c* are any data type. It works in the way that the stream of *cout* i.e. << is left associative. It means first *cout* << *a* is executed and to further execute it, the second << should have *cout* on its left hand side. So here, in a way, in this operator overloading, a reference to the calling object that is *cout* is being returned to the stream insertion <<. Thus a reference of *cout* is returned, and (as a reference to *cout* is returned) the << sees a *cout* on left hand side and thus the next << *b* is executed and it returns a reference to *cout* and with this reference the next << *c* works. This all work is carried out by *this* pointer.

We have seen that value can be returned from a function with *this* pointer. We used it with assignment operator. Lets consider our previous example of *Date* class. In that class we defined increment operator, plus operator, plus equal operator, minus operator and minus equal operator. Suppose we have *Date* objects *d1*, *d2* and *d3*. When we write like *d2 = d1++* ; or *d2 = d1 + 1* ; here we realize that the + operator and the ++ operator should return a value. Similarly, other operators should also return a value. Now let's consider the code of *Date* class. Now we have rewritten these operators. The difference in code of these is not more than that now it returns a reference to an object of type *Date*. There are two changes in the previous code. First is in the declaration line where we now use & sign for the reference and we write it like

Date& Date::operator+=(int days)

We write this for the all operators (i.e. +, ++, - and -=). Then in the function definition we return the reference of the left hand side *Date* object (i.e. calling object) by writing

*return *this* ;*

Now we can rewrite the *operator+=* of the *Date* class as follows.
The declaration line in the class definition will be as

Date& operator+=(int days);

And the function definition will be rewritten as the following.

© Copyright Virtual University of Pakistan

```
Date& Date::operator+=(int days)  // return type  reference to
object
{
        for (int i=0; i < days; i++)
        *this++;
        return *this;    // return reference to object
}
```

This concludes that whenever we are writing arithmetic operator and want that it can be used in chained statements (compound statements) then we have to return a value. The easiest and most convenient way of returning that value is by returning a reference to the calling object. So, by now we can easily write the statements of *date* object, just like we write for integers or floats. We can write

```
        date2 = date1 + 1 ;
Or      date2 = date1++ ;
```

and so on.

Conversions

Being in the C language, suppose we have an integer *i* and a float *x*. Now in the program we write *x = i* ; As we know that the operations of *int* and *float* are different in the memory. Therefore, we need to do some kind of conversion. The language automatically converts *i* (int) to a float (or to whatever type is on the L.H.S.) and then does the assignment.

Both C and C++ have a set of rules for converting one type to another. These rules are used in the following situations

- When assigning a value. For example, if you assign an integer to a variable of type long, the compiler converts the integer to a long.
- When performing an arithmetic operation. For example, if you add an integer and a floating-point value, the compiler converts the integer to a float before it performs the addition.
- When passing an argument to a function; for example, if you pass an integer to a function that expects a long.
- When returning a value from a function; for example, if you return a float from a function that has double as its return type.

In all of these situations, the compiler performs the conversion implicitly. We can make the conversion explicit by using a cast expression.

Now the question arises that can we do conversion with objects of our own classes. The answer is yes. If we go to the basic definition of a class it is nothing but a user defined data type. As it is a user defined data type, we can also define conversion on it. When we define a class in C++, we can specify the conversions that the compiler can apply when we use instances of that class. We can define conversions between classes, or between a class and a built-in type

© Copyright Virtual University of Pakistan

There is an example of it. Suppose we have stored date in a serial number form. So now it is not in the form of day, month and year but it is now a long integer. For example if we start from January 1, 1900 then 111900 will be the day one, second January will be the day 2, third January will be the day 3 and so on. Going on this way we reached in year 2000. There will be a serial number in year 2000. This will be a single long integer that represents the number of days since a particular date. Now if we have a *Date* object called *d* and an integer *i*. We can write *d = i* ; which means we want that *i* should go in the serial part of *d*. This means convert the integer *i* into *date* object and then the value of *i* should go into the serial number and then that object should be assigned to *d*. We can make this conversion of *Date* object. We do this in the constructor of the class. We pass the integer to the constructor, here convert it into a date, and the constructor then returns a *Date* object. On the other hand, if there is no constructor then we can write conversion function. We have used the conversion operator with *cast*. The way of casting is that we write the name of the cast (type to which we want to convert) before the name of variable. Thus if we have to write *x = i* ; where *x* is a float and *i* is an integer then we write it with conversion function as *x = (float) i* ; The (float) will be the conversion operator. Normally this conversion is done by default but sometimes we have to force it. Now we want to write a conversion operator, which converts an integer to a *Date* object. (here Date is not our old class, it's a new one). We can write a conversion function. This function will be a member of the *Date* class. This function will return nothing, as it is a conversion function. The syntax of this function will be *Date ()* that means now this is a conversion function.

In the body of this function we can write code of our own. Thus we can define the operator, and it will work like that we write within the parentheses the name of the conversion operator. The conversion functions are quiet interesting. They allow us to manipulate objects of different classes. For example, we have two classes one is a truck and other is a car. We want to convert the car to a truck. Here we can write a conversion function, which says take a car convert it into a truck and then assign it to an object of class truck.

Sample Program (conversion by constructor)

Lets take a look at an example in which the conversion functions are being used. There is a class *Fraction*. Lets talk why we called it *Fraction* class. Suppose we have an assignment

double x = 1/3 ;

When we store 1/3 in the computer memory, it will be stored as a double precision number. There is a double precision division of 1 by 3 and the answer 0.33333… is stored. Here the number of 3s depends upon the space in the memory for a double precision number. That value is assigned to a *double* variable. What happens if we multiply that *double* variable by 3, that means we write *3 * x*; where x was equal to 1/3 (0.33333…). The answer will be 0.99999…, whatever the number of digits was. The problem here is that the answer of 3 * 1 / 3 is 1 and not 0.99999. This problem occurs, when we want to represent the numbers exactly. The fraction class is the class in which we provide numerator and denominator to the object and it stores them separately. It will always keep them as an integer numerator and an integer denominator and we can do all kinds of arithmetic with it. Now if 1 and 3 are stored

© Copyright Virtual University of Pakistan

separately as numerator and denominator, how we can add them to some other fraction. We have enough tools at our disposal. One of which is that we can overload the addition operator. We can add 1/3 and 2/5, the result of which is another fraction i.e. numerator and denominator. In this way we have no worries of round off errors, the truncation and conversions of int and floats.

Considering the *Fraction* class we can think of a constructor which takes an integer and converts it into a fraction. We do not want that as a fraction there should be some value divided by zero. So we define the default constructor that takes two integers, one for numerator and one for denominator. We provide a default value for the denominator that is 1. It means that now we can construct a fraction by passing it a single integer, in which case it will be represented as a fraction with the passed integer as a numerator and the default vale i.e. 1 as the denominator. If we have a fraction object *f* and we write *f = 3*; Then automatically this constructor (i.e. we defined) will be called which takes a single integer as an argument. An object of type fraction will be created and the assignment will be carried out. In a way, this is nothing more than a conversion operation. That is a conversion of an integer into a fraction. Thus a constructor that takes only one parameter is considered a conversion function; it specifies a conversion from the type of the parameter to the type of the class.

So we can write a conversion function or we can use a constructor of single argument for conversion operation. We cannot use both, we have to write one or the other. Be careful about this. We don't use conversion functions often but sometimes it is useful to write them. The conversion operators are useful for defining an implicit conversion from the class to a class whose source code we don't have access to. For example, if we want a conversion from our class to a class that resides within a library, we cannot define a single-argument constructor for that class. Instead, we must use a conversion operator.

It makes our code easier and cleaner to maintain. It is important that pay more attention while defining a class. A well-defined class will make their use easy in the programming.
Following is the code of the example stated above.

```
/* This program defines a class Fraction which stores numerator and
denominator of a fractional number separately. It also overloads the
addition operator for adding the fractional numbers so that exact results
can be obtained.
*/

#include <stdlib.h>
#include <math.h>
#include <iostream.h>

// class definition
class Fraction
{
        public:
                        Fraction();
                        Fraction( long num, long den );
```


© Copyright Virtual University of Pakistan

```
                    void display() const;
                    Fraction operator+( const Fraction &second ) const;
        private:
                    static long gcf( long first, long second );
                    long numerator, denominator;
};

//------------ Default constructor
Fraction::Fraction()
{
        numerator = 0;
        denominator = 1;
}
//------------ Constructor
Fraction::Fraction( long num, long den )
{
        int factor;
        if( den == 0 )
          den = 1;
        numerator = num;
        denominator = den;
        if( den < 0 )
        {
                    numerator = -numerator;
                    denominator = -denominator;
         }
        factor = gcf( num, den );
        if( factor > 1 )
        {
                    numerator /= factor;
                    denominator /= factor;
        }
}
//------------ Function to print a Fraction
void Fraction::display() const
{
        cout << numerator << '/' << denominator;
}

//------------ Overloaded + operator
Fraction Fraction::operator+( const Fraction &second ) const
{
        long factor, mult1, mult2;
        factor = gcf( denominator, second.denominator );
        mult1 = denominator / factor;
        mult2 = second.denominator / factor;
        return Fraction( numerator * mult2 + second.numerator * mult1,
                                denominator * mult2 );
}
```


© Copyright Virtual University of Pakistan

```
//------------ Greatest common factor
// computed using iterative version of Euclid's algorithm
long Fraction::gcf( long first, long second )
{
        int temp;
        first = labs( first );
        second = labs( second );
        while( second > 0 )
        {
                temp = first % second;
                first = second;
                second = temp;
        }
        return first;
}

//main program
void main()
{
        Fraction a, b( 23, 11 ), c( 2, 3 );
        a = b + c;
        a.display();
        cout << '\n';
        system("pause");
}
```

The output of the program is as follows

```
91/33
```

Here is an example from the real world. What happens if we are dealing with currency? The banks deal with currency by using computer programs. These programs maintain the accounts by keeping the track of transactions and manipulating deposits and with drawls of money. Suppose the bank declares that this year the profit ratio is 3.76 %. Now if the program calculates the profit as 3.67 % of the balance, will it be exactly in rupees and paisas or in dollars and cents? Normally, all the currencies have two decimal digits after decimal point. Whenever we apply some kind of rates in percentage the result may become in three or four decimal places. The banks cannot afford that the result of 90 paisas added to 9 rupees and 10 paisas become 10 rupees and 1 paisa. They have to accurate arithmetic. So they do not rely on programs that use something like *double* precisions to represent currencies. They would have rather written in the program to treat it as a string. Thus 9 is a string, 10 is a string and the program should define string addition such that the result of addition of the strings .10 and .90 should be 1.00. Thus, there are many things that happen in real world that force us as the programmer to program the things differently. So we do not use native data types.

The COBOL, COmmon Business Oriented Language has a facility that we can represent the decimal numbers exactly. Internally it (the language) keeps them as strings in the memory. There is no artificial computer representation of numbers. Now a day, the languages provide us the facility by which we can define a data type

© Copyright Virtual University of Pakistan

according to a specific requirement, as we defined *fraction* to store numerator and denominator separately. By using this *fraction* data type, we never loose precision in arithmetic. The same thing applies to the object like currency, where we can store the whole number part and the fractional part both as separate integers and never loose accuracy. But whenever we get into these classes, it is our responsibility to start writing all of the operators that are required to make a complete class.

Lecture No. 34

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 8***

Summary

- Arrays of Objects
- Dynamic Arrays of Objects
- Overloading new and delete Operators
- Example of Overloading new and delete as Non-members
- Example of Overloading new and delete as Members
- Overloading [] Operator to Create Arrays

Arrays of Objects

A class is a user-defined data type. Objects are instances of classes the way *int* variables are instances of *int*s. Previously, we have worked with arrays of *int*s. Now, we are going to work with arrays of objects.

The declaration of arrays of user-defined data types is identical to the array of *primitive* data types.

Following is a snapshot of our veteran *Date* class:

```
/* Snapshot of Date class discussed in previous lectures */
class Date
{
    private:
      int  day, month, year;

    public:
     /* Parameterless constructor, it is created by the compiler automatically when we
```


© Copyright Virtual University of Pakistan

```
    don't write it for any of our class. */
  Date( )
  {
      cout << "\n Parameterless constructor called ...";
      month = day = year = 0;
  }
  /* Parameterized constructor; has three ints as parameters. */
  Date(int month, int day, int year)
  {
      cout << "\n Constructor with three int parameters called ...";
      this->month = month;
      this->day = day;
      this->year = year;
  }

  ~Date ( )
  {
      cout << "\n Destructor called ...";
  }
  . . .
  . . .
};
```

Consider the example of declaring an array of *10* date objects of *Date* class. In this case, the declaration of arrays will be as under:
Following is the declaration:

```
Date myDates [10] ;
```

With this line (when this line is executed), we are creating 10 new objects of *Date* class. We know that a constructor is called whenever an object is created. For every object like *myDate[0], myDate[1],.... myDate[9]*, the constructor of the *Date* class is called. Theimportant thing to know here is that which constructor of Date class is being called to construct objects of the array *myDates*. As we are not doing any initialization of the array objects explicitly, the default constructor (parameterless constructor) of the Date class is called. Remember, the default constructor is defined by the C++ compiler automatically for every class that has no parameterless constructor defined already. In our case of *Date* class, we have defined a parameterless constructor, therefore, the compiler will not generate default constructor automatically.

We can also initialize the array elements at the declaration time. This initialization is similar to that done for native data types. For *int* array, we used to do initialization in the following manner:

```
int array [10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } ;
```

Similarly, we initialize *Date* array while declaring it:

```
Date yourDate [3] = { Date(10, 24, 1980), Date(06, 14, 1985), Date(07, 09,1986) };
```


© Copyright Virtual University of Pakistan

The above statement will call the parameterized constructor *Date (int month, int day, int year)* of *Date* class to create three objects of *myDate* array. This parameterized constructor carries out initialization of the objects data member (*month, day, year*) with the values supplied as arguments to it.

It will be interesting to know, how the following statement works:

Date myDate [10] = { Date(09, 03, 1970), Date(08, 23, 1974) } ;

We are trying to declare an array of 10 Date objects while supplying only initialization values for the first two elements. At first, we might be doubtful if the statement is compiled successfully. Not only it compiles successfully but also does the initialization of the first two objects (*myDate[0], myDate[1]*). What will happen to the remaining objects in the array? Actually, all the *10* objects are created successfully by the above statement. The parameterized constructor is called for the first two objects (*myDate[0], myDate[1]*) and parameterless constructor is called for the remaining objects (*myDate[2], myDate[3], …, myDate[9]*).

You might have noticed that at the array initialization stage, we have explicitly called parameterized constructor of *Date* for every object. We may specify only the argument when a constructor with only one parameter is called.

```
/* A snapshot of String class discussed in previous lectures */

class String
{
private :
        char  *buf ;
public:
        // Constructors
        String ();
        String( const char  *s )
        {
                buf  =  new  char [ 30 ];
                strcpy (buf,s);
        }
        . . .
        . . .
};
```

For example, in the above-mentioned case of *String* class, we have a constructor that is accepting one argument of type *char **. While writing our code, we can declare and initialize an array of *String*s as follows:

```
String message [10] = {     "First line of message\n",
                            "Second line of message\n",
                            String( "Third line of message\n"),
                            String( )
```


© Copyright Virtual University of Pakistan

```
};
```

See the initializing arguments for first two objects i.e, (*message[0], message[1]*) in the array. Here only one string is being passed. Therefore, for the first two objects, constructor with one parameter of type *char* * of *String* class is called automatically. That constructor is *String (char * str)*. For the third object (*message[2]*), the same constructor with one *char* * as parameter is being called explicitly. For fourth object (*message[3]*), parameterless constructor i.e., *String ( )* is being called explicitly, though, this was optional as parameterless constructor is called up automatically when no initialization is made. As there is no explicit initialization for the remaining six objects, the parameterless constructor is called up automatically.

Can we create arrays of objects dynamically? As usual, the answer is yes. Let's discuss it in detail.

Dynamic Arrays of Objects

Consider the following statement:

```
1.  String  *text ;
2.  text  =  new  String [5] ;
```

In line 1, we have declared a pointer *text* of *String* type.

In line 2, we are creating an array of 5 objects of *String* type. This statement allocates space for each object of the array, calls the parameterless constructor for each object and starting address of the first object is assigned to the pointer *text*.

The important point to be noted here is that in line 2, we can't initialize objects because there is no way to provide initializers for the elements of an array allocated with *new*.

The default constructor (parameterless constructor) is called for each element in the array allocated with *new*. Remember, the default constructor for a class is generated by C++ compiler automatically if it is not defined already in the class definition.

To deallocate these arrays of objects, the *delete* operator is used in the same way as it is used for the native data types.

There are few cautions that should be taken care of while performing these operations of allocation and deallocation with arrays of objects.

Firstly, while deallocating an array allocated with *new* operator, it is important to tell the compiler that an array of objects is being deleted. The brackets (*[]*) are written in our *delete* statement after the *delete* keyword to inform the *delete* operator that it is going to delete an array. The consequences of using the wrong syntax are serious. For example, if we want to delete previously created array of five *String* objects using the following statement:

```
delete  text;  // Incorrect syntax of deleting an array
```

The delete operator in this case will not be aware of deleting (deallocating) an array of objects. This statement will call the destructor only for the object pointed by the ***text*** pointer i.e. *String[0]* and deallocate the space allocated to this object. The requirement is to call the destructor for all the objects inside the array and deallocate the space allocated to all of these objects. But on account of the wrong syntax, only the first object is deleted and the remaining four objects (*String[1], String[2], String[3], String[4]* pointed by *text[1], text[2], text[3], text[4]* respectively) remain

© Copyright Virtual University of Pakistan

in the memory intact. The memory space occupied by these four objects results in *memory leak* as the same program or any other program on the same computer cannot use it unless it is deallocated.

Calling the destructor while destroying an object becomes essential when we have allocated some memory in free store from inside the object (usually from within the constructor).

To destroy an array of objects allocated on free store using the *new* operator, an array equivalent of *delete* operator is used. The array equivalent of *delete* operator is to write empty square brackets after the *delete* keyword (*delete []*). So the correct statement is:

```
delete [] text ;
```

This statement destroys the whole array properly. It calls destructor for each object inside the array and deallocates the space allotted to each object. Actually, by looking at the brackets (*[]*) after *delete*, the compiler generates code to determine the size of the array at runtime and deallocate the whole array properly. Here, it will generate code to deallocate an array of *5* objects of *String* type.

If we create an array of *Date* objects and want to delete them without specifying array operator: It will look as under:

```
// Bad Technique: deleting an array of objects without []
// for a class that is not doing dynamic memory allocation internally
Date * ppointments;
appointments = new Date[10];
. . .
delete appointments; // Same as delete [] appointments;
```

Although, this is good to deallocate an array of objects without specifying array operator ([]) as there is no dynamic memory allocation occurring from inside the *Date* class. But this is a bad practice. In future, the implementation of this class may change. It may contain some dynamic memory allocation code. So it is always safer to use array operator (*[]*) to delete arrays.

Can we overload *new* and *delete* operators? Yes, it is possible to overload *new* and *delete* operators to customize memory management. These operators can be overloaded in global (non-member) scope and in class scope as member operators.

Overloading of new and delete Operators

Firstly, we should know what happens when we use *new* operator to create objects. The memory space is allocated for the object and then its constructor is called. Similarly, when we use *delete* operator with our objects, the destructor is called for the object before deallocating the storage to the object.

When we overload *new* or *delete* operators, it can only lead to a change in the allocation and deallocation part. The call to the constructor after allocating memory while using *new* operator and call to the destructor before deallocating memory while

© Copyright Virtual University of Pakistan

using *delete* operator will be there. These calls to constructors and destructors are controlled by the language itself and these cannot be altered by the programmer.

One of the reasons of overloading *new* and *delete* operators can be their limited current functionality. For example, we allocate space on free store using the *new* operator for *1000 int*s. It will fail and return *0* if there is no contiguous space for *1000 int*s in free store. Rather the free store has become fragmented. The total available memory is much more than *1000 int*s , but in fragments. There is no contiguous segment of at least *1000 int*s. The built-in *new* operator will fail to get the required space but we can overload our own *new* operator to de-fragment the memory to get at least *1000 int*s space. Similarly, we can overload *delete* operator to deallocate memory.

In the embedded and real-time systems, a program may have to run for a very long time with restricted resources. Such a system may also require that memory allocation always takes the same amount of time. There is no allowance for heap exhaustion or fragmentation. A custom memory allocator is the solution. Otherwise, programmers will avoid using *new* and *delete* altogether in such cases and miss out on a valuable C++ asset.

There are also downsides of this overloading. If we overload *new* or *delete* operator at global level, the corresponding built-in *new* or *delete* operator will not be visible to whole of the program. Instead our globally written overloaded operator takes over its place all over. Every call to *new* operator will use our provided *new* operator's implementation. Even in the implementation of *new* operator, we cannot use the built-in *new* operator.
Nonetheless,when we overload *new* operator at a class level then this implementation of *new* operator will be visible to only objects of this class. For all other types (excluding this class) will still use the built-in *new* operator. For example, if we overload *new* operator for our class *Date* then whenever we use *new* with *Date*, our overloaded implementation is called.

```
Date* datePtr = new Date;
```

This statement will cause to call our overloaded *new* operator. However, when we use *new* with any other type anywhere in our program as under:

```
int* intPtr = new int [10];
```

The built-in *new* operator is called. Therefore, it is safer to overload *new* and *delete* operators for specific types instead of overloading it globally.

An important point to consider while overloading *new* operator is the return value when the *new* operator fails to fulfill the request. Whether the operator function will return *0* or throw an exception.

Following are the prototypes of the *new* and *delete* operators:

```
void * operator new ( size_t size ) ;
void operator delete ( void * ptr) ;
```

The *new* operator returns a *void* * besides accepting a parameter of whole numbers *size_t*. This prototype will remain as it is while overloading *new* operator. In the implementation of overloaded *new* operator, we may use *calloc( )* or *malloc( )* for memory allocation and write some memory block's initialization code.

© Copyright Virtual University of Pakistan

The delete operator returns nothing (*void*) and accepts a pointer of *void* * to the memory block. So the same pointer that is returned by the *new* operator, is passed as an argument to the *delete* operator. Remember, these rules apply to both if operators (*new* and *delete*) are overloaded as member or non-member operators (as global operators). Importantly, whenever we use these operators with classes, we must know their sequence of events that is always there with these operators. For *new* operator, memory block is allocated first before calling the constructor. For *delete* operator, destructor for the object is called first and then the memory block is deallocated. Importantly, our overloaded operators of *new* and *delete* only takes the part of allocation and deallocation respectively and calls to constructors and destructors remain intact in the same sequence.

Because of this sequence of events, the behavior of these *new* and *delete* operators is different from the built-in operators of *new* and *delete*. The overloaded *new* operator returns *void* * when it is overloaded as non-member (global). However, it returns an object pointer like the built-in *new* operator, when overloaded as a member function. It is important to understand that these operator functions behave like *static* functions when overloaded as member functions despite not being declared with *static* keyword. *static* functions can access only the *static* data members that are available to the class even before an object is created. As we already know that *new* operator is called to construct objects, it has to be available before the object is constructed. Similarly, the *delete* operator is called when the object has already been destructed by calling destructor of the object.

Example of Overloading new and delete as Non-members

Suppose we want *new* to initialize the contents of a memory block to zero before returning it. We can achieve this by writing the operator functions as follows:

```
/* The following program explains the customized new and delete operators */

#include <iostream.h>
#include <stdlib.h>
#include  <stddef.h>

// ------------- Overloaded new operator
void *  operator  new ( size_t   size )
{
   void * rtn = calloc( 1, size ); // Calling calloc() to allocate and initialize memory
   return  rtn;
}


// ------------ Overloaded delete operator
void  operator  delete  ( void *   ptr )
{
   free( ptr );  // Calling  free() to deallocate memory
}
void  main()
```


© Copyright Virtual University of Pakistan

```
{
   // Allocate a zero-filled array
   int *ip = new int[10];
   // Display the array
   for ( int i = 0; i < 10; i ++ )
      cout << " " << ip[i];
   // Release the memory
   delete [] ip;
}
```

The output of the program is as follows.

```
0 0 0 0 0 0 0 0 0 0
```

Note that the *new* operator takes a parameter of type *size_t*. This parameter holds the size of the object being allocated, and the compiler automatically sets its value whenever we use *new*. Also note that the *new* operator returns a *void* pointer. Any *new* operator we write must have this parameter and return type.
In this particular example, *new* calls the standard C function *calloc* to allocate memory and initialize it to zero.
The *delete* operator takes a *void* pointer as a parameter. This parameter points to the block to be deallocated. Also note that the *delete* operator has a *void* return type. Any *delete* operator we write, must have this parameter and return type.
In this example, *delete* simply calls the standard C function *free* to deallocate the memory.

Example of Overloading new and delete as Members

```
// Class-specific new and delete operators

#include <iostream.h>
#include <string.h>
#include <stddef.h>
const int MAXNAMES = 100;

class Name
{
public:
   Name( const char *s ) { strncpy( name, s, 25 ); }
   void display() const { cout << '\n' << name; }
   void * operator new ( size_t size );
   void operator delete( void * ptr );
   ~Name() {};   // do-nothing destructor
private:
   char name[25];
};
```


© Copyright Virtual University of Pakistan

```
// ---------Simple memory pool to handle fixed number of Names
char  pool[MAXNAMES] [sizeof( Name )];
int inuse[MAXNAMES];

// ---------Overloaded new operator for the Name class
void *  Name :: operator  new( size_t size )
{
   for( int p = 0; p < MAXNAMES; p++ )
         if( !inuse[p] )
         {
            inuse[p] = 1;
            return  pool + p;
         }
   return  0;
}

// ---------- Overloaded delete operator for the Names class
void  Name :: operator  delete( void *ptr )
{
   inuse[((char *)ptr - pool[0]) / sizeof( Name )] = 0;
}

void main()
{
   Name *  directory[MAXNAMES];
   char name[25];
   for( int i = 0; i < MAXNAMES; i++ )
   {
      cout  <<  "Enter name # "  <<  i+1  <<  ": ";
      cin  >>  name;
      directory[i]  =  new   Name( name );
   }
   for( i = 0; i < MAXNAMES; i++ )
   {
      directory[i]->display();
      delete directory[i];
   }
}
```

The output of the above program is given below.

```
Enter name # 1: ahmed
Enter name # 2: ali
Enter name # 3: jamil
Enter name # 4: huzaifa
Enter name # 5: arshad
Enter name # 6: umar
Enter name # 7: saleem
Enter name # 8: kamran
```


© Copyright Virtual University of Pakistan

```
Enter name # 9: babar
Enter name # 10: wasim

ahmed
ali
jamil
huzaifa
arshad
umar
saleem
kamran
babar
wasim
```

This program declares a global array called *pool* that can store all the *Name* objects expected. There is also an associated integer array called *inuse*, which contains *true/false* flags that indicate whether the corresponding entry in the *pool* is in use.
When the statement *directory[i] = new Name(name)* is executed, the compiler calls the class's *new* operator. The *new* operator finds an unused entry in *pool*, marks it as used, and returns its address. Then the compiler calls *Name*'s constructor, which uses that memory and initializes it with a character string. Finally, a pointer to the resulting object is assigned to an entry in directory.
When the statement *delete directory[i]* is executed, the compiler calls *Name* 's destructor. In this example, the destructor does nothing; it is defined only as a placeholder. Then the compiler calls the class's *delete* operator. The *delete* operator finds the specified object's location in the array and marks it as unused, so the space is available for subsequent allocations.
Note that *new* is called before the constructor, and that *delete* is called after the destructor.

Overloading [ ] Operator to Create Arrays

We know that if we overload operators *new* and *delete* for a class, those overloaded operators are called whenever we create an object of that class. However, when we create an array of those class objects, the global *operator new( )* is called to allocate enough storage for the array all at once, and the global *operator delete( )* is called to release that storage.
We can control the allocation of arrays of objects by overloading the special array versions of *operator new[ ]* and *operator delete[ ]* for the class.

Previously, while employing global *new* operator to create an array of objects, we used to tell the *delete* operator by using the array operator(*[]*) to deallocate memory for an array. But it is our responsibility to provide or to overload different type of *new* and different type of *delete*.

There is a common problem when working with arrays. While traversing elements from the array, we might run off the end of the array. This problem might not be caught by the compiler. However, some latest compilers might be able to detect this.

© Copyright Virtual University of Pakistan

```
int  iarray [10] ;
for ( int  i = 0;  i  <  100;   i++ )
{
        // Some code to manipulate array elements
}
```

If a variable is used in the condition of the loop instead of the constant value, the probability of that error increases. Variable might have an entirely different value than that anticipated by the programmer.

We can overcome this problem of array bound by overloading array operator '*[]*'. As usual before overloading, we should be clear about the functionality or semantics of the array operator. We use array operator to access an element of array. For example, when we write *iarray[5]*, we are accessing the 6th element inside array *iarray*. As we want to check for validity of index every time, an array element is accessed. We can do this by declaring the size of the array using *#define* and checking the *index* against the size every time the array is accessed.

```
#define  MAXNUM  1000
int  iarray [MAXNUM];
```

Below is the syntax of declaration line of overloaded array operator:

```
int& operator  [] ( int   index ) ;
```

In the body of this operator, we can check whether the *index* is greater or equal to the *MAXNUM* constant. If this is the case, the function may throw an exception. At the moment, the function only displays an error message. If *index* is less than *MAXNUM* and greater than or equal to zero, a reference to the value at the *index* location is returned.

Let's write a class *IntArray* and see the array manipulation.

```
/*
The following example defines the IntArray class, where each object contains
an array of integers. This class overloads the [] operator to perform
range checking.
*/

#include <iostream.h>
#include  <string.h>

class  IntArray
{
public:
    IntArray( int len );
    int  getLength( ) const;
    int &  operator[] ( int index );
    ~IntArray( );
private:
    int  length;
    int  *aray;
```


© Copyright Virtual University of Pakistan

```
};


// ------------ Constructor
IntArray :: IntArray( int len )
{
   if( len  >  0 )
   {
      length  =  len;
      aray  =  new  int[len];
      // initialize contents of array to zero
      memset( aray, 0, sizeof( int ) * len );
   }
   else
   {
      length  =  0;
      aray  =  0;
   }
}

// ------------ Function to return length
inline int  IntArray :: getLength()  const
{
   return length;
}
// ------------ Overloaded subscript operator
// Returns a reference
int & IntArray :: operator []( int index )
{
   static int dummy  =  0;
   if( (index  >=  0)  &&
      (index < length) )
      return aray[index];
   else
   {
      cout  <<  "Error: index out of range.\n";
      return  dummy;
   }
}

// ------------ Destructor
IntArray :: ~IntArray()
{
   delete aray;
}
void main()
{
   IntArray numbers( 10 );
   int  i;
   for( i = 0; i < 10; i ++ )
```


© Copyright Virtual University of Pakistan

```
    numbers[i]  =  i;        // Use numbers[i] as lvalue
  for( i  =  0; i  <  10; i++ )
    cout  <<  numbers[i]  <<  '\n';
}
```

This program first declares *numbers* of type *IntArray* object that can hold ten integers. Later, it assigns a value to each element in the array. Note that the array expression appears on the left side of the assignment. This is legal as the *operator[]* function returns a reference to an integer. This means the expression *numbers[i]* acts as an alias for an element in the private array and it can be the recipient of an assignment statement. In this situation, returning a reference is not simply more efficient but also necessary.

The *operator[]* function checks whether the specified *index* value is within range or not. If it is within the range, the function returns a reference to the corresponding element in the private array. If it is not, the function prints out an error message and returns a reference to a *static* integer. This prevents out-of-range array references from overwriting other regions of memory while causing unexpected program behavior.

Tips

- The *default constructor* is defined by the C++ compiler automatically for every class that has no default constructor (parameterless constructor) defined already.
- The default constructor (parameterless constructor) is called for each element in the array allocated with *new*.
- The *new* operator returns a *void* *, accepts a parameter of type *size_t*.
- The *delete* operator returns nothing (*void*) and accepts a pointer of *void* * to the memory block.
- With *new* operator function, a block of memory is allocated first and then constructor is called.
- With *delete* operator, destructor of the object is called first and then memory block is deallocated.
- By overloading *new* and *delete* operators, only allocation and deallocation part can be overridden.
- The same pointer that is returned by the *new* operator, is passed as an argument to the *delete* operator. These rules apply to both, if operators (*new* and *delete*) are overloaded as member or non-member operators (as global operators).

© Copyright Virtual University of Pakistan

- By overloading the array operator ([]), one can implement mechanism to check for array bound.



© Copyright Virtual University of Pakistan

Lecture No. 35

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11

11.1, 11.2, 11.3, 11.3.1, 11.3.2,

11.4, 11.4.1, 11.4.2, 11.4.3, 11.5

Summary

30) Streams
31) Source and Destination of streams:
32) Formatted Input and Output
33) Recap Streams
34) Buffered Input/Output
35) Methods with streams
36) Examples using streams

Streams

We have been discussing the concept of 'Streams' from the very beginning of the course. In this lecture, various aspects of this concept will be discussed. There are two types of streams i.e. input streams and output streams. Before going into minute details, we will see what actually these two types are. You are well aware of the terms 'cin' and 'cout', used several times in our previous programs. We have used *cout* for output and *cin* for input. Similarly, the terms of file input and file output are very much known to us. We leaned how to write in files and how to read from files. These are also streams. Let's have a look on the things and functions we can do with streams. There are some specific functions for input and output in C. *printf* and *scanf* are normally used in C. In these functions, we have to tell what type of data we are using and in which variable. Streams are counterpart of this in C++. The input output system in C++ is streams. As the name applies, it's a stream of bytes. As told earlier, it may also be termed as a door through which program can communicate with the outside world. For entering some data, we use *cin* stream while the data is read from the keyboard and stored in some variable. To display data on the screen, we can take help of *cout*.

For making the things more comprehensive, we will consider the example of a class-room. Suppose you are sitting in the class and listening to this lecture. All of a

© Copyright Virtual University of Pakistan

sudden, the students are asked by the instructor to go out of the classroom. How would you do it? You will get up from the seat and walk through the door. But you can see that all of you cannot go through the door simultaneously. You would go through the door one by one. Similarly if you have to come to the classroom, you will enter one by one and sit on the seats. In a way you are forming a sequence of people in this case. Stream is a sequence of bytes. It is an ordered sequence. Let's compare it with the door example. The person who enters first will go out of the door first. The person who enters behind someone will go out behind that person. Similarly streams are also ordered sequence. The thing that enters first into the stream will go out first. You should think streams as ordered sequence of bytes. Byte is a unit of measure. A byte can store one character, so you can think of an ordered sequence of characters.

As programmers, we communicate with our programs in English through the keyboard. We may be typing the letters, abc or the numbers, 012 on the keyboard. These all are the characters. In the programs, we store these characters in variables of different data types. Sometimes, these may be in some of our objects. On the keyboard, we type the character 'a' that is stored in some variable *c* in our program. How these two are linked? This link is formed in *cin* stream. Consider *cin* as a pipe or a door. The character 'a' is entered from one side and then a conversion takes place i.e. character is converted into its binary representation, stored in the character variable named *c*. So there is an implicit conversion happening. The same thing happens, if we have an integer *i* and press the key 1 from the keyboard. The digit 1 travels as a character but inside it is stored as number. You have to be careful while dealing with this concept. We have talked about the ASCII characters. On the keyboard, we have alphabets, numbers and symbols. When you press the number key from the keyboard, it goes at number inside the computer. In general terms, it is a character. It means that when you enter a key on your keyboard, a character sequence is generated that later goes into the computer. This character sequence has some binary representation. It does not mean a sequence of characters, but a code that goes inside the computer on pressing a key on the keyboard. This code is called as ASCII code. It is the binary representation of a character. Here both the characters 'A' and 'B' have some binary representation. Similarly '0','1','2' have some binary representation. It does not mean that the character representation of '1' is also '1'. If you are aware of the ASCII table (you have already written a program to display the ASCII table), it will be evident that the binary representation of character '1' is some other value. Similarly all the numbers 1, 2, 3 etc have some ASCII value. So whenever you press any key on the keyboard, its ASCII code goes inside the computer.

Now when we use *cin* stream to read some number from the keyboard and store it in the integer variable, its binary representation is ignored and the value is stored. So *cin* is performing this transformation operation. It is taking a character ASCII code and knows that it is supposed to represent some number. It has the ability to convert it into the appropriate number before putting it into the integer variable *i*. What happen if we use *cin* to read some value and store it in some integer variable and press some alphabet key instead of numeric keys. Some error will occur. So in the *cin* stream, an error can be detected.

Let's us look at the collection of input and output classes. These are objects having functions. This collection of classes and their functions are known as input-output

© Copyright Virtual University of Pakistan

streams in the C++. The most common things that we have been using are *cin*, used to get input from the keyboard and *cout* employed to display something on the screen. So one is input stream and the other one is output stream. Since we are not going to discuss the object- oriented programming in this course. We will not discuss the hierarchy through which these classes are derived. We have some objects for input stream and out put stream e.g. *cin* and *cout* respectively. Being objects, they have member methods that we can call. They also have the operators like '<<' and '>>', as used in the earlier programs. These operators '<<', '>>' are heavily overloaded. What does this mean? If we write *cin >> i;* here *i* is an integer. Automatically *cin* will take a character in ASCII code from the keyboard, convert it into number and store it into the integer variable. On the other hand, if we write *cin >> c;* where *c* is a character data type. When we press a key from the keyboard, it will be stored in *c* as a character. So the stream extractor operator of cin (i.e. >>, which gets the data from the stream and stores it into the variable) is already overloaded. It knows how to behave with int, char, float etc data type and what sort of conversion is required. In case of float number, we have decimal point, *cin* knows how to treat it besides converting and storing it into a float variable. Similarly if we use any character pointer i.e. string, the same >> operator has the capability of reading strings too. Obviously, one operator can't perform all these functions. It seems that we are using the same operator. Internally, this operator is overloaded. It is not the same for int, char, float, string and so on. But due to the operator overloading, its usage is very simple. We just write *cin >> i;* it works perfectly.

Source and Destination of streams

As earlier said that streams are sort of door or pipe between two things. What are these two things? For every stream, there must be some source and some destination. For *cin*, the source is normally keyboard and the destination can be an ordinary variable i.e. native-data type variable. It could be some area of memory or our own data type, i.e. object for which we have overloaded the operator and so on. So always there is a source and there is a destination.

cout is output stream. It takes the data from the program and presents it in human readable form. It also has some source and destination. The source may be some file, or the region in memory or the processor or a simple variable or our own object of our data type. The destination is normally screen. The destination can be a file, screen, or printer etc. You have used file input and file output so you know how it works. When we talk about area in memory, it may be an array that we read or write. It could also be a character string which is itself an area in the memory.

"Every stream has an associated source and a destination"

Now we will talk about yet another concept i.e. the state of stream. What does it mean? We say that *cin >> i* ; where *i* is an integer. When we give it the input 'a' and press the key 'enter', the stream knows that it is a bad input. So it is capable of signaling and setting its state specifying that some thing not good has been done. So from a program, we can always test whether the state of stream is right or not. We should carry out all kinds of error checking, debugging and error handling while writing programs. We don't want to manipulate bad data. So checking for this everywhere will be good. For example, if we write a simple program that takes two

© Copyright Virtual University of Pakistan

integers from key-board, divides one number by the other and displays the result. Following will be the code segment.

```
int i, j ;
cin > > i ;
cin > > j ;
cout < < i / j ;
```

Now we have to see what happens if the user gives a value 0 for j. We don't want to divide it by zero as even the computer does not know how to do it. When we have zero in j, our program probably will work through an exception or error before coming to a halt. If we trap this error inside the program, it would be much nicer. We can say if j is not zero, it will be good to carry out the division. So error checking and handling is always important. The same thing applies to I/O streams. When we execute input or output operation, we should check whether the operation has been carried out correctly or not. To do this, we can check the state of the stream.

Here is a simple example showing the simple use of streams.

```
/* Avoiding a precedence problem between the stream-insertion operator and the
conditional operator. */

#include<iostream>

int main()
{
        int x,y;
        cout<< "Enter two integers: ";
        cin>>x>>y;
        cout<<x << (x ==y ? " is" : " is not") <<" equal to "<< y;
        return 0;
 }
```

The output of the program:

```
Enter two integers: 3  3
3 is equal to 3
```

Formatted Input and Output

Other things that the streams provide us are a capability of formatted input and output. We have been using *cin* and *cout* very simply without any formatting. As the output of a program, we do not want that numbers should be printed in a way that makes it difficult to read and understand. We want to format the output in a way that the numbers are placed correctly at the correct position on the paper. You might have seen the electricity bills or telephone bills printed by the computers. First the empty bills are printed in the printing press containing headings with blank boxes to put in the bill entries. Then the computer prints the entries in these boxes from the system. These entries (data regarding the bill) are printed at correct places on the bill. Sometimes, you see that the entries are properly printed. That is not a computer-fault

© Copyright Virtual University of Pakistan

but only due to poor paper adjustment in the printer. The printing of these entries is carried out by with the use of formatted output.

The second example is the display of a matrix on the screen. Suppose, we want that the numbers of a column are displayed up and down in a column. Similarly the second column should be displayed and so on. At first, we will format these things.

When we do word processing, we type a paragraph. The lines of the paragraph are left justified but ragged on right hand side. In word processing, we have a choice to justify the paragraph. By doing this, the left and right margins of the paragraph are put in a straight line while adjusting the space between the words. Now look what happens if we want to print a string with *cout* and want it left or right justified. This is what we call formatting the output. Similarly, we want to print the value of *pi* which is stored in a variable as 3.1415926. But we want that it should be printed as 3.141 that means up to three decimal places.
There should be a method of formatting it. Thus by formatting the output, the presented representation (which we read as human being) can be different from the internal representation. So we can do a lot of formatting with these streams.

Besides, there are member functions with the streams. Let's look at the member functions of *cin*. The first one is the *get* function. We can use it by writing:

cin.get();

The notation explains that *cin* is an object (cin is an object of input stream) and *get* is a member function of it. This function reads a character. In this case, it reads a single character from key board and returns it as *cin* is calling it. We have two variants of this get function with *cin*. One is that *cin.get* returns a character. W e can write it as under:

c = cin.get() ;

The second method is *cin.get(character variable) i.e.* one character at a time. It works with characters, not through number or string. It is one character at a time.

The second function of *cin* is the *read* function. This function differs from the *get* function in the way that it returns a buffer instead of a single character. So we can point to a buffer and tell the number of characters to be read. We normally provide a delimiter, a specific character up to which we want to read. Normally we use the new line character as a delimiter and read a single line at a time.

Thus, we have three ways of obtaining input with *cin*, which is an object of type input stream (istream). If we create an object of type *istream*, it will also get these functions as it is derived from the same class.

We have seen that there are many methods and operators associated with *cin*. The same thing applies to *cout*. *cout* is the output stream which usually, presents the data from the computer in human readable form. The operator associated with *cout* is the stream insertion (<<). That means we insert the operator in the stream and the stream displays the output on the screen. So the operator with cout is << and it displays the

© Copyright Virtual University of Pakistan

value of the data variable that we provide it after the << sign. Thus to display the value of an integer variable i, we can write *cout* << *i* ; If we want to format the output, it can also be done here.

The cout has a function *write* with it. This function can be used if we want to write a chunk of data from the buffer. Similarly, to output a single character, *cout* has the function named *put*. It can be written as:

cout.put(character variable) ;

Here, it will display the value of the character variable.

Recap streams

Streams are nothing but an ordered sequence of bytes.
They allow data to move from one part of the computer to another which may be the screen or key board from and to, or from memory or files on disc and so on.
Byte stream is used to connect the source and the destination.
These byte streams are implemented as objects. Being objects, they do have their member functions and have their member operators. The member operators are heavily overloaded to allow these streams to handle a variety of data types.
The streams have a state that can be checked by us. We have used eof (end of file) with the file reading. This is a way to check the state of the stream.

While using I/O streams, we have to include some header files. Whenever we use *cin* and *cout*, the file *iostream.h*, is included in which all these classes and objects have been defined. For the formatted input and output, we manipulate the streams. To do stream manipulations, we have to include a header file having the name *iomanip.h*. We can understand that *iomanip* is a short hand for input output manipulation.

Now let's take a look at the standard streams which are provided to our programs. Whenever we write a C++ program and include *iostream.h* in it, we get a stream for input (reading) that is *cin.* This is a built in thing. We can use this object. Similarly, for output (writing), we get *cout*. Other than these, we get some other streams by default. These include *cerr* (read as c error) and clog. To understand these streams, we have to talk about buffered input and output.

Buffered Input/Output

In computers, most of the components relate to electronics like chips, memory, micro processor etc. There are also electro-mechanical things like disc. The key board itself is an electro mechanical accessory. The electro mechanical parts of the computer are normally very slow as compared to the electronic components. So there is a difference between the two in terms of speed. Secondly, every input/output operation costs computer time. Input/output costs and the I/O devices (keyboard, monitor and disc etc) are slower as compared to the speed of the microprocessor and the memory being used. To overcome this speed difference, we use the mechanism, called buffered input/output. Suppose, we have a program which executes a loop. That loop outputs a number in each of iteration to store in a file on the disc. If we write the output number

© Copyright Virtual University of Pakistan

to the disc in each iteration, it will be the horrendously wastage of computer time. It means that the disc is electro mechanical device. Similarly in each iteration, the mechanical movement takes time. But if we gather the data and write it to the disc, there will be one mechanical movement. The heads of the disc will move mechanically once to a point and the whole chunk of data will be written on the disc. This is the more efficient way of using the disc. So whenever we have a program that writes the output data to the disc, it will be nice to collect the output data (numbers) and write it on the disc in one write operation instead of writing the numbers one by one. The area where we gather the numbers is known as buffer. The example stated in this case is the buffered output. In this case, the output does not go directly to the disc. We first gather the data in a buffer and then write it on the disc.

Now think about another situation. Suppose we have a program that performs very complex calculations. It means that there is a *while* loop that performs so heavy calculations that each of the iteration takes, say one minute and then provides the result. Now we want to write that output to a file on the disc and see the iteration number of the loop on the screen. We do not want to write the output to the disc after each iteration. We gather the data in a buffer. In the meantime, we want to see the loop counter on the screen. If we gather this output of counter number in a buffer, it may happen that the buffer gathers the iteration numbers for 250 iterations before displaying it on the screen. Thus, we see on the screen the numbers 1, 2, 3 …..250, when 250 iterations have been performed. There are again 250 numbers gathered at one time. We see numbers 251, 252 ……500, when 500 iterations have been performed. When we start the program, there will be two buffers gathering data. One buffer gathers the data to write to the disc and the other gets the data of iteration numbers to display on the screen. As we said that each iteration takes one minute, meaning that the iteration numbers will not be seen for a long time. Rather, these will be shown after 250 iterations (i.e. 250 minutes). During this period, we do not show any thing on the screen. Here, we are not sure whether the program is executing properly as we do not see any iteration number on the screen. We want to know after every minute that loop has executed once. So after every minute, the loop counter should be displayed on the screen, so that at any time we could see how many iterations have been performed. For this, we need unbufffered output on the screen. Thus, in the same program we require buffered and unbuffered output.

Now these requirements are contradictory with different issues. These are met in our system with the *cerr* object. *cout* is a buffered output. We cannot see it as nowadays compilers are very intelligent. If you know UNIX, or command prompt and input output redirection, we can actually see this in operation. Perhaps, you can create an example to understand this. For the moment just try to understand that *cout* is buffered output. It gathers data and sends it to the screen. On the other hand, *cerr* is unbuffered output. It will show the data on the screen at the same time when it gets it. So *cerr* is an output stream, an *ostream* object but unbuffered. It shows data immediately. So we can use something like the *cerr* object that will show how many loops have been executed. It will use something like *cout* to write on the disk to buffer the output. In this case, we are getting efficiency in addition to information. Normally we use *cerr* object in C++ for this purpose. Besides, we also have *clog*. It is also known as standard log having detailed information of the log. To collect information of the program, we write it with *clog*. Normally when we execute our programs- *cout, cerr* and *clog*, all are connected to screen. We have ways to direct them at different

© Copyright Virtual University of Pakistan

destinations. That depends on the operating system. Suppose, we specify the buffer size, normally the operating system or compiler does this for us. A typical size of buffer is 512 bytes. When the information is of 512 byte size, output will take place. But in the program, we may want at some point that whatever is in the buffer, show them. Is there a way of doing that? The normal mechanism is flush. Flush the stream. The flush command forces the data from the buffer to go to its destination which is normally a screen or file and make the buffer empty.

Uptil now, we have been using two things to end the line. One is new line character i.e. "\n". When we are displaying something on the screen, it makes the next output to start from the next line. The cursor moves to the left margin of the next line on the screen. New line is just a character. The other one was *endl*. If we write *cout << endl;* It seems that the same thing happens i.e. the cursor moves to the left margin of the new line. But *endl* actually does something else. It flushes the output too. As a result, it seems that *cout* is unbuffered i.e. its output is immediately available on the screen. Depending on the compiler and operating system, you may or may not see the buffered effect. But one thing is clear that while reading the source code, you will know where *cout* is used and where *cerr*. Typically, that is also the programming style where the output of *cerr* is informative and small. It shows that where is the control in the program. The output of *cout* is more detailed and the actual output of the program. There are benefits of these things in the code. In case of *cin*, it is alone. For output, we have *cout*, *cerr* and *clog*. In DOS, we have two more output streams i.e. *caux* (auxiliary input output stream) and *cprn* (printer output). These are no more relevant now.

Predefined Stream Objects:

Object	**Meaning**
cin	Standard input
cout	Standard output
cerr	Standard error with unbuffered output.
clog	Standard error with buffered output
caux	Auxiliary (DOS only)
cprn	Printer(DOS only)

Now let's take a look at the operators associated with these streams. We have been using the stream insertion operators '<<' with *cout*. We need to understand that how these operators are implemented. Using *cout*, we can chained the output. It means that we can write as *cout << "The value of the first integer is " << i;* This is the single *cout* statement. How does that work? What happens is the first part goes to *cout*. In this case, it is the string "The value of the first integer is". The data travels in the direction of the arrows. This string is inserted in the stream and displayed on the screen. What about the rest of the statement i.e. *<< i;* this again needs *cout* on the left side to be executed. It should look like as *cout << i;* Once, we have defined that this is the behavior expected by us. Then, we understand that this is exactly the way it has been programmed. The stream insertion operator '<<' is overloaded for the output stream and it returns the reference of the output stream. The syntax of stream insertion operator is:

```
ostream& ostream::operator << (char *text);
```


© Copyright Virtual University of Pakistan

The important thing to note is that this operator returns the reference to the *ostream* object itself. Whenever we write a chained output statement, it is executed from left to right. So *cout << " The value of the first integer is"* is processed first from left to right. The process is that this character string is displayed on the screen. As per the prototype and definition, it returns the reference to the *ostream* object. In this case, the object was *cout*, so a reference to the *cout* object is returned. Now the rest of the statement becomes as *cout << i;* It is processed quite nicely. This allows the stream insertions to be chained. It is also applicable to the input. So if we say something like *cin >> i >> j;* both *i* and *j* are integers. Now again, it is processed from left to right. This is the *istream* and the extraction operator will return the reference to the *istream* object i.e. *cin*. So at first, the *cin >> i* is processed which will return the reference to the *cin* object. The rest of the statement seems as *cin << j;* It is important to understand how these operators work. You can see their prototypes that they return *iostream* objects themselves. That is the why, we can chain them. Now let's see what are the other various methods associated with these input output streams.

Methods of streams

There are some other functions associated with *cin* stream. We have used some of them. We have used *get()* and *read()* methods with input stream. Another member function of *cin* is *getline()*. It reads a complete buffer i.e. the number of character specified up to a delimiter we specify. We can write something like:

```
cin.getline(char *buffer, int buff_size, char delimiter = '\n')
```

The character data is stored in **buffer*. *buff_size* represents the number of characters to be read. If we specify its value 100, then *getline* will read 99 characters from the keyboard and insert a null character in the end. As you know, in C++ every character string ends with a null character. We can also give it a delimiter. Sometimes, we may want to read less character. Normally, the delimiter is the new line character. So while typing on the keyboard, if we press the enter key then it should stop reading further and put the data into the variable *buffer*. So there is a *getline* fiunction.

There are some other interesting functions also. When we use *cin.get()*, a character is read. We can throw back the character gotten by this *get* function by using the *unget()* function. So we can use *cin.unget() that* will return the most recently (last) gotten single character.

We have a function *peek()*, also written as *cin.peek()*; The purpose of this function is that we can see the next character that we are going to get. This function returns the next character that would be read if we issue *cin.get()*.

All these functions (getline, get, read, unget and peek) are implemented as member functions of the input class.

Similarly, there are functions associated with *cout*. We have *cout.putline()*; which outputs a buffer. Actually we have no need of this function because *cout*, itself, knows how to handle character strings. Then we have *cout.write()*; which can perform a raw, unformatted output. The function *cout.put()*; is like a formatted output. It performs character by character output. We can do many formatting conversions by using the stream insertion operator (i.e. <<) with *cout*. We can write an overloaded function of

© Copyright Virtual University of Pakistan

stream insertion (<<) to input or output a complex number. We know that a complex number has two parts i.e. real and imaginary. We can write the overloaded function such that if we give two numbers with space between them it could read it. We can also write it as that it could read two numbers (that are real and imaginary parts of a complex number) separated by comma. Thus there may be different ways to write the overloaded operator.

The white space character is very significant. We can show it by a simple example. Suppose we have an array *name* of 60 characters. We get a name from the user in this array by using *cin* and then display this string by *cout*. The code segment for this purpose can be written as:

```
char name [60] ;
cin >> name ;
cout << name ;
```

Now when the user enters the name, suppose it enters 'naveed malik' that is a name containing two words with a space between them. When we display this name by using *cout*, only 'naveed' is displayed on the screen. It means that only one word 'naveed' was stored in the array. The reason for it that the streams (cin, cout) are sensitive to white space character that is treated as a delimiter. Now where is the second word 'malik'. It has not got deleted yet. It is in the buffer of the stream. This example will read like the following:

```
char nam1 [30], name2 [30] ;
cin >> name1 >> name2 ;
```

Thus, we have two character arrays now. We can write 'naveed malik' and press enter. The first part before space (naveed) will go to the first array *name1* when that array is used with *cin*. We will write another *cin* with *name2* and the second part (malik) will go to the second array *name2*. So things don't disappear. They stay in the buffer till you actually expect them. We have to be careful about that.

Examples using streams

A simple example showing the use of getline function.

```
// A simple example showing the use of getline function.
#include <iostream.h>

int main()
{
   const int SIZE = 80;
   char buffer[SIZE];

   cout << " \n Enter a sentence: \n" ;
   cin.getline(buffer, SIZE);

   cout << " The sentence entered is: \n" << buffer << endl;
   return 0;
}
```


© Copyright Virtual University of Pakistan

Output of the program.

```
Enter a sentence:
this is a test
The sentence entered is:
this is a test
```

A simple example showing the use of read and write functions.

```
// A simple example showing the use of read and write functions.
#include <iostream.h>

int main()
{
   const int SIZE = 80;
   char buffer[SIZE];

   cout << " \n Enter a sentence: \n" ;
   cin.read(buffer, 20);

   cout << " The sentence entered was: \n";
   cout.write(buffer, cin.gcount());
   cout << endl;
   return 0;
}
```

Output of the program.

```
Enter a sentence:
This is a sample program using read and write functions
The sentence entered was:
This is a sample pro
```


© Copyright Virtual University of Pakistan

Lecture No. 36

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11

11.6, 11.7

Summary

37) Stream Manipulations
38) Manipulators
39) Non Parameterized Manipulators
40) Parameterized Manipulators
41) Format State Slags
42) Formatting Manipulation
43) Showing the Base
44) Scientific representation
45) Exercise

Stream Manipulations

After having a thorough look into the properties and object definition of I/O streams, we will discuss their manipulations. Here, there is need of some header files to include in our program the way, *cin* and *cout* are used. We include *iostream.h* and *fstream.h* while using file manipulations. In case of manipulation of the I/O streams, the header file with the name of *iomanip.h* is included. It is required to be included whenever there is need of employing manipulators.

As discussed earlier, we can determine the state of a stream. The states of the stream can be determined. For example, in case of *cin*, we can check where the end of file comes.

For state- checking, these stream objects have set of flags inside them. These flags can be considered as an integer or long integer. The bit position of these integers specifies some specific state. There is a bit for the end of file to test. It can be written as under:

```
cin.eof() ;
```

It will return the state of end of file. The bit will be set if the file comes to an end. Similarly, there is a *fail* bit. This bit determines whether an operation has failed or not.

© Copyright Virtual University of Pakistan

For example, an operation could be failed due to a formatting error. The statement *cin.fail*; will return the value of the *fail* bit. As this statement returns a value, we can use it in an 'if statement' and can recover the error. Then there is a *bad* bit. This bit states that data has lost. The presence of this bit means that some data has lost during I/O operation. So we can check it in the following manner.

```
cin.bad();
```

It can be used in parentheses as a function call that will allow us to check whether the operation failed or successful. Similarly we can also check for' *good',* that is a bit showing that everything is good. This bit will be set if *fail* and *bad* bits are not set. We can check this bit as *cin.good* ; and can find out whether the input operation was successful. If some bit like *bad* has been set, there should also be a mechanism to clear it. For this, we have

```
cin.clear() ;
```

as a member function for these objects. This will reset the bits to their normal good state. This is a part of checking input stream.

Manipulators

Whenever carrying out some formatting, we will want that the streams can manipulate and a number should be displayed in a particular format. We have stream manipulators for doing this. The manipulators are like something that can be inserted into stream, effecting a change in the behavior. For example, if we have a floating point number, say pi (л), and have written it as *float pi = 3.1415926* ; Mow there is need of printing the value of pi up to two decimal places i.e. 3.14 . This is a formatting functionality. For this, we have a *manipulator* that tells about width and number of decimal points of a number being printed. Some manipulators are parameter less. We simply use the name of the manipulator that works. For example, we have been using *endl*, which is actually a manipulator, not data. When we write *cout << endl* ; a new line is output besides flushing the buffer. Actually, it manipulates the output stream. Similarly *flush* was a manipulator for which we could write *cout << flush* that means flushing the output buffer. So it manipulates the output.

A second type of manipulators takes some argument. It can be described with the help of an example. Suppose we want to print the value of pi up to two decimal places. For this purpose, there should be some method so that we can provide the number i.e. two (2) up to which we want the decimal places. This is sent as a parameter in the manipulators. Thus we have the parameterized manipulators.

Let's have a look on what streams do for us. We know that streams are like ordered sequence of bytes and connect two things i.e., a source and a destination. In the middle, the stream does some conversion. So it may take some binary representation of some information and convert it into human readable characters. It may also take characters and convert them into an internal representation of data. With in the conversion of data, we can do some other things. For example, you might have seen that if a system prints computerized cheques, it puts some special characters with the numbers. If there is a cheque for four thousand rupees, this amount would be written on it as ****4000.00. The idea of printing * before the amount is that no body could

© Copyright Virtual University of Pakistan

insert some number before the actual amount to change it. As we don't want that somebody has increased the amount on the cheque from Rs 4000 to Rs 14000.The printing of * before the amount is a manipulation that we can do with input or output objects. We can also tell the width for a number to be printed. So there are many conversions that we can do. We can use fill characters like * as mentioned in the example of cheque printing. To accomplish all these tasks, there are different methods. So it becomes a little confusing that the same work is being done through 2-3 different methods. Some are inline manipulators like *endl*. We can use it with << and write inline as *cout << endl* ; The same work could be done with the *flush* method. We could also write *cout.flush* ; Thus it is confusing that there is an inline manipulator and a function for the same work.

Non-Parameterized Manipulators

Let's start with simple manipulators. We have been dealing with numbers like integers, floats etc for input and out put. We know that our number representations are associated with some base. In daily life, the numbers of base 10 are used in arithmetic. When we see 4000 written on a cheque, we understand that it is four thousands written in the decimal number system (base 10). But in the computer world, many systems are used for number representation that includes binary (base 2), octal (base 8), decimal (base 10) and hexadecimal (base 16) systems. A simple justification for the use of these different systems is that computers internally run on bits and bytes. A byte consists of eight bits. Now if we look at the values that can be in eight bits. 256 values (from 0 to 255) can be stored in eight bits. Now consider four bits and think what is the highest number that we can store in four bits. We know that the highest value in a particular number of bits can be determined by the formula $2^n - 1$ (where n is the number of bits). So the highest value that can be stored in four bits will be $2^4 - 1$ i.e. 15. Thus the highest value, we can store in four bits is 15 but the number of different values that can be stored will be 2^n i.e. 16 including zero. Thus we see that while taking half of a byte i.e. four bits, 16 (which is the base of hexadecimal system) different numbers can be stored in these four bits. It means that there is some relationship between the numbers that have a base of some power of two. So they can easily be manipulated as bit format. Thus four bits are hex. What about eight (octal)? If we have three bits, then it is $2^3 = 8$, which is the base of octal system. Thus, we can use three bits for octal arithmetic.

We can use manipulators to convert these numbers from one base to the other. The manipulators used for this purpose, can be used with *cin* and *cout*. These are non-parameterized manipulators. So if we say the things like *int i = 10* ; Here *i* has the decimal value 10. We write *cout << i* ; and 10 is being displayed on the screen. If we want to display it in octal form, we can use a manipulator here. If we write

```
cout << oct << i ;
```

it will display the octal value of i (10) which is 12. This manipulator converts the decimal number into an octal number before displaying it. So the octal representation of 10 which is 12, will be displayed on the screen. Similarly if we write

```
cout << hex << i ;
```


© Copyright Virtual University of Pakistan

Here hex stands for the hexadecimal. *hex* is the manipulator that goes into the out put stream before *i* and manipulates the stream by converting the decimal number into a hexadecimal number. As a result, the hexadecimal value of 10 is displayed on the screen. If we have a number in octal or hexadecimal system , it can be converted into a decimal number by putting the *dec* manipulator in the stream like

```
cout << dec << i ;
```

These (oct, hex and dec) are very simple inline manipulators without any argument. There is also a manipulator for white space. The white space has a special meaning. It is a delimiter that separates two numbers (or words). In *cin* and *cout*, the white space acts as a delimiter. If we want that it should not act as a delimiter, it can used as a *ws* manipulator. This manipulators skips white space. This manipulator takes no argument. This *ws* manipulator is sometime useful but not all the times. The following table shows the non-parameterized manipulators and their description.

Manipulator	**Domain**	**Effect**
dec	In / Out	Use decimal conversion base
hex	In / Out	Use hexadecimal conversion base
oct	In / Out	Use octal conversion base
endl	Output	Inserts a new line and flush the stream
ends	Output	Terminate a string with NULL
flush	Output	Flush the stream
ws	Input	Skip leading whitespace for the next string extraction only

The base becomes important while doing programming of scientific programs. We may want that there is the hexadecimal presentation of a number. We have discussed the justification of using hexadecimal or octal numbers, which is that they match with bits. Here is another justification for it. Nowadays, computers are just like a box with a button in front of them. A reset button is also included with the main power button. While seeing the pictures or in actual Miniframe and mainframe computers, you will notice that there is a row of switches in front of them. So there are many switches in front of these computers that we manipulate. These switches are normally setting directly the values of registers inside the computer. So you can set the value of register as 101011 etc by switching on and off the switches. We can do that to start a computer or signaling something to computer and so on. There are a lot of switches in front of those computers ranging between 8 to 16. You have to simply remember what is the value to start the computer. Similarly, it will require the reading the combinations of switches from the paper to turn on the computer. This combination tells you which switch should be on and which should be off. As a human being instead of remembering the whole pattern like 10110000111 etc, it could be easy to remember it as 7FF. Here we are dealing with HEX numbers. For the digit 7, we need four bits. In other words, there is need to set four switches. The pattern of 7 is 0111. So we set 7 with this combination. For F, all the four bits should be on as 1111 and so on. Thinking octal and hexadecimals straight away maps to the bits. It takes a little bit of practice to effectively map on the switches. On the other hand, decimal does not map to those bits. What will be its octal number in case of decimal number 52.? You have to calculate this. What is the binary representation of 52? Again you have to

© Copyright Virtual University of Pakistan

calculate. There are a lot of things which you have to calculate. On the hand, if we say 7ABC in case of HEX, we are mapping the number straight away on four bits. In octal system, we map the number on three bits. A is ten so it will be 1010 so you can quickly set the switches. It may not be relevant today. But when you are working with big computers, it will become quite relevant. There are many times when we have to manipulate binary data using mechanical device while thinking in hexadecimal and octal terms. In the language, you have the facility to set the base. You can use *setbase(), hex, oct, dec* and *setf*. There are many ways of doing the same thing. Programmers write these languages. Therefore they make this facility available in the language as built in.

Parameterized Manipulators

Suppose we want to print the number 10 within a particular width. Normally the numbers are written right justified. In case of no action on our part, *cout* displays a number left justified and in the space required by the number. If we want that all numbers should be displayed within the same particular width, then the space for the larger number has to be used. Let's say this number is of four digits. Now we want that there should be such a manipulator in the output that prints every number in a space of four digits. We have a manipulator *setw* (a short for set width), it takes as an argument the width in number of spaces. So to print our numbers in four spaces we write

```
cout << setw(4) << number ;
```

When printed, this number gets a space of four digits. And this will be printed in that space with right justification. By employing this mechanism, we can print values in a column (one value below the other) very neat and clean.

Now in the example of printing a cheque, we want that the empty space should be filled with some character. This is required to stop somebody to manipulate the printed figure. To fill the empty space, there is need of manipulator *setfill*. We can write this manipulator with *cout* as the following

```
cout << setfill (character) ;
```

where the *character* is a single character written in single quotes. Usually, in cheque printing, the character * is used to fill the empty spaces. We can use any character for example, 0 or x. The filling character has significance only if we have used *setw* manipulator. Suppose, we are going to print a cheque with amount in 10 spaces. If the amount is not of 10 digits, the empty space will be filled with *. Thus the usage of *setfill* is there where we use *setw* for printing a number in a specific width. So if we want to print an amount in 10 spaces and want to fill the empty spaces with *, it can be written as under.

```
cout << setfill(*) << setw(10) << amount ;
```

Thus the manipulators can also be of cascading nature. The stream insertion operator (<<) is overloaded and every overload of it returns a reference to the *cout* object itself. This means that while working from left to right, first the fill character will be set returning a reference to *cout* . Later, its width will be set to 10 character and return a

© Copyright Virtual University of Pakistan

reference to *cout again*. Finally, the amount will be displayed in the required format. Thus, we have manipulated the stream in two ways. Example of a pipe with two bends can help it understand further. Now whatever figure goes into it, its width and the fill character is set and things are displayed in the space of 10 characters. If we want to print an amount of Rs 4000 on the cheque, it will be printed on the cheque as ******4000. Thus, we have two manipulators, *setw* and *setfill* which are used with *cout*.

Let's further discuss the same example of cheque. In real world, if we look at a computer printed cheque , the amount is printed with a decimal point like 4000.00 even if there is no digit after decimal point. We never see any amount like 4000.123, as all the currencies have two- digit fractional part. Thus, we examine that the fractional part has been restricted to two decimal places. The decimal digits can be restricted to any number. We have a manipulator for this purpose. The manipulator used for this purpose is *setprecision*. This is a parameterized manipulator. It takes an integer number as an argument and restrict the precision to that number. If we write

```
cout << setprecision (2) << float number ;
```

The above statement will display the given float number with two decimal places. If we have the value of pi stored in a variable, say *pi*, of type float with a value of 3.1415926 and want to print this value with two decimal places. Here, manipulator *setprecision* can be used. It can be written as under.

```
cout << setprecision (2) << pi ;
```

This will print the value of *pi* with two decimal places.

Now think about it and write on the discussion board that whether the value of pi is rounded or truncated when we print it with setprecision manipulator. What will be the value of pi with five decimal places and with four decimal places? Will the last digit be rounded or the remaining numbers will be truncated?

At this point, we may come across some confusion. We have learned the inline manipulators that are parameter less. For these, we simply write *cout << hex << number;* which displays the number in hexadecimal form. There is also a parameterized manipulator that performs the same task. This manipulator is *setbase*. It takes the base of the system (base, to which we want to format the number) as an argument. Instead of using *oct, dec* and *hex* manipulators, we can use the *setbase* manipulator with the respective base as an argument. So instead of writing

```
cout << oct << number ;
```

we can write

```
cout << setbase (8) << number ;
```

The above two statements are equivalent in the way for having the same results. It is a matter of style used by one of these manipulators. We can use either one of these, producing a similar effect. The *cout << setbase(8)* means the next number will be printed in the base 8. Similarly *cout << setbase(16)* means the next number will be printed in hexadecimal (base 16) form. Here a point to note is that *setbase (0)* is the same as *setbase(10).*

© Copyright Virtual University of Pakistan

Following is the table, in which the parameterized manipulators with their effect are listed.

Manipulator	Domain	Effect
resetioflags(long f)	In / Out	Clear flags specified in f
setbase (int b)	In / Out	Set numeric conversion base to b (b may be 0, 8, 10 or 16)
setfill (int c)	Output	Set fill character to c
setiosflags(long f)	In / Out	St flags specified in f
setprecision (int p)	Output	Set floating point precision to p
setw (int w)	Output	Set field width to w

Format State Flags

We have discussed that there are flags with the stream objects. This set of flags is used to determine the state of the stream. The set includes *good, fail, eof* etc that tells the state of the stream. There is also another set of flags comprising the ones for input/output system (ios). We can use *setioflag,* and give it as an argument a *long* number. Different bit values are set in this number and the flags are set according to this. These flags are known as format state flags and are shown in the following table. These flags can be controlled by the *flags*, *setf* and *unsetf* member functions.

Format state flag	**Description**
ios::skipws	Skip whitespace character on an input stream.
ios::left	Left justify output in a field, padding characters appear to the right if necessary.
ios::right	Right justify output in a field, padding characters appear to the left if necessary.
ios::internal	Indicate that a number's sign should be left justified in a field and a number's magnitude should be right justified in that same field (i.e. padding characters appear between the sign and the number).
ios::dec	Specify that integers should be treated as decimal (base 10) values.
ios::oct	Specify that integers should be treated as octal (base 8) values.
ios::hex	Specify that integers should be treated as hexadecimal (base 16) values.
ios::showbase	Specify that the base of a number is to be output ahead of the number(a leading 0 for octals, a leading 0x or 0X for hexadecimals).
ios::showpoint	Specify that floating-point numbers should be output with a decimal point. This is normally used with ios::fixed.
ios::uppercase	Specify that uppercase letters (i.e X and A through F) should be used in the hexadecimal integers and the uppercase E in scientific notation.
ios::showpos	Specify that positive and negative numbers should be preceded by a + or - sign, respectively.

© Copyright Virtual University of Pakistan

ios::scientific	Specify output of a floating-point value in scientific notation.
ios::fixed	Specify output of a floating-point value in fixed point notation with a specific number of digits to the right of the decimal point.

Let's talk about more complicated things. We discussed a parameterized manipulator *setw* that sets the width to print in the output. There is an alternative for it i.e. the member function, called '*width()'*. This function also takes the same parameter and an integer, in which width the things are to display or read. This function applies to both input and output stream. For this, we write *cin.width (7).* This will create a format field of the width of 7 characters for an input. Now we write *cout.width (10)* ; this will set the width of output field to 10. With it, the next number to be printed will be printed in 10 spaces. Thus *setw*, inline manipulator has the alternative function *cin.width* and *cout.width* with single argument.

It equally applies to the *setprecision*. This is the parameterized, inline- manipulator that sets the places after the decimal point. There is a member function as well in these objects that is *precision*. The *setprecision* is an inline manipulator, used along with stream insertion (<<). If we want to do the same thing with a function call, *cout.precision(2)* is written. It has the same effect as that of *cout << setprecision (2)*. Thus we have different ways of doing things.
We have used *setfill* manipulator. Here is another member- function i.e. *cout.fill*. The behavior of this function is exactly the same. We simply write *cout.fill('*')* ; identical to *cout << setfill('*')*. The filling character is mostly used whenever we use financial transactions but not necessarily. We can also use zero to fill the space.
So *fill* and *setfill*, *width* and *setw*, *precision* and *setprecision* and almost for every inline manipulator, there are member functions that can be called with these streams.
The member functions are defined in *iostream.h. However*, the manipulators are defined in *iomanip.h*. Normally we have been including *iostream.h* in our programs to utilize the member functions easily. But inclusion of a header file 'iomanip.h file is must for the use of manipulators.
We should keep in mind that when we can write inline manipulators in the following fashion.

```
cout << setw (7) << i ;
```

And in the next line we write

```
cout << j ;
```

Here the *setw* manipulator will apply to *i* only and not after that to *j*. This means that inline manipulators apply only to the very next piece of data i.e. output. It does not apply to subsequent output operations.

Formatting Manipulation

We can adjust the output to left side, right side or in the center. For this purpose, we have a member function of the object whose syntax is as under:

cout.setf(ios:: *flag*, ios:: adjust field)

The *setf* is a short for set flag. The flags are long integers, also the part of the objects. They are the bit positions, representing something. Here we can set these flags. The

© Copyright Virtual University of Pakistan

flags of *adjustfield* are set with values i.e. left, right, left | right and internal. The description of these is as follows.

Value of flag	Meaning	Description
left	Left-justify output	Justifies the output to left side
right	Right-justify output	Justifies the output to right side
left | right	Center output	Centralized the output
internal	Insert padding	Places padding between signs or base indicator and the first digit of a number. This applies only to number values and not to character array.

Following is the code of a program that shows the effects of these manipulators.

```
//This program demonstrate the justified output

#include <iomanip.h>
#include <iostream.h>

void main()
{
        int i = -1234;
        cout.setf(ios::left, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::right, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::internal, ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cout.setf(ios::left | ios::right,
        ios::adjustfield);
        cout << "|" << setw(12) << i << "|" << endl;
        cin >> i ;
 }
```

Following is the output of the above program.

```
|-1234       |
|       -1234|
|-       1234|
|       -1234|
```

We have discussed two types of manipulators for base, the parameter less manipulator in which we look for *oct*, *dec* and *hex*. On the other hand, there is a parameterized manipulator *setbase* which takes an integer to set the base. It uses 0 or 10 for decimal, 8 for octal and 16 for hexadecimal notations.
Now we have a generic function *setf* that sets the flags. We can write something like

cout.setf(ios::hex)

© Copyright Virtual University of Pakistan

The hex is defined in *ios*. It has the same effect. It sets the output stream, in this case *cout*, to use hexadecimal display for integers. So it is the third way to accomplish the same task. The use of these ways is a matter of programming style.

Showing the base

Now there should be someway to know which base has the number output by the programmer. Suppose we have a number 7ABC, then it be nothing but hexadecimal. What will be the nature of 7FF. It is hexadecimal. However, the number 77 (seven seven) is a valid number in all of different basis. We have a built-in facility *showbase*. It is a flag. We can set the *showbase* for output stream that will manipulate the number before displaying it. If you have the *showbase* falg on (by default it is off), a number will be displayed with special notations. The *setf* function is used to set the flag for the base field. Its syntax is as under:

cout.setf(ios::*base*, ios::basefield);

Here base has three values i.e. oct, dec and hex for octal, decimal and hexadecimal systems respectively. If the basefield is set to *oct* (octal), it will display the number with a preceding zero. It shows that the number is in octal base. If the basefield is set to *hex* (hexadecimal), the number will be displayed with a preceding notation 0x. The number will be displayed as such if the basefield is set to *dec* (decimal). If there is a number, say 77, it will be difficult to say that it is in octal, decimal or hexadecimal base, a valid number for all the three systems. However, if we output it with the use of *showbase*, it will be easy to understand in which base the output number is being represented. The following example, demonstrates this by showing the number (77) along with the base notation.

```
/* This program demonstrate the use of show base.
It displays a number in hex, oct and decimal form.
*/

#include <iostream.h>

void main()
{
   int x = 77;
   cout.setf(ios::showbase);
   cout.setf(ios::oct,ios::basefield);   //base is 8
   cout << x << '\n';                    //displays number with octal notation
   cout.setf(ios::hex,ios::basefield);   //base is 16
   cout << x << '\n';                    //displays number with hexadecimal notation
   cout.setf(ios::dec,ios::basefield);
   cout << x << '\n';
}
```

Following is the output of the program.

```
 0115
0x4d
77
```


© Copyright Virtual University of Pakistan

Scientific Representation

When the numbers get bigger, it becomes difficult to write and read in digits format. For example, one million will be written as 1000000. Similarly hundred million will be 100000000 (one with eight zeros). How will we display the number which is of 20-digit long? For this, we use scientific notation. To do it, a manipulator *ios:: scientific* can be used. If the flag in *setf* to the *scientific* is set, it can be written as

```
cout.setf(ios::scientific, ios::floatfield) ;
```

Then the floating point numbers will be displayed in scientific notation. A number in scientific is like 1.946000e+009. So we can set the state of output stream to use scientific notation for outputting a number.
To do the scientific notation off and restore the default notation, we set the flag in *setf* function to *fixed* (which is a short for fixed point notation). This can be written as

```
cout.setf(ios::fixed, ios::floatfield) ;
```

Uppercase/Lowercase Control

Similarly, we have a manipulator *ios::uppercase*. While using this manipulator, the e in scientific notation is written in uppercase i.e. E. If we are using hexadecimal numbers, then the characters of it will be displayed in uppercase letters as A, B, C, D, E and F.

Exercise

- We have been using matrices i.e. a two dimensional array. As an exercise, try to print out a matrix of three rows and three columns, in such a way that it should be nicely formatted. The numbers should be aligned as we write it in a neat and clean way on the paper. You can use the symbol | at the start and end of each row as we don't have a so big square bracket to put around three rows. To be more elegant to print a matrix, we can use a proper graphic symbol to put square brackets around the matrix instead of using | symbol. In the ASCII table, there are many symbols that we can use to print in our programs. We have the integer values of these symbols in the table. Suppose you have a value 135 of a symbol. Now to print this symbol, press the 'alt' key and keeping the key pressed enter the integer value i.e. 135 from the num pad of the key board, release the 'alt' key. Now you will see that symbol on the screen. For the value 135, the symbol is ç. In programming, we can provide this symbol to be printed as a single character in single quotes. For this, put a single quote and then enter the symbol in the way stated above and then put the single quote. It will be written as 'ç'. Find out proper symbols from the ASCII table that can comprise to put a square bracket around the matrix.
- Write simple programs to demonstrate the use of different manipulators and examine their effects.

© Copyright Virtual University of Pakistan

Lecture No. 37

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 11***

11.3, 11.3.1, 11.4, 11.4.1

Summary

- Overloading Insertion and Extraction Operators
- Example 1
- Example 2
- Tips

Overloading Insertion and Extraction Operators

We are already aware that while overloading operators, functions are written and spirit or behavior of the operators (+, -, *, /) is maintained in their implementations. Similarly the operator's spirit is kept intact while overloading stream insertion and extraction operators.

We get an integer as input by writing the following lines:

```
int i;
cin >> i;
```

Have a look on the stream extraction operator's (>>) behavior here. The similar behavior is maintained when we overload this stream extraction operator (>>) or stream insertion operator (<<).

There are couple of important things to take care of, before starting implementation for overloading an operator:

The first thing to see is the type of the operator i.e. whether the operator is binary or unary. The binary operator takes two operands while unary operator takes one. The number of operands for an operator cannot be changed while overloading it.
Secondly, the programmer has to take care of, what an operator is returning back. For example, in case of addition (+), it returns back the result of addition. So the cascading statement like *a + b + c;* can be executed successfully. In this case, at first, *b + c* is executed. The result of this operation is returned by the *operator +*, to add it in the variable *a*. So in actual, the operation is carried out as:

© Copyright Virtual University of Pakistan

a + (b + c).

We want to overload stream extraction (>>) and insertion (<<) operators which are actually already overloaded. See the code lines below:

```
1.  int i = 123;
2.  double d = 456.12;
3.  float f = 789.1;
4.
5.  cout << i << " ";
6.  cout << d << " ";
7.  cout << f;
```

You can see the lines 5, 6 and 7. The same stream insertion operator (<<) has been used with different data types of *int*, *double* and *float*. Alternatively, these lines (5, 6 and 7) can be written within statement of one line:

```
cout << i << " "<< d << " "<< f;
```

Similarly, the stream extraction operator (>>) is used with different data types in the following manner:

```
cin >> i;
cin >> d;
cin >> f;
```

Here, stream extraction operator is used with different data types of *int*, *double* and *float*. The three lines given above can be written in one cascading line:

```
cin >> i >> d >> f;
```

The file *iostream.h* contains the operator overloading declarations for these stream insertion (<<) and extraction (>>) operators for native data types. The declarations inside this file look like the following:

```
istream& operator>>(char*);
istream& operator>>(unsigned char* p)
 {
   return operator>>((char*)p);
 }
istream& operator>>(signed char*p)
 {
    return operator>>((char*)p);
    }
istream&  operator>>(char& c);
istream& operator>>(unsigned char& c)
      {
       return operator>>((char&)c);
      }
```


© Copyright Virtual University of Pakistan

```
istream& operator>>(signed char& c)
 {
  return operator>>((char&)c);
   }
istream& operator>>(int&);
istream& operator>>(long&);
#if defined(__GNUC__)
__extension__ istream& operator>>(long long&);
__extension__ istream& operator>>(unsigned long long&);
#endif
istream&  operator>>(short&);
istream& operator>>(unsigned int&);
istream& operator>>(unsigned long&);
istream&  operator>>(unsigned
short&); #if _G_HAVE_BOOL
istream& operator>>(bool&);
#endif
istream& operator>>(float&);
istream& operator>>(double&);
istream&  operator>>(long
double&); istream& operator>>(
__manip func)
 {
  (*func)(*this); return *this;
 }
istream& operator>>(__imanip func)
  {
   return (*func)(*this);
  }
istream& operator>>(streambuf*);
```


© Copyright Virtual University of Pakistan

function returns. Therefore, it does not make sense to return references of the objects, passed by value to the function.

As we are declaring this operator function as *friend* of our class *Vehicle*, the *private* members of *Vehicle* will be accessible to this operator function. For example, *tyre* is a *private* data member of type *int* inside *Vehicle* class and inside the operator function's implementation, we can access it by simply writing *v.tyre* as:

```
output << v.tyre;
```

The above statement actually is:

```
cout << v.tyre;
```

tyre is of native data type *int*. The *output* will work for native data types as it is actually *cout*, which is overloaded for all native data type. We are constructing a building using the basic building blocks. We can use the already used bricks to construct new walls. Similarly, while writing out programs, we implement our overloaded operators using the already available functionality of native data types.

Here is how we overload stream insertion operator (<<) for our *Date* class:

```
#include <iostream.h>
class Date
{
   friend ostream& operator << ( ostream & os, Date d );
          // this non-member function is a friend of class date
  . . .
  . . .
};


ostream & operator << ( ostream & os, Date d )
{
  os << d.day << "." << d.month << "." << d.year; // access private
       data
                                                  // as friend
  return os;
};
```

Likewise, we can overload stream extraction operator (>>). All the conditions for overloading this operator are similar to that of stream insertion operator (>>). It cannot be a member operator, always a non-member operator function, declared as *friend* of the class to be overloaded for. It returns an object of type *istream &*, accepts first parameter of type *istream &*. There is one additional restriction on extraction operator (>>) i.e. the second parameter is also passed by reference as that object is modified by this operator function. For our *Date* class, it is declared as:

```
istream & operator >> ( istream & input, Date & d );
```

Note that second parameter can also be passed by reference for insertion operator (<<) but that is not mandatory and may be used to gain performance. But in case of extraction operator (>>), it is mandatory to have second parameter of reference type.

© Copyright Virtual University of Pakistan

Example 1

Following is our* Date *class containing the overloaded insertion (<<) and extraction (>>) operators:

```
/* Date class containing overloaded insertion and extraction operators. */
# include <iostream.h>

class Date
{
    public:
    Date( )
    {
            cout << "\n Parameterless constructor called ...";
            month = day = year = 0;
     }

    ~Date ( )
    {
//          cout << "\n Destructor called ...";
     }

   // Methods, not directly related to the example have been taken out from the class

      friend ostream & operator << ( ostream & os, Date d );
      friend istream & operator >> ( istream & is, Date & d );

     private:
                int month, day, year;
};

ostream & operator << ( ostream & os, Date d )
{
      os << d.day << "." << d.month << "." << d.year; // access private data of
                                                      //Date being a friend
      return os;
};

istream & operator >> ( istream & is, Date& d )
{
  cout << "\n\n Enter day of the date: ";
  cin >> d.day;
  cout << " Enter month of the date: ";
  cin >> d.month;
  cout << " Enter year of the date: ";
  cin >> d.year;
```


© Copyright Virtual University of Pakistan

```
  return  is;
};

main(void)
{

   Date date1, date2;
   cout <<  "\n\n Enter two dates";
   cin  >> date1 >> date2;
   cout  <<  "\n Entered date1 is: "   <<   date1 << "\n Entered date2 is: " << date2;

}
```

The output of the program is:

```
Parameterless constructor called ...
Parameterless constructor called ...

Enter two dates: ...

Enter day of the date: 14
Enter month of the date: 12
Enter year  of the date: 1970

Enter day of the date: 05
Enter month of the date: 09
Enter year  of the date: 2000

Entered date1 is: 14.12.1970
Entered date2 is: 5.9.2000
```

Example 2

Following is an example of a *Matrix* class, where until now, we have not overloaded insertion (<<) and extraction operators (>>).

```
/* Matrix class, which is without overloading stream operators */
#include <iostream.h>
#include <stdlib.h>

class Matrix
{
    private :

        int numRows, numCols ;
        float elements [30] [30] ;

    public :
        Matrix( int rows , int cols ) ;
```


© Copyright Virtual University of Pakistan

```
        void getMatrix ( ) ;
        void displayMatrix ( ) ;

};

Matrix :: Matrix ( int rows = 0 , int cols = 0)
{
   numCols = cols ;
        numRows = rows ;

   for ( int i = 0 ; i < numRows ; i ++ )
   {
          for ( int j = 0 ; j < numCols ; j ++ )
        {
                elements [ i ] [ j ] = 0 ;
        }
   }
}

void Matrix :: getMatrix ( )
{
    for ( int i = 0 ; i < numRows ; i ++ )
    {
       for ( int j = 0 ; j < numCols ; j ++ )
       {
          cin >> elements [ i ] [ j ] ;
       }
    }
}


void Matrix :: displayMatrix ( )
{
    for ( int i = 0 ; i < numRows ; i ++ )
    {
       cout << "|  " ;
       for ( int j = 0 ; j < numCols ; j ++ )
       {
          cout << elements [ i ] [ j ] << "  " ;
       }
       cout << "|" << endl ;
    }
}

void main ( )
{

    Matrix matrix (2, 2) ;

    matrix.getMatrix ( ) ;
```


© Copyright Virtual University of Pakistan

```
    matrix.displayMatrix ( ) ;

    system ( "PAUSE" ) ;

}
```

The operator functions (<<, >>) are not overloaded for this program. A specific function *getMatrix()* has been called to get the values for the *matrix* object this entirely a different way than we used to do for primitive data types. For example, we used to get *int* i as; cin >> i. Similarly, we called a method *displayMatrix()* to display the values in the *matrix* object. We can see here, if we overload insertion (<<) and extraction (>>) operators then the user of our class, does not need to know the specific names of the functions to input and display our objects.

The changed program after overloading insertion, extraction operators and few additional statements to format the output properly:

```
/* Matrix class, with overloaded stream insertion and extraction operators. */
#include <iostream.h>
#include <stdlib.h>

class  Matrix
{
        float elements[30][30];
        int numRows, numCols;

public:
        Matrix ( int rows = 0 , int cols = 0 )
        {
                numRows = rows;
                numCols  =  cols;
        }

        friend ostream & operator << ( ostream & , Matrix & );
        friend istream &   operator    >> ( istream & , Matrix & );
};

istream &  operator  >> ( istream &  input , Matrix &   m )
{
   for ( int i = 0;  i < m.numRows;  i ++ )
   {
      for ( int j = 0;  j < m.numCols;  j ++ )
                {
                        input  >>  m.elements [ i ] [ j ] ;
                }
        }
        return  input;
}
```


© Copyright Virtual University of Pakistan

```
ostream &  operator  << ( ostream &   output , Matrix &   m )
{
        for ( int r = 0;  r  <  m.numRows;  r++ )
        {
                for ( int c = 0;  c < m.numCols;  c++ )
                {
                        output  <<  m.elements [ r ] [ c ]   <<   '\t' ;
                }
                output   <<    endl;
        }
        return  output ;
}

int  main ( )
{

        Matrix  matrix ( 3 ,3 );
        cout << "\nEnter a 3 * 3 matrix \n\n";
        cin  >>  matrix ;
        cout    << "\nEntered matrix is: \n";
        cout  <<  matrix;

        system ( "PAUSE" );
       return 0;
}
```

The output of the program is:

```
Enter a 3 * 3 matrix

45
65
34
23
72
135
90
78
45

Entered matrix is:
45      65      34
23      72      135
90      78      45
Press any key to continue . . .
```

You can see both the operators are declared *friend*s of the *Matrix* class so that they can directly access the *private* members of the *Matrix*.

The insertion operator (<<) is accepting both the parameters left and right by reference. We already know that for insertion operator (<<), it is not really required

© Copyright Virtual University of Pakistan

to pass the second parameter (the *Matrix* object in this case) by reference but we have used here to gain efficiency. The function is returning an object *ostream &*, i.e., it is returning a reference to a *ostream* object, that actually is the required in order to support cascaded operations using this operator.

The extraction operator (>>) is also accepting both the parameters by reference. But for this operator, it is mandatory to accept the *Matrix* object by reference because this function is modifying that object. Similar to the insertion operation, this function is also returning a reference to *istream* object in order to support cascaded operations.

Clearly after overloading the operators << and >>, it is more convenient for the programmer to use these already familiar operators to display and input the object data members. Readability of the program has also comparatively increased.

Tips

- ***Stream insertion (<<) and extraction operators (>>) are always implemented as non-member functions.***
- **operator << *returns a value of type* ostream & *and* operator >> *returns a value of type* istream & *to support cascaded operations.***
- ***The first parameter to* operator << *is an* ostream & *object.* cout *is an example of an* ostream *object. Similarly first parameter to* operator >> *is an* istream & *object.* cin *is an example of an* istream *object. These first parameters are always passed by reference. The compiler won't allow you to do otherwise.***
- ***For operator >>, the second parameter must also be passed by reference.***
- The second parameter to *operator* << is an object of the class that we are overloading the operator for. Similar is the case for *operator* >>.

© Copyright Virtual University of Pakistan

Lecture No. 38

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 11, 7, 3

11.6.4, page 181, 7.7, 3.10

Summary

46) User Defined Manipulator
47) Examples of user defined manipulator
48) Static keyword
49) Static Objects
50) Static data members of a class

Today, we will discuss the concepts like 'user-defined manipulators' and 'static keywords'. Despite being considered minor subjects, these become very important while carrying out complex programming. Let's start with 'User-defined manipulators'.

User Defined Manipulators

We have talked a lot about the manipulators that are provided with the streams in the C++. These are similar to 'setw' function, used to set the width of the output. These are employed as inline like *cout << endl << i;* Remember that these functions work for only the immediately next output. How can we write our own manipulator? To determine it, it is better to understand what parameter-less manipulators are? These are the manipulators without any parameter like *endl*. This is a parameter-less built-in manipulator that inserts the new line besides flushing the buffer. If we want to write our own manipulator, how can we do this? In case of operator overloading, it is pre-requisite to know that where the operator will be used, what will be on its left-hand and right-hand sides. On reviewing the manipulators, you will find a stream object, normally on the left-hand side. Here, we are talking about *ostream*, an output stream. So that object will be *cout*. The *cout* will take this manipulator to carry out some manipulation. These are written in cascading style as *cout << manipulator << "some data" << end*l. With this cascading style, you can get a hint about the operation of this manipulator and its requirements. The point is, the left-hand side is going to be *ostream* object that will call the manipulator. What will be passed to the manipulator and what will be the return type.

© Copyright Virtual University of Pakistan

Normally on the right-hand side of the manipulator, we have another stream insertion operator i.e. <<. Here we are considering a parameter-less manipulator, that is no argument or number will be passed to it. It may be something like inserting a tab between two numbers for formatting or a manipulator to end the line or to make a sound of bell and so on. The left hand side is *ostream* object. There are no other parameters. The right-hand side is normally a stream insertion operator. We use it as *cout << manipulator* which is itself an action. We overload the stream insertion operator in such a way that the cascading works. So we return an *ostream* object. More accurately, a reference to *ostream* objects is returned. Manipulator is also going to be used in the same way, so that it returns a reference to an object of type *ostream*. Therefore we want to return the *cout* object or whatever stream we are using. Secondly it also needs the object that is calling it. Here we are not talking about our own class. *ostream* class is built-in and not under our control. So it can not be modified. We can only extend it by defining external things. So it is not a member function or member operator, but only a standalone operator. Normally the declaration of this manipulator is as:

```
ostream& manipulator_name (ostream& os)
```

This is also not a friend function. We cannot define friends for the classes that are already written and not in our control. The argument *os* here is the same object which is calling this function. We have to explicitly declare it. After this, we have to define this. Definition is just as another function. You can always write whatever you want inside the function. But we have to look at the spirit of the manipulator. When we are talking about the spirit of the manipulator, it means that the manipulator should only do something regarding output and return. It is normally very simple. Its return type is *ostream* object. In case of tab character, we can write as return *os << '\t';* It can be bell or something else. We can write useful manipulators to leave single or double blank lines or formatting the strings etc. Remember that it has to return a reference of object of type *ostream*. It automatically gets that object as parameter passed in to the function.

Examples of user defined manipulator

Here is the sample program using the manipulators.

```
/*  A small program which uses the user defined manipulators.
*/

#include <iostream.h>
#include <stdlib.h>

// Gives System Beep
ostream & bell ( ostream & output ) // Manipulator
{
   return output << '\a' ;
}
```


© Copyright Virtual University of Pakistan

```
// Gives Tab
ostream & tab ( ostream & output )     // Manipulator
{
   return output << '\t' ;
}

// Takes the cursor to next line
ostream & endLine ( ostream & output )  // Manipulator
{
   return output << '\n' << flush ;
}

void main ( )
{
    cout << "Virtual " << tab << "University" << bell << endLine ; // Use of Mainpulator
    system ( "PAUSE" ) ;
}
```

Lets see another example of matrix using the user defined manipulators for displaying the matrix on the screen.

Here is the code:

```
/*
A small program showing the use of user defined manipulators.
The display function of matrix is using these manipulators to
format the display.
*/

#include <iostream.h>
#include <stdlib.h>
#include <iomanip.h>

// definition of class matrix
class Matrix
{
     private:
        int numRows;
        int numCols;
        float elements[3][3];

     public:
       // constructor
       Matrix(int rows = 0, int cols = 0)
       {
          numRows = rows ;
          numCols = cols;
       }

       // overloading the extraction and insertion operators
       friend ostream & operator << ( ostream & , Matrix & );
```


© Copyright Virtual University of Pakistan

```
        friend istream & operator  >> ( istream & , Matrix & );
        // defining the user defiined manipulators
        friend ostream & spaceFirst ( ostream & );
        friend ostream & spaceBetween ( ostream & );
        friend ostream & line ( ostream & );
        friend ostream & newLine ( ostream & );
        friend ostream & star ( ostream & );
        friend ostream & sound ( ostream & );
};
//defining the operator >>
istream & operator >> ( istream & input , Matrix & m )
{
   for ( int i = 0 ; i < m.numRows ; i ++ )
   {
      for ( int j = 0 ; j < m.numCols ; j ++ )
      {
       input >> m.elements [ i ] [ j ] ;
      }
   }
   return input;
}

//defining the operator <<
ostream & operator << ( ostream & output , Matrix & m )
{
   for ( int i = 0 ; i < 60 ; i ++ )
   {
      if ( i == 30 )
      {
          output << "Displaying The Matrix" ;
      }
      else
      {
          output << star ;
      }
   }
   output << newLine;
   for ( int r = 0 ; r < m.numRows ; r++ )
   {
      output << spaceFirst << line;
      for ( int c = 0 ; c < m.numCols ; c++ )
      {
          output << spaceBetween << m.elements [ r ] [ c ] << sound << spaceBetween ;
      }
      output << spaceBetween << line;
      output << newLine;
   }
   output << newLine;
   return output;
}
```


© Copyright Virtual University of Pakistan

```
//defining the user defined manipulator, inserting the space
ostream & spaceFirst ( ostream & output )
{
   output << setw(33);
   return output;
}

//defining the user defined manipulator, inserting the space
ostream & spaceBetween ( ostream & output )
{
   output << setw ( 4 );
   return output;
}
//defining the user defined manipulator, inserting the | sign
ostream & line ( ostream & output )
{
   output << "|" ;
   return output ;
}
//defining the user defined manipulator, inserting the new line
ostream & newLine ( ostream & output )
{
   output << endl;
   return output;
}
//defining the user defined manipulator, inserting the *
ostream & star ( ostream & output )
{
   output << "*" ;
   return output ;
}
//defining the user defined manipulator, making sound
ostream & sound ( ostream & output )
{
   output << "\a" ;
   return output ;
}

// the main function
int main ( )
{
     // declaring a matrix of 3*3, taking its input and displaying on the screen
     Matrix matrix( 3, 3);
     cin >> matrix;
     cout << matrix;
     system("PAUSE");
     return 0;
}
```

The output of the program:

© Copyright Virtual University of Pakistan

```
3
5
1
8
7
6
2
5
2
******************************Displaying  The Matrix******************************

                    |  3   5   1  |
                    |  8   7   6  |
                    |  2   5   2  |

Press any key to continue . . .
```

Static keyword

We have been using static keyword in our examples. What is the meaning of static? The word refers to something that is stationary, stopped and not moveable. What are the types of these variables, declared as static? How can we make use of them? Static as the word implies are variables which exist for a certain amount of time, much longer than that by ordinary automatic variables. Let's consider the example about the lifetime of data variables. One of the variable types is global variable. Global variables are those that are defined outside of main. They are written as standalone statements before main function as *int i;* the variable *i* is a global variable. It is not only accessible in main but also in all the functions. They can assign some value to *i* or obtain the value of *i*. Global variables come into existence whenever we execute the program and the memory is allocated for *i*. It exists all the time when the program is running. At the end of the program execution, the memory will be de-allocated and returned to the operating system. So it has a very long lifetime. We need a value which exists for the complete execution of the program and is available in all the functions. We use global variables. It is not a good idea to do that unless it is absolutely necessary. The major plus point of these variables is that these are accessible from everywhere in the program. The inconvenience is that theses variables are visible in those functions too which does not need them.

Suppose, we have a global variable *i* declared as *int i;* and in some function we are writing a *for* loop as *for(i = 0; i < n; i++);* Now which *i* is being used here. This is the variable *i*, declared as global. This global *i* may have some valuable value, like the number of cars or the number of students etc. Here, in the function when we run the loop, the value of *i* will be changed. The global variables have this bad habit of being around, even when we don't need them. What will happen if we declare another *i* variable inside the function? A local variable will be created inside the function at run time and the global *i* is not going to be accessible. But this can be very subtle and hard to track programming errors. These are not syntax errors but logical ones. So beware of using too many global variables. Now have a look on the other side of the picture. While writing functions, we pass values to them. So instead of passing the value of *i*

© Copyright Virtual University of Pakistan

again and again, we declare it as global. Now it is available in the function, leaving no need of passing it.

Let's now come to the next variety of variables. The variables, defined in the main function are local to the function main. It means that they are accessible in all parts of the main function. Their values can be assigned, used in computations and later displayed. When we enter some function other than main, these variables are not accessible there. They are hidden. The global and the local variables, declared in a function are visible. The arguments passed to a function, are also visible. We pass the parameters through stack. Parameters are written on the stack. Later, the function is called which reads from the stack and makes a temporary copy for its use. The variables, declared and used inside the function are called automatic variables. They automatically come into being when the function is called. When the function finishes, these variables are destroyed. So automatic variables are created constantly and destroyed all the time. Here, we are talking about variables ordinary as well as user defined objects. Their behavior is same. They are automatic when the function is called, memory is allocated normally on the stack at the same time and used. When the function exits, these variables are destroyed. What happens if we want that when the function exits, some value, computed inside the function, is remembered by the function and not destroyed. This should not be visible by the other parts of the program.

Let's consider the example of a refrigerator. When we open the door of a refrigerator, the light turns on and we can see the things inside it. However, on closing the door, the light turns off. Do we know that light is off because whenever we open the door the light is on. When we close the door what is inside. We do not know. May be things magically disappear. When we open the door, magically, the things are at their position. You can think of this like a function. When we enter in the function, these automatic variables are available there and visible. When we came out of the function, it is like closing the door of the refrigerator and the light is turned off. We cannot see anything. Function goes one step ahead of this and it actually destroys all the variables. Whereas, in the refrigerator, we know that things are there. Somehow we want the function to behave like that. Outside the refrigerator, these things are not available. We can not access them. Let's say there is a bottle of water inside the refrigerator. You open the door and place it some other place. Next time, when you will open the door, the bottle is seen at the same position where you have moved it. It would not have moved to some other position. If you think of automatic variables, suppose we say inside the body of the function *int i = 0;* Every time the function is called and you go into the function where *i* is created. It has always the value 0 to start with and later on we can change this value.

What we want is that whenever we go back into the function, once we call the function like we open the door of the refrigerator and move the bottle to some other place and close the door. So we made one function call. Next time, when we call the function, the bottle is found in its new place. In other words, if we have defined an integer variable, its value will be set at 10 in the function when we return from the function. Next time when we call the function, the value of that integer variable should be 10 instead of 0. We want somehow to maintain the state of a variable. We want to maintain its previous history.

© Copyright Virtual University of Pakistan

If we declare a global variable, the state would have been maintained. The global variable exists all the time. Whatever value is set to it, it is there and accessible from any function. The drawback is that variable exists even when we don't want it. Static keyword allows us a mechanism from getting away of the downside of the global variables and yet maintaining a state inside a function. When we visit, it is found out what are its values before that we go ahead with this value. For this, whenever we declare a variable inside the function, *static* keyword is employed before the variable declaration. So we write as:

```
static int i;
```

That declares *i* to be a static integer inside the function. Think about it. Should we declare static variables inside the main function? What will happen? 'main' itself is a function so it is not illegal. There is no objective of doing this in main because main is a function from where our programs start and this function executes only for once. So its state is like an ordinary variable, declared inside main. It is only relevant for the called functions. We write inside the function as *static int i;* while initializing it once. It will be created only once irrespective of the number of function calls. Now once it is created, we increment or decrement its value. The function should remember this value. The programmer may go out of the function and come back into it. We should get the value that should be same as that at the time of leaving the function. It is necessary for the static variables that when these are created, they should be initialized. This initialization will be only for once for the complete life cycle of the program. They will be initialized only once.

Here, we have to take care of the subtle difference. In case of ordinary variable declaration, we should initialize them before using. If you have to initialize an *int* with zero, it can be written as *int i;* and on the next line *i = 0;* But in case of static variables, we have to use a different type of initialization. We have to use it as *static int i = 0;* It means that creation of *i* and the allocation of memory for it takes place simultaneously. It is initialized and the value 0 is written. This is initialization process. If somewhere in the function, we have statement *i = 10;* it will not be treated as initialization. Rather, it is an assignment statement. Here we want that as soon as the variable is created, it should be initialized. This initialization will be only for once for the lifetime of the program and it takes place when first time we enter in to the function. However we can manipulate this variable as many times as we want. We can increment or decrement it. However, it will remember its last value. How does this magic work? So far, we have been talking about the stack and free store. There is another part of memory, reserved for the variables like static variables. On the stack, automatic variables are being created and destroyed all the time. The heap or free store has the unused memory and whenever we need memory, we can take it from there and after use return it. This is the third part which is static memory area where static variables are created and then they exist for the rest of the program. These variables are destroyed on the completion of the program. So they are different from automatic variables which are normally created on stack. They are different from dynamic variables that are obtained from free store.

To prove this whole point let's write a simple program to fully understand the concept and to see how this works. Write a small function while stating that *static int i = 0;* Here, we are declaring *i* as a static integer and initializing it with zero. Then write

© Copyright Virtual University of Pakistan

i++; print the value of *i* using *cout*. Now this function just increments the value of *i*. This *i* is a static integer variable inside the function. Now write a main function. Write a loop inside the main and call this function in the loop. Let's say the loop executes for ten times. You will notice that whenever you go inside the function, the value of *i* is printed. The value of *i* should be printed as 1.2.3…10. If you remove the word static from the declaration of *i*, you will notice that every time 1 is printed. Why 1? As *i* is now automatic variable, it is initialized with zero and we increment it and its value becomes 1. *cout* will print its value as 1. When we return from the function *i* is destroyed. Next time when function is called, *i* will be created again, initialized by zero, incremented by 1 and *cout* will print 1. By adding the static keyword, creation and initialization will happen once in the life time of our program. So *i* is created once and is initialized once with the value of zero. Therefore *i++* will be incrementing the existing value. At first, it will become 1. In this case, function will return from the loop in the main program**,** call this function again**.** Now its value is 1, incremented by 1 and now the value of *i* becomes 2 and printed by *cout*. Go back to main, call it again and so on, you will see it is incrementing the last value. You can prove that static works.

Here is the code of the program:

```
/*  This is a simple program. This shows the use of static variables inside a function.
*/

#include <iostream.h>

void staticVarFun();
void  nonstaticVarFun();

void main(void)
{
   cout << "\nCalling the function which is using static variable \n";
   for(int i = 0; i < 10; i++)
      staticVarFun();

   cout << " \nCalling the function which is using automatic variable \n";
   for(int i = 0; i < 10; i++)
      nonstaticVarFun();
}
// function definiition using static variables
void staticVarFun()
{
   static int i = 0;
   i++;
   cout << "The value of i is:" << i << endl;
}

// function definiition using automatic variables
void nonstaticVarFun()
{
   int i = 0;
```


© Copyright Virtual University of Pakistan

```
    i++;
    cout << "The value of i is:" << i << endl;
}
```

The output of the program:

```
Calling the function which is using static variables
The value of i is:1
The value of i is:2
The value of i is:3
The value of i is:4
The value of i is:5
The value of i is:6
The value of i is:7
The value of i is:8
The value of i is:9
The value of i is:10

Calling the function which is using automatic variables
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
The value of i is:1
```

Static Objects

Let us look at some more uses of this keyword. As mentioned earlier that the user defined data types are the classes and objects that we created. These are now variables as for as we are concerned. If these are variables, then we can declare them as static. Now we have to be careful when we think about it. When we declared static int, we said that it should be initialized there. We initialized it with zero. What is the initialization of objects? We have defined a class and instantiated an object of that class. So we can say something like vehicle A or truck B where vehicle and truck are the classes which we have defined. 'A' and 'B' are their objects, being created in some function or main. When are these objects initialized? You know that the initialization is done in constructors. So normally C++ provides a default constructor. Here we have to write our own constructors as initialization can happen only once, if declared static. Again we are talking about these static objects inside a function instead of the main. These objects should maintain their values while getting out of the function.

© Copyright Virtual University of Pakistan

Whenever we create a static object, it must be initialized. Most of the time, we want that when the object of our class is created, its data members should be initialized by some value. For this purpose, we have to provide a constructor so that whenever an object is created, its data members are initialized. Only then it will work. Otherwise we will have problems. We may want to do as truck A, but our constructor takes some arguments. Now how this object will be created. How many wheels this truck will have? How many seats will be there? We have a solution to overcome this problem. Define a constructor which takes arguments and provides the default value to it simultaneously. If you provide a constructor with default values, then the object which is created will automatically get these values. If you write *truck A(4, 6)*, there may be some constructor which will initialize it with 4 wheels and 6 seats. But the point to remember is if you ever go to use a static object, it is necessary to provide a constructor with default arguments so that the object which you have created is initialized properly. Other than that the whole behavior of a static object is exactly the same as we have a static variable of an ordinary data type or native data type. Static variable means maintaining the state of a variable. It exists and lives around even when we are outside the function. It is an alternative to using a global which exists even when we don't want it. Now we try to learn about the destructors of static objects. If you create an object inside a function as *truck A*, when the function finishes, the object *A* will be destroyed. Destructor for this static object will be called.

To prove this write a class, inside the constructor. Also write a *cout* statement which should print 'inside the constructor of 'and the name of the object which will be passed as an argument. In the destructor write a cout statement as *cout << " Inside the destructor of " << name*, where name will tell us that which object is this. Now experiment with it. Declare a global variable of this class before main as *truck A('A')*. When the constructor for this object is called the line 'Inside the constructor of A' will be displayed. Now within the main function, declare another object as ordinary variable i.e. *truck B('B')*. Its constructor will also be called. You will see it. Write a small function and create another object within that function as *truck C('C')*. Define another function and declare a static object in it as *truck D('D')*. Call these two functions from main. Now compile and execute this program, as we have written *cout* statements inside the constructor and destructor. Now you will be able to determine which object is being created and which one being destroyed. Here you will also notice that first of all global object *A* will be created. There is going to be a line 'Inside the constructor of object A'. After that, object *B* will be created, followed by the display of constructor *cout* line. From main, we are calling function *F* which is creating object *C*. So object *C* will be created then. What next? The function *F* will finish and the control go back to main. If the function *F* finishes, its local data will be destroyed. So the object *C* will be destroyed. Here, you see it on the screen 'Inside the destructor C'. After this the function *G* will be called and we will have 'Inside the constructor for D'. This object D is a static object. Now when the function *G* finishes, you will not see the destructor line for object *D*. After this, the main function finishes and the destructors will be called for objects *A* (which is global), object *B* (which is inside the main) and object *D* (which is created as static inside the function G). In which order these will be called?. If you look at this very simple program, you will find that the last object to be created was the static object inside the function *G*. Should that deleted first i.e. the destructor of object *D* should be called? Well actually not true, the local variables of main function will be first destroyed. Static objects remain for longer period of time. Later, the static object *D* will be destroyed and the

© Copyright Virtual University of Pakistan

thing finally destroyed is the global object, which was created first of all. You will find that the destructor for object *A* is called. With this exercise, you will know the sequence in which things are created and destroyed. Another thing that you will notice is that when the function *G* finishes the static object is not destroyed.

The code of the program;

```
// An example of static objects, notice the sequence of their creation and destruction
#include <iostream>

// defining a sample class
class truck {
  private:
    char name; // Identifier
  public:
    // constructor displaying the output with the object name
    truck(char cc):name(cc) {
        cout << "inside the constructor of " << name << endl;
    }

    // distructor displaying the output with the object name
    ~truck() {
        cout << "Inside the destructor of " << name << endl;
    }
};
// defining a global object
truck A('A');

// a simple function creating an object
void f() {
  truck C('C');
}
// a simple function creating a static object
void g() {
  static truck D('D');
}

// main function
int main() {
  // an ordinary object
  truck B('B');
  // calling the functions
  f();
  g();
}
```

The output of the program:

```
inside the constructor of A
inside the constructor of B
inside the constructor of C
Inside the destructor of C
```


© Copyright Virtual University of Pakistan

```
inside the constructor of D
Inside the destructor of B
Inside the destructor of D
Inside the destructor of A
```

Lets recap these concepts. When you declare a static variable (native data type or object) inside a function, it is created and initialized only once during the lifetime of the program and therefore it will be destroyed or taken out of memory only once during the lifetime of the program. So it is a good way of maintaining state. It is an alternative to using a global data type which has some side effects. In the main, we can write static variables but it is a meaningless exercise because these are exactly like ordinary variables inside main.

Static data member of a class

Lets talk about the keyword static inside the class. Static variables are used to maintain state. We are talking about the state in which we left the function. While extending the concept, we will go inside an object. Here, we should find certain things left exactly the way they were initially. So now we are talking of static data members inside a class. What does it mean?

Literally speaking, the word 'Static' means the stationary condition of things. Stationary for object or class? Here it will be stationary for the class. That means that static data will be created once and initialized once for that class. Therefore it is not related to the objects of that class. There is only one copy of the static data member inside a class. The copy is not repeated for the objects. Whenever we create an object of a class, the complete data structure is copied for that object and there is one copy of functions which the objects may use. Static members are single for the whole class in the static memory area. It will not be repeated whenever we create an object of the class.

Now the question arises when it will be created? When it will be initialized? And when it will be destroyed? Now these are on class level and not on object level. To understand this, we have to talk about the lifetime of the static data member. The lifetime of the static data member of a class is the lifetime of the program. In other words, when you include a class in the program as a class definition, the memory is allocated for its static data members. We have some techniques to initialize it. We initialize it only once. Initialization is done at file scope which means almost at the global scope. We initialize it outside of the main. The memory is allocated for these static members. No other copy can be created for them. Therefore we can create and initialize them outside of main. There is no object so far. How can we initialize its static data members?

Suppose we have a class truck as:

```
class truck{
  public:
        int wheels;
        int seats;
```


© Copyright Virtual University of Pakistan

```
}
```

Now we refer the data members with the object as:

```
truck A;
A.wheels = 6;
A.seats = 4;
```

That's a way to refer to a data member. Here we are saying that we have some static data member of class and the object *A* has not been created yet. But we have the memory for the static members. Now we want to initialize that memory. How can we do that? We do this by using the scope resolution operator (::) and on its left hand side, we have class name and not the object name. On the right side, we write the name of the static data member. Suppose we have some static integer data member *i* in the class truck, so we can write it as:

```
truck::i = 10;
```

This initialization is taking place at file scope outside of the main .As it is happening only once in the program, it will not be executed again. It is being initialized once for the class. You can create as many object as you want. Objects can read and change that value.

Static data members of a class can be public or private. The objects of the class have access to them. They can manipulate it. But it is created and initialized only once. There is a single copy of these static data members regardless of how many objects of the class you create.

Let's take a look at the problems having static data members of a class. Suppose we have a class as 'savingsAccount'. We deposit money in that account. Some profit is also earmarked for it. Over a period of time, bank declares the rate of the profit. Profit rate is same for all PLS accounts. We have defined a class *savingsAccount* which have the information like person name, account number, current balance etc. We also have to keep the profit rate so that we can apply that on the account. Do we have different profit rate of every account? No, the bank has declared say 3% profit rate. That will be applied to all the PLS accounts. So we want that the profit rate should be defined at one place. It should be the part of the class but not defined for each object. So it is a good place to use a static variable as a data member of the class. We can initialize it at file scope as:

```
savingsAccount::profit_rate = 3.0;
```

We will write this before main function. As soon as, we compile the program and try to run it, the space is created in the static storage area. The above statement will initialize that static memory with 3.0. No savings account has been created yet. We will be creating saving accounts (object of class savingsAccount) in the main or some other function as account1, account2, etc. This profit rate will be available to every account. We can access it as:

```
account1.profit_rate;
```


© Copyright Virtual University of Pakistan

and can use it in computations. This is legal but a bad usage. Why it is a bad usage? Suppose we write as;

```
account1.profit_rate = 4.0;
```

What will happen? Does the profit rate for only *account1* has been changed? No. There is only one copy of profit_rate for all the objects of this class. That means if an object manipulates the static data member, which it can through the member functions or directly depending on it is private or public. It is actually modifying the value of that static data member for all objects of this class. So don't assume that it will change profit_rate for one object. It is a legal but a bad usage. Always use it with the class name and not with the object. So you should access it as *savingsAccount::profit_rate*. It means you are resolving it at class scope. Be careful while applying it.

Let's consider another example. We have a class as *student* and a data member for how many students in the class. Now every time, a student enrolls in the course, we want that number of students should be incremented. Whenever a student withdraws, fails or passes out from the course, the number of students should be decremented. We want this to be inside the *student* class. How does it work? We define a static data member as *static int how_many;* and initialize it to zero as:

```
student::how_many = 0;
```

In the constructor of the class we write as:

```
how_many++;
```

This way, whenever a student object is created, *how_many* will be incremented. Whenever a student leaves the course, its destructor should be called. We will write in the destructor as:

```
how_many--;
```

So it's a good way of keeping track of how many objects of a particular type exist at this time inside the program. To display that we can write a member function that will display 'how_many'. The merit of this technique is that we have done all this work inside the class. We did not use two classes or global variable or go through the source code to count the number of students. Using these, you can make your program more and more dynamic. The usage of static is very import. So you should understand it clearly. Static is maintaining the state. The state may be how many students are in the class.

Today we have covered the parameter-less manipulators which will return the *ostream* object and *ostream* object is passed as an argument to them. Then we discussed about the static data, both at the ordinary level and then the static data members inside the class. These are very useful. As you write bigger and more complex programs, you will find that these concepts are very useful. Again from a generic prospective, you will be working hopefully in your professional career with many different languages. You have to understand that every language might

© Copyright Virtual University of Pakistan

represent the static concept in a different way. But just knowing that concept empowers you and help you to understand more complex programming languages as well.

Lecture No. 39

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 5, 6, 7

Revise old topics

Summary

51) Pointers
52) References
53) Call by Value
54) Call by Reference
55) Dynamic Memory Allocation
56) Assignment and Initialization
57) Copy constructor
58) Example
59) Rules for Using Dynamic Memory Allocation
60) Usage of Copy Constructor
61) Summary
62) Exercise

In this lecture, we will review the concepts like pointers, references and memory allocation, discussed in the previous lectures. The review of these topics will help enhance our understanding of the programming. Let's discuss these topics one by one.

Pointers

Pointer is a special type of variable that contains a memory address. It is not a variable that contains a value, rather an address of the memory that is contained inside a pointer variable.

In C++ language, variables can be without type. Either we can have a void pointer or these can be typed. So we can have a pointer to an integer, a pointer to a character and a pointer to a float etc. Now we have a user-defined data type, which we call classes, so we can have pointers to classes.

© Copyright Virtual University of Pakistan

While talking about pointers, we actually refer to pointing to an area in memory. A pointer to an integer, points to a location opted by an integer in memory. If there is an array of integers in the memory, we can still use a pointer to point to the beginning of the array. This is a simple manipulation. To have the address of a memory location, we use & sign. The '&' sign is also used in references. So we have to be very cautious while making its use. To further overcome this ambiguity, we will now recapture the concept of reference.

References

A reference can be considered as a special type of pointer as it also contains memory address. There are some differences between pointers and references. Pointers may point to nothing while references always have to point to something. A reference is like an alias for an object or a variable. The references should be used when we are implementing the call by reference. This helps us make our syntax easier as we can implement the call by reference with out using the * operator.

Call by Value

Whenever we call a function and pass an argument, an object or variable to the function, then by the default rule of C and C++, it is a call by value. It means that the original data remains at its place and a temporary copy of it is made and passed to the function. Whatever the function does with this copy, the original value, in the calling function, remains intact. This is a call by value.

Call by Reference

If we want a function to change something in the original object variable or whatever, that variable or object by reference would be passed. To do this, we don't make temporary copy of that object or variable. Rather, the address of the variable is sent. When the function manipulates it, the original object will be manipulated, effecting change in its values. The use of call by reference is also important for the sake of efficiency. If we have a large object, sending of its copy will be something insufficient. It will occupy a large space on the stack. Here, we can use call by reference instead of call by value only for efficiency while we need not to change the original object. For this, we use a keyword *const* that means that a *const* (constant) reference is being passed. The function can use its values but cannot change it.
Now we come to the dynamic memory allocation.

Dynamic Memory Allocation

In C language, we have a method to allocate dynamic memory. In it, while executing the program, we allocate some memory from the free store (heap) according to our need, use it and after using, send it back to the free store. This, dynamic memory allocation, is a very common function in programming. While writing C++ language, it was decided that it should not be implemented by a function call. There should be native operators, supposed to be very efficient. These operators are *new* and *delete*. We allocate memory with the *new* operator from the free store. It returns a pointer. For example, if we say *p* is a pointer to an integer, the statement will be written as

```
int *p ;
```

Now we write

```
p = new int ;
```


© Copyright Virtual University of Pakistan

This statement performs the task in the way that it gets memory for an integer from the free store. There is no variable name for that memory location but the address of this location is stored in the pointer *p*. Now by using the syntax **p* (which means whatever p points to), we can manipulate that integer value. We can also write to and read from that location. This memory allocation is done during the execution of the program. Whenever we allocate memory with the *new* operator, it is our responsibility to de-allocate this memory after the termination of the program. To do this de-allocation, we have an operator *delete*. To de-allocate the memory, allocated with *p = new int ;* we will write

```
delete (p) ;
```

It will not delete the *p* rather, it will send the memory gotten and pointed by *p* back to the free store.

Same thing happens when we try to allocate more than a simple variable i.e. when we are trying to allocate arrays. When we use *new* operator to allocate a space for an array, we tell the size of the array in square brackets (i.e. []). We can write it like

```
p = new int [10] ;
```

This statement says *p* is pointing to an area of memory having the capability to store 10 integers. So there is an array of 10 integers whereas *p* is pointing to the beginning point of the array. Now we can access the elements of the array by manipulating the pointer *p*. To make the memory allocated for an array free, the syntax is a little bit different. Whenever, we allocate an array then to free it we write

```
delete [] p ;
```

This will free the array that is pointed by *p*. In this case, the space of 10 integers that was pointed by *p,* will be de-allocated despite the fact that we write empty brackets with the *delete* operator. This is due to the fact that C++ internally has the record that how much memory was allocated while allocating an array. After *delete,* the pointer points to nothing. So it's a free pointer and can be reused.

Now let's go on and look at a special type of objects. These are the objects with the data members as the pointers. In the previous lectures, we had discussed the example of a class Matrix. In that class, we said that it will be a two dimensional matrix while talking about its size etc. In this example, it will be a generic class besides being a matrix of 3 x 3. Now we want to see the Matrix class defined such a way that the programmer can take an object of it of any size at any time, say a matrix of 3 x 3, 5 x 5 or 10 x 10. It means that the size of the matrix should be variable. Now we have to bring every thing together in terms of how we declare and manipulate arrays inside a C++ program.

Whenever we declare an array, we have to mention its size. It is necessary, as otherwise no memory will be allocated, it's about the static memory allocation. In it, we declare like that it will be a two dimensional array of *m* rows and *n* columns. The compiler will allocate a memory for this array at the start of the program. Now we are talking about that at the compilation time, we don't know the size of the array. We want to allocate an array of the required size at run time. When the constructor of the class is called, the memory required by that object should be allocated at that time.

© Copyright Virtual University of Pakistan

For example, in case of Matrix class, we will like to tell the number of rows and columns of the object of the Matrix in the constructor, so that the constructor could allocate the memory for that object. A keyword *new* is used for this purpose. Here, we realize that in case of data member of this class Matrix, fixed rows and columns cannot be used in the class definition. We cannot define a two-dimensional array inside the class as it negates the concept of extensibility. Here, inside the class we define something like *int *m;* which means *m* is a pointer to an integer. It is the data member of the class. In the constructor of the class, we say that it will take two integer arguments, rows and columns. Whenever we declare an object, which is an instantiation of the class, the constructor of the class is called. Now in the constructor of this class we want to allocate memory from the free store equal to the memory required by *m* x *n* (m multiply by n) integers. So if we say a Matrix of 3 rows and 3 columns then we require memory for 9 (3 * 3) integers. If we say four rows and five columns then we require a memory for 20 (4 * 5) integers. Now it is very simple, inside the constructor we will write

```
m = new int [rows * columns] :
```

Thus we created a Matrix to which we tell the number of rows and columns, through variables, during the execution of the program. When we created an object of the class its constructor is called to which the variable values are passed and by multiplying these variables (number of rows and columns), we allocate a memory for these integers by the *new* operator and its address is assigned to *m*. Thus the object is initialized and the required memory is available to it. Now we can use it as a two dimensional array and can put values inside it. Now the class definition of the class Matrix can be written as under.

```
class Matrix
{
        private:
                int *m;
                int row, col;
        public:
        Matrix(int rows, int cols)
        {
                        m = new int[rows * cols];
        }
};
```

There is a requirement that if the constructor of a class allocates the memory, it is necessary to write a destructor of that class. We have to provide a destructor for that class, so that when that object ceases to exist, the memory allocated by the constructor, is returned to the free store. It is critically important. Otherwise, when the object is destroyed, there will be an unreferenced block of memory. It cannot be used by our program or by any other program. It's a memory leak that should be avoided. So whenever we have a class in which the constructor allocates dynamic memory, it is necessary to provide a destructor that frees the memory. Freeing the memory is an easy process. We have no need to remember that how many rows and columns were used to allocate the memory. We simply use the *delete* operator with empty brackets and the pointer that points to the allocated memory. We write this as follows

© Copyright Virtual University of Pakistan

```
delete [] m ;
```

This statement frees the allocated memory (whatever its size is) that is being pointed by *m*.

Assignment and Initialization

Let us discuss the assignment and initialization. We do initialization as

```
int i = 0 ;
```

This is declaring an integer and initializing it. The second way to do this is

```
int i ;
i = 0 ;
```

This has the same effect as the first statement but the behavior is different. At first, a space is allocated for *i* before assigning a value to it..
The same applies whenever we create an object. We can either create an object, initialize it at the creation time that means constructor is being called, or we can create an object and then assigns values to its data members later. This is usually done either by set functions or with the assignment statements. Here the thing to differentiate is that if we have two objects of a class, say Matrix, *m1* and *m2*. We have, in some way, created *m1*, its rows and columns have been allocated, and values have been put in these rows and columns. Now somewhere in the program, if we write

```
m2 = m1 ;
```

It is an assignment statement. If we have not defined the overloaded operator for assignment, the default assignment of C will be carried out. The default assignment is a member-to-member assignment. Now let's again look at the construct for the Matrix class. In it, we have only one data member i.e. a pointer to an integer. We have written *int *m ;* in the class definition. So in *m1*, there is a pointer to an integer but *m1* is a fully qualified developed object. It is, let's say, a 5 x 5 matrix and has values for its entities. Now when we write

```
m2 = m1 ;
```

m2 has also a pointer *m* to an integer as its own data member. If we have not written an overloaded assignment operator for this class, the value of *m* of the object *m1* will be assigned to *m* of the object *m2*. Now we have two objects, having pointer variables that hold the same address. So these are pointing to the same region in the memory. There arise many problems with this. Firstly, if we destroy *m1*, the destructor of *m1* is called and it frees the memory. So the memory being pointed by *m* of the object *m1* has gone back to the free store. Now what happens to *m2*? There is a pointer in *m2*, pointing to the same memory, which has been sent to the free store by the destructor of *m1*. The memory is no longer allocated. It has been sent to the free store. So we have a serious problem. We don't want two objects to point to the same region in the memory. Therefore, we write an assignment operator of our own. This assignment operator is such that whenever we do some object assignment, it allocates separate

© Copyright Virtual University of Pakistan

memory for one object and copies the values of the second object into that space. Thus the second object becomes an independent object. So we have to be careful. Whenever we have a class with a dynamic memory allocation, there is need for writing an assignment operator for it.

Copy Constructor

Now we will see what a copy constructor is? We have discussed the dangers that a programmer may face during the process of assigning the objects. The same danger comes to the scene, when we pass an object like the Matrix class to a function that does some manipulations with it. Suppose, we have a function that takes an object of a class Matrix as an argument. The default mechanism of calling a function in C or C++ is call by value. Now what is the value for this object, being passed to the function? The values of the data members of the object will be placed on the stack. The function will get a temporary object, as it is a call by value. The original object remains intact. The values of data members of that temporary object will be the same as the values in the original object. Now if it is a simple class, there will be no problem. However, if there is a class with a pointer as its data member and that pointer has allocated some memory, then the value of that pointer is copied. This value is passed to the function and not the memory. Now in the function, we have a temporary object that has a pointer as data member, pointing to the same memory location as the original object. If we are just reading or displaying that object, there is no problem with it. If we change the values of the object, the values in the temporary object get changed. However, when we manipulate the pointer, it changes the values in the memory of the original object. This change in the original values is the mechanism of the call by reference and here we have done call by value. We know that a temporary object is passed to the function. So how we get around this problem? The way to resolve this problem is to create a complete copy of the object. We want that in this copy of the object the pointer value must not point to the same memory location. Rather, it must point to the memory location of its own. So we want to have a copy constructor. That means a constructor that will create a new object with a full copy of the other object. This is also known as deep copy as opposed to shallow copy. The shallow copy makes a member-to-member copy of the object without taking care whether the pointer is a pointer or an ordinary variable. The copy of the ordinary variables is perfectly valid and legal. But if we make a member copy of a pointer, there may be the problem in which a new pointer variable is created with the same value, without creating a new area of memory. Therefore, we have to write something special that is called copy constructor.

The basic line in the syntax of the copy constructor is that we are trying to create a new object by passing an object of the same class as the argument. So we should think of a constructor. For example if we have the class Matrix, its prototype will be as under.

```
Matrix (Matrix &) ;
```

The ‘&’ sign shows that a reference of the object of type Matrix will be passed. Now whenever we write a copy constructor, there is need to be very cautious. We have to allocate a new memory for that copy. When we go into this copy constructor, at first, it is determined that how much memory the original object has allocated? Since we pass the object, so all the queries might have answers. For example, in case of Matrix class, we can find the number of rows and columns. We create a temporary object

© Copyright Virtual University of Pakistan

inside the constructor and allocate it a memory. Then the values of memory of the original object are copied into this memory. Now the pointer of this new object will point to this new location. As the constructor returns nothing and just creates a new object, so there is no return statement in the constructor. Thus, this copy constructor is completed. The syntax of this constructor is given below

```
Matrix::Matrix ( const Matrix  &other )
{
        size = other.size ;                    // size is a function
to determine the
    memory allocated by object
        m = new int [size] ;
        copyvalues ( m, other ) ;
}
```

In this case, it creates a new object that actually creates memory. It does not make the shallow copy so there is no copy of the pointer value. In fact, the pointer has a new value. But the values pointed to by this pointer (i.e. the values in the new allocated memory) are the copy of the values in the memory of the original object, and then this fully constructed object is returned.

Now we have the facility of copy constructor. With it, we can define new objects based on the existing objects. This copy constructor is necessary for the objects in which the memory is allocated dynamically. We can use this copy constructor without causing any problems. Suppose we have written as

```
Matrix a (3,3) ;
```

We have defined a constructor that takes two arguments rows and columns, as a matrix 'initializer'. We allocate memory for these rows and columns and create an object. Then somewhere, later in the program, We write

```
Matrix b (a) ;
```

This statement makes a complete copy of already created object *a* and a new object *b* is created. This new object has its own pointer and own memory allocation. This memory location is different from the memory allocated inside the matrix *a*. Now if *a* dies then *b* is still a valid object. So a copy constructor is critically useful. It is used when we want to create a duplicate copy of an object. It is always used whenever we want to do a call by value into a function to which an object is being passed, and the object is of a class in which we have allocated the memory dynamically.

Example

Let's look at an example to further understand these concepts. We write a class *String*. It has a data member *c* that is a pointer to a character. We write a member function (copy function) of the class, which can copy values in the character array of the class. There is a constructor that will allocate a space for the character arrays i.e. string. The starting address of this array will be stored in data member of the class i.e. in a pointer. We have not written any copy constructor. Now we want to make two objects of this *String* class, say, *s1* and *s2*. We create object *s1* and assign a string to it. We write it as

© Copyright Virtual University of Pakistan

String s1 (“test1”) ;

Now after this we write

String s2 = s1 ;

Thus we create a new object *s2*. The values of *s2* are initialized with the values of *s1*. As we have written no copy constructor, C will provide the default copy constructor. Now if we display the string of *s2*, it will be the same as of *s1*. Now use the copy function to assign new values to the string inside the object *s1*. So we write

s1.copy(“A new string”) ;

Thus we write a new string in *s1*. Now again if we display the string of *s2* by writing

s2.print ;

We will see that it displays the same value, assigned to *s1* in the previous statement. The reason is that the default copy constructor has done member-to-member copy. It has copied the value of the character pointer to the pointer of *s2* and thus both pointers are pointing to the same memory location.
Now there is need of providing a copy constructor for a class like this. We also have to provide a destructor as we are doing memory allocation inside the constructor of the class.

Following is the code of the example in which we provide a copy constructor. We create an object based on an existing object. The copy constructor creates an object with full copy of the existing object with its values in a new memory location.

```
/*This program has a copy constructor and demonstrate the use of it.
We create a new object by passing it an existing object, this calls
the copy constructor and thus creates a complete copy of the passing
object, and has its values in new location of memory.
*/

#include <iostream.h>
#include <stdlib.h>

// class definition
class String
{
        char *c;
    public:
         // copy function
         void copy (char *s)
        {
             c = s ;
         }
         // getting the length of the string
```


© Copyright Virtual University of Pakistan

```
        int length ()const
        {
            return strlen(c);
         }
         //constructors
        String ();
        String (const char *s)
        {
              c = new char [ 30 ];
               strcpy (c, s);
         }
        // copy constructor
        String( const String &other );

        //display the string
        void print()
        {
            cout << c << endl ;
         }
         //destructor
         ~String()
         {
             delete []c ;
          }
};
// definition of copy constructor
String::String( const String &other )
{
        int length;
        length = other.length();
        c = new char[length + 1];
        strcpy( c, other.c );

}

main ()
{
        String s1("test1");
        cout << "The string of s1 is    "  ;
        s1.print();
        String s2(s1);
        cout << "The string of s2 is    " ;
        s2.print();
        s1.copy("A new string"); // assign new value to string s1
        cout << "The string of s1 is     " ;
        s1.print();
        cout << "The string of s2 is    " ;
        s2.print();             //s2 has its own previous value
}
```


© Copyright Virtual University of Pakistan

The following is the output of the program which shows the use of copy constructor.

```
The string of s1 is    test1
The string of s2 is    test1
The string of s1 is     A new string
The string of s2 is    test1
```

The other affected part is the assignment operator itself. We know that there are dangers in the assignment operators of a class in which memory is being allocated. We cannot write an assignment operator for such a class blindly. When we write an assignment operator for such a class, that operator must first look at the fact whether there is self-assignment being done.

Suppose we have an integer *i*. We have written as

```
int i ;
i = 10 ;
```

And down to this we write

```
i = i ;
```

There is no problem with it. It is a legal statement. It is complicated if we do such assignment through pointers. In such a case, pointer is pointing to itself and even it has no problem. But when we do this with objects that have allocated dynamic memory, the method of assignment is changed. Let's take the example of Matrix. We have an object of Matrix, say *m1*, which has three rows and three columns. Another object, say *m2*, has five rows and five columns. Somewhere in the program we write

```
m1 = m2 ;
```

Here *m2* is a big object while m1 is a small one. We want to assign the big object to the smaller one. The assignment operator for this type of class, first de-allocates the memory reserved by the left-hand side object. It frees this by the *delete* operator. Then it will determine the memory required by the object on right hand side. It will get that memory from the free store by using *new* operator. When it gets that memory, it will copy the values and thus the statement *m1 = m2*; becomes effective. So assignment has a requirement.

Now if we say

```
m1 = m1 ;
```

We have defined assignment operator. This operator will delete the memory allocated by *m1* (i.e. object on L.H.S.). Now it wants to determine the memory allocated by the object on the right hand side, which in this case, is the same i.e. *m1*. Its memory has been deleted. So here we get a problem. To avoid such problem, whenever we write an assignment operator, for objects of the class that has done memory allocation. After this, we do other things.

We have discussed the example in which we create an object of Matrix. We create it using a copy constructor by giving it another object. The syntax of it we have written as

```
Matrix m2 (m1) ;
```

This is the syntax of creating an object based on an existing object. We can write it in the following fashion.

```
Matrix m2 = m1 ;
```

While this statement, we should be very clear that it is not an assignment only. It is also a construction. So whenever we are using initialization, the assignment operator

© Copyright Virtual University of Pakistan

seems as equal to operator. But actually assignment operator is not called. Think about it logically that why assignment operator is not called? The assignment operator is called for the existing objects. There should be some object on the left-hand side, which will call the assignment operator. When we have written the declaration line

```
Matrix m2 = m1 ;
```

The m2 object has not constructed yet. This object of Matrix does not exist at the time of writing this statement. So it cannot be calling the assignment function or assignment operator. This is an example of the use of a copy constructor. Thus, there are two different ways to write it. Remember that whenever we create an object and initialize it in the declaration line, it calls the copy constructor.

Let's talk about another danger faced by the programmers when they do not provide copy constructor. The ordinary constructor is there which allocates memory for the objects of this class. Suppose we do a call by value to a function. Here, we know that a temporary copy of the object is made and provided to the function. The function does manipulations with this copy. When the function returns that temporary copy is destroyed. As no copy constructor is there, a shallow copy, with values of pointers, is made. The destructor should be there as we do memory allocation in the class. Now suppose that there is a destructor for that class. Now when this temporary object destroys its destructor executes and de-allocates the memory. Now as it was a shallow copy so its pointers were pointing to the same memory as of the original object. In this way, actually, the memory of the original object is de-allocated. So the pointer of the original object now points to nothing. Thus, in the process of function call, we destroyed the original object as it is an invalid object now. Its pointer is pointing to an unknown memory location. This is a subtle but very critical. This can be avoided by providing a copy constructor, which actually constructs a fully formed object with its own memory. That temporary object will go to the function . When it is destroyed, its destructor will de-allocate this memory. However, the original object will remain the same.

Rules for Using Dynamic Memory Allocation

Whenever we have a class in which we do dynamic memory allocation, there are some rules that should be followed.

First, we must define a constructor for it. Otherwise, we will not be able to carry out dynamic memory allocation. This constructor should be such that it gets memory from the free store, initializes the object properly, sets the value of the pointer and returns a fully constructed object.

Secondly, we must write an assignment operator for that class. This assignment operator should first check the self-assignment and then make a deep copy.. So that a properly constructed object should be achieved..

Thirdly, as we are doing dynamic memory allocation in the constructor, it is necessary to provide a destructor. This destructor should free the allocated memory.
These three rules are must to follow.

© Copyright Virtual University of Pakistan

Usage of Copy Constructor

Let us see where the copy constructors are being used?
First, it is used explicitly at some places, where we write

```
Matrix m2 (m1) ;
```

This is an explicit call to a copy constructor. Here *m1* is being passed by reference. If we want that there should be no change in *m1*, then it is necessary to use the key word *const* with it to prevent any change in *m1*. The presence of this key word means that the constructor will do nothing with the object, being passed to it. The use of copy constructor in this explicit way is clear.

The second way to use the copy constructor is by writing the declaration line as

```
Matrix m2 = m1 ;
```

Here the use of copy constructor is not clear. It is not clear by the statement that copy constructor is being used. It seems an assignment operator is being used. Be careful that it is not an assignment operator. It is a copy constructor.

The third use of the copy constructor is calling a function and passing it an object by value. If we have provided a copy constructor, it will be called automatically and a complete temporary copy (with memory allocation) of the object is given to the function. If we do not provide copy constructor, the call by value functions will create problems. In the function, if we change any value of the object, it will change the value in the original object.
In function calling, when we do the call by value, the copy constructor is called. On the other hand, in call by reference, copy constructor is not called and the address of the original object is passed.

Summary

A pointer is a variable that holds memory address. The & operator is used to get the address of a variable or an object. The & sign is also used as a short hand for a reference. Whenever we have a & sign in the declaration, it implies a reference. Whenever we have & sign on right hand side of an assignment operator, it is taken as address of an object. We can do dynamic memory allocation by using pointers.
In C++ language, we have two very efficient operators provided which are *new* and *delete*. We use the new operator for obtaining memory from the free store. It returns a pointer to the allocated memory. We store the value of this pointer in a pointer variable.
In a class, which allocates memory dynamically, there is a data member i.e. a pointer. When we create an object of the class at run time, it will allocate memory according to our requirement. So there is no waste of memory and the situations in which we want to store large data in small memory or vice versa are prevented. So we do dynamic memory allocation inside these classes.
Whenever we have dynamic memory allocation inside a class, we have to provide few things. We must provide a constructor that does the memory allocation for us producing a well-formed object.

© Copyright Virtual University of Pakistan

We must provide a copy constructor that is able to create fully formed copies of the objects. That means is should not only make the copies of the values of the pointers but it should give the pointers new values by allocating new memory for the object. And should copy the values of the original object into this new memory location.
We must provide an assignment operator. This operator should be able to check the self-assignment and can assign one object to the other in a proper fashion using the concept of deep copy and not a shallow copy. So we allocate memory space then copy element by element in this allocated memory.
And finally we must do de-allocation which means whenever we destroy an object and it goes out of scope, we should free the memory allocated by that object. To do the memory de-allocation we must provide the destructor of the class in which we free (delete) the memory by using deletes operator.

Exercise

You should write small programs to examine the working of these rules. You can check this if we allocate memory and do not delete it in the destructor. Then the next time, when we execute the program it will allocate a new memory. We can find that which memory is assigned by displaying the value of the pointer (not the value it points too). It will be a number with 0x-notation i.e. it will be in hexadecimal. We don't care about the exact value but we will find that if we have provided a proper destructor. Then on the same computer, in the same session, we execute the program, a specific address of memory will be assigned to the program. With the proper destructor, we stop the program and then again start it. Nine out of ten times, we get the same memory. That means we will see the same address. Nine times out of ten is because the operating system can use this memory somewhere else between the times of two executions of the program. If we do not provide a destructor i.e. we do not deallocate the memory, it is necessary that each time we will get a new memory. The previous memory is being wasted. You can prove it by yourselves by writing small programs.

© Copyright Virtual University of Pakistan

Lecture No. 40

Reading Material

Deitel & Deitel - C++ How to Program ***Chapter 7***

7.3, 7.4

Summary

- Objects as Class Members
- Example 1
- Example 2
- Advantages of Objects as Class Members
- Structures as Class Members
- Classes inside Classes
- Tips

Objects as Class Members

A class is a user defined data type and it can be used inside other classes in the same way as native data types are used. Thus we can create classes that contain objects of other classes as data members.

When one class contains objects of other classes, it becomes mandatory to understand how and in what sequence the contained and containing objects are constructed. An important point in construction of an object is that the contained data members of the object (regardless whether they are native or user defined data types) are constructed before the object itself. The order of destruction of an object is reverse to this construction order, where the containing object is destroyed first before the contained objects.

To elaborate the construction and destruction orders of objects, we take a class *A* and contain its instance (object) in another class *B*.

```
/* This program illustrates the construction and destruction orders of objects. */

#include <iostream.h>

class A
{
```


© Copyright Virtual University of Pakistan

```
    public:
    A()
    {
       cout << "\n A Constructor ...";
    }

    ~A()
    {
       cout << "\n A Destructor ...";
    }
};

class B
{
    public:
            B()
            {
               cout << "\n B Constructor ...";
            }

            ~B()
            {
               cout << "\n B Destructor ...";
            }

    private:
            A  a;
};

void main(void)
{
    B  b;
}
```

The output of this code is as follows:

```
A Constructor ...
B Constructor ...
B Destructor ...
A Destructor ...
```

In the code above, we have contained an instance of the class *A* inside class *B*. In the *main* function, we have only created an object of the class *B*.

From the output, we can see the first line that the contained object *a*'s default constructor is called before the default constructor of the class *B*. At destruction time, the destructor of the class *B* is called first before *A*'s. Note that the contained object '*a'* of class *A* is constructed by calling the default constructor. Hence, we have found one way of constructing contained objects by means of default constructors and then setting the values of data members by calling setter methods of the object. But this is cumbersome and wasteful, we have a better way provided by the language to initialize

© Copyright Virtual University of Pakistan

contained objects at construction time using the *initializer list*. Initializer list is used to initialize the contained objects at the construction time.

Example 1

Let's take a class of *PersonInfo* that stores *name, address* and *birthday* of a person. This *PersonInfo* class contains an instance of our veteran *Date* class to store *birthday* of a person.

```
class PersonInfo
{
    public:
        // public member functions...
    private:
      char name[30];
     char address[60];
     Date   birthday;          // member object
};
```

This declaration specifies a *Date* object *birthday* as a *private* data member. Note that no arguments are specified in the declaration of *birthday*. However, this does not mean that the default constructor is called when the *PersonInfo* object is constructed but we can always specify a member initializer to call a parameterized constructor.
A colon is placed after the parameter list of the containing class's constructor, followed by the name of the member and a list of arguments as shown below:

```
class PersonInfo
{
      public:
         PersonInfo( char * nm, char * addr, int  month, int   day, int  year );
         // ...
      private:
         // ...
  };

  PersonInfo::PersonInfo( char * nm, char * addr, int month, int day, int year )
     : birthday( month, day, year ) // Member initializer
  {
     strncpy( name, nm, 30 );
     strncpy( address, addr, 60 );
  }
```

Note that there are five parameters inside *PersonInfo* constructor including the three parameters *month*, *day* and *year* parameters to be passed to *Date* class's parameterized constructor as *birthday(month, day, year)*. We are using the initializer list, therefore, there is no need to call setter methods of the *Date* class to initialize the *birthday* object. Similarly, multiple contained objects can be initialized by using comma separated initializers. The order of the execution of initializers is the same as the order of declarations of objects inside the outer class. To confirm about the order

© Copyright Virtual University of Pakistan

of execution, let us have another *Date* object *drvLicenseDate* declared after *birthday* object in the *PersonInfo* class:

```
 /* This program illustrates the initializer list, order of execution of constructor's inside
the list. */

#include <iostream.h>
#include  <string.h>

class  Date
{
          public:
            Date( );
            Date(int  month, int   day, int   year);
            ~Date ( );

          private:
                    int  month, day, year;
};

Date::Date( )
{
   cout << "\n Date -- Default constructor called ...";
   month  =  day  =  year  =  0;
 }

Date::Date(int  month, int  day, int   year)
 {
   cout  <<   "\n Date -- Constructor with month="   <<   month
          << ", day= "  <<  day  <<  ", year= "  <<  year  <<  " called ...";this-
   >month  =  month;
   this->day = day;
   this->year  =  year;
  }

Date::~Date ( )
 {
          cout  <<   "\n Date -- Destructor called ...";
 }

class PersonInfo
{
public:
   // public member functions...
   PersonInfo( char  * nm, char  * addr, int  month, int  day, int  year,
                     int  licMonth, int  licDay, int  licYear );
   PersonInfo::~PersonInfo();

private:
   char  name[30];
```


© Copyright Virtual University of Pakistan

```
    char address[60];
    // member objects
    Date birthday;
    Date drvLicenseDate;
};

PersonInfo::PersonInfo( char * nm, char * addr, int month, int day, int year,
                        int licMonth, int licDay, int licYear )
    : drvLicenseDate( licMonth, licDay, licYear), birthday( month, day, year )
        // Above line is initializer list
{
    cout << "\n PersonInfo -- Constructor called ...";
    strncpy( name, nm, 30 );
    strncpy( address, addr, 60 );
}

PersonInfo::~PersonInfo()
{
    cout << "\n PersonInfo -- Destructor called ...";
}

main(void)
{
    PersonInfo pi("Abbas", "12-Y, DHS, Lahore, Pakistan", 12, 12, 1972, 12, 10, 1992);
}
```

The output of this program is:

```
Date -- Constructor with month=12, day= 12, year= 1972 called ...
Date -- Constructor with month=12, day= 10, year= 1992 called ...
PersonInfo -- Constructor called ...
PersonInfo -- Destructor called ...
Date -- Destructor called ...
Date -- Destructor called ...
```

Because *birthday* is declared before *drvLicenseDate*, it is clear from the output that the constructor for *birthday* is called first and then for the *drvLicenseDate* object, although *drvLicenseDate* is present before *birthday* in the initializer list.

Example 2

Let's take another example to work with the size of a matrix. We declare a *Column* class first then a *Row* class. *Row* class contains an instance of *Column* class to store the number of columns (number of elements) inside one *Row* instance. Further, the *Matrix* class contains an instance of *Row* class. See the code below.

```
/* Program to illustrate the initialization lists, construction and destruction sequences of
contained and containing objects. */
```


© Copyright Virtual University of Pakistan

```
#include <iostream.h>
#include  <stdlib.h>

class  Column
{
   private :
      int  size ;

   public :
      Column ( int  size )
      {
            cout  <<  "Column created"  <<  endl  <<  endl ;
            this->size  =  size ;
      }
      ~Column ( )
      {
            cout  <<   "Column destroyed "  <<   endl   <<   endl ;
      }

      void showSize ( ) ;
      void  setSize ( int ) ;
};

void Column :: showSize ( )
{
   cout  <<   "Column size is : "   <<   size   <<   endl   <<   endl ;
}
void Column :: setSize ( int  sz )
{
   size  =   sz ;
}

class Row
{
   private :
      int size ;
      Column  col ;
   public :
      Row ( int  rowSize, int   colSize ) : col( colSize )
      {
            cout  <<  "Row created"  <<  endl  <<  endl ;
            this->size  =  rowSize ;
      }
      ~Row ( )
      {
            cout  <<   "Row destroyed "   <<   endl   <<   endl ;
      }
      void showSize ( ) ;
      void  setSize ( int ) ;
};
```


© Copyright Virtual University of Pakistan

```
void  Row :: showSize ( )
{
    col.showSize ( ) ;
    cout  <<  "Row size is : "   <<   size   <<   endl   <<   endl ;

}
void Row :: setSize ( int sz )
{
   size  =  sz ;
}

class  Matrix
{
   private :
      Row row ;
   public :
      Matrix ( int  rowSize, int  colSize ) : row( rowSize,  colSize )
      {
            cout  <<  "Matrix created"   <<   endl   <<   endl ;
      }
      ~Matrix ( )
      {
            cout  <<  "Matrix destroyed"   <<   endl   <<   endl ;
      }
      void  displayMatrixSize ( ) ;

} ;

void Matrix :: displayMatrixSize ( )
{
   row.showSize ( ) ;
}

void  f( )
{
   Matrix matrix(3, 4) ;
   matrix.displayMatrixSize  ( ) ;
}

int  main()
{
   f( );
   system("PAUSE");
   return  0;
}
```

The output of the program is as follows:

```
Column created
```


© Copyright Virtual University of Pakistan

```
Row created

Matrix created

Column size is : 4

Row size is : 3

Matrix destroyed

Row destroyed

Column  destroyed

Press any key to continue . . .
```

Notice the construction sequence of objects. In order to create a *Matrix* object, a *Row* object is created first and to create a *Row* object, a *Column* object is created. So the contained object *Column* is constructed first of all, then comes the *Row* object and finally the *Matrix* object. At destruction time, the very first object to destroy is the last object constructed, which is the *Matrix* object. The second object destroyed is *Row* object and then the *Column* object at the end. See also the use of initializer list in the code, how the *colSize* and *rowSize* arguments are passed to the constructors.

The *public* data members of a contained object can also be accessed from outside of the containing class. For example, if *row* object inside *Matrix* class is declared as *public* and has a *public* variable named *size* then it can be accessed using the dot operator (“.”) as:

```
int  main ( void )
{
        Matrix matrix ( 4, 5 ) ;
        Matrix.row.size  =  8 ;
}
```

Advantages of Objects as Class Members

© Copyright Virtual University of Pakistan

It is a way of reusing the code when we contain objects of our already written classes into a new class. For example,* Date *class can be used as data member of* Student, Employee *or* PersonInfo *class. In this approach, we don't have to test our previously written classes again and again. We write a class, test it once and add it into our components library to use it later.

It gives clarity and better management to the source code of our programs when we break up problems into smaller components. The smaller components can be managed independently from their contained objects forming their own classes. For example, in the previous example program, *Matrix* was subdivided into *Row* and *Column* classes.

When we declare an object as a constant data member inside a class then that constant object is initialized using the initializer list. Therefore, a class, whose object is contained as *const* object, must have a parameterized constructor.

Structures as Class Members

We have already studied that structures and classes are very similar in C++ except the default scope of members. The default scope for members of structures is *public* whereas the default visibility for class members is *private*.
Likewise, objects of different classes can act as data members, structures and unions can also act as data members of a class. In fact, all the discussion above for *Class Objects as Class Members* applies to this topic of *Structure Objects as Class Members*.

```
#include <iostream.h>
#include <stdlib.h>

struct VehicleParts
{

   int wheels;
   int seats;

   VehicleParts()
   {
      cout << "\n VehicleParts - default constructor";
   }

   VehicleParts(int wheels, int seats)
   {
      this->wheels = wheels;
      this->seats  = seats;
      cout << "\n VehicleParts - parameterized constructor";
   }
```


© Copyright Virtual University of Pakistan

```
   ~VehicleParts()
   {
      cout << "\n VehicleParts - destructor" << endl;
   }

} ;

class  Vehicle
{

   private :
         VehicleParts vehicleParts ;

   public :
      Vehicle( )
      {
            cout << "\n Vehicle - default constructor" << endl;
      }
      Vehicle( int  a, int  b ) : vehicleParts( a, b )
      {
            cout << "\n Vehicle - parameterized constructor";
      }
      ~Vehicle( )
      {
            cout << "\n Vehicle - destructor";
      }
      void setPartsNum ( int a, int b )
      {
         vehicleParts.wheels = a ;
         vehicleParts.seats = b ;

      }
      void displayNumVehicleParts ( )
      {      /* The data members of the structure are public,
            therefore, directly accessible from outside. */
            cout << "\n Number of wheels for this vehicle are "
                        << vehicleParts.wheels;
            cout << "\n Number of seats for this vehicle are "
                        << vehicleParts.seats << endl;

      }

} ;

void f()
{
  Vehicle car( 4, 2 ) ;
  car.displayNumVehicleParts( ) ;
}
```


© Copyright Virtual University of Pakistan

```
void main ( )
{
   f();
   system ( "PAUSE" ) ;
}
```

The output of the program is:

```
VehicleParts - parameterized constructor
 Vehicle - parameterized constructor
 Number of wheels for this vehicle are 4
 Number of seats for this vehicle are 2

 Vehicle - destructor
 VehicleParts - destructor
 Press any key to continue . . .
```

Classes inside Classes

In C language, structures can be defined inside structures, Similarly in C++, we can have structures or classes defined inside classes. Classes defined within other classes are called nested classes.

A nested class is written exactly in the same way as a normal class. We write its data members, member functions, constructors and destructors but no memory is allocated for a nested class unless an instance of it is created. C++ allows multiple levels of nesting. Importantly, we should be clear about the visibility of the nested class. If a class is nested inside the *public* section of a class, it is visible outside the outer (enclosed) class. If it is nested in the *private* section, it is only visible to the members of the outer class. The outer class has no special privileges with respect to the inner class. So, the inner class still has full control over the accessibility of its members by the outer class. Interestingly, the *friend* operator can be used to declare enclosed class as a friend of inner class to provide access to inner class's *private* members. This operator is used in the same way as we use it for other classes that are not nested. We can also make the inner class to access the *private* members of enclosed class by declaring the inner class as a *friend* of outer class.

The reason of nesting classes within other classes is simply to keep associated classes together for easier manipulation of the objects.

```
/* This program illustrates the nested classes */

#include <iostream.h>
#include <stdlib.h>

class Surround
{
        public :
                class FirstWithin
```


© Copyright Virtual University of Pakistan

```
        {
            public:
                FirstWithin ()
                {
                   cout << "\n FirstWithin - default constructor";
                }
                ~FirstWithin()
                {
                   cout << "\n FirstWithin - destructor";
                }
                int getVar() const
                {
                   return (variable);
                }
            private:
                int variable;
        };

    FirstWithin myFirstWithin;

    private:
        class SecondWithin
        {
            public:
                SecondWithin()
                {
                   cout << "\n SecondWithin - default constructor";
                 }
                ~SecondWithin()
                {
                   cout << "\n SecondWithin - destructor ";
                }
                int getVar() const
                {
                   return (variable);
                 }
            private:
                int variable;
        };

// other private members of Surround

};

void f(void)
{
    Surround::SecondWithin a;
    Surround::FirstWithin b;
    Surround c;
    c.myFirstWithin.getVar();
```


© Copyright Virtual University of Pakistan

```
}

int main()
{
    f();
    cout << endl << " ";
    system("PAUSE");
    return 0;
}
```

The output of the program is as follows:

```
SecondWithin - default constructor
 FirstWithin - default constructor
 FirstWithin - default constructor
 FirstWithin - destructor
 FirstWithin - destructor
 SecondWithin - destructor
 Press any key to continue . . .
```

Notice the access specifier (::) usage in function *f()* to access the members of inner class.

The class *FirstWithin* is visible both outside and inside *Surround*. The class *FirstWithin* has therefore global scope. The constructor *FirstWithin()* and the member function *getVar()* of the class *FirstWithin* are also globally visible. The *int variable* data member is only visible for the members of the class *FirstWithin* as it is declared *private*. Neither the members of *Surround* nor the members of *SecondWithin* can access the variable of the class *FirstWithin* directly. The class *SecondWithin* is visible only inside *Surround*. The *public* members of the class *SecondWithin* canalso be used by the members of the class *FirstWithin*, as nested classes can be considered members of their surrounding class.

The constructor *SecondWithin()* and the member function *getVar()* of the class *SecondWithin* can also only be reached by the members of *Surround* (and by the members of its nested classes). The *int variable* data member of the class *SecondWithin* is only visible to the members of the class *SecondWithin*. Neither the members of *Surround* nor the members of *FirstWithin* can access the variable of the class *SecondWithin* directly.

The nested classes can be considered members of the surrounding class, but the members of nested classes are not members of the surrounding class. So, a member of the class *Surround* may not access *FirstWithin::getVar()* directly. The nested classes are only available as type names. They do not imply as objects containment by the surrounding class. If a member of the surrounding class uses a (non-static) member of a nested class then a pointer to a nested class object or a nested class data member is defined in the surrounding class. The pointer is further used by the members of the surrounding class to access members of the nested class.

It is important to know how do we define Member functions of nested classes. They may be defined as inline functions or they can also be defined outside of their surrounding class. Consider the constructor of the class *FirstWithin* in the previous example.

© Copyright Virtual University of Pakistan

The constructor *FirstWithin()* is defined in the class *FirstWithin*, which is, in turn, defined within the class *Surround.*
Consequently, the class scopes of the two classes must be used to define a *constructor* as the following:

```
Surround :: FirstWithin :: FirstWithin ( )
{
        variable = 0 ;
}
```

The classes *FirstWithin* and *SecondWithin* are both nested within *Surround*, and can be considered members of the surrounding class. Since members of a class may directly refer to each other, members of the class *SecondWithin* can refer to *public* members of the class *FirstWithin* but they cannot access *private* members of the *FirstWithin* unless *SecondWithin* is declared as a *friend* of *FirstWithin.*
See the code snippet below, we have used *friend* operator here extensively so that all the three classes *Surround*, *FirstWithin* and *SecondWithin* can access *private* members of each other.

```
class Surround
{
        class SecondWithin ;
        public :
                class FirstWithin
                {
                        friend class Surround ;
                        friend class SecondWithin ;
                        public :
                                int getValue()
                                {
                                    Surround :: variable = SecondWithin :: variable ;
                                    return (variable);
                                }
                        private :
                                static int variable ;
                } ;
                friend class FirstWithin ;
                int getValue ( )
                {
                        FirstWithin :: variable = SecondWithin :: variable ;
                        return (variable) ;
                }
        private :
                class SecondWithin
                {
                        friend class Surround ;
                        friend class FirstWithin ;
                        public :
                        int getValue ( )
                        {
```


© Copyright Virtual University of Pakistan

```
                    Surround::variable = FirstWithin::variable;
                    return (variable) ;
                }
                private:
                    static int variable;
                };
        friend class SecondWithin ;
        static int variable;
};
```

We can also define structures inside classes in the same manner as we defined classes within classes. Again, all the above discussion is valid for structures inside classes except the default scope of members in structures is *public* unless explicitly declared otherwise.

Tips

- ***A class can contain instances of other classes as its data members.***
- It is a way of reusing the code when we contain objects of our already written classes into a new class.
- The inner data members of the object are constructed and then the object itself. The order of destruction of an object is reverse to this construction order, where the outer object is destroyed first before the inner data members.
- Initializer list is used to initialize the inner objects at the construction time.
- In C++, we can have structures or classes defined inside classes. Classes defined within other classes are called nested classes.

© Copyright Virtual University of Pakistan

Lecture No. 41

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 12

12.2, 12.3

Summary

63) Template Functions
64) Overloading Template Functions
65) Explicitly Specifying the Type in a Template Function
66) Template Functions and Objects
67) Recap

In the previous lecture, we talked about the objects used as data members within classes. In this case, you will find that there is a lot of code reuse. A completely debug code is used as building blocks for developing new and more complex classes. Today's discussion will mainly focus on a new way of code reuse with an entirely different style of reuse. This method is called templates.

Template Functions

There are two different types of templates in C++ language i.e.' function templates and class templates. Before going ahead, it will be sagacious to know what a template is? You have used a lot of templates in the childhood. There are small scales being marketed at the stationary shops having some figures on them like circle, a square, rectangle or a triangle. We have been using these articles to draw these shapes on the paper. We put the scale on the paper and draw the lines with the pencil over that figure to get that shape. These engraved shapes are generally called stencils. But in a way, these are also templates. We may also take these 'cut-outs' as sketches. So a template is a sketch to draw some shape or figure. While drawing a special design, say of furniture, we develop a template for this, which is not an actual piece of furniture. We try that its shape should be like the outline. Later, the cut out prepared out of wood in line with the template, is actual piece of furniture. We can think of making a triangular template and then drawing it on a piece of wood and shaping it into a triangle. We can use the same template and put it on a piece of metal and can cut it into a triangle and so on. In a way, that template is allowing us the reuse of a certain shape. This is the concept we are going to try and build on here.

© Copyright Virtual University of Pakistan

Here we are going to discuss the benefits of the function templates. We have been using a *swap* function. We want to interchange two things. You know the technique that we need a third temp-place holder. If we want to swap two integers *i* and *j*, the code will be as under:

```
void swap(int &i, int &j)
{
        int tmp;
        tmp = i;
        i = j;
        j = tmp;
}
```

This is a very generic way of interchanging two values. We have written a swap function to interchange two integers. To interchange two doubles, we have to come up with some other swap function for doubles and so on. Whenever, a need to use this swapping technique for different data type arises, we have to write a new function. Can we write such functions? Yes, we can. These functions can be overloaded. We can have functions with the same name as long as the types or the number or the arguments are different. Compiler can detect which function should be used. It will call that function appropriately. So you can define *swap* for integers, floats and doubles. There is also no problem in defining multiple versions of this function with different data types. Depending on what is required, the compiler will automatically make a call to the correct function. This is the overloading. The code for every data type looks like:

```
void swap(SomeDataType &firstThing, SomeDataType &secondThing)
{
        SomeDataType tmp;
        tmp = firstThing;
        firstThing = secondThing;
        secondThing = tmp;
}
```

This is a sort of generic code, we are writing again and again for different data types. It will be very nice if somehow we can write the code once and let the compiler or language handle everything else. This way of writing is called templates or function templates. As seen in the example of a template of a triangle, we will define a generic function. Once it is defined and determined where it will be called for some specific data type, the compiler will automatically call that function.

As discussed in the example of overloaded functions, the automatic part is also there. But we wrote all those functions separately. Here the automatic part is even deeper. In other words, we write one template function without specifying a data type. If it is to be called for *int* data type, the compiler will itself write an *int* version of that function. If it is to be called for double, the compiler will itself write it. This does not happen at run time, but at compile time. The compiler will analyze the program and see for which data type, the template function has been called. According to this, it will get the template and write a function for that data type.

© Copyright Virtual University of Pakistan

Now, we will see the idea or technique for defining template function. Here you will come across some new keywords. First keyword is "template". These are the recent addition to the language. Some old compilers may not have these features. However, now almost all of the compilers implement these features. Another keyword is *class* that is quite different than we have been using for defining the classes. This is another use of the same keyword. Normally, when we define a generic function, it is independent of the data type. The data type will be defined later in the program on calling this function. The first line will be as *template<generic data type>*. This generic data type is written while using the *class* key word as *template<class variable_name>*. So the first line will be as;

```
template<class T>
```

We generally use the variable name as *T* (T evolves from template). However, it is not something hard and fast. After the variable name, we start writing the function definition. The function arguments must contain at least one generic data type. Normal function declaration is:

```
return_type    function_name(argument_list)
```

return_type can also be of generic type. There should be at least an argument of generic type in the *argument_list*. Let's take a very simple example. This is the function *reverse*. It takes one argument and returns its minus version. The *int* version of this function is as:

```
int reverse(int x)
{
        return (-x);
}
```

Similarly its double version will be as:

```
double reverse(double x)
{
        return (-x);
}
```

Similarly, we can define it for other data types.
Let's see how can we make a template of this function. The code is as:

```
template<class T>

T reverse(T x)
{
  return (-x);
}
```

In the above function definition, we have used *T* as generic type. The return type of the function is *T* which is accepting an argument of type *T*. In the body of the function, we just minus it and return *x* that is of type *T*. Now in the main program, if

© Copyright Virtual University of Pakistan

we write it as *int i* and then call *reverse(i), what* will happen? The compiler will automatically detect that *i* is an *int* and *reverse* is a template function. So, an *int* version of *reverse* function is needed in the program. It uses the template to generate an *int* version. How does it do that? It replaces the *T* with *int* in the template function. You will get exactly the same function as we have written before as *int reverse(int x).* This copy is generated at compile time. After the compilation, all the code is included in the program. A normal function call will happen. When we write *reverse(i)* in the main or some other function, it is not required to tell that an *int* version is needed. The compiler will automatically detect the data type and create a copy of the function of the appropriate data type. This is important to understand. Similarly if we have *double y;* and we call the *reverse* function as *reverse(y)*; the compiler will automatically detect that this program is calling *reverse(i)* and *reverse(y). Here i* is an *int* and in *reverse(y), y* is a *double*. So the compiler will generate two versions of *reverse* function, one with *int* and the other with *double*. Then it will be compiled and the program will execute correctly. This is the classic example of code reuse. We have to pay attention to writing the template. It should be generic in nature.

For a programmer, there are facilities of the macros and *#define* which have the limitations. Macro is a code substitution while *#define* is a value substitution. Here, in templates, we write a generic code and the compiler generates its copies of appropriate types. It is always better than ordinary function overloading. Now let's take the previous example of *reverse*. When we write the function of *reverse* and give it a value of type *double*, a version of the *reverse* function for *double* is created, compiled and used. If we write the same template in some other program and call it for an integer, it will still work. It will automatically generate code for *int*. We should write a template while doing the same functionality with different data types. The rule for templates is that at least one argument in the function should be of generic data type. Other wise, it is not a template function. We write a template *class T* and use *T* as a new data type. Being a template data type, it does not really exist. The compiler will substitute it on its use besides generating an appropriate code. There are some limitations that should be kept in mind. We cannot store the declarations and definitions of these functions in different files. In classes, we have this for certain purposes. In case of a class, we put the declaration of the class and its basic structure in the header file to facilitate the users to know what the class does implement. The definition of the class, the actual code of its functions and the manipulations are provided along with as object code. Here in the template case, the compiler makes a copy of the source code and converts it to object code. We cannot give the declaration of the template function in one file and the definition in some other. If we store these in different files, it will not compile. It does not have real data type and still has parameterized or generic data type in it. So the declaration and definition of a template function should be in the same file. We will include this file or keep the template with our main program. When it will be used, the copies of code will be automatically generated. So it is a slight limitation with templates. In any case, template class or template functions are for our own use. We do not write template functions as libraries for other people as it is like giving away our source code.

For template functions, we must have at least one generic argument. There may be more than one generic arguments. We have to pass it to pieces of data to be swapped. We can write swap function as:

© Copyright Virtual University of Pakistan

```
template<class T>

 void swap(T &x, T &y)
{
                T tmp;
                tmp = x;
                x = y;
                y = tmp;
}
```

In the above function, we are passing both arguments of generic type and declared a *tmp* variable of generic type. We can also mix generic data types and native data types including user defined data types. This template version of swap function can be used for integer, double, float or char. Its copy will be created after writing the *swap(a, b)* in the program. If we have *int a, b;* in the program, *int* copy of *swap* function will be generated. If you have written *char a, b;* a *char* copy of *swap* function will be generated. A copy is simple substitution of *char* with *T*. Just replace *T* with *char* in the *swap* function and remove the first line i.e. *template<class T>*, this is the function that the compiler will generate. Now we have seen examples of one generic and two generic arguments functions. You can write template functions with more than two generic arguments.

So far, we have been using only one generic data type. However, the things can not be restricted to only one generic data type. We can use more than one generic data types in template functions. We can do that by extending the *template<class T>*. The use of two generic types can be written as:

```
template <class T, class U>
```

We can use any name in place of *T* and *U*. Two data types can be mixed here. So we can write a function that takes an *int* and *float* and can multiply these two. We can use *T* as *int* and *U* as *float* or whatever is the function requirement. Let's look at another example of template function. We want to write a function that takes two arguments of same type and tells which of the two is greater. The code will be as below:

```
// A small program shows the use of template function
#include<iostream.h>

// template function of deciding a larger number
template<class T>
T larger(T x, T y)
{
   T big;
   if (x > y)
      big = x;
   else
      big = y;
   return(big);
}
```


© Copyright Virtual University of Pakistan

```
// the main function
void main()
{
   int i = 7, j = 12;
   double x = 4.5, y = 1.3;
   cout << "The larger of " << i << " and " << j << " is " << larger(i, j)<< endl;
   cout << "The larger of " << x << " and " << y << " is " << larger(x, y)<< endl;
   //cout << "The larger of " << x << " and " << y << " is " << larger(i, y)<< endl;

}
```

The output of the program is:

```
The larger of 7 and 12 is 12
The larger of 4.5 and 1.3 is 4.5
```

The function *larger* is very simple. We have two arguments in it, compared these two arguments and set the variable *bigger*. You have noticed that the definition of *larger* is not exactly correct. In the *if* condition, we check that *x* is greater than *y*. So in the else-part *x* can be equal to or lesser than *y*. Let's see their use in the main. We declare two integers *i* and *j* and two doubles *x* and *y*. Then we use the *cout* to display the result. The *larger* function will return the bigger argument. When we write *larger(i, j)*, the compiler will detect it and generate an *int* version of the larger function. In the next line, we have used *larger(x, y)* as *x* and *y* are double. Here, the compiler will generate a double version of the *larger* function. Now compile the program and it is ready to be executed. The two versions of *larger* functions will be executed .We get the larger of two integers and larger of two doubles. You have noticed that the last line of the code is commented out. In that line, we are trying to call the *larger (i, y)*. There is a problem that if you uncomment this line, it will not be compiled. We have only defined one generic class type in the templatized function i.e. *class T*. Here we are trying to call it with an *int* and a *double*. The compiler does not know what to do with it. Either it should promote the *int* to *double* and call the *double* version or demote the *double* into *int* and call the *int* version. But the compiler will not make this decision. It will be a compile time error. We can write another template function that handles two data types or try to call this function with one data type. Be careful about these fine points. Template is a nice thing to use but needs to be used carefully. The compiler may give an error, so you have to correctly use it. Here in the larger function, we have to provide both arguments of the same data type.

Following is an example of *larger* function using two different generic types.

```
// A template function example using two generic types
#include<iostream.h>

// template function
template <class T, class U>
void larger(T val1, U val2)
{
  if (val1 > val2)
    cout<<"First is larger"<<endl;
```


© Copyright Virtual University of Pakistan

```
  else
     cout<<"First is not larger"<<endl;
}

// main function
void main()
{
  larger(2.1, 9);
  larger('G', 'A');
}
```

The output of the program is:

```
First is not larger
First is larger
```

Overloading Template Functions

Let's take benefit of our knowledge and discuss the things of the next level i.e. function overloading. Under the techniques employed in function overloading, the functions have the same name but differ either by the number of arguments or the type of the arguments. Remember that the return type is not a differentiator when you are overloading the functions. Now if the number or type of the arguments is different and the function name is same, the compiler will automatically call the correct version. The same rule applies to the template function. We can write overloaded template functions as long as there is use of different number or type of arguments.

We have written a templatized *swap* function. Let's rename that function as *inverse*. It will swap the variables. We have another *inverse* function that takes one argument and return the minus of the argument supplied. We have two template functions named *inverse*. Here is the code of the program:

```
//  An example of overloaded template functions.

#include<iostream.h>

// template function
template<class T>
void inverse(T &x, T &y)
{
   T temp;
    temp = x;
    x = y;
    y = temp;
}

// overloaded inverse fucntion
template<class T>
T inverse(T x)
{
  return (-x);
```


© Copyright Virtual University of Pakistan

```
}

// the main fucntion
void main()
{
  int i = 3, j = 5;
  // calling the templatized functions
  inverse(i);
  inverse(i, j);
  cout << "i = " << i << ", j = " << j << endl;
}
```

The output of the program is:

```
i = 5.6, j = -3.4
```

In the above program, we have overloaded template functions. When we write *invers(i)*, the compiler will detect the *inverse* function with one argument and generate its *int* code. However, on writing *inverse (i, j)*, it will generate an *int* version of the *inverse* function which takes two parameters. This is not a good example as the function names are confusing. The function which does swapping should be named as swap while the one doing negative should be named as negative. There might be good occasions where you might want to use overloaded templates. The same rule of ordinary function overloading applies on template function overloading.

Explicitly Specifying the Type in a Template Function

In the template functions, sometimes we want to see which version of the template function should be used. Let's take the example of *reverse* function. We call that function for *double* data type. A function for the *double* would have been generated and its negative value will be returned. Suppose we want to pass it a *double* but return an *integer*. We want to return a negative integer that was a part of the double variable, passed to the function. We can force the compiler to generate an *int* version of this function while not passing it an *int*. It can take place when we are going to call the function in the program. We write the data type in the angle brackets between the function name and argument list. For example, if we have a template *reverse* function, which returns *-x*. In the program, we have *double a*. As soon as we write *reverse(a)*, the compiler will generate a *double* version of *reverse* function. Now we want that 'a' should be passed to this function while returning an *int*. The prototype of the function is *T reverse(T x)*. We want that *T* should be replaced by *int*. At the same time, we want to pass it *double*. To obtain this, we will write as *reverse <int> (a);* writing *<int>* forces the compiler to also generate an integer version of the function. There may be instances where this technique is useful.

Suppose, we have a template of *reverse* function that depends on two generic data types. The function template is as follows:

```
template <class T, class U>

T reverse (U x)
{
        return -x;
```


© Copyright Virtual University of Pakistan

```
}
```

Now the return type is *T* while the argument is of type *U*. In the body of the function, we return *-x* and the conversion automatically takes place. In this function template, we are using two generic types. The return type cannot force anything as it is used later in the assignment statement. If we have *double a* in the program, and say *reverse(a);* What version will be generated? What will be replaced with *T* and *U*? We can force it as *reverse<int>(a);* In that case, it will force *T* to become *int*. It will force *U* to become of the type *a* i.e. *double*. It will take a double number, reverse and convert it into *int* and return it. You can explicitly specify it as *reverse<int, double> (a);* so we have specified both *T* and *U*. We are specifying this to the compiler so that when the compiler generates the code, it carries out it for these versions. It is like the default argument list. You can not force the second part only i.e. you can not force *U* only while missing *T*. It has to go left to right. You can do as *revere<double, double> (a)*; or *reverse(double, int>(a)*. The appropriate versions will be generated. Actually, you can force what type of versions should be generated in your code. Normally, we do not need to force the template functions. Normally the template function is used for different data types while generating appropriate versions by the compiler.

Here is the code of the above-explained program:

```
// An example of forcing the template functions for some specific data type
#include<iostream.h>

template <class T, class U>
T reverse (U x)
{
  return (-x);
}

// main function
void main()
{
   double amount = -8.8;
   // calling the function as double reverse(int)
  cout << reverse<double, int>(amount) << endl;

   // calling the function as double reverse(double a)
   cout << reverse<double>(amount) << endl;

   // calling the function as double reverse(double a)
   cout << reverse<double, double>(amount) << endl;

   // calling the function as int reverse(int a)
   cout << reverse<int, int>(amount) << endl;
}
```

The output of the code is as follows:

```
8
8.8
```


© Copyright Virtual University of Pakistan

```
8.8
8
```

Template Functions and Objects

We have seen the template functions and know that classes also have member functions. Can we use these templates with the member functions of a class? Yes, we can templatize member functions. The operations used within template functions should be present in the public part of the class. Let's see an example to understand this. Suppose we have created a class *PhoneCall*. We have a *lengthOfCall* data member in the class that tells about the duration of the call. Another character data member is *billCode*. The *billCode* will tell us that this call is local, domestic or international. Suppose, we browse the bill and notice a wrong call. What should we do? We will pick up the phone and call the phone company or go the phone company office to get the bill corrected. How will they do that? They will verify it with the record and see there is no such call or the duration is not chargeable. So far, we have been using the *reverse* function to minus the input argument. Here we want to reverse the phone call. Suppose, we define that when the *billCode* is 'c', it means that call has been reversed or cancelled. Can we use this concept and write a *reverse* function for this class. Let's revisit the *reverse* function template.

```
template<class  T>

T reverse(T x)
{
  return (-x);
}
```

Here *T* is the generic type that will be replaced with int, float etc. Can we replace *T* with the object of the class *PhoneCall*. How will that work? Let's replace the *T* with *PhoneCall*, the code looks like:

```
PhoneCall reverse(PhoneCall x)
{
  return (-x);
}
```

The declaration line shows that it returns an object of *PhoneCall* and takes an argument of type *PhoneCall*. Inside the body of the function, we are returning *-x*. What does *–PhoneCall* mean? When we are using template functions in the classes, it is necessary to make sure that whatever usage we are implementing inside the template function, the class should support it. Here we want to write *–PhoneCall. So* a minus operator should be defined for the *PhoneCall* class. We know how to define operators for classes. Here the minus operator for *PhoneCall* will change the *billCode* to 'c' and return the object of type *PhoneCall*. Let's have a look on the code.

```
// A simple program to show the usage of the template functions in a class

#include<iostream.h>

// reverse template function
```


© Copyright Virtual University of Pakistan

```
template<class T>
T reverse(T x)
{
  return (-x);
}

// definition of a class
class PhoneCall
{
  private:
        int lengthOfCall; // duration of the call
        char billCode;  // c for cancelled, d for domestic, i for international, l for local

  public:
    PhoneCall(const int l, const char b); // constructor
    PhoneCall PhoneCall::operator-(void); // overloaded operator
    int getLengthOfCall(){ return lengthOfCall;}
        void showCall(void);
};

PhoneCall::PhoneCall(const int len=0, const char b='l')
{
        lengthOfCall = len;
        billCode = b;
}

void PhoneCall::showCall(void)
{
   cout <<"The duration of the call is " << lengthOfCall << endl;
   cout <<"The code of the call is " << billCode << endl;
}
// overloaded operator
PhoneCall PhoneCall::operator-(void)
{
   PhoneCall::billCode='c';
   Return (*this);
}

// main function
void main()
{
        PhoneCall aCall(10, 'd');
        aCall.showCall();
        aCall = reverse(aCall);
        aCall.showCall();
}
```

The output of the code is:

```
The duration of the call is 10
The code of the call is d
```


© Copyright Virtual University of Pakistan

```
The duration of the call is 10
The code of the call is c
```

We have overloaded the minus operator in the *PhoneCall* class. We cannot change the unary or binary nature of operators. The minus operator is lucky due to being unary as well as binary simultaneously. Here, we have overloaded unary minus operator so, it can be written as *-x*. The definition of operators is same as ordinary functions. We can write whatever is required. Here we are not taking negative of anything but using it in actual meaning that is reversing a phone call. In the definition of the minus operator, we have changed the *billCode* to 'c' that is cancelled and the object of type *PhoneCall* is returned. Now look at the definition of the *reverse* template function and replace the *T* with *PhoneCall*. In the body of the function where we have written *–x*, the minus operator of the *PhoneCall* class will be called. Since it is a member operator and the calling object is available to it. Now let's look at the main program. We take an object of *PhoneCall* as *PhoneCall aCall(10, 'd');* The object *aCall* is initialized through the constructor. Now we display it as *aCall.showCall();* that shows the length of the call and bill code. After this, we say *reverse(aCall);* The reverse function should not be changing *aCall* and should return an object of type *PhoneCall*. It means that *aCall = reverse(aCall);* the object returned is assigned to *aCall*. Reverse is a template function, so the compiler will generate a *reverse* function for *PhoneCall*. When it will call *-x*, the member minus operator of the class *PhoneCall* will be called. It will change the *billCode* of that object and return the object. As we have written *aCall = reverse(aCall)* so the object returned from *reverse* having *billCode* as 'c' will be assigned to *aCall*. Now while displaying it using the *aCall.showCall()*, you will see that the *billCode* has been changed. So reverse works.

Recap

Let's just recap what we just did in this lecture. We have defined a template function *reverse*. In the template definition, this function returns *-x* whatever *x* is passed to it. After this, we wrote a *PhoneCall* class and defined its minus operator. Whenever we have to take minus of the *PhoneCall*, this operator will be called. This action is based on the phone call domain and is to change the bill code to 'c'. So the minus operator returns an object of type *PhoneCall* after changing its bill code. Now these two are independent exercises. The class *PhoneCall* does not know about the *reverse* function. It only has defined its minus operator. On the other hand, the *reverse* template function has nothing to do with the *PhoneCall* class. It is the main program, linking these two things. The main function declared an object of type *PhoneCall* and called the *reverse* function with that object. When we write the statement *aCall = reverse (aCall);* the compiler automatically detects that we have got a situation where *reverse* template function is called with the object of type *PhoneCall*. It needs to generate a copy of this template function that will work with *PhoneCall*. When it goes to generate that copy, it encounters with *return(-x)*. It has to know that a minus operator exists for that class. If we have not defined the minus operator what will happen. The compiler may give an error that it does not know what is *-PhoneCall*. On the other hand, sometimes default substitution takes place. It may not be what you want to do. You have to be careful and look at the implications of using a template function with a class and make sure all the operations within template function should be defined explicitly for the class so that function should work correctly. Once this is done, you realize that life has become simpler. The same *reverse* function works for this class. Now you can extend the concept and say how to reverse a car, how to

© Copyright Virtual University of Pakistan

reverse a phone call, how to reverse an int and so on. The idea is combining two very powerful techniques i.e. operator overloading and template mechanism which provides for writing the code at once. In the normal overloading, the facility is that we can use the same name again and again. But we have to write the code each time. Normally, the code is different in these overloaded functions. In this case, we are saying that we have to write identical code i.e. to reverse something, swap two things. So the code is same only data type is different then we should go and define a template for that function and thus, template is used again and again. We started with the template function, used at program level. The use of template with class was also demonstrated. This combination has some rules. This is that all the operations that template function is using should be defined in the class. Otherwise, you will have problems.



© Copyright Virtual University of Pakistan

Lecture No. 42

Reading Material

Deitel & Deitel - C++ How to Program Chapter. 12, 20

12.4, 12.5, 12.7, 12.8, 20.1

Summary

68) Class Templates
69) Class Templates and Nontype Parameters
70) Templates and Static Members
71) Templates and Friend Functions
72) Example
73) Sample Program
74) Advantages and Disadvantages of Templates
75) Standard Template Library

As discussed earlier, template functions are utilized while writing functions for generic data type. We take benefit of templates in case of writing the same function repeatedly. In this case, the writing code seems very similar, in fact identical. But the data type is changed for different versions. So we write a function for generic data type whose syntax we have used as under

template <class T>

Here T is a generic data type. Now in the function we write T while dealing with a data type. This becomes a generic template of the function. During the process of using this template, this function with a particular data type is called. The compiler automatically detects the type of the data passed (say we passed an int) and generates a new copy of the function with *int*. Here T is written in the original template. The copy is compiled to the object code while existing in the program. The same thing applies to other data types, used for the same function. Thus, we create a family of functions with a single template. The functionality of these functions is the same but with different data types. For example, if we want to find the square of a number, a template square will be written first. It will be called with int, double or float. Otherwise, we have to write the over loaded versions of the square function for different data types. So template functions are of good use for the programmers. They promote code reuse. The major advantage of their use is that once we write correct code with correct logic for a template function, there will be no need to re-write it.

© Copyright Virtual University of Pakistan

How can we test a template? We write the template in reverse technique at the moment. When it is known what the template has to do, we take a data type for example *int*. A complete function for int is written and tested. After the ascertainment of the accuracy of function's working, the *int* in the function is replaced with T and declared a template by writing *template <class T>*. We cannot see the code generated by the compiler in the editor. So it becomes difficult to debug the template code. We have to read and check the code carefully while writing it.
After having a detailed discussion on the template function, we will now talk about template classes.

Class Templates

Creation of a data type of our own with the same behavior for *int*, *float* and *double* etc, is the case of defining a complete interface and implementation in a generic fashion. To further understand this concept, let's talk about a data structure called stack. We will now try to understand how does stack work, what its properties are and can we make it generic.
You might have seen the plates, kept together in a orderly manner i.e. one on the other. This is a stack. Now if someone wants to add a plate on the pile, he will have to put it on the top of the stack. So, there is only one way to add something on the stack. Similarly, if we want to pick a plate from the pile, it will be taken from the upper-most tier. Thus the property of the stack is that the last placed thing will be picked first. This phenomenon is called 'Last-in, first out' or LIFO. In programming, we can understand what thing we want to add, the required thing is added to the top of the stack. When we pick up a thing from it, the last placed item is picked first. Following this rule of stack (last in first out), we can make a stack of integers, floats and doubles etc. Here, the stack is a class with a defined behavior, interface and the data, it holds. Now we say that the data held by the class is variable to help make a stack of integers, floats or doubles. Thus, stack is a good candidate for a template class. It means that when we instantiate the class for creating objects, a stack of integers or floats is required. The behavior of the compiler in template classes is the same as in template functions. If we want to instantiate a template class with a new data type, the compiler will generate a new version of the class with the specific data type at the place of T in the template class.
We know that a class is a user-defined data type. With the help of a template class, we make another class of the user defined data type. In other words, things are not restricted to creating copies of class only for native data type. Copies of class of our own data type can also be created. It is a case of a real extensibility.
Let's see the syntax of this generic template class. It is similar to the simple template function in which we write *template <class T>*. Here T is the placeholder that will be replaced by the data type when we use it. The syntax of the template class is

```
template <class T>
class class-name()
{
        definition of class
};
```

In the definition of the class where the generic data type is required, we write T. For example, there is a class in which we want to write *int* data type. The *int* is the data

© Copyright Virtual University of Pakistan

type that may be a *float* or *double* at different times. For this, T is written wherever we are using *int* in the class definition. Be careful that some times, we may use integers as it in a class. For example, when we create a vector or array, its size will be an integer. But the array will be of integers, floats or chars. So don't confuse it them. It is better to replace T whenever necessary.
We start writing of a template with the following line

```
template <class T>
```

Later, definition of the class in ordinary fashion begins. We should use T wherever in case of employing the generic data type. T is not something fixed for this purpose. We can use a, b or c or whatever needed. However, T is normally used.
The member functions are normally defined out side the class. To define the member functions of the template class, we write

```
template <class T>
class name <T>::function name (argument list)
{
        // function body
}
```

In the function body, the programmer will write T wherever it is needed. This way, the template-class and its member functions are defined. However, when we use the class in main program or other function, the objects of this class are declared. Suppose there is a class Number, say *Number x*; As Number is a template class, we will have to tell the type of the number. Let's see the code of this simple template class Number. The Number class can store and display a number. The definition of the class is written as under.

```
template<class T>
class Number
{
        private:
                T myNumber;
        public:
                Number( T n );
                display();
};
```

We create an object of this class in main program by telling the type of the number in the following way

```
Number <data type>
```

Here data type may be *int, float* or *double*. Now we can create an object of this class for an integer type as follows.

```
Number <int> x ;
```

We can read the line as *x* is an object of class Number and its parameter is an *int*. The way of analyzing it is that wherever we wrote T in the class, now *int* is written there.

© Copyright Virtual University of Pakistan

Compiler does it automatically. We will not see the code of this class written for *int*. The compiler generates a copy of the class for *int* and uses it. Similarly, if we write

```
Number <double> y ;
```

Here *y* will be an object with the data member of type *double*. Again, the entire copy of the class will be created with *double* everywhere instead of T. The program will compile and run. So it is quiet useful and a big shortcut.

Class Templates and Nontype Parameters

There is a little variation, which is we can also use non-type parameters in templates. What do non-type parameters mean? We have been writing *template <class T>*. However, while writing *template <class T, int element>*, the non-generic type (i.e. int) will be treated as a constant in the class definition. In the class definition, wherever we use the name *element*, it will be replaced by the value passed to it. Arrays when declared and given a number in square brackets, the number will be a constant. Similarly, while using with the # sign, we associate a name with a number which is a constant. Here the non-type parameter in a way behaves like a constant. We can use it to give a dimension to the things. Instantiating a class, we not only replace T but also provide a value for the non-type parameter defined in the class definition.
By using templates, we save a lot of effort and labor. The other big motivating factor is the reuse of tested and tried code for a particular problem.

Templates and Static Members

Now let's talk about the implications of the template classes. To understand the behavior of templates with static members, we have to comprehend the concept of static member variables. Static variable members are used to define ordinary classes. The static variable has a single copy for the whole class. So there are not separate copies of the static data variable for each object like ordinary data members.
Now let's see what happens when a static member is a part of a template class. The instantiation of the class has two parts i.e. one is creating an object while the other is the type of the object. For example, from the previous class *Number*, we write

```
Number <int> x ;
```

Here *x* is an object of generic class Number with a specific type *int*. We can also use *float* or *double* instead of *int*. We suppose, there is a static variable in the *Number* class. On instantiating the class with *int*, there will be a copy of static variable for *int* set-off objects. If we instantiate the class for *float*, there is going to be a copy of the static member for float numbers. So the member is static (i.e. there is one copy) for that type of the objects. There will be one static value for different object created with type *int* while another static value for different objects created for type *double*. So, this static value is not class wide. It is something of specific nature. In simple words, the compiler reads the code of program (main or other function) and generates a copy of the template class accordingly. It also gives a name of its own to this copy. Thus in

© Copyright Virtual University of Pakistan

a way, the compiler generates a new unique class, replacing T with *int* (or any other data type we want). The static member of this unique class behaves exactly like the ordinary class. Similarly the compiler generates a copy for *double* with a unique name. Here the static member of this copy will affect the objects created for *double* data type. The static variables are instantiated once for each type whenever we instantiate a class while replacing generic data type with a specific data type.

It is pertinent to note that we can replace the generic data type with our own data type. This is slightly tricky. Suppose, we have written a class, '*Person'*. There is also a generic class Array, which can be instantiated with *int, float* or *double* data type that means it may be an array of integers, floats and doubles respectively. Can we do so with an array of persons? If we have defined a class called *Person*, there may be an array of *Person. Person* now behaves like another data type. At the moment, it does not matter whether the data type is user defined or not.
We have to be careful that when we are using our own object i.e. our own class in a template, it must support the functions and interfaces, needed for this generic structure of the class. So don't put in something that cannot be used by this generic structure. We have discussed an example of phoneCall where *reverse* returns *x* by converting it to *–x*. In that example, we had to define the minus (-) operator for phone call. Similarly, in that example, billCode is changed to 'c'. If number is passed, the negative number will be returned. Its behavior was changed in phoneCall. So we have to take care of these things.
Whenever we use a template class and instantiate it for one of our own classes, it is necessary to have compatible function calls in it. It means that member functions behave properly as per requirements.

Templates and Friend Functions

Now we will have a look on another concept i.e. friend functions. We have read in classes that a programmer can declare functions as friend of the class. Let's first look at the need of friend functions. Suppose we have defined an operator for our class, say + operator. We know that + is a binary operator that takes two arguments. It implements a + b. While implementing + operator for a class, we see that the calling object is on the left side of the operator (i.e. a). The + operator gets this calling object through *this* pointer. It has an argument on the right hand side i.e. *b*. Here, *a* is an object of our class and *b,* an ordinary data type. Similarly, we have *a + 2 ;* where *a* is an object of a class and 2, an ordinary *int*. Now this + operator has to behave intelligently. This way, we have over loaded it within the class definition. What happens when we say *2 + a ;* ? In case of ordinary integers, 2 + 3 is same as 3 + 2. So we want the same behavior in a class. We want *2 + a* behaving the same way as *a+ 2*. We cannot carry out overloading of a member function for this. When we write *2 + a;* there will be an *int* on left- hand side of the + operator. It is not a member function of the class. For a member function or member operator, the object on left- hand side should be that of the class. So if we want *2 + a* ; we have to write a friend function for it. Here, the private data of the class is manipulated by this function. For this purpose, there is need to have access of this function to the private data of the class. The only way through which an external function can have access to the private data of the class is declaring that function to be a friend of the class. So this was the motivation of the friend function.

© Copyright Virtual University of Pakistan

Now what will be the behavior of friend functions in a template class. For example if we write in our template class

friend f ();

Here *f* is function name. Thus in above statement, we say that *f* is a friend function of the class. It is declared in a template class. While creating an object of a template class, we tell the type (*int, float or user defined data type* etc) of which the class is to be generated. Now when we have written *friend f()*, *f* becomes a friend function for all classes generated by using this template. So it is very global that means *f* will have an access to the private data structure of any and all different classes which are instantiated by this template class.
Now we write T in the argument list of the friend function. If we have instantiated a class for an integer, and we have written the friend function *f* with T as *f* <*int*>. This will mean that this function is a friend for all *int* versions of this class. This is an interesting concept. If we have a *double* version of the class, this *f* (i.e. f<int>) will not be a friend of it. It is only a friend of the integer versions of the class.
Similarly, if we have a friend class declared in the template class as

friend class Y;

then it means that all the member functions of the class *Y* can access the private data of any class-type generated with the help of this template. For example, if we generate a class for *int*, the member functions of class *Y* can handle the data structure of the object of this class of type *int*. If we instantiate a class for *double* from this template, the member functions of *Y* can handle the data of the object with *double* version.
Similarly if we write in a template class

friend A :: f ()

Here *f* is a function. It means that the member function *f* of a class *A* is a friend of this template class. We have not granted access to the whole class, but only to a function of that class. That access applies to all classes generated using this template.
Finally, if we use <T> with the function *f*, it becomes specific. In other words, this friend function (i.e. written with <T>) will be a friend of classes generated by the template with T data type. It will not be a friend of the other versions of the class. Here T may be *int*, *float*, *double* or any user defined data type.

Example

Let's apply these concepts on one specific example. We may create a class called *Stack* in a generic fashion. There are some properties of the stack. Firstly, we should be able to know that at what position in the stack we are at a particular time. So this is a concept of stack pointer. We take an array for stack. The stack pointer always points to the top of the stack. It will be good to make the array generic so that we can make an array of any data type. Then there are few questions like is stack empty or full etc.
Here the code seems fairly straight- forward. We start with

template class <T>

© Copyright Virtual University of Pakistan

and then write

class Stack

In the class definition, An integer variable called *size* is declared for the size of the array. Then we declare the array and write its data type as T, which is a generic type .It will be replaced by *int*, *float*, *double* or *char* when we will use this array. We can use the dynamic memory allocation for the array. But we use a fixed size array for the sake of simplicity. To declare an array, we need a constant value. Normally, this constant value is not written in the class definition. It will go to the constructor and be required when the constructor will be called for an object. We can use the constructor to actually define the array for us. We need some utility functions. The function *push()* is used to push an element on the stack. We use the function *pop()* to get an element from the stack. The *push()* and *pop()* functions put and get things of type T. So *pop()* should return something of type T. That means it will return *int* if *int* is pushed and returns *double* if *double* is pushed and so on. So we need *push()* and *pop()* which are parameterized with T. After this, there is need of some functions for generic manipulation like if stack is full or if stack is empty. We can write function *isempty() that* returns a Boolean. If it returns TRUE, the stack will be empty. However, presence of something in the stack will turn it FALSE. Similarly we can write a utility function *isfull()* to check whether the stack is full. We cannot store elements more than that size in the stack. The *isfull()* returns TRUE, if the stack is full. The code of the class definition is very simple. We write T wherever we need a generic data type. It can be written as under.

```
template <class T>
class Stack
{
        private :
                int size ;
                T array [ ] ;
        public :
                Stack ( ) ;
                void push ( T ) ;
                T pop ( ) ;
                bool isEmpty ( ) ;
                bool isFull ( )
} ;
```

In the definition of the functions of the class, we again use <T> immediately after the name of the class. It will be followed by the resolution operator (::) and the function name and finally we write T, wherever we want to use generic data type. It's a definition of the class Stack. While using it, say for *int*, we write *Stack <int>* and provide a initializer value so that it can determine the size of the array. We read it as 'create a stack of type int'. Similarly *Stack<double>x* will mean *x* is a stack of type *double*. The main advantage of this process is that we write the *Stack* class once as the behavior is common regardless of the type of the data we want to put on. Stack class can be used for *int*, *double* or *char* data type.

This is the analysis of the example of Stack class, now as a programmer, it is left to you to write the complete code of the example.

© Copyright Virtual University of Pakistan

Sample Program

Here is a sample program that demonstrates the use of template class.

```
/* This program defines a template class and shows its use for different data types.
There is also the use of template function. It also overloads the << operator.
*/
#include<iostream.h>

template<class T>
class Generic
{
    private:
        T instance;
    public:
        Generic(T i);
        void print(void);
};

//generic constructor
template<class T>
Generic<T>::Generic(T i=0)
{
        instance=i;
}

template<class T>
void Generic<T>::print(void)
{
        cout<<"Generic printing: "<<endl;
        cout<<instance<<endl;
}

class Employee
{
   private:
        int idNum;
        double salary;
   public:
        Employee(int id);
        friend ostream& operator <<(ostream& out, const Employee &e);
};

Employee::Employee(int id=0)
{
        idNum=id;
        salary=4.9;
}

ostream& operator<<(ostream &out, const Employee &emp)
```


© Copyright Virtual University of Pakistan

```
{
        out<<"Employee number "<<emp.idNum;
        out<<" Salary "<<emp.salary;
        return(out);
}

void main()
{
        Generic<int>anInt(7);
        Generic<double>someMoney(6.65);
        Generic<Employee> aWorker(333);
        anInt.print();
        someMoney.print();
        aWorker.print();
}
```

Following is the output of the program.

```
Generic printing:
7
Generic printing:
6.65
Generic printing:
Employee number 333 Salary 4.9
```

Advantages and Disadvantages of Templates

Although, most of the following uses can also be implemented without templates; templates do offer several clear advantages not offered by any other techniques:

- Templates are easier to write than writing several versions of your similar code for different types. You create only one generic version of your class or function instead of manually creating specializations.
- Templates can be easier to understand, since they can provide a straightforward way of abstracting type information.
- Templates are type-safe. This is because the types that templates act upon are known at compile time, so the compiler can perform type checking before errors occur.
- Templates help in utilizing compiler optimizations to the extreme.

Then of course there is room for misuse of the templates. On one hand they provide an excellent mechanism to create specific type-safe classes from a generic definition with little overhead. On the other hand, if misused

- Templates can make code difficult to read and follow depending upon coding style.
- They can present seriously confusing syntactical problems esp. when the code is large and spread over several header and source files.
- Then, there are times, when templates can "excellently" produce nearly meaningless compiler errors thus requiring extra care to enforce syntactical

© Copyright Virtual University of Pakistan

and other design constraints. A common mistake is the angle bracket problem.

Standard Template Library (STL)

Templates are a major code reuse feature. History of C++ language reveals that the template feature was introduced later, relative to other features. But it is a very important feature. We will realize that it makes a lot more sense to keep total code base very small and very concise. It also helps ensure that the same tested code is used everywhere. We had earlier referred to this concept while writing classes. We separated the interface and implementation and sealed the implementation after testing it. Afterwards, we created different objects of the class and every object knew its behavior. Thus there was an abstraction of details. The template functions and template classes go one-step even further. With templates, we can perform different tasks while using one base code. Objects of different types staying with one particular framework can be instantiated. This framework (template) is so important that a couple of researchers actually sat down and started looking at that in programming we often are using the one concept which applies to so many things that we should templatise it. For example, with the help of arrays, we do different manipulations like, 'next element', go to the end of the array, add something at the end etc. Now suppose that the size of array is 100. We want to add the 101st element in the array. We can do it by copying the same array in a new big array and adding the element to that array. Thus we have solutions for different problems, but these are the things of very common use. Their every day use is so important that two researchers wrote a whole library of common use functions. This library is a part of the official standard of C++. It is called STL i.e. Standard Template Library. As a library, it is a tested code base. Some one has written, tested and compiled for the ultimate use of programmers. We can use these templates and can implement different concepts for our own data types. Equally is true about the use of the array data type. Our code will become very small with the use of this tested facility. Similarly, there is no bug or error in it. Thus, if we have a tested and tried code base, we should try our best to write programs by using it. STL is a lot of important code, pre-developed for us. It is available as a library. We can write programs by using it. Thus our programs will be small and error free.

© Copyright Virtual University of Pakistan

Lecture No. 43

Reading Material

Lecture 1, Lecture 25 - Lecture 42

Summary

- Programming Exercise - Matrices
- Design Recipe
- Problem Analysis
- Design Issues and Class Interface

Programming Exercise - Matrices

Mathematics is a good domain to develop different classes and programs. For example, solutions for Complex numbers, Matrices and Quadratic Equations can be sought for developing our own classes. In this lecture, we will take a problem to manipulate and perform different operations on Matrices. Matrices are used in lot of real world problems. We will perform problem analysis, design and implementation.

Let's take a look at analysis and design phases first by using our design recipe.

Design Recipe

Firstly we do analysis and try to come up with a problem statement. Express its essence, abstractly and with examples. After describing the problems in few sentences, we try to formulate the problem with examples. It is emphasized to pay attention to the details. We do analysis of the data structures to be used in the program and choose the best fit to the program requirements. The code is written to implement the program. After implementation is completed, we do its testing to verify that it is behaving properly in all scenarios. If any bugs are found, they are fixed. This cycle of testing and bug fixing continues until the program is working perfectly without any problem.

We are going to write a program to manage operations on Matrices.

At the start of the problem analysis phase, let's try to understand the problem domain first.

Problem Analysis

A matrix is nothing but a two-dimensional array of numbers. It is normally represented in rows and columns. A matrix is represented as:

© Copyright Virtual University of Pakistan

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

It is a matrix *A* with 3 rows and 4 columns. So order of the matrix is 3 * 4.
Before going further, let's consider what are the operations normally performed on matrices.

- A matrix is added to another matrix.
- A scalar value (an ordinary number) is added to a matrix.
- A matrix is subtracted from another matrix.
- A scalar number is subtracted from a matrix.
- A matrix is multiplied with another matrix.
- A scalar number is multiplied with a matrix.
- A matrix is divided by a scalar.
- A matrix is transposed.

Now, we will define what these operations are and if there are any restrictions on matrices performing these operations.

The sum or addition of two matrices of the same order is found by adding the corresponding elements of the two matrices. If *A* and *B* are two matrices of order m * n to be added then their resultant matrix will also have the same order m * n.

$$A_{ij} + B_{ij}$$

Where i varies from 1 to m (max number of rows) and j varies from 1 to n (max number of cols).
Clearly, there is a restriction on the matrices performing this addition operation that they should have same numbers of rows and columns, in other words their order should be the same.
There is another operation of addition of scalar number to a matrix. In this operation, a number is added to all elements of the matrix.

Subtraction operation works in the same fashion that two matrices of the same order takes part in this operation and resultant matrix with similar order is obtained by subtracting each element of one matrix from the corresponding element of other matrix. For example, see the subtraction operation and assignment below:

$$C_{ij} = A_{ij} - B_{ij}$$

$$\begin{bmatrix} -2 & -4 & -5 \\ -2 & 2 & 0 \\ 0 & 0 & 10 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix} - \begin{bmatrix} 3 & 6 & 8 \\ 7 & 4 & 7 \\ 9 & 10 & 1 \end{bmatrix}$$

Not to confuse your understanding with assignment in computer programs, the resultant matrix is put on the left of assignment operator otherwise in Mathematics it is located on the right.
Each element of matrix *B* is subtracted from the corresponding element of the matrix *A* and the resultant goes to matrix C. *C* will have the same number of rows and columns as *A* and *B*.
Similar to the addition, there is another operation for subtracting a scalar from a matrix. In this case, a number is subtracted from each element of the matrix.

© Copyright Virtual University of Pakistan

For Division of a matrix by a scalar, the scalar number divides each element of the matrix. Let *x* be a scalar number and *A* be a matrix then division is represented as:

$$C_{ij} = A_{ij} / x$$

Each element of matrix *A* is divided by the number *x* to produce the corresponding number in the resultant matrix *C*. For example, A_{11} (element in first row and first column of matrix A) is divided by the scalar number *x* to provide C_{11} (element in first row and first column of matrix C).

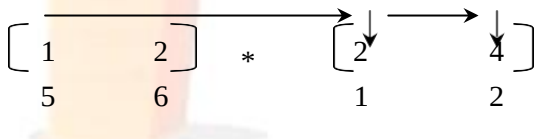
The multiplication operation is bit complicated as compared to the above discussed operations. We will discuss simple case first, when a scalar is multiplied by a matrix. Suppose, this time we want to multiply the scalar *x* with the matrix *A* as:

$$C_{ij} = x * A_{ij}$$

Each element of matrix *A* is multiplied with the scalar *x* and the resultant number is put in the corresponding location inside the matrix C.

Now, we will see how a matrix is multiplied with another matrix. Firstly, there is a restriction on order of the matrices involved in this operation. The number of columns of the first matrix should be equal to the number of rows of the second matrix.
Two matrices are multiplied in the following manner:
We take the first row of first matrix and multiply it with the first column of the second matrix. The multiplication is done in such a way that the first element of the row is multiplied with the first element of the column, second element is multiplied with the second element and so on. The results of all these multiplication operations are added to produce one number. The resultant number is placed at the corresponding position (i.e. 1st row 1st col in this case) in the resultant matrix.
Further the same first row is multiplied with the second column of the second matrix and the resultant number is placed at intersecting position of first row and second column in the resultant matrix. This process goes on till the last column of the second matrix.
Then comes the second row of first matrix and whole operation is repeated for this row, this row is multiplied with all the columns of the second matrix. This process goes on till the last row of the first matrix.

$$\begin{bmatrix} (1)(2)+(2)(1) & (1)(4)+(2)(2) \\ (5)(2)+(6)(1) & (5)(4)+(6)(2) \end{bmatrix}$$

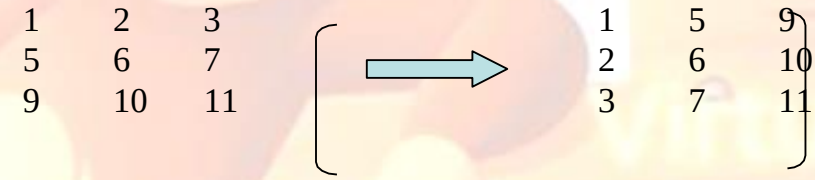


Note the resultant matrix is put on the left of the =. In Mathematics, this is put on right but not to confuse your understanding with assignment concept in computer programs, it is put on left.

© Copyright Virtual University of Pakistan

If a matrix with order *m* rows, *n* columns is multiplied with another matrix of *n* rows and *p* columns then the resultant matrix will have *m* rows and *p* columns. In the above diagram, the first matrix has two rows and second matrix has two columns, therefore, the resultant matrix has two rows and two columns.

Now comes the last operation, we are thinking of implementing i.e. Transpose of a matrix. Transpose of a matrix is obtained by interchanging its rows and columns. How do we interchange rows and columns for transposing the matrix? We take the first row of the matrix and write it as a first column of the new matrix. The second row of the original matrix is written as second column of the new matrix and similarly the last row of the original matrix is written as last column of the new matrix. At the end of this operation, when all rows of the original matrix are finished, we have new matrix as transpose of the original matrix. There is no change in the size (order or number of rows and cols of a matrix) of the transposed matrix when the original matrix is a square matrix. But when the original matrix is not a square matrix, there is a change in the order of the transposed matrix. The number of rows of the original matrix becomes the number of columns of the transposed matrix and the number of columns of the original matrix becomes the number of rows of the transposed matrix.



Until now in this problem analysis phase, we have analyzed the problem in order to understand what are the matrices and what are their operations to be implemented. Now at the next stage, we try to determine the followings:

- What are the constants to be used in our class?
- What are going to be the data structures to cater to the different sized matrices?
- How much memory is required and how it will be allocated?
- What is going to be the interface of the class?

Design Issues and Class Interface

We want to specify the size of the matrix at creation time and allocate the memory for that. So we don't see any use of constants inside our class named *Matrix*.

The size of the memory to be allocated is not going to be huge, as we are not catering to the very huge sized matrices. Therefore, the memory for a matrix is going to be allocated dynamically bluntly after the size of the matrix is specified in terms of rows and columns without worrying about the size of the matrix.

For the interface of our *Matrix* class, we will declare a constructor that will accept integer number of rows and columns of the matrix to be created as parameters.

```
Matrix ( int  rows, int   cols ) ;
```

The constructor function will be doing the memory allocation for the matrix.

© Copyright Virtual University of Pakistan

As part of the interface, we will declare a display function inside our *Matrix* class that will display the elements on the screen.

void display (Matrix &);

To perform already discussed different operations on matrices, we need to overload operators. For example to perform addition of two matrices, + *operator* will be overloaded as a member function of the *Matrix* class. The + *operator* function will be called for the *Matrix* object on the left of the + and the *Matrix* object on the right to it will be passed as a parameter to it. This function will add the corresponding elements of the both matrices and returns the resultant back.

Matrix operator + (Matrix &) const;

The same thing applies to the subtraction operation of two matrices. – *operator* function will be overloaded for that as a member function of the Matrix class.

Matrix operator - (Matrix &) const;

The situation changes a bit, when we want to write the functions to cater to different operations where both the operands are not matrix objects rather one of them is scalar. For example, when we want to do the following operation:

A + x ;

Where A is a matrix and x is a scalar.
Then we write a member function that accepts a scalar number as a parameter instead of a Matrix object.

Matrix operator + (Scalar) const;

The Scalar can be an *int*, *double* or *float*, that we will cover later.

But the situation is more different, when we want to perform the scalar addition operation in the following manner:

x + A ;

By now we should be clear that member function cannot be written to handle this operation because there is a scalar number on the left of +. Therefore, we need to write a *friend operator* function for this type of operation. The friend functions are non-members and therefore, defined outside of the class.

friend Matrix operator + (Scalar , Matrix &) ;

Similarly, when a scalar is subtracted from a *Matrix* object like the following:

A - x ;

A member function is written to cater to this operation.

© Copyright Virtual University of Pakistan

Matrix operator - (Scalar) const;

But again, when a matrix is subtracted from a scalar number:

x - A ;

Then we have to write a *friend operator* to handle this operation.

friend Matrix operator - (Scalar , Matrix &) ;

In order handle the multiplication operations of two Matrix objects like the following:

*A * B ;*

A member *operator* * function is defined.

*Matrix operator * (const Matrix &) ;*

This operator is called for the *Matrix* object on the left of * and the object on the right is passed as an argument. The function multiplies both the matrices and returns the resultant matrix.
When a scalar is multiplied with a scalar like:

*A * x ;*

The following member operator * handles this:

*Matrix operator * (Scalar) const;*

But for operation like the following:

*x * A;*

following friend *operator* function is written:

*friend Matrix operator * (const Scalar , const Matrix &) ;*

For division operation like the following:

A / x;

A member operator / is overloaded as:

Matrix operator / (const Scalar);

Now we will talk about transpose of a matrix. For this operation, we will write a member function *transpose* that will transpose the original matrix.

Matrix & transpose(void) ;

© Copyright Virtual University of Pakistan

Now we are left with few more things to cover to complete the rudimentary interface of our class *Matrix*.

Operators += and -= are overloaded as member operators. These composite operators use the assignment operator (=).

We will also overload stream insertion and extraction operators as *friend* functions to our *Matrix* class as follows:

```
friend ostream &  operator  << ( ostream & , Matrix & ) ;
friend istream &  operator  >> ( istream & , Matrix & ) ;
```

So here is how we declare our *Matrix* class. The interface of the class is the *public* methods of the class. Here is one important point to understand that what we are concerned about here is the class interface and not about the program interface to the user of the program. A programmer can develop user interface by writing his/her code while using the class interface.

```
/* Declaration of the Matrix class. This class is containing the double type elements */

class Matrix
{
   private:
      int numRows, numCols;
      double **elements;
   public:
      Matrix(int=0, int=0);         // default constructor
      Matrix(const Matrix & );      // copy constructor
       ~Matrix();                   // Destructor

       int getRows(void) const;      // Utility fn, returns no. of rows
       int getCols(void) const;      // Utility fn, returns no. of columns
       const Matrix & input(istream &is = cin);  // Read matrix from istream
       const Matrix & input(ifstream &is);       // Read matrix from istream
       void output(ofstream &os) const;          // Utility fn, prints matrix with graphics
       void output(ostream &os = cout) const;    // Utility fn, prints matrix with graphics

       const Matrix& transpose(void);      // Transpose the matrix and return a ref

       const Matrix & operator = (const Matrix &m);     // Assignment operator

       Matrix operator+( Matrix &m) const;       // Member op + for A+B; returns matrix
       Matrix operator + (double d) const;
       const Matrix & operator += (Matrix &m);
       friend Matrix operator + (double d, Matrix &m);

       Matrix operator-( Matrix & m) const;       // Member op + for A+B; returns matrix
       Matrix operator - (double d) const;
       const Matrix & operator -= (Matrix &m);
```


© Copyright Virtual University of Pakistan

```
        friend Matrix operator - (double d, Matrix& m);

        Matrix operator*(const Matrix & m);
        Matrix operator * (double d) const;
        friend Matrix operator * (const double d, const Matrix& m);

        Matrix operator/(const double d);
        friend ostream & operator << ( ostream & , Matrix & );
        friend istream & operator >> ( istream & , Matrix & );
        friend ofstream & operator << ( ofstream & , Matrix & );
        friend ifstream & operator >> ( ifstream & , Matrix & );
        void  display( ) ;
};
```

In the above declarations, we should note how we are passing and returning *Matrix* objects. We are passing and returning the Matrix objects by reference because passing the Matrix objects by value will be a overhead that will affect performance and more memory will be allocated and de-allocated on stack.

Notice that we are doing dynamic memory allocation inside the *constructor* of the class. You must be remembering that wherever the dynamic memory allocation is made, it has to be freed explicitly. To de-allocate the memory, we will write code inside the *destructor* of the class *Matrix*. The other consideration when we are allocating memory on free store from within constructor is that the default *assignment operator* will not work here. Remember, the default *assignment operato*r makes *shallow copy* of the object members, therefore, we will have to write our own *assignment operator* (=) in order to make *deep copy* of the object data members. Remember that a *copy constructor* is called when a new *Matrix* object is initialized and constructed based on an already existent *Matrix* object. Therefore, we have to write our own *copy constructor* in order to make deep copy of the object data members.

There is one very important point to mention about this class *Matrix*. A *Matrix* can be composed of *int*s, *float*s or *double*s as their elements. Instead of handling these data types separately, we can write *Matrix* class as a template class and write code once for all native data types. While writing this template class, the better approach to write will be, to go with a simple data type (e.g. *double*) first to write a *Matrix* class and then extend it to a template class later. Another thing that can be templatized in the *Matrix* class is the *Scalar* number. Actually, this Scalar number can be an *int*, *float* or *double*; therefore, we may also use a template for this.

We have to perform certain checks and make decisions inside the implementation of member functions. For example, while writing the division operator member function, we will check against the number that it should be non-zero. Before adding two matrices, we will check for their number of rows and columns to be equal. Also in this exercise, we have declared only one class *Matrix* to manipulate matrices. There are alternate approaches to this. For example, we could declare a *Row* class first and then contain multiple objects (same in number as number of rows required for the matrix object) of *Row* class inside the *Matrix* class making a matrix of a certain size. To

© Copyright Virtual University of Pakistan

make it simple, we have selected to manage matrices using only one class *Matrix*. The objective here is to practice the already studied programming constructs as much as possible.



© Copyright Virtual University of Pakistan

Lecture No. 44

Reading Material

Lecture 25 - Lecture 43

Summary

- Matrix Class
- Definition of Matrix Constructor
- Destructor of Matrix Class
- Utility Functions of Matrix
  - Input Function
  - Transpose Function
- Code of the Program

Matrix Class

After talking at length about the concept of matrices in the previous lecture, we are going to have a review of 'code' today with special emphasis on concepts of constructors and destructors. We may also briefly discuss where should a programmer return by value, return by reference besides having a cursory look on the usage of the pointers.

The data structure of the *Matrix* class is very simple. We have defined an arbitrary number of functions and operators. You may add and subtract more functions in it. In this lecture, we have chosen just a few as it is not possible to discuss each and every one in a brief discourse. As discussed earlier, the code is available to use that can be compiled and run. This class is not complete and a lot of things can be added to it. You should try to enhance it, try to improve and add to its functionality.

One of the things that a programmer will prefer to do with this class is its templatization. We have implemented it for type *double*. It was done due to the fact that the elements of the *Matrix* are of type *double*. You may like to improve and make it a templatized class so that it can also handle integer elements. This class is written in a very straightforward manner. The *double* is used only for elements to develop it into a *Matrix* class for integers if you replace all the *double* word with the *int*. Be careful, you cannot revert it back to *double* by just changing all the *int* to *double* as integers are used other than element types.

Let's discuss the code beginning with the data structure of the class. In keeping the concepts of data hiding and encapsulation, we have put the data in the private section of the class. We have defined number of rows (i.e. *numRows*) and number of columns

© Copyright Virtual University of Pakistan

(i.e. *numCols*) as integers. These will always be whole number. As we cannot have one and a half row or column, so these are integers.

The private part of the *Matrix* class is:

```
int numRows, numCols;
double **elements;
```

These are fixed and defined. So whenever you have an object of this class, it will have a certain number of rows and certain number of columns. These are stored in the variables- *numRows* and *numCols*. Where will be the values of the elements of this matrix? The next line is *double **elements;* i.e. *elements* is an array of pointers to *double*. First * is for array and second * makes it a pointer. It means that '*elements'* is pointing to a two-dimension array of *double*. When we say *elements[i]*, it means that it is pointing to an array of *double*. If we say *elements[i][j]*, it means we are talking about a particular element of type *double*. We have not taken the two-dimension array in the usual way but going to dynamic memory allocation. We have developed a general *Matrix* class. The objects created from it i.e. the instances of this class, could be small matrices as 2*2. These may also be as big matrix as 20*20. In other words, the size of the matrix is variable. In fact, there is no requirement that size should be square. It may not be 20*20. It may be a matrix of 3*10 i.e. three rows and ten columns. So we have complete flexibility. When we create an object, it will store the number of rows in *numRows* and number of columns in *numCols*. The elements will be dynamically allocated memory in which the *double* value is stored.

While using the dynamic memory, it is good to keep in mind that certain things are necessary to be implemented in the class. 1) Its constructor should make memory allocation. We will use the *new* operator, necessitating the need of defining its destructor also. Otherwise, whenever we create an object, the memory will be allocated from the free store and not de-allocated resulting in the wastage of the memory. Therefore a destructor is necessary while de-allocating memory. 2) The other thing while dealing with classes having dynamic memory allocation, we need to define an assignment operator. If we do not define the assignment operator, the default will do the member wise copy i.e. the shallow copy. The value of pointer will be copied to the pointer but the complete data will not be copied. We will see in the code how can we overcome this problem..

Let's discuss the code in the public interface of the class. The first portion of the public interface is as:

```
Matrix(int=0, int=0);        // default constructor
Matrix(const Matrix & ); // copy constructor
~Matrix();                   // Destructor
```

In the public interface, the first thing we have is the constructors. In the default constructor, you see the number of columns and number of rows. The default values are zero. If you just declare a matrix as *Matrix m*, it will be an object of class *Matrix* having zero rows and zero columns i.e. memory is not allocated yet. Then we have also written a copy constructor. Copy constructor becomes necessary while dealing

© Copyright Virtual University of Pakistan

with dynamic memory allocation in the class. So we have to provide constructor, destructor, assignment operator and the copy constructor. The declaration line of copy constructor is always the same as the name of the class. In the argument list, we have a reference to an object of the same class i.e. *Matrix*(const *Matrix* &); The & represents the reference. This is the prototype. After constructors, we have defined a destructor. Its prototype is also standard. Here it is ~*Matrix*(); it takes no argument and returns nothing. Remember that constructors and destructors return nothing.

After this, we have utility functions for the manipulation of the *Matrix*.

```
int getRows(void) const;                    // Utility fn, returns no. of rows
int getCols(void) const;                    // Utility fn, returns no. of columns

const Matrix & input(istream &is = cin);  // Read from istream i.e. keyboard
const Matrix & input(ifstream &is);         // Read matrix from ifstream

void output(ofstream &os) const;            // Utility fn, prints matrix with graphics
void output(ostream &os = cout) const;  // Utility fn, prints matrix with graphics

const Matrix& transpose(void);            // Transpose the matrix and return a ref
```

We have defined two small functions as *getRows* and *getCols* which will return the number of rows and number of columns of the matrix respectively. Here, you are writing the class and not the client which will use this class. During the usage of this class, there may be need of some more functions. You may need to add some more functionality depending on its usage. At this point, a function may be written which will return some particular row as a vector or the nth column of a matrix. These things are left for you to do.

We need some function to input values into the matrix. There are two input functions both named as *input*. One is used to get the value from the keyboard while the other to get the values from some file. There is a little bit difference between the declaration of these two that will be discussed later. The *input* function which will get the input from the keyboard, takes an argument of type *istream*. Remember that *cin*, that is associated with the keyboard, is of type *istream*. These are member functions and called by some object of *Matrix*. The *Matrix* object will be available to it through *this* pointer. The *istream* is passed as argument and we have given a temporary name to the input argument i.e. *is* and its default value is *cin*. It is a nice way of handling default values. If you write in the main program as *m.input()* where *m* is an object of type *Matrix*, it will get the input from the keyboard. This is due to the fact that it is getting *cin* by default. We have defined another *input* function and that is an example of function overloading. This function takes *ifstream* as input argument i.e. a file stream. If we want to read the matrix data from some file, this *input* function may be employed. There is no default argument in it.

Similarly we have two *output* functions. The names are chosen as arbitrary. You may want to use print or display. One output function will display the matrix on the screen. Its argument is *ostream* while the default value will be *cout*. The other output function will be used when we want to write the matrix in some file. It takes *ofstream* as

© Copyright Virtual University of Pakistan

argument and we have not provided any default argument to it. You will have to provide a file handle to use this function.

Let's continue to talk about the arithmetic manipulations we want to do with the matrices.

```
Matrix operator+( Matrix &m) const;  // Member op + for A+B; returns matrix
Matrix operator + (double d) const;  // Member op + for A+d; returns matrix
const Matrix & operator += (Matrix &m); // Member op += for A +=B
friend Matrix operator + (double d, Matrix &m); // friend operator for d+A

Matrix operator-( Matrix & m) const;  // Member op - for A-B; returns matrix
Matrix operator - (double d) const;       // Member op - for A-d;
const Matrix & operator -= (Matrix &m);  // Member op -= for A-=B;
friend Matrix operator - (double d, Matrix& m); // Friend op - for d-A;

Matrix operator*(const Matrix & m); // Member op * for A*B;
Matrix operator * (double d) const; // Member op * for A*d;
friend Matrix operator * (const double d, const Matrix& m);//friend op*,  d*A

const Matrix& transpose(void);      // Transpose the matrix and return a ref
const Matrix & operator = (const Matrix &m);     // Assignment operator
```

We have defined different functions for plus. Some of these are member operators while the others called as friend. The first plus operator is to add two matrices. It is a member operator that takes a *Matrix* object as argument. The second one is to add some *double* number to matrix. Remember that we are having a class of *Matrix* with *double* elements. So we will add *double* number to it. It is also a member operator. When you write something like *m + d*, where *m* is an object of type *Matrix* and *d* is a *double* variable, this operator will be called. Here *Matrix* object is coming on the left-hand side and will be available inside the operator definition by this pointer. On the other hand, the *double* value *d* is presented as argument to the operator.

There is another variety of adding double to the matrix i.e. if we write as *d + m*. On the left-hand side, we don't have *Matrix*. It cannot be a member function as the driving force is not an object of our desired class. If not a member function, it will have two arguments. First argument will be of type *double* while the second one is going to be of *Matrix* object in which we will add the *double* number. As it is not a member function and we want to manipulate the private data members of the class, it has to be a friend function. Function will be defined outside, at file level scope. In other words, it will not be a member function of class *Matrix* but declared here as a friend. It is defined as:

```
Friend Matrix operator + (double d, Matrix &m);
```

Its return type is *Matrix*. When we add a *double* number, it will remain as *Matrix* .We will be able to return it. The final variant is included as an example of code reuse i.e. the += operator. We write in our program *i += 3;* and it means *i = i + 3*; It would be nice if we can write *A += B* where *A* and *B* both are matrices.

© Copyright Virtual University of Pakistan

After plus, we can do the same thing with the minus operator. Having two matrices-*A* and *B*, we want to do *A-B*. On the left hand side, we have *Matrix*, so it will be a member operator. The other matrix will be passed as argument. In *A-B*, *A* is calling this operator while *B* being passed as argument. We can also do *A-d* where *A* is a *Matrix* and *d* is of type double. For this, we will have to write a member operator. All these operators are overloaded and capable of returning *Matrix*. In this overloaded operator, *double* will be passed as argument. Then we might want to do it as *d - A*, where *d* is *double* variable and *A* is of type *Matrix*. Since the left hand side of the minus (-) is *double*, we will need a friend function, as it is not possible to employ a member function. Its prototype is as:

```
friend Matrix operator - (double d, Matrix& m);
```

Let's now talk about multiplication. We have discussed somewhat about it in the previous lecture. For multiplication, the first thing one needs to do is the multiplication of two matrices i.e. *A*B* where *A* and *B* both are matrices. As on the left hand side of the operator * we have a *Matrix* so it will be a member function so *A* can be accessed through *this* pointer. *B* will be passed as argument. A matrix should be returned. Thus, we have a member operator that takes an argument of type *Matrix* and returns a *Matrix*. You may like to do the same thing, which we did with the plus and minus. In other words, a programmer will multiply a matrix with a *double* or multiply a *double* with a matrix. In either case, we want that a matrix should be returned. So at first, *A * d* should be a member function. Whereas *d * A* will be a friend function. Again the return type will be a *Matrix*.

In case of division, we have only one case i.e. the division of the matrix with a *double* number. This is *A / d* where *A* is a *Matrix* while *d* is a *double* variable. We will divide all the elements of the matrix with this *double* number. This will return a matrix of the same size as of original.

Taking benefit of the stream insertion and extraction operator, we can use double greater than sign (>>) and write as >> *m* where *m* is a *Matrix*. For this purpose, we have written stream extraction operator. The insertion and extraction functions will be friend functions as on the left-hand side, there will be either input stream or output stream.

We have also defined assignment function in it. As we are using dynamic memory allocation in the class, assignment is an important operator. Another important function is transpose, in which we will interchange the rows into columns and return a *Matrix*. This is the interface of our *Matrix* class. You may want to add other functions and operators like +=, -=, *=. But it is not possible to add /= due to its very limited scope.

We have used the keyword const in our class. You will find somewhere the return type as *Matrix* and somewhere as *Matrix* &. We will discuss each of these while dealing with the code in detail.

© Copyright Virtual University of Pakistan

Definition of Matrix Constructor

Let's start with the default constructor. Its prototype is as under:

```
Matrix(int = 0, int = 0);  // default constructor
```

We are using the default argument values here. In the definition of this function, we will not repeat the default values. Default values are given at one place. The definition code is as:

```
Matrix::Matrix(int row, int col)        //default constructor
{
   numRows = row;
   numCols = col;

   elements = new (double *) [numRows];

   for (int  i = 0; i < numRows; i++){
      elements[i] = new double [ numCols];

      for(int j = 0; j < numCols; j++)
            elements[i][j] =0.0; // Initialize to zero
   }
}
```

Two integers are passed to it. One represents the number of rows while the other is related to the number of columns of the *Matrix* object that we want to create. At first, we will assign these values to *numRows* and *numCols* that form a part of the data structure of our class. Look at the next line. We have declared *elements* as ***elements* i.e. the array of pointers to *double*. We can directly allocate it by getting the elements of *numRows* times *numCols* of type *double* from free store. But we don't have the *double* pointer. Therefore, first of all, we say that element is array of pointer to *double* and allocate pointers as much as needed. So we use the *new* operator and use *double** cast. It means that whatever is returned is of type pointer to *double*. How many pointers we need? This is equal to number of rows in the matrix. There is one pointer for each of the rows i.e. now a row can be an array. This is a favorable condition, as we have to enter data in the columns. Now *elements* is *numRows* number of pointers to *double*. After having this allocation we will run a loop.

In C/C++ languages, every row starts from zero and ends with *upper-limit – 1*. Now in the loop, we have to allocate the space for every *elements[i];* How much space is needed here? This will be of type *double* and equal to number of columns. So when we say *new double[numCols],* it means an array of *double*. As *elements[i]* represents a pointer and now it is pointing to an array. Remember that pointers and arrays are synonymous. Now a pointer is pointing to this array. We have the space now and want to initialize the matrix. For this, we have written another loop. To assign the value, we will write as:

```
elements[i][j] = 0.0;
```


© Copyright Virtual University of Pakistan

Let's review this again. First of all we have assigned the values to *numRows* and *numCols*. Later, we allocated the pointers of *double* from the free store and assigned to *elements*. Then we got the space for each of this pointer to store *double*. Finally, we initialized this space with 0.0. Now we have a constructive matrix.

Is there any exceptional value? We can think of assigning negative values to number of rows or number of columns. If you want to make sure that this does not happen, you can put a test by saying that *numRows* and *numCols* must be greater than or equal to zero. Passing zero is not a problem as we have zero space and nothing happens actually. Now we get an empty matrix of dimension 0*0. But any positive number supplied will give us a constructor and initialize zero matrix.

Let's discuss the other constructor. This is more important in the view of this class especially at a time when we are going to do dynamic memory allocation. This is copy constructor. It is used when we write in our program as *Matrix A(B);* where *A* and *B* both are matrices. We are going to construct *A* while *B* already exists. Here we are saying that give us a new object called *A* which should be identical to the already existing object *B*. So it is a copy constructor. We are constructing an object as a copy of another one that already exists. The other usage of this copy constructor is writing in the main function as *Matrix A = B;* Remember that this is declaration and not assignment statement. Here again copy constructor will be called. We are saying that give us a duplicate of *B* and its name should be *A*. Here is the code of this function:

```
Matrix::Matrix(const Matrix &m)
{
   numRows = m.numRows;
   numCols = m.numCols;
   elements = new (double *) [numRows];
   for (int  i = 0; i < numRows; i++){
      elements[i] = new double [ numCols];
      for(int j = 0; j < numCols; j++)
            elements[i][j] = m.elements[i][j];
   }
}
```

We are passing it a constant reference of *Matrix* object to ensure that the matrix to be copied is not being changed. Therefore we make it *const*. We are going to create a brand new object that presently does not exist. We need to repeat the code of regular constructor except the initialization part. Its rows will be equal to the rows of the object supplied i.e. *numRows = m.numRows*. Similarly the columns i.e. *numCols= m.numCols*. Now we have to allocate space while using the same technique earlier employed in case of the regular constructor. In the default constructor, we initialize the elements with zero. Here we will not initialize it with zero and assign it the value of *Matrix m* as *elements[i][j] = m.elements[i][j]*. Remember that we use the dot operator to access the data members. We have not used the dot operator on the left hand side. This is due to the fact that this object is being constructed and available in this function through *this* pointer. Therefore the dot operator on the left hand side is not needed. We will use it on the right hand side to access the data members of *Matrix m*. This is our copy constructor. In this function, we have taken the number of rows and columns of the object whose copy is being made. Then we allocate it the space

© Copyright Virtual University of Pakistan

and copy the values of elements one by one. The other thing that you might want to know is the use of nested loop both in regular constructor and the copy constructor.

Destructor of Matrix Class

'Destructor' is relatively simple. It becomes necessary after the use of new in the constructor. While creating objects, a programmer gets memory from the free store. So in the destructor, we have to return it. We will do it as:

```
delete [] elements;
```

Remember that '*elements*' is the variable where the memory has been allocated. The [] simply, indicates that it is an array. The compiler automatically takes care of the size of the array and the memory allocated after being returned, goes back to the free store. There is only one line in the destructor. It is very simple but necessary in this case.

Utility Functions of Matrix

The functions *getRows()* and *getCols()* are relatively simple. They do not change anything in the object but only read from the object. Therefore we have made this function constant by writing the *const* keyword in the end. It means that it does not change anything. The code of the *getRows()* functions is as follows:

```
int Matrix :: getRows ( ) const
{
   return numRows;
}
```

This function returns an *int* representing the number of rows. It will be used in the main function as *i = m.getRows();* where *i* is an *int* and *m* is a *Matrix* object. Same thing applies to the *getCols()* function. It is of type *const* and returns an *int* representing the number of columns.

Let's talk about little bit more complicated function. It is the output to the screen functions. We want that our matrix should be displayed on the screen in a beautiful way. You have seen that in the books that matrix is written in big square brackets. The code of the function is as:

```
void Matrix::output(ostream &os) const
{
   // Print first row with special characters
   os.setf(ios::showpoint);
   os.setf(ios::fixed,ios::floatfield);
   os << (char) 218;

   for(int j = 0; j < numCols; j++)
         os << setw(10) << " ";

   os << (char) 191 << "\n";
```


© Copyright Virtual University of Pakistan

```
// Print remaining rows with vertical bars only
for (int i = 0; i < numRows; i++){
    os << (char) 179;
    for(int j = 0; j < numCols; j++)
          os << setw(10)<< setprecision(2) << elements[i][j];
    os << (char) 179 << "\n";
}

 // Print last row with special characters
 os << (char) 192;
 for(int j = 0; j < numCols; j++)
       os << setw(10) << " ";

 os << (char) 217 << "\n";
}
```

We have used special characters that can be viewed in the command window. We have given you an exercise of printing the ASCII characters on the screen. After ASCII code 128, we have special graphic symbols. We have used the values of those symbols here. To print those in the symbol form, we have forced it to be printed as *char*. If we do not use the *char*, it would have printed the number 218 i.e. it would have written an integer. We have forced it to print the character whose ASCII value is 218. Now it prints the graphic symbol for that character. We have referenced the ASCII table and seen which symbol will fit in the left corner i.e. 218. So we have written it as:

```
os << (char) 218;
```

os is the output stream. *char* is forcing it to be print as character. The left corner will be printed on the screen as ⌈. Now we need the space to print the columns of the matrix. In the first line we will print the spaces using a loop as:

```
for(int j = 0; j < numCols; j++)
  os << setw(10) << " ";
```

Here we have changed the width as 10. You can change it to whatever you like. Then we print nothing in the space of ten characters and repeat that for the number of columns in the matrix. After this, we printed the right corner. This is the first line of the display. Other lines will also contain the values of the elements of the matrix. These lines will start with a vertical line and then the element values of the row and in the end we have a vertical line. For each row, we have a vertical bar and the number values which will be equal to number of columns (elements in each row equals to the number of columns) and then a vertical bar in the end. In the beginning of this code, we have used two other utilities to improve the formatting. Here we have a matrix of type *double* so every element of the matrix is *double*. For *double*, we have used *os.setf(ios::fixed,ios::floatfield);* that means that it is a fixed display. Scientific notation will not be used while decimal number is displayed with the decimal point. After this, we have set the precision with two number of places. So we have a format and our decimal numbers will always be printed with two decimal places. The numbers are being printed with a width of ten characters so the last three places will

© Copyright Virtual University of Pakistan

be as *.xx*. Rest of the code is simple enough. We have used the nested loops. Whenever you have to use rows and columns, it will be good to use nested loops. When all the rows have been printed, we will print the below corners. We referenced the ASCII table, got the graphic symbol, printed it, left the enough space and then printed the other corner. The matrix is now complete. When this is displayed on the screen, it seems nicely formatted matrix with graphic symbols. That is our basic output function.

Let's look at the file output function. While doing the output on the screen, we made it nicely formatted. Now you may like to store the matrix in a file. While storing the matrix in the file, there is no need of these lines and graphic symbol. We only need its values to read the matrix from the file. So there is a pair of functions i.e. output the matrix in the file and input from the file. To write the output function, we actually have to think about the input function.

Suppose, we have declared a 2*2 *Matrix m* in our program. Somewhere in the program, we want to populate this matrix from the file. Do we know that we have a 2*2 matrix in the file. How do we know that? It may 5*5 or 7*3 matrix. So what we need to do is somehow save the number of rows and columns in the file as well. So the output function that puts out on the file must put out the number of rows and number of columns and then all of the elements of the matrix. Following is the code of this function:

```
void Matrix::output(ofstream &os) const
{
    os.setf(ios::showpoint);
    os.setf(ios::fixed,ios::floatfield);

    os << numRows << " " << numCols << "\n";

    for (int i = 0; i < numRows; i++){

        for(int j = 0; j < numCols; j++)
            os << setw(6) << setprecision(2) << elements[i][j];

        os << "\n";
    }
}
```

The code is shorter than the other output function due to non-use of the graphical symbols. First of all, we output the number of rows and number of columns. Then for these rows and columns, data elements are written. We have also carried out a little bit formatting. While seeing this file in the notepad, you will notice that there is an extra line on the top that depicts the number of rows and columns.

Input Functions

Input functions are also of two types like output functions. The first function takes input from the keyboard while the other takes input from the file. The function that takes input from the keyboard is written in a polite manner because humans are interacting with it. We will display at the screen "Input Matrix size: 3 rows by 3

© Copyright Virtual University of Pakistan

columns" and it will ask "Please enter 3 values separated by spaces for row no. 1" for each row. Spaces are delimiter in C++. So spaces will behave as pressing enter from the keyboard. If you have to enter four numbers in a row, you will enter as number (space) number (space) number (space) number before pressing the enter key. We have a loop inside which will process input stream and storing these values into *elements[i][j];* So the difference between this function and the file input function is 1) It prompts to the user and is polite. 2) It will read from the keyboard and consider spaces as delimiter.

The other input function reads from the file. We have also stored the number of rows and number of columns in the file. The code of this function is:

```
const Matrix & Matrix::input(ifstream &is)
{
    int Rows, Cols;
    is >> Rows;
    is >> Cols;
    if(Rows > 0 && Cols > 0){
        Matrix temp(Rows, Cols);
        *this = temp;
        for(int i = 0; i < numRows; i++){
            for(int j = 0; j < numCols; j++){
                is >> elements[i][j];
            }
        }
    }
    return *this;
}
```

First of all, we will read the number of rows and number of columns from the file. We have put some intelligence in it. It is better to check whether *numRows* and *numCols* is greater than zero. If it is so, then do something. Otherwise, there is nothing to do. If rows and columns are greater than zero, then there will be a temporary matrix specifying its rows and columns. These values are read from the file, showing that we have a matrix of correct size. Now this matrix is already initialized to zero by our default constructor. We can do two things. We can either read the matrix, return the value or we can first assign it to the matrix that was calling this function. We have assigned it first as **this = temp;* here *temp* is a temporary matrix which is created in this function but **this* is whatever this points to. Remember that this is a member function so *this* pointer points to the matrix that is calling this function. All we have to do is to assign the *temp* to the matrix, which is calling this function. This equal to sign is our assignment operator, which we have defined in our *Matrix* class. If the dimensions of the calling matrix are not equal to the *temp* matrix, the assignment operator will correct the dimensions of the calling matrix. It will assign the values, which in this case is zero so far. Now we will read the values from the file using the nested loops. The other way is to read the values from the file and populate the *temp* matrix before assigning it to the calling matrix in the end. That is the end of the function. Remember that the *temp* matrix, which we have declared in this function, will be destroyed after the exit from the function. This shows that the assignment operator is important here. All the values will be copied and it will perform a deep

© Copyright Virtual University of Pakistan

copy. Does this function return something? Its return type is reference to a *const Matrix*. Its ending line is return **this* that means return whatever *this* points to and it is returned as reference. The rule of thumb is whenever we are returning the *this* pointer, it will be returned as a reference because this is the same object which is calling it. When you are returning a matrix that is not a reference, it is a returned by value. The complete matrix will be copied on the stack and returned. This is slightly wasteful. Yet you cannot return a reference to the *temp* object in this code. The reference of the *temp* will be returned but destroyed when the function is finished. The reference will be pointing to nothing. So you have to be careful while returning a reference to *this*.

Transpose Function

The transpose of a matrix will interchange rows into columns. There are two alternative requirements. In the first case, we have a square matrix i.e. the number of rows is equal to number of columns. In this situation, we don't need extra storage to do this. If the number of rows is not equal to the number of columns, then we have to deal it in a different way. We can use general case for both purposes but you will notice that it is slightly insufficient. Here is the code of the function.

```
const Matrix & Matrix::transpose()
{
    if(numRows == numCols){    // Square matrix
        double temp;
        for(int i = 0;i < numRows; i++){
            for(int j = i+1; j < numCols; j++){
                temp = elements[i][j];
                elements[i][j] = elements[j][i];
                elements[j][i] = temp;
            }
        }
    }
    else  // not a square matrix
    {
        Matrix temp(numCols, numRows);
        for(int i = 0; i < numRows; i++){
            for(int j = 0; j < numCols; j++){
                temp.elements[j][i] = elements[i][j];
            }
        }
        *this = temp;
    }
    return *this;
}
```

In the beginning, we checked the case of square matrix i.e. if the number of rows is equal to number of columns. Here we are dealing with the square matrix. We have to change the rows into columns. For this purpose, we need a temporary variable. In this case, it is a variable of type *double* because we are talking about a *double* matrix. Look at the loop conditions carefully. The outer loop runs for *i = 0* to *i < numRows*

© Copyright Virtual University of Pakistan

and the inner loop runs from *j = i+1* to *j < numCols*. Then we have standard swap functionality. We have processed one triangle of the matrix. If you start the inner loop from zero, think logically what will happen. You will interchange a number again and again, but nothing will happen in the end, leaving no change in the matrix. This is the case of the square matrix. But in case of non-square matrix i.e. the code in the else part, we have to define a new matrix. Its rows will be equal to the columns of the calling matrix and its columns will be equal to the number of rows. So we have defined a new *Matrix temp* with the number of rows and columns interchanged as compared to the calling matrix. Its code is straightforward. We are doing the element to element copy. The difference is, in the loop we are placing the *x row, y col* element of the calling matrix to *y row, x col* of the *temp* matrix. It is an interchange of the rows and columns according to the definition of the transpose. When we have all the values copied in the *temp*. We do our little magic that is **this = temp*. Which means whatever *this* points to, now assigned the values of the matrix *temp*. Now our horizontal matrix becomes vertical and vice versa. In the end, we return *this*. This is the basic essence of transpose code.

We will continue the discussion on the code in the next lecture. We will look at the assignment operator, stream operator and try to recap the complete course.

Code of the Program

The complete code of the matrix class is:

```
#include <iostream.h>
#include <iomanip.h>
#include <stdlib.h>
#include <stdio.h>
#include <fstream.h>

class Matrix
{
   private:
      int numRows, numCols;
      double **elements;
   public:
      Matrix(int=0, int=0);         // default constructor
      Matrix(const Matrix & );      // copy constructor
       ~Matrix();                   // Destructor

       int getRows(void) const;     // Utility fn, returns no. of rows
       int getCols(void) const;     // Utility fn, returns no. of columns
       const Matrix & input(istream &is = cin);  // Read matrix from istream
       const Matrix & input(ifstream &is);       // Read matrix from istream
       void output(ofstream &os) const;          // Utility fn, prints matrix with graphics
       void output(ostream &os = cout) const;    // Utility fn, prints matrix with graphics

       const Matrix& transpose(void);      // Transpose the matrix and return a ref

       const Matrix & operator = (const Matrix &m);     // Assignment operator
```


© Copyright Virtual University of Pakistan

```
        Matrix operator+( Matrix &m) const;       // Member op + for A+B; returns matrix
        Matrix operator + (double d) const;
        const Matrix & operator += (Matrix &m);
        friend Matrix operator + (double d, Matrix &m);

        Matrix operator-( Matrix & m) const;      // Member op + for A+B; returns matrix
        Matrix operator - (double d) const;
        const Matrix & operator -= (Matrix &m);
        friend Matrix operator - (double d, Matrix& m);

        Matrix operator*(const Matrix & m);
        Matrix operator * (double d) const;
        friend Matrix operator * (const double d, const Matrix& m);

        Matrix operator/(const double d);
        friend ostream & operator << ( ostream & , Matrix & );
        friend istream & operator >> ( istream & , Matrix & );
        friend ofstream & operator << ( ofstream & , Matrix & );
        friend ifstream & operator >> ( ifstream & , Matrix & );


};
Matrix::Matrix(int row, int col)       //default constructor
{
   numRows = row;
   numCols = col;
   elements = new (double *) [numRows];
   for (int  i = 0; i < numRows; i++){
      elements[i] = new double [ numCols];
      for(int j = 0; j < numCols; j++)
            elements[i][j] = 0; // Initialize to zero
   }
}

Matrix::Matrix(const Matrix &m)
{
   numRows = m.numRows;
   numCols = m.numCols;
   elements = new (double *) [numRows];
   for (int  i = 0; i < numRows; i++){
      elements[i] = new double [ numCols];
      for(int j = 0; j < numCols; j++)
            elements[i][j] = m.elements[i][j];
   }
}

Matrix::~Matrix(void)
{
    delete [] elements;
```


© Copyright Virtual University of Pakistan

```
}

int Matrix :: getRows ( ) const
{
 return numRows;
}

int Matrix :: getCols ( ) const
{
 return numCols;
}
void Matrix::output(ostream &os) const
{
    // Print first row with special characters
   os.setf(ios::showpoint);
   os.setf(ios::fixed,ios::floatfield);
    os << (char) 218;
    for(int j=0; j<numCols; j++)
          os << setw(10) << " ";
    os << (char) 191 << "\n";
    // Print remaining rows with vertical bars only
    for (int i=0; i<numRows; i++){
       os << (char) 179;
       for(int j=0; j<numCols; j++)
             os << setw(10)<< setprecision(2) << elements[i][j];
       os << (char) 179 << "\n";
     }
     // Print last row with special characters
    os << (char) 192;
    for(int j=0; j<numCols; j++)
          os << setw(10) << " ";
    os << (char) 217 << "\n";

}

void Matrix::output(ofstream &os) const
{
          os.setf(ios::showpoint);
    os.setf(ios::fixed,ios::floatfield);
    os << numRows << " " << numCols << "\n";
    for (int i=0; i<numRows; i++){
       for(int j=0; j<numCols; j++)
             os << setw(6) << setprecision(2) << elements[i][j];
       os << "\n";
     }

}

const Matrix & Matrix::input(istream &is)
{
```


© Copyright Virtual University of Pakistan

```
    cout << "Input Matrix size: " << numRows << " rows by " << numCols << " columns\n";
    for(int i=0; i<numRows; i++){
        cout << "Please enter " << numCols << " values separated by spaces for row no." << i+1 << ": ";
        for(int j=0; j<numCols; j++){
            cin >> elements[i][j];
        }
    }
    return *this;
}

const Matrix & Matrix::input(ifstream &is)
{
    int Rows, Cols;
    is >> Rows;
    is >> Cols;
    if(Rows>0 && Cols > 0){
        Matrix temp(Rows, Cols);
        *this = temp;
        for(int i=0; i<numRows; i++){
            for(int j=0; j<numCols; j++){
                is >> elements[i][j];
            }
        }
    }
    return *this;
}

const Matrix & Matrix::transpose()
{
    if(numRows == numCols){    // Square matrix
        double temp;
        for(int i=0; i<numRows; i++){
            for(int j=i+1; j<numCols; j++){
                temp = elements[i][j];
                elements[i][j] = elements[j][i];
                elements[j][i] = temp;
            }
        }
    }
    else
    {
        Matrix temp(numCols, numRows);
        for(int i=0; i<numRows; i++){
            for(int j=0; j<numCols; j++){
                temp.elements[j][i] = elements[i][j];
            }
        }
        *this = temp;
```


© Copyright Virtual University of Pakistan

```
    }
    return *this;
}

const Matrix & Matrix :: operator = ( const Matrix & m )
{
  if( &m != this){
    if (numRows != m.numRows || numCols != m.numCols){
      delete [] elements;
      elements = new (double *) [m.numRows];
      for (int  i = 0; i < m.numRows; i++)
         elements[i]=new double[m.numCols ];
    }
    numRows = m.numRows;
    numCols = m.numCols;
    for ( int i=0; i<numRows; i++){
       for(int j=0; j<numCols; j++){
            elements[i][j] = m.elements[i][j];
       }
    }
  }
  return *this;
}

Matrix Matrix::operator + (  Matrix &m ) const
{
  // Check for conformability
  if(numRows == m.numRows && numCols == m.numCols){
    Matrix temp(m);
    for (int i = 0; i < numRows; i++){
      for (int j = 0; j < numCols; j++){
          temp.elements[i][j] += elements[i][j];
      }
    }
    return temp ;
   }
}

Matrix Matrix::operator + (  double d ) const
{
    Matrix temp(*this);
    for (int i = 0; i < numRows; i++){
      for (int j = 0; j < numCols; j++){
          temp.elements[i][j] +=  d;
      }
    }
  return temp ;
}

const Matrix & Matrix::operator += (Matrix &m)
```


© Copyright Virtual University of Pakistan

```
{
      *this = *this + m;
      return *this;
}

Matrix Matrix::operator - (  Matrix &m ) const
{
 // Check for conformability
 if(numRows == m.numRows && numCols == m.numCols){
   Matrix temp(*this);
   for (int i = 0; i < numRows; i++){
      for (int j = 0; j < numCols; j++){
          temp.elements[i][j] -=   m.elements[i][j];
      }
   }
   return temp ;
  }
}

Matrix Matrix::operator - (  double d ) const
{
   Matrix temp(*this);
   for (int i = 0; i < numRows; i++){
      for (int j = 0; j < numCols; j++){
          temp.elements[i][j] -=  d;
      }
   }
  return temp ;
}

const Matrix & Matrix::operator -= (Matrix &m)
{
      *this = *this - m;
      return *this;
}

Matrix Matrix::operator* ( const Matrix& m)
{
   Matrix temp(numRows,m.numCols);
   if(numCols == m.numRows){
       for ( int i = 0; i < numRows; i++){
          for ( int j = 0; j < m.numCols; j++){
             temp.elements[i][j] = 0.0;
             for( int k = 0; k < numCols; k++){
                 temp.elements[i][j] += elements[i][k] * m.elements[k][j];
             }
          }
      }
   }
   return temp;
```


© Copyright Virtual University of Pakistan

```
}

Matrix Matrix :: operator * ( double d) const
{
   Matrix  temp(*this);
   for ( int i = 0; i < numRows; i++){
      for (int  j = 0; j < numCols; j++){
         temp.elements[i][j] *= d;
      }
   }
   return temp;
}

Matrix operator * (const double d, const Matrix& m)
{
   Matrix   temp(m);
   temp = temp * d;
   return temp;
}

Matrix Matrix::operator / (const double d)
{
      Matrix temp(*this);
      for(int i=0; i< numRows; i++){
            for(int j=0; j<numCols; j++){
                  temp.elements[i][j] /= d;
            }
      }
      return temp;
}

Matrix operator + (double d, Matrix &m)
{
      Matrix temp(m);
      for(int i=0; i< temp.numRows; i++){
            for(int j=0; j<temp.numCols; j++){
                  temp.elements[i][j] *= d;
            }
      }
      return temp;
}

Matrix operator - (double d, Matrix& m)
{
      Matrix temp(m);
      for(int i=0; i< temp.numRows; i++){
            for(int j=0; j<temp.numCols; j++){
                  temp.elements[i][j] = d - temp.elements[i][j];
            }
      }
```


© Copyright Virtual University of Pakistan

```
    return temp;
}

ostream & operator << ( ostream & os, Matrix & m)
{
    m.output();
    return os;
}

istream & operator >> ( istream & is, Matrix & m)
{
    m.input(is);
    return is;
}

ofstream & operator << ( ofstream & os, Matrix & m)
{
    m.output(os);
    return os;
}

ifstream & operator >> ( ifstream & is, Matrix & m)
{
    m.input(is);
    return is;
}

int main()
{
  // declaring two matrices
  Matrix m(4,5), n(5,4);

  // getting input from keyboard
  cout << "Taking the input for m(4,5) and n(5,4) \n";
  m.input();
  n.input();

  // displaying m and taking its transpose
  cout << "Displaying the matrix m(4,5) and n(5,4)\n";
  m.output();
  n.output();

  cout << "Taking the transpose of matrix m(4,5) \n";
  m.transpose();

  cout << "Displaying the matrix m(5,4) and n(5,4) \n";
  m.output();

  cout << "Adding matrices n into m \n";
  m = m + n;
```


© Copyright Virtual University of Pakistan

```
    m.output();

    cout << "Calling m + m + 4 \n";
    m = m + m + 4;
    m.output();

    cout << "Calling m += n \n";
    m += n;
    m.output();

    cout << "Calling m = m - n \n";
    m = m - n;
    m.output();

    cout << "Calling m = m - 4 \n";
    m = m - 4;
    m.output();

    cout << "Calling m -= n \n";
    m -= n;
    m.output();

    m.transpose();

    Matrix c;

    cout << "Calling c = m * n \n";
    c = m * n;
    c.output();

    cout << "Calling c = c * 4.0 \n";
    c = c * 4.0;
    c.output();

    cout << "Calling c = 4.0 * c \n";
    c = 4.0 * c ;
    c.output();

    cout << "Testing stream extraction \n";
//  cin >> c;

    cout << "Testing stream insertion \n";
// cout << c;

    cout << "Writing into the file d:\\junk.txt \n" ;
    ofstream fo("D:/junk.txt");
    fo << c;
    fo.close();

    cout << "Reading from the file d:\\junk.txt \n";
```


© Copyright Virtual University of Pakistan

```
    ifstream fi("D:/junk.txt");
    fi >> c;
    fi.close();

    cout << c;

    system("PAUSE");
    return 0;
}
```

The output of the program is:

```
Taking the input for m(4,5) and n(5,4)
Input Matrix size: 4 rows by 5 columns
Please enter 5 values separated by spaces for row no.1:   1.0  2.0   3.0   4.0  5.0
Please enter 5 values separated by spaces for row no.2:   7.0  5.5   2.3   2.0  1.0
Please enter 5 values separated by spaces for row no.3:   3.3  2.2   1.1   4.4  5.5
Please enter 5 values separated by spaces for row no.4:   9.9  5.7   4.3   2.3  1.5
Input Matrix size: 5 rows by 4 columns
Please enter 4 values separated by spaces for row no.1:   11.25  12.25  13.25   14.25
Please enter 4 values separated by spaces for row no.2:   25.25  50.50  75.75   25.50
Please enter 4 values separated by spaces for row no.3:   15.15   5.75   9.99  19.90
Please enter 4 values separated by spaces for row no.4:   25.50  75.75  10.25   23.40
Please enter 4 values separated by spaces for row no.5:   50.50  75.50  25.25   15.33
Displaying the matrix m(4,5) and n(5,4)

[    1.00    2.00    3.00    4.00    5.00]
[    7.00    5.50    2.30    2.00    1.00]
[    3.30    2.20    1.10    4.40    5.50]
[    9.90    5.70    4.30    2.30    1.50]


[   11.25   12.25   13.25   14.25]
[   25.25   50.50   75.75   25.50]
[   15.15    5.75    9.99   19.90]
[   25.50   75.75   10.25   23.40]
[   50.50   75.50   25.25   15.33]

Taking the transpose of matrix m(4,5)
Displaying the matrix m(5,4) and n(5,4)

[    1.00    7.00    3.30    9.90]
[    2.00    5.50    2.20    5.70]
[    3.00    2.30    1.10    4.30]
[    4.00    2.00    4.40    2.30]
[    5.00    1.00    5.50    1.50]

Adding matrices n into m

[
```


© Copyright Virtual University of Pakistan

```
[  12.25   19.25   16.55   24.15
   27.25   56.00   77.95   31.20
   18.15    8.05   11.09   24.20
   29.50   77.75   14.65   25.70
   55.50   76.50   30.75   16.83]
```

Calling m + m + 4

```
[  28.50   42.50   37.10   52.30
   58.50  116.00  159.90   66.40
   40.30   20.10   26.18   52.40
   63.00  159.50   33.30   55.40
  115.00  157.00   65.50   37.66]
```

Calling m += n

```
[  39.75   54.75   50.35   66.55
   83.75  166.50  235.65   91.90
   55.45   25.85   36.17   72.30
   88.50  235.25   43.55   78.80
  165.50  232.50   90.75   52.99]
```

Calling m = m - n

```
[  28.50   42.50   37.10   52.30
   58.50  116.00  159.90   66.40
   40.30   20.10   26.18   52.40
   63.00  159.50   33.30   55.40
  115.00  157.00   65.50   37.66]
```

Calling m = m - 4

```
[  24.50   38.50   33.10   48.30
   54.50  112.00  155.90   62.40
   36.30   16.10   22.18   48.40
   59.00  155.50   29.30   51.40
  111.00  153.00   61.50   33.66]
```

Calling m -= n

```
[  13.25   26.25   19.85   34.05
   29.25   61.50   80.15   36.90
   21.15   10.35   12.19   28.50
   33.50   79.75   19.05   28.00
   60.50   77.50   36.25   18.33]
```

Calling c = m * n

```
[  5117.55  8866.42   4473.54 3066.94
   7952.36 15379.14   7884.15 5202.50]
```


© Copyright Virtual University of Pakistan

```
 4748.18  8540.74  7566.73  3570.75
 3386.23  5949.35  4280.89  2929.51
```

Calling c = c * 4.0

```
 20470.19  35465.70  17894.15  12267.75
 31809.46  61516.55  31536.59  20810.01
 18992.71  34162.97  30266.91  14283.00
 13544.91  23797.41  17123.54  11718.05
```

Calling c = 4.0 * c

```
 81880.76 141862.80  71576.62  49071.00
127237.84 246066.20 126146.34  83240.04
 75970.86 136651.88 121067.65  57132.02
 54179.64  95189.64  68494.16  46872.18
```

Testing stream extraction
Testing stream insertion
Writing into the file d:\junk.txt
Reading from the file d:\junk.txt

```
 81880.76     0.81     0.62     0.00
127237.84     0.20     0.35     0.04
 75970.86     0.88     0.66     0.02
 54179.65     0.65     0.16     0.18
```

Press any key to continue . . .

© Copyright Virtual University of Pakistan

Lecture No. 45

Reading Material

Lecture 43 - Lecture 44

Summary

- Example (continued)
  - Assignment Operator Function
  - Addition Operator Function
  - Plus-equal Operator Function
  - Overloaded Plus Operator Function
  - Minus Operator Function
  - Multiplication Operator Function
  - Insertion and Extraction Operator Function
  - exercise
- Rules for Programming
- Variables and Pointers
- Arrays
- Loops and decisions
- Classes and Object
- Garbage Collection
- Truth Table
- Structured Query Language

Example (continued)

This is a sequel of the discussion on 'matrix class' made in the previous lectures.

Assignment Operator Function

Before going into minute details, we will talk about the assignment operator of this class. This operator occupies very important place in the field of programming. When we want to write code like *a = b;* where *a* and *b* both are matrices, the assignment operator attains a critical role as our class does dynamic memory allocation. Here we don't know whether the size of *a* and *b* will be the same or not. If these are of different size, then it will be better to reallocate the memory. So it warrants the existence of some checks and balances. At first, we look at the declaration line of the assignment operator, it is written below.

```
const Matrix & operator = (const Matrix &m);
```


© Copyright Virtual University of Pakistan

The declaration line states that it must return a reference to a matrix. Why do we want to return a matrix? The return of the matrix is necessary to enable us write the statement i.e. *a = b = c;* We know, in C and C++ languages, every expression has a value of its own. Now if we write *a = b ;* it will mean that the whole action will have a value. This value itself will be a reference to a matrix. On the other hand, we have made it *const*. In other words, the reference that is being returned is a constant.
Whenever we return a reference of a thing that thing can become on left-hand side of an assignment statement, which can lead to some very funny behavior. If we write *(a = b) = c ;* It will result in the execution of the statement, *a = b.* It will return a value that is a reference to a matrix. This reference will be assigned the value of *c*. this means that some minor things can take place. We will like to have parentheses only on right hand side. It is advisable not to do an assignment to the reference that is being returned. To avoid this, we write *const* with it. Thus, we get the efficiency as reference is being returned. We also enjoy safety due to the fact that no value can assign to this reference in the same statement. Due to the reference returned, we can write the statement like *a = b = c ;*

Let's have a look on the next implication i.e. the size of the matrix. Whenever there is dynamic memory allocation in a class, we have to check against self-assignment. Self-assignment means statements like *a = a ;*. If we take ordinary variables, say integer, writing *i = i ;* is quiet right. But if we write something like *a = a ;* it will be possible to free the memory of object on the left hand side as it is doing some dynamic memory allocation. Now we can assign new memory and copy the right hand side in it. If we write *a = a;* it will lead to a very tricky situation. This means that we, at first, delete it (the left-hand side), as right hand side is the same object with same memory, so it is also deleted. Then, we try to copy the right hand side that has been deleted. So we must check against self-assignment. While dealing with the code of the assignment operator, we will first check whether it is for the self-assignment or not. To ascertain it, we will write the following line.

```
if( &m != this)
```

This way, the self-assignment check has been carried out. In case of not finding self-assignment, the programmer will have to do further process..
After checking the self-assignment, the program checks the size of both the matrices and sees whether these are the same or different. If their size is same, then it will copy the matrix of right hand side element-by-element in the matrix on left-hand side. But if their size is different in terms of number of rows or columns, then we have to create a new matrix for the left-hand side. This code is similar to the one that we wrote in the constructor. We free the memory and reallocate the memory of the correct size. This size is equal to the size of the matrix on right hand side. After re-allocating the memory, we readjust the number of rows and columns of the lef- hand side object. So we write it as

```
numerous = m.numRows ;
numCols = m.numCols ;
```

While defining rows and columns, we execute a *for* loop and copy the elements of right hand side at corresponding positions on left hand side. This way, we define the assignment operator, which can be used in different functions. Here, we can write

© Copyright Virtual University of Pakistan

statement like *a = b which will* work properly. The same thing is applied to our main code when we come to a client function of the class. The code of the function of assignment operator is written as below.

```
const Matrix & Matrix :: operator = ( const Matrix & m )
{
    if( &m != this)
    {
        if (numRows != m.numRows || numCols != m.numCols)
        {
            delete [] elements;
            elements = new (double *) [m.numRows];
            for (int i = 0; i < m.numRows; i++)
            elements[i]=new double[m.numCols ];
        }
        numRows = m.numRows;
        numCols = m.numCols;
        for ( int i=0; i<numRows; i++)
    {
            for(int j=0; j<numCols; j++)
            {
            elements[i][j] = m.elements[i][j];
            }
        }
    }
    return *this;
}
```

Addition Operator Function

Now we will discuss the addition operator. We have discussed a variety of addition operators. We come across one of these while writing *a + b* where *a* and *b* are matrices. While adding two matrices, it is ensured that these are compatible. It means that the matrices are conformable for addition. Their number of rows and columns should be equal. The code, we have just, written is very simple. It first checks whether the matrices are compatible. If so, it does the element-to-element addition. If these are not compatible, it returns the old matrix. Here the thing to remember is, what is being returned? The addition operator returns a new matrix after adding two matrices. The matrices, which were added, remain the same. So, if we add two matrices *a* and *b*, these will remain as it is and addition operator will return a resultant matrix by adding them. Therefore, if we find the matrices are compatible, a new matrix is defined. Having defined the new matrix, we can apply some tricks to it. We can define the new matrix by using copy constructor. So a complete matrix will be copied to it. This is reflective from the following statement.

```
Matrix temp (m) ;
```

When we a copy of one matrix, i.e. *temp*, the elements of the other matrix are added to it. We do this with a loop. After this, temp is returned. This temporary matrix (temp) which was created in the addition operator, is returned. However, its reference can not be returned. Here we have to return a matrix, ignoring the problem of

© Copyright Virtual University of Pakistan

efficiency. So this whole matrix will be copied on the stack and assigned wherever it is needed. If we have written *c = a + b ;* it will be assigned to *c*. The things where the reference or matrix is being returned, should be carried out carefully. It is important for us to know what thing should be returned to where, and what is its usage and behavior? The code of the addition operator function is written in the program as below.

```
Matrix Matrix::operator + (  Matrix &m ) const
{
        // Check for conformability
        if(numRows == m.numRows && numCols == m.numCols)
        {
                Matrix temp(*this);
                for (int  i = 0; i < numRows; i++)
                {
                        for (int j = 0; j < numCols; j++)
                        {
                        temp.elements[i][j] +=   m.elements[i][j];
                        }
                }
                return temp ;
        }
        Matrix temp(*this);
        return temp;
}
```

Plus-equal (+=) Operator Function

Now we will discuss the += operator. Whenever a programmer writes *a += b*, he will come across a different scenario as compared to the one witnessed in the case of the addition operator. In a way, now '*a'* itself is being changed. So if *a* is going to change, we can return a reference to *a*. The conformability check remains the same as in ordinary addition. That means both matrices must have the same number of rows and columns. There is no need of creating a temporary matrix. Here we can return reference to left-hand side matrix. Here one finds that there is reuse and efficiency in the code. The += operator is defined as under.

```
const Matrix & Matrix::operator += (Matrix &m)
{
        *this = *this + m;
        return *this;
}
```

Overloaded plus Operator Function

Next concept to be discussed the overloading of the overloaded plus (+) operator. It is very simple. If we want to add a *double* variable in a matrix, there will be no problem of conformability. All we need to do is add a value to every element. Here we are doing *a + d* (*a* is a matrix while *d* is a *double*). Here, *a* will not change. A new matrix will be returned by adding the value of *d* to the elements of *a*. So it is not returning a reference, but a matrix. If it is returning a matrix, it must return a matrix that is created inside this function. Thus, we can use copy constructor and write

© Copyright Virtual University of Pakistan

```
Matrix temp (* this) ;
```

In this matrix *temp*, we add the value of *d* by a nested loop and return it. This way, the addition of a matrix with a *double* variable is defined. The code of it is given below.

```
Matrix Matrix::operator + ( double d ) const
{
        Matrix temp(*this);
        for (int  i = 0; i < numRows; i++)
        {
                for (int j = 0; j < numCols; j++)
                {
                        temp.elements[i][j] +=  d;
                }
        }
        return temp ;
}
```

Next function is *d + a* (i.e. double variable + matrix). There is a *double* variable and not a matrix on left hand side, leaving no option for having it as a member function. It is a friend function and defined outside the class. We don't use the scope resolution operator (::) with it. It is defined as an ordinary stand-alone function. It still returns a matrix .So it is written as

```
Matrix operator + (double d, Matrix &m)
```

Two arguments are passed to it that are the variables on left and right side of the operator. The remaining code is almost the same as that of *a + d*, and is written as below.

```
Matrix operator + (double d, Matrix &m)
{
        Matrix temp(m);
      for(int i=0; i< temp.numRows; i++)
        {
        for(int j=0; j<temp.numCols; j++)
                {
                temp.elements[i][j] += d;
        }
         }
        return temp;
}
```

Minus Operator (-) Function

The same discussion of plus (+) operator applies to the minus (-) operator as both are identical. We see the difference between these operators when we do *a + d*. In case of addition, it will be the same as that of d + a. However, while dealing with minus case, the result of *a - d* will be obviously different from the result of *d - a*.

© Copyright Virtual University of Pakistan

Multiplication Operator (*) Function

The most complicated operator out of all these arithmetic manipulators for matrices is the multiplication (*) operator. How do we multiply two matrices together? We have already discussed it while defining matrices. We have discussed that the size of the resultant matrix will be the number of rows of the first matrix multiplied by the number of columns of second matrix. We also have discussed the method to calculate the element of the resultant matrix. Obviously, before doing this, we check the conformability of the matrices for multiplication. The code of the function for * operator is defined as below.

```
Matrix Matrix::operator* ( const Matrix& m)
{
        Matrix temp(numRows,m.numCols);
        if(numCols == m.numRows)
        {
                        for ( int i = 0; i < numRows; i++)
                        {
                                        for ( int j = 0; j < m.numCols; j++)
                                {
                                temp.elements[i][j] = 0.0;
                                for( int k = 0; k < numCols; k++)
                                {
                                        temp.elements[i][j]     +=      elements[i][k]     *
m.elements[k][j];
                                }
                                        }
                        }
         }
        return temp;
}
```

The multiplication of a matrix with a *double* is nothing more complicated as that of doing addition of a matrix with a *double*. So code used in both cases is similar. In the case of division of a matrix by a double, the only thing that differs is that while dividing a matrix with a *double*, we have to check the division by zero. After having that little check, we divide the matrix. Without going for some complex method, we simply return the original matrix if it encounters a division by zero. Mathematically speaking, it is not correct. We should actually throw an exception so that program should stop in such a case. But we do not throw an exception and return the original matrix without trying to divide it by zero. So our program does not generate a run time error. There may be logical errors. So we have to be careful in such a case.

Insertion (<<) and Extraction (>>) Operator Function

The last set of functions that we have defined is the stream insertion (<<) and extraction (>>) operators. These operators may be taken as the example of code reuse. We have already defined input and output functions for this class. These input and output can handle matrices with files or on screen. Now we want an operator to write *cin >> m* ; where *m* is an object of type Matrix. It means that we have to read from the keyboard and store these values in the matrix *m*. Inside the code, we can reuse the input function and write *m.input* in the function body. This is the overloaded stream

© Copyright Virtual University of Pakistan

extraction operator. The only difference is that we have to return a reference to the stream. Thus it is a two- line function and is a good example of code reuse. We have written the input function, which can be used here. Same thing applies if we have to take input from the file and put it into matrix *m*. We have declared these functions as friend functions and in the following their code is written

```
istream & operator >> ( istream & is, Matrix & m)
{
        m.input(is);
        return is;
}
```

We will now use the input function, written to take input from file.

```
ifstream & operator >> ( ifstream & is, Matrix & m)
{
        m.input(is);
        return is;
}
```

Similarly, the pair of output functions can be reused to overload the stream insertion operator.

```
ostream & operator << ( ostream & os, Matrix & m)
{
        m.output();
        return os;
}
```

And for the file output the code is as follows.

```
ofstream & operator << ( ofstream & os, Matrix & m)
{
        m.output(os);
        return os;
}
```

Exercise

Now you should study and understand this whole code of the class and use it. Its use is very simple. You can write the main function and in it write

```
Matrix m (3,3) ;
```

When you will execute it, a matrix of three rows and three columns will be created and values of it will be zero. To display this matrix on the screen, you can write

```
m.output ;
```

It will display a properly formatted matrix on the screen. Similarly you can define other matrices and can get their values from the keyboard. You can multiply them and

© Copyright Virtual University of Pakistan

see that multiplication is done only if the matrices are conformable for multiplication. Similarly, addition will work only if the matrices are conformable for addition. You can write a little test program. You should also try to extend the class by adding new functions and features into it. In the code, there are not proper error messages. You can write code to do more error checking and to display proper error messages wherever an error encounters.

It is very simple to change the whole class from *double* to *int*. More complicated one would may be used to write a template for this class. In the class, wherever there is *double*, you will write <T> there and on the top, there will be template <class T>. The remaining things will almost look identical. You will have to take care in friend functions. So there is a lot of stuff you can do with it.

Review

Now we will review the different topics of the course briefly and some discussion in respect of languages and programming. In the beginning of the course, we came across a few programming guidelines. We have read about design recipe. Then we went on and developed the way of thinking.
To begin with the review of the previously discussed subjects, we will now talk about the rules of programming.

Rules for Programming

We need simply three constructs to solve any problem.

1) Things should be executed sequentially. That means the statements are executed in a sequence i.e. the second statement follows the first and so on.

2) We need to have a decision. The decision means if something is true, the program executes it. Otherwise, it tries doing something else. So there is simple decision or if-else decision.

3) The third construct is loop, which is a repetition structure that performs the same task repeatedly with different values.

So the availability of sequences, decisions and loops can help us write any program. The code, we write, should be short and concise. It need to be self-contained and understandable. Comments should be placed liberally. The comments should explain the logic, not the mechanics. Try to avoid fancy programming. The indentation of the code has no means programmatically as it does not mean any thing at all. What actually matters is how you structure the code using braces and semicolons i.e. the structure of the language. Otherwise, C and C++ are free-format languages. We can write the whole code in a single line. But it will be very difficult to read. We format our code so that it could be read easily. So indentation is for us, not for the compiler. Similarly, any language does not dictate it. On the other hand, if we put our code inside braces and blocks, it will ensure a logical syntax.

Variables and Pointers

© Copyright Virtual University of Pakistan

After constructs, the concept of variables and pointers holds very important position in programming. The variable is a name for a value. It is like a label on a box in the memory, which contains a value. We can use this label to manipulate the value, instead of using the address of the memory that contains the value. There are different types of variables.

We discussed earlier, the pointers are much more specific to C and C++. A pointer is an address of a location in the memory. We also have talked about their manipulations.

Arrays

An array is a type of data structure. We use an array to store multiple values of the same data type. In C, C++ and FORTRAN languages, the arrays are of the same data type i.e. every element of the array is of the same data type. There can be an array of integers, an array of characters and so on. We cannot have elements of different types in an array. There are the languages in which we can have arrays of mixed-type. FoxPro is the quotable example in this regard. We can store in a variable whatever we want. For example if we write *a = 3* then *a* is a numerical value. On other hand suppose, if we write *a = "This is a string"*. Here '*a'* becomes a character string. Similarly in Visual Basic, there is a data type, called *variant* that can store data of all kinds. So remember that whenever we talk of arrays and variables, different languages behave differently. There are no hard and fast rules.

Loops and Decisions

The loops and decisions are 'bread and butter' for a programmer.

While talking about decisions, we read that in C and C++ languages, there is a simple *if* statement. There is also '*if-else* statement'. We can write a big structure by using the nested *if-else* statements. There is also *switch* statement. These statements (if, if-else and switch) are language specific. Almost all the modern programming languages provide a decision structure. The exact syntax of which can be got from language reference.

While talking about repetition structure, we come across the concept of loops. The loops are of three different types in C ++. These include *while*, *do-while* and *for* loop. There is a subtle difference between them. While using the *while* loop, if its condition is false at the start, its body will not execute even once. In other words a *while* loop executes zero or more times. On the other hand, if we write a *do-while* loop, the block of code written after the *do* will execute at least once. So a *do-while* loop executes one or more times. The *for* loop is more like the *while* loop. It will execute zero or more times. Every loop has a basic structure that is independent of the language. There is some initialization, a condition that is tested for the execution of the loop. Then there is the body of the loop in which it performs its task. These are almost same in the languages. But the syntax is particular to the language.

Classes and Objects

In this course, we discussed, only the concept of, classes and objects. The study of rudiments of classes and objects can help us understand the difference between implementation and interface besides comprehend the concept of the encapsulation. We combine the data and code to form an object. It is a new type of variable, a user-defined data type. It not only has its data structure but also the code that manipulates

© Copyright Virtual University of Pakistan

the data. The major advantage of data hiding and encapsulation is that it makes every thing tested, debugged and ready to use. When we come in the main program, which uses these classes or objects, our code becomes very simple. We can reuse this code repeatedly. When we put all of this together, the concept of doing object-oriented programming becomes clear. How can a class be made from another class? We will talk about polymorphism in the course of object oriented programming. It can determine what function is to call at the execution time of the program not at the compile time. These are very important and powerful methods. There will be whole idea of thinking objects. Here we only covered mechanics. When we were talking about mechanics, we have to understand how can we implement a member function and a member operator. We use the sequences, decisions and repetition structures while writing the member or friend functions. So we build on our previous knowledge and introduce the concepts of classes and objects.

Garbage Collection

The whole concept of using objects and their notation which is *object.member*, where member could be a data type or a function, that is what we have been exercising. We also mentioned that we could have pointers to objects. During the manipulation of the data variable or data member with pointers, we use the arrow (->) notation rather than the dot (.) notation. The concept of pointers is very important but quite limited to C and C++. The modern languages, for example JAVA, describe pointers as dangerous. We can go anywhere in the memory and can change a value. There is another problem with pointers, which is that these could be pointing to nowhere. For example, we allocate memory and de-allocate it there or in some other function, without reassigning a new memory to the pointer that was pointing to that memory. Thus, a dangling pointer is there that points to nothing. There is also reverse case of it that we assign a memory through a pointer where the pointer is destroyed, the memory remains allocated and is wasted. To address these things, there are only references in JAVA instead of pointers. JAVA gives the concept of garbage collection with the use of references. Due to this garbage collection, we are free from the headache of de-allocating the memory. We allocate and use the memory. When it is no longer in use, JAVA automatically deletes (frees) it through garbage collection. But in C and C++ languages, we have to take care of de-allocating the memory. In classes where we use dynamic memory, we have to provide destructors to free this memory. The languages keep evolving, new constructs will keep evolving in existing or new languages. So the foundations of our knowledge must be strong. We have to know what is programming. We have to know how can we take the essence of a problem by analyzing it. We should repeat the design recipe as many times as needed.

Truth Table

There are some areas where the decision structures become very complicated. Sometimes, we find it difficult to evaluate a complicated logical expression. Sometimes the logic becomes extremely complicated so that even writing it as a simple syntax statement in any language. It becomes complicated to determine what will be evaluated in what way. We know the concept of truth table. The truth tables are very important. These are still a tool available for analyzing logical expressions. We will read logic design in future, which is actually to do with chips and gates. How we put these things together. In logic design, there are certain techniques that are

© Copyright Virtual University of Pakistan

known as minimization techniques. These are used to make a big circuit with the use of minimum chips. These minimization techniques deal with Boolean algebra i.e. logic. These techniques are also used in programming. So we should keep breadth in our vision while maintaining a horizontal integration. We should always think outside the box. There is a way of thinking for us as programmers. We always look at problems, slice and dice them and come up with solutions. Programming as a skill is infact important. It helps us think, from a logical perspective. How can we do it is something else. We can get it from the reference books of the language or from online help in the compiler. This part that how can we do is always changing. New languages will be evolved for our help. On the other hand, what is to be done depends on our logical skills and fundamental knowledge. We have to develop this thing.

Structured Query Language

In the business world, most of the programming is database-oriented. In today's databases, like Oracle and SQL Server, a different kind of language is used. These are the languages that are called as structured query languages i.e. SQL. SQL, is so important that a standard has been developed for it. So there is an ANSI standard for this language. There is a major difference between SQL and the conventional language. The SQL by law says 'tell me what do you want and I will determine how to do it'. Whereas in our conventional languages like C or C++, we have to tell the languages what we want to do and how to do it. These are differentiated in the terminology like third generation languages and fourth generation languages.

There are optimizers built in those languages. Optimizers mean how a query or question can be executed more efficiently. In the same way, there are optimizers in our compilers. When we write the code, the compiler looks into it to determine how this code can be executed more efficiently. The modern compilers do a large optimization. Different software companies or computer manufacturers write the compilers. The standard is the same for writing compilers. The difference is that how much fast the executable version of a program executes and how much memory it uses, when compiled by different compilers. The speed and memory usage is the two yard sticks of output code.

The fundamentals are important. Keeping of a breadth of vision is also critically important. We have to constantly keep up with literature, keep up with new development, and experiment with more and more new tools.

Talking about languages is not that important. However, talking about programming is critically more important. If we have a sound fundamental knowledge, no new language can frighten us. We will never feel over powered by any new language. The fundamentals can become strong only by practicing more and experimenting to the maximum.

© Copyright Virtual University of Pakistan